OPERATION KOMOREBI · REALIZATION CALCULUS

Projector Resolved Seed Calculus III

Release scope and verification record

Version 1.0.0-rc1 · 20 October 2026 · Private prepublication candidate

Publication decision

Technical package assembled and source identities authenticated. Unrestricted formal closeout is not certified. The expanded manuscript is supplied in full, with the historical Lean formal core and the Section 16 continuation. Section 17 gains a new full ordinary proof supplement. Two candidate Lean modules remain uncompiled and are not imported into the accepted development.

What the evidence supports

Layer	Accepted boundary
Finite algebraic core	Historical 41-row formal map and 143 mapped targets, retained with its original evidence.
Section 16	12 recorded production modules, with 558 logical declarations replayed in the inherited record; 14 generated runtime helpers separately inventoried.
Section 17	Ordinary proofs of continuous packet jets, norm estimates, Taylor remainders, and higher composition. No new Lean acceptance.
Independent tests	15,331 retained Section 16 controls, 2,622 Section 17 controls, and 677 new higher-order symbolic checks passed in this closeout.

Critical restriction

The 16 October visual edition asserts 2,693 replayed declarations and 98+2 accepted results. Its matching Lean source and independent receipts were not recovered. Those claims are **not** release certified here. The 8 October visual guide is the selected reproducible edition.

Licensing and distribution

No project-wide manuscript or code license was selected in the preserved materials. This package is a private release candidate, not permission for public redistribution. No license has been inferred or assigned.

Evidence guide and reading order

A bounded, reproducible reading edition

What each document is

Traditional monograph (98 pages). The complete October 8 mathematical edition, unchanged. It includes established theorems, qualified reductions, research directions, and ordinary proofs across the expanded paper.

Analytic proof supplement (5 pages). Additional rigorous Section 17 arguments. In particular, the literal range jet has a sharper norm bound than its ambient extension. No new Lean result is claimed.

Visual guide (26 pages). The source-bound finite companion, with 24 worked plates and 22 exact model figures. This is the earlier edition, deliberately selected over an October 16 edition with unauthenticated formal claims.

Lean and evidence guide (49 pages). Documents the historical finite 41-row contract. It is preserved, not silently retrofitted to claim expanded monograph coverage.

Statement register (167 environments). The JSON and CSV map identifies source hashes and evidence classes, including 98 theorem-like environments and two classical standard-fact environments.

Reproducibility

Run `python VERIFY_RELEASE.py` after extraction to validate all packaged bytes and PDF structure. The exact symbolic tests and inherited proof source/object identity checks have separate commands in the release README. Fresh Lean kernel verification requires the absent pinned 2.9 GB toolchain and dependencies.

What remains open

The two Section 17 draft Lean modules, the analytic derivative API correspondence, exact higher-order formalization, and comprehensive Lean correspondence for the expanded manuscript. The public license decision also remains outstanding. An accepted historical proof transcript is not a new compilation in this environment.

Interpretation

The mathematical language of packet reconstruction and obstruction is usable now. Closing a narrowly stated theorem and proving general existence are different achievements. This release preserves that distinction, including all known failures and unrecovered evidence.

PAPER III

Projector Resolved Seed Calculus III

Typed transfer, nonlinear packets, and obstruction theory

Typed operations, nonlinear coordinates, and correction maps give a common language for reconstruction, transport, and obstruction.

Komorebi P.

Modern Monograph • 8 October 2026

How to use this volume

Begin with typed edge words and exact coordinate transport. A chosen connection introduces defects, holonomy, and packet obstruction witnesses. Part II develops nonlinear packet jets, tree expansions, correction complexes, filtered lifting, elimination, and smooth transport.
Part I retains the finite v1.0.1 formal contract. Part II gives the strengthened nonlinear and analytic development with ordinary proofs. Formal coverage stays attached to the released declarations. The visual atlas at the end of this volume provides twenty-four worked plates. The separate Lean guide explains the formal contract and its evidence.

Overview

Finite projector packets turn linear transfer into typed walks and arbitrary maps into exact coordinate representations. Canonical coordinate transport satisfies an exact cocycle law. A chosen connection can fail to compose, producing holonomy and packet obstruction witnesses. Nonlinear packet jets lead to higher composition, convergent rooted tree expansions, graded operations, and nonlinear refinement defects. A specified correction map produces a canonical cokernel obstruction. Relative exact sequences, mapping cones, Hilbert splitting, nonlinear correction, and filtered lifting describe how these obstructions behave. Exact elimination produces Schur formulas, and finite transport reduces gluing to fundamental cycle holonomy. For smooth orthogonal packet families, direct packet rotations converge in operator norm to canonical unitary transport. The finite formal contract and the broader ordinary proof development retain their separate scopes throughout.

This edition preserves the existing mathematical development and all ninety-five proof bodies. Worked examples and reading boxes explain how to use the constructions. The included Lean evidence comes from the preserved release. Current checks establish source consistency and replay finite computations.

Contents

I Finite typed transfer and the formal contract **1**

1 Purpose and formal status 1
1.1 Statement labels . . . 1

2 Finite typed packet graphs 2
2.1 Orientation and matrix convention . . . 2

3 Operator words and directed walks 4
3.1 Typed words . . . 4
3.2 The checked recursive expansion . . . 4
3.3 Exact expansion from words to walks . . . 5

4 Canonical exact coordinate transport 7
4.1 The Paper III frame model . . . 7
4.2 Exact packet coordinates . . . 8

5 A distinct noncanonical connection API 9
5.1 The minimal compiled API . . . 9
5.2 A stronger reversible pair API . . . 11

5.3 Cocycle defect and triangle holonomy . . . 12
5.4 Paths and endpoint independence . . . 13

6 Gauge covariance and comparison with canonical transport 14

7 Obstruction packets: local certificates and holonomy blocks 15
7.1 The generic compiled certificate . . . 15
7.2 Specialization to holonomy blocks . . . 16
7.3 Gauge behavior of packet blocks . . . 18

8 Products supported in packets and holonomy walks 19

9 A minimal exact example 20

10 Canonical coordinates for arbitrary maps 20

11 Nonlinear operation trees: locality and the formalization frontier 22
11.1 The compiled foundation of unary and binary trees . . . 22
11.2 Typed colored extension beyond the finite contract . . . 22
11.3 Packet labelings and the multilinear first target . . . 23
11.4 Connection defects at nonlinear nodes . . . 24

12 Formalization ledger for version 1.0.1 24
12.1 Kernel-backed Paper III surface . . . 27

13 The finite contract and the nonlinear transition 28

II Nonlinear packet calculus, obstruction complexes, and holonomy 29

14 Introduction 29

15 A map of the nonlinear calculus 30

16 Exact nonlinear packet coordinates 31

17 Packet jets and higher composition 33

18 Rooted tree expansions 36

19 Graded nonlinear operations 39

20 Nonlinear refinement defects 41

21 Canonical obstruction objects 43
21.1 Relative obstruction under coarsening . . . 46

22 Split Hilbert obstructions and nonlinear correction 49

23 Filtered lifting 52

24 Filtered obstruction complexes 56

25 Nonlinear elimination 57

26 Finite transport and holonomy 59
26.1 Intertwining two transport systems 60

27 Smooth projector transport 62

28 Realization data and limits of the calculus 67

A Visual atlas 69

List of Figures

1 Rows record target packets and columns record source packets. This convention fixes execution order. Composable words collapse to endpoint blocks, while one broken packet interface annihilates the entire operator product. 3

2 Exact coordinate transport and connection transport are distinct. The canonical loop telescopes to the identity. A chosen connection may carry nontrivial holonomy, which transforms by conjugation under gauge. 11

3 A finite complete packet resolution reconstructs the total holonomy obstruction from blocks indexed by input and output packets. A nonzero off-diagonal block is an exact witness of packet mixing. 18

4 Second packet jets for $F(x, y) = (x + xy, y + x^2)$ at the origin. Coordinate axis projectors give $DF(0) = I$. The first output has mixed second derivatives equal to one, while the second has a repeated first input derivative equal to two. Each grid records the ordered pair of input packets. The mixed entries retain an interaction that diagonal restrictions would lose. 34

5 A specified correction range in $\mathbb{R}^2$. For $L(a, b) = (a, 0)$, the image is the horizontal axis. The defect $(2, 3)$ splits into $(2, 0)$ in that image and the remaining representative $(0, 3)$. Subtracting the image component leaves the same cokernel class. The functional $(x, y) \mapsto y$ vanishes on the correction range and detects the value three. The correction equation $Lu = -(2, 3)$ therefore has no solution. 45

6 A rotating rank one packet and its orthogonal complement. With $u(\theta) = (\cos\theta, \sin\theta)$ and $P(\theta) = u(\theta)u(\theta)^{\mathsf{T}}$, the canonical generator is $[P', P] = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$. For a positive step $\delta < \pi/2$, the polar unitary is the usual rotation $R(\delta)$. On each displayed mesh, the direct rotation product equals $R(\pi/3)$ exactly. This special planar example illustrates the construction. The general theorem supplies convergence for families whose step products need not already equal the continuous transport. 66

Part I Finite typed transfer and the formal contract

The finite release supplies the algebraic foundation and a contract for its individual declarations. Here, "roadmap" and "outside the contract" describe the scope of the finite Lean formalization. Ordinary mathematical results in Part II may lie beyond that scope. The thirty original result environments and their proof bodies are retained. The source contract has 41 mapping rows because it also includes definitions and other interfaces.

The preserved v1.0.1 release records Lean v4.34.0-rc2 against Mathlib revision `aeb72f1e9981b6d35b426b16dca7759be3e7ba06` [3, 17]. The compiler and kernel check descriptions in Part I refer to that inherited evidence. The companion verification files record the editorial checks of this edition separately.

1 Purpose and formal status

Paper I fixed typed seeds, operator grammars, exact projector packets, generated hulls, and reconstruction [13]. Paper II added coherent refinement coordinates, weighted transfer, arithmetic separation, and linear defect identities [14]. The present paper begins the next layer: explicit edge words, optional connection data, and obstructions within individual packets.

Canonical coordinate transport and a connection play different mathematical roles. Canonical transport compares two exact descriptions of the *same* ambient object and is forced by analysis and synthesis. A connection chooses transports that may carry extra modeling, discretization, or gauge information. Canonical transport obeys a cocycle theorem. A connection may fail to obey a cocycle, and that failure is meaningful data. Keeping separate notation lets us distinguish the automatic coherence of exact coordinates from the possible holonomy of a chosen connection.

1.1 Statement labels

Every substantial theorem, lemma, corollary, proposition, definition, API, and roadmap item carries one or more of the three labels below. When separate clauses use a Paper III result and an inherited bridge, the statement carries both applicable labels. Illustrative examples do not require a label.

S1. **Lean-backed in Paper III**: the displayed scope is no stronger than the named compiled declaration in the released Paper III module.

S2. **Inherited Lean-backed**: the displayed result is supplied by the frozen formal contract of Paper I or II.

S3. **Roadmap only**: the text states a future interface or theorem target and is excluded from the completion denominator.

Boundary 1.1 (Final theorem denominator). The completion denominator of the finite v1.0.1 contract consists of the thirty result environments in Part I. Of these, 26 resolve to compiled Paper III declarations and four resolve to the frozen Papers I–II contracts. The authoritative theorem map names the exact declarations used by each statement. The kernel audit checks those targets and their supporting theorems. The namespace gate requires every declaration in the compiled `Komorebi.PaperIII` namespace to pass its checks. Roadmap items are excluded from the denominator and remain visibly marked.

2 Finite typed packet graphs

2.1 Orientation and matrix convention

Let V be a finite packet type, E a finite edge type, and R a semiring. A directed multigraph is a pair of endpoint functions

$$\mathrm{src}, \mathrm{tgt} : E \longrightarrow V.$$

Parallel edges remain distinct elements of E. Rows are targets and columns are sources. Thus an edge $e : i \to j$ is represented by a matrix supported at row j, column i.

Definition 2.1 (Packet graph and scalar edge system)**.** For $a, b \in V$ and $r \in R$, define

$$\mathrm{packetMatrix}(a, b; r)_{yx} = \begin{cases} r, & y = b \text{ and } x = a, \\ 0, & \text{otherwise.} \end{cases}$$

An edge system is a coefficient function $c : E \to R$. Its elementary edge operator and total transfer operator are

$$L_e = \mathrm{packetMatrix}(\mathrm{src}(e), \mathrm{tgt}(e); c(e)), \qquad T = \sum_{e \in E} L_e.$$

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.EdgeOperators.PacketGraph;Komorebi.PaperIII.EdgeOperators.PacketGraph.packetMatrix;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp`

Words are written in execution order. In $L_f L_e$, the rightmost operator L_e acts first. For a written word $(e_1, \dots, e_n)$, e_1 acts first and the operator product is $L_{e_n} \cdots L_{e_1}$.

Reading the execution order

Read the edge list in the order of execution. The matrix product writes later actions farther to the left. The target packet of one edge must equal the source packet of the next before multiplication can contribute. Over a noncommutative semiring, this order also fixes the order of the scalar coefficients.

Lemma 2.2 (Declared support)**.** *For every edge e, $(L_e)_{yx} = 0$ unless $y = \mathrm{tgt}(e)$ and $x = \mathrm{src}(e)$.*

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp_eq_zero_off_support`

Theorem 2.3 (Typed interface annihilation)**.** *If $\mathrm{tgt}(e) \neq \mathrm{src}(f)$, then*

$$L_f L_e = 0.$$

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp_mul_edgeOp_eq_zero`

Proof. For a matrix entry (y, x),

$$(L_f L_e)_{yx} = \sum_{z \in V} (L_f)_{yz} (L_e)_{zx}.$$

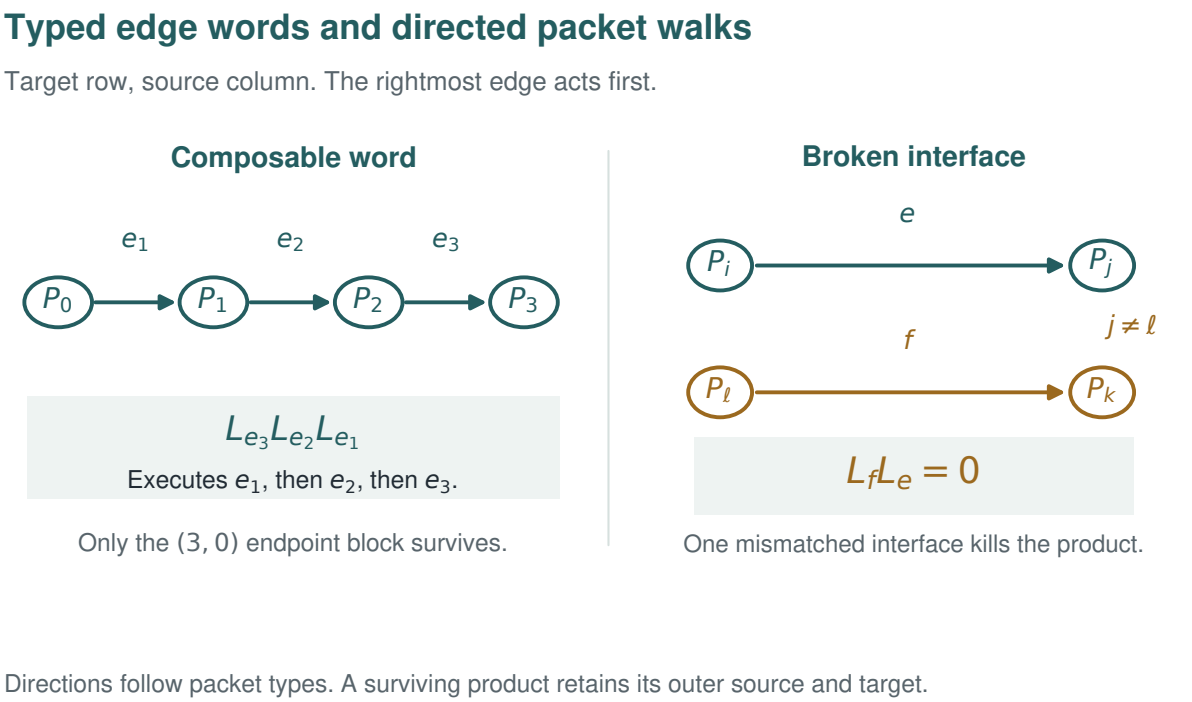


Figure 1. Rows record target packets and columns record source packets. This convention fixes execution order. Composable words collapse to endpoint blocks, while one broken packet interface annihilates the entire operator product.

A potentially nonzero summand would require simultaneously $z = \mathrm{src}(f)$ and $z = \mathrm{tgt}(e)$, contrary to the hypothesis. □

Theorem 2.4 (Collapse of a matched edge pair)**.** *If* $\mathrm{tgt}(e) = \mathrm{src}(f)$*, then*

$$L_f L_e = \mathrm{packetMatrix}(\mathrm{src}(e), \mathrm{tgt}(f); c(f)c(e)).$$

Together with theorem 2.3, this gives both cases for the composition of two edge operators.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp_mul_edgeOp_of_composable`

Proof. The matrix-product sum has exactly one potentially nonzero intermediate index, namely $\mathrm{tgt}(e) = \mathrm{src}(f)$. At that index the coefficients occur in operator order as $c(f)c(e)$, and the outer source and target are $\mathrm{src}(e)$ and $\mathrm{tgt}(f)$. □

Boundary 2.5 (Current fiber scope). The compiled Paper III model above has one scalar coordinate per packet. For a family of modules $(X_i)_{i \in V}$, an intrinsic edge map would instead have type $\ell_e : X_{\mathrm{src}(e)} \to X_{\mathrm{tgt}(e)}$, and its totalization on $\bigoplus_i X_i$ would be $\iota_{\mathrm{tgt}(e)} \ell_e \pi_{\mathrm{src}(e)}$. The same support proof applies, but this generalization with dependent fibers is not attributed to the released Paper III modules for scalar edge systems.

Figure 1 summarizes the support convention and shows how composable words differ from words with a broken interface. The next section uses this distinction.

3 Operator words and directed walks

3.1 Typed words

Definition 3.1 (Composable word and execution operator)**.** A finite word $w = (e_1, \dots, e_n)$ is composable when

$$\mathrm{tgt}(e_r) = \mathrm{src}(e_{r+1}) \qquad (1 \le r < n).$$

Set

$$L_w = L_{e_n} \cdots L_{e_1}, \qquad L_\emptyset = I.$$

This definition records the written order of execution even though matrix composition is read right to left.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.Composable;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp`

Theorem 3.2 (First broken interface)**.** *If a word begins $(e, f, \dots)$ and $\mathrm{tgt}(e) \neq \mathrm{src}(f)$, then its word operator is zero.*

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_eq_zero_of_first_break`

Theorem 3.3 (Noncomposable words vanish)**.** *If w is not composable, then $L_w = 0$.*

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_eq_zero_of_not_composable`

Proof. Choose the first broken adjacent pair. The corresponding consecutive factor is zero by theorem 2.3, and associativity absorbs that zero into the complete product. The Lean proof implements the same argument by induction on the list. □

3.2 The checked recursive expansion

Definition 3.4 (Recursive word expansion)**.** Define

$$\mathcal{W}_0 = I, \qquad \mathcal{W}_{n+1} = T\mathcal{W}_n.$$

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordExpansion`

Theorem 3.5 (Transfer power equals recursive word expansion)**.** *For every $n \in \mathbb{N}$,*

$$T^n = \mathcal{W}_n.$$

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_eq_wordExpansion`

Proof. The zero case is the identity matrix. If $T^n = \mathcal{W}_n$, then the left-successor power convention gives $T^{n+1} = TT^n = T\mathcal{W}_n = \mathcal{W}_{n+1}$. □

Remark 3.6. The formal module also defines an explicit sum over tuples $w : \mathrm{Fin}(n) \to E$ and a second sum in which noncomposable tuples are replaced by zero. It proves that the recursive matrix, the explicit tuple sum, and the sum filtered by composability agree. The separate declarations make the finite enumeration claim directly checkable.

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.explicitWordExpansion;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.walkExpansion;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordExpansion_eq_explicitWordExpansion;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.explicitWordExpansion_eq_walkExpansion`

3.3 Exact expansion from words to walks

For a nonempty composable word $w = (e_1, \dots, e_n)$, define its reverse coefficient product

$$c^{\leftarrow}(w) = c(e_n)c(e_{n-1}) \cdots c(e_1).$$

The reversal is essential over a noncommutative semiring.

Lemma 3.7 (One word is one endpoint block)**.** *If $w = (e_1, \dots, e_n)$ is nonempty and composable, then*

$$L_w = \mathrm{packetMatrix}(\mathrm{src}(e_1), \mathrm{tgt}(e_n); c^{\leftarrow}(w)).$$

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.reverseCoefficient;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.terminalTarget;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_eq_packetMatrix_of_composable`

Proof. Induct on n. The case $n = 1$ is the definition of L_e. Suppose the formula holds for the prefix $(e_1, \dots, e_n)$ and append f. Composability gives $\mathrm{tgt}(e_n) = \mathrm{src}(f)$. Multiplying the two packet matrices leaves one intermediate index and changes the coefficient to $c(f)c(e_n) \cdots c(e_1)$, with source $\mathrm{src}(e_1)$ and target $\mathrm{tgt}(f)$. □

Write $\mathrm{Walk}_n(i, j)$ for the finite set of composable edge tuples of length n whose first source is i and whose final target is j. For $n = 0$, it has one empty walk when $i = j$ and is empty otherwise. Assign the empty walk the coefficient 1.

> **The empty walk**
>
> The empty walk records doing nothing at a chosen packet. It contributes only to a diagonal entry. Thus the convention for length zero matches the identity matrix and lets the walk sum include every nonnegative power.

Theorem 3.8 (Finite operator expansion from words to walks)**.** *For every $n \in \mathbb{N}$,*

$$T^n = \sum_{w \in E^n} L_w = \sum_{w \in E^n} \begin{cases} L_w, & w \text{ composable}, \\ 0, & w \text{ not composable}. \end{cases}$$

> *Parallel edges occur as distinct summands.*

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_eq_explicitWordExpansion;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_eq_walkExpansion`

Proof. Induction and finite distributivity give

$$\left(\sum_{e\in E} L_e\right)^n = \sum_{(e_1,\dots,e_n)\in E^n} L_{e_n}\cdots L_{e_1}.$$

Every noncomposable summand is zero by theorem 3.3, so inserting the composability filter does not change the sum. In Lean, tuple concatenation and the successor sum are handled by `tupleWordOp_snoc` and `explicitWordExpansion_succ`; the two bridge equalities then yield the displayed theorem. □

Corollary 3.9 (Endpoint coefficient walk sum). *For all $n \in \mathbb{N}$ and $i, j \in V$,*

$$\boxed{(T^n)_{ji} = \sum_{w\in \mathrm{Walk}_n(i,j)} c^{\leftarrow}(w).}$$

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.Connects;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.endpointWalkCoefficientSum;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_apply_eq_sum_walks`

Proof. Take the (j, i) entry in theorem 3.8. By lemma 3.7, a surviving word contributes $c^{\leftarrow}(w)$ exactly when its endpoints are i and j, and contributes zero otherwise. At $n = 0$, the identity matrix gives the unique empty diagonal walk. □

Corollary 3.10 (Reachability obstruction at fixed length). *If $\mathrm{Walk}_n(i, j) = \emptyset$, then $(T^n)_{ji} = 0$.*

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_apply_eq_zero_of_no_walk`

Proof. This is immediate from the empty sum in the preceding corollary. □

Corollary 3.11 (Orbit support obstruction). *Let $S \subseteq V$, and let $x \in R^V$ be supported on S. If j is not reachable by a directed path from any vertex of S, then*

$$(T^n x)_j = 0 \qquad \text{for every } n \in \mathbb{N}.$$

Hence the orbit under T of x cannot generate a vector with a nonzero coordinate at j.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.Reachable;Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_mulVec_apply_eq_zero_of_unreachable_support`

Inherited formal contract: Papers I–II.
`Komorebi.PacketNaturalityCompletion.Reachability.algebraicOrbitHull_le_reachable_support;Komorebi.PacketNaturalityCompletion.Reachability.unreachable_nonzero_packet_obstruction`

Proof. Expand $(T^n x)_j = \sum_i (T^n)_{ji} x_i$. Terms with $i \notin S$ vanish by the support of x, and terms with $i \in S$ vanish by corollary 3.10. The final claim follows because the j-coordinate also vanishes on every finite linear combination of orbit vectors. □

4 Canonical exact coordinate transport

4.1 The Paper III frame model

Let every chart a carry a linear frame equivalence $F_a : M \xrightarrow{\sim} M$. The formal Paper III module defines transport from a to b by

$$J_{b \leftarrow a} = F_b^{-1} F_a.$$

This frame formula means *synthesize in chart a, then analyze in chart b.*

Theorem 4.1 (Exact frame transport is coherent). *For every a, b, c,*

$$J_{a \leftarrow a} = \mathrm{id}, \qquad J_{a \leftarrow b} J_{b \leftarrow a} = \mathrm{id}, \qquad J_{c \leftarrow b} J_{b \leftarrow a} = J_{c \leftarrow a},$$

and therefore

$$J_{a \leftarrow c} J_{c \leftarrow b} J_{b \leftarrow a} = \mathrm{id}.$$

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.exactTransport_self;Komorebi.PaperIII.Transport.exactTransport_roundTrip;Komorebi.PaperIII.Transport.exactTransport_cocycle;Komorebi.PaperIII.Transport.exactTransport_triangleLoop`

Proof. Every identity is cancellation of adjacent $F_r F_r^{-1}$ factors. The Lean definitions use `LinearEquiv.trans`; the displayed formulas use the equivalent right-to-left function-composition notation fixed in this paper. □

Theorem 4.2 (Every finite canonical path telescopes). *Let $p = (a_0, a_1, \dots, a_n)$ be any finite vertex list and put*

$$J_p = J_{a_n \leftarrow a_{n-1}} \cdots J_{a_1 \leftarrow a_0}, \qquad J_{(a_0)} = \mathrm{id}.$$

Then

$$J_p = J_{a_n \leftarrow a_0}.$$

In particular, if $a_n = a_0$, then $J_p = \mathrm{id}$.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.pathEndpoint;Komorebi.PaperIII.Transport.exactPathTransport;Komorebi.PaperIII.Transport.exactPathTransport_eq_endpoint;Komorebi.PaperIII.Transport.exactPathTransport_loop`

Proof. Induct on the list of successive vertices and apply the cocycle in theorem 4.1 at each step. If the endpoint is the start, the endpoint transport is the identity. □

4.2 Exact packet coordinates

The graph model above resolves a fixed scalar matrix. Coordinate transport is a separate construction. Let X be an R-module, and let

$$\mathcal{P}^a = (P_i^a)_{i \in I_a}$$

be a finite complete family of pairwise orthogonal linear idempotents:

$$P_i^a P_j^a = \delta_{ij} P_i^a, \qquad \sum_{i \in I_a} P_i^a = I_X.$$

Define the exact packet coordinate module

$$\mathrm{Coord}(\mathcal{P}^a) = \left\{ (x_i)_{i \in I_a} : P_i^a x_i = x_i \right\}.$$

Its analysis and synthesis maps are

$$\mathcal{A}_a(x) = (P_i^a x)_i, \qquad \mathcal{S}_a((x_i)_i) = \sum_i x_i.$$

Completeness and orthogonality give

$$\mathcal{S}_a \mathcal{A}_a = \mathrm{id}_X, \qquad \mathcal{A}_a \mathcal{S}_a = \mathrm{id}_{\mathrm{Coord}(\mathcal{P}^a)}.$$

> **What a packet coordinate contains**
>
> A packet coordinate is a whole vector fixed by its projector. Analysis splits an ambient vector into these components, and synthesis adds them back. The transport between exact systems keeps the ambient vector and changes its packet presentation.

Definition 4.3 (Canonical exact coordinate transport)**.** For two exact packet systems on the same ambient module, define

$$J^X_{b \leftarrow a} = \mathcal{A}_b \mathcal{S}_a : \mathrm{Coord}(\mathcal{P}^a) \longrightarrow \mathrm{Coord}(\mathcal{P}^b).$$

The arrow points from the source coordinate system a to the target coordinate system b.

Inherited formal contract: Papers I–II.

`Komorebi.PaperII.NestedRefinement.refineCoordinates;Komorebi.PaperII.NestedRefinement.coordinateRefinementEquiv`

Theorem 4.4 (Identity, inverse, and cocycle for canonical transport)**.** *For any exact packet systems a, b, c on X,*

$$J^X_{a \leftarrow a} = \mathrm{id}, \qquad \left(J^X_{b \leftarrow a}\right)^{-1} = J^X_{a \leftarrow b}, \qquad J^X_{c \leftarrow b} J^X_{b \leftarrow a} = J^X_{c \leftarrow a}.$$

Inherited formal contract: Papers I–II.

`Komorebi.PaperII.NestedRefinement.refineCoordinates_self;Komorebi.PaperII.NestedRefinement.coarsenCoordinates_refineCoordinates;Komorebi.PaperII.NestedRefinement.refineCoordinates_coarsenCoordinates;Komorebi.PaperII.NestedRefinement.refineCoordinates_comp;Komorebi.PaperII.NestedRefinement.coordinateRefinementEquiv_trans`

Proof. Insert $\mathcal{S}_b\mathcal{A}_b = \mathrm{id}_X$:

$$J^X_{c\leftarrow b}J^X_{b\leftarrow a} = \mathcal{A}_c\mathcal{S}_b\mathcal{A}_b\mathcal{S}_a = \mathcal{A}_c\mathcal{S}_a = J^X_{c\leftarrow a}.$$

The identity statement is $\mathcal{A}_a\mathcal{S}_a = \mathrm{id}$. Setting $c = a$ in the cocycle gives $J^X_{a\leftarrow b}J^X_{b\leftarrow a} = \mathrm{id}$; interchanging a and b gives the other composition, so the two transports are inverses. □

If $F : X \to Y$ is linear and $\mathcal{P}^a, \mathcal{P}^b$ are source packet systems while $\mathcal{Q}^r, \mathcal{Q}^s$ are target packet systems, put

$$\widehat{F}_{r\leftarrow a} = \mathcal{A}^Y_r F \mathcal{S}^X_a.$$

Theorem 4.5 (Canonical coordinate naturality)**.** *The exact coordinate representation preserves typed composition and satisfies*

$$J^Y_{s\leftarrow r}\widehat{F}_{r\leftarrow a} = \widehat{F}_{s\leftarrow b}J^X_{b\leftarrow a}.$$

Inherited formal contract: Papers I–II.

`Komorebi.PaperII.NestedRefinement.coordinateOperator_comp;Komorebi.PaperII.NestedRefinement.coordinateOperator_naturality`

Proof. Both sides reduce to $\mathcal{A}^Y_s F \mathcal{S}^X_a$ after inserting the appropriate synthesis–analysis identity. The composition law is the same calculation with the middle ambient module inserted. □

Corollary 4.6 (Canonical triangle holonomy is trivial)**.** *For every triple a, b, c,*

$$J^X_{a\leftarrow c}J^X_{c\leftarrow b}J^X_{b\leftarrow a} = \mathrm{id}.$$

Inherited formal contract: Papers I–II.

`Komorebi.PaperII.NestedRefinement.refineCoordinates_comp;Komorebi.PaperII.NestedRefinement.refineCoordinates_self`

Boundary 4.7 (Refinement is not reduction). Although Paper II uses the name *refineCoordinates*, J is an equivalence between two complete coordinate presentations. It neither truncates packets nor projects to a smaller state space. Any lossy restriction, interpolation, or learned identification is extra data and cannot inherit theorem 4.4 without proof.

5 A distinct noncanonical connection API

5.1 The minimal compiled API

The compiled Paper III API assumes only normalization at each chart. All chart transports act on one module M, so chart labels distinguish routes but not Lean types.

Definition 5.1 (Noncanonical connection normalized at each chart)**.** A minimal connection is a family

$$\Gamma_{b\leftarrow a} \in \mathrm{Aut}_R(M) \qquad (a, b \in B)$$

with only

$$\Gamma_{a\leftarrow a} = \mathrm{id}_M.$$

Neither inversion on reversed edges nor the cocycle law is a field of the structure.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.Connection`

At a triple (a, b, c), the compiled cocycle predicate is

$$\Gamma_{c\leftarrow b}\Gamma_{b\leftarrow a} = \Gamma_{c\leftarrow a}.$$

Flatness means this equality holds for every triple. The formal multiplicative relative defect and triangle holonomy are

$$D^{\times}_{c,b,a} = (\Gamma_{c\leftarrow b}\Gamma_{b\leftarrow a})\Gamma^{-1}_{c\leftarrow a}, \qquad H_{c,b,a} = \Gamma_{a\leftarrow c}\Gamma_{c\leftarrow b}\Gamma_{b\leftarrow a}.$$

The defect is based at the target c. The holonomy is based at a.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.Connection.CocycleAt;Komorebi.PaperIII.Transport.Connection.Flat;Komorebi.PaperIII.Transport.Connection.cocycleDefect;Komorebi.PaperIII.Transport.Connection.triangleHolonomy`

Theorem 5.2 (Checked flatness consequences)**.** *For the minimal connection API, flatness implies*

$$D^{\times}_{c,b,a} = \mathrm{id}_M, \qquad H_{c,b,a} = \mathrm{id}_M$$

for every a, b, c.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.Connection.cocycleDefect_eq_refl_of_flat;Komorebi.PaperIII.Transport.Connection.triangleHolonomy_eq_refl_of_flat`

Proof. Flatness replaces the two-edge route in $D^{\times}$ by the direct route, which cancels its inverse. For H, flatness first collapses $a \to b \to c$ to $a \to c$, then collapses $a \to c \to a$ to the normalized transport $a \to a$. □

Proposition 5.3 (The minimal API admits nontrivial holonomy)**.** *There is a connection on three points over $\mathbb{Q}$ for which transport $0 \to 1$ is multiplication by 2, every other transport between distinct endpoints is the identity, and*

$$H_{2,1,0}(1) = 2.$$

In particular $H_{2,1,0} \neq \mathrm{id}$.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.twistedThreePointConnection;Komorebi.PaperIII.Transport.twisted_triangleHolonomy_apply_one;Komorebi.PaperIII.Transport.twisted_triangleHolonomy_ne_refl`

Boundary 5.4 (Why the minimal defect and loop are distinct). Without reverse-edge inversion, $\Gamma_{a\leftarrow c}$ need not equal $\Gamma^{-1}_{c\leftarrow a}$. Therefore the multiplicative relative defect $D^{\times}_{c,b,a}$ and triangle holonomy $H_{c,b,a}$ are different comparisons. The formal module proves that global flatness trivializes both; it does not identify them for an arbitrary connection.

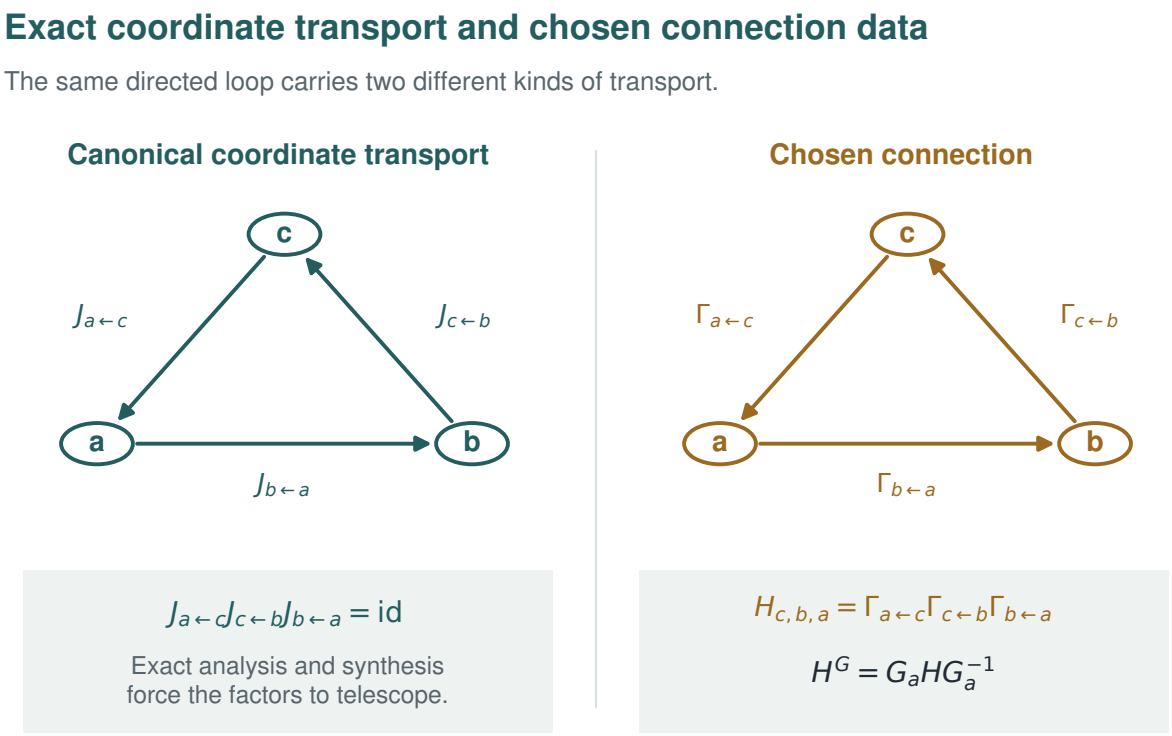


Figure 2. Exact coordinate transport and connection transport are distinct. The canonical loop telescopes to the identity. A chosen connection may carry nontrivial holonomy, which transforms by conjugation under gauge.

Figure 2 records this separation visually: the exact coordinate route telescopes, whereas the chosen connection route may carry a nonidentity loop.

> **Coordinate transport and connection choice**
>
> Canonical transport compares exact coordinate descriptions of the same object. Its loops telescope. A connection supplies a separate family of comparison maps, so its loops can carry information about that choice. The distinction keeps coordinate identities separate from connection assumptions.

5.2 A stronger reversible pair API

Let R be a commutative ring, let B be a type of charts or refinement states, and let each C_a be an R-module. The chart type is not assumed finite. The following definition is deliberately not derived from analysis and synthesis.

> **Definition 5.5** (Normalized reversible connection)**.** A normalized reversible connection is a family of linear equivalences
>
> $$\Gamma_{b\leftarrow a} : C_a \xrightarrow{\sim} C_b \qquad (a, b \in B)$$
>
> such that
>
> $$\Gamma_{a\leftarrow a} = \mathrm{id}, \qquad \Gamma_{a\leftarrow b} = \Gamma_{b\leftarrow a}^{-1}.$$

No cocycle equation is included in the structure.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.ReversibleConnection`

This adds heterogeneous fibers and inversion on reversed edges to the homogeneous minimal connection. The connection is still a separate choice of maps, rather than a definition of canonical coordinate transport.

Definition 5.6 (Calibration and flatness)**.** When the C_a are exact packet coordinates of one ambient module, the connection is *canonically calibrated* if

$$\Gamma_{b\leftarrow a} = J_{b\leftarrow a} \quad \text{for every } a, b.$$

It is *flat* if

$$\Gamma_{c\leftarrow b}\Gamma_{b\leftarrow a} = \Gamma_{c\leftarrow a} \quad \text{for every } a, b, c.$$

The canonical connection is flat. A general flat connection is proved in theorem 6.3 to be related to it by a gauge change.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.ReversibleConnection.CocycleAt;Komorebi.PaperIII.Transport.ReversibleConnection.Flat;Komorebi.PaperIII.Transport.ReversibleConnection.canonicalConnection;Komorebi.PaperIII.Transport.ReversibleConnection.canonicalConnection_flat`

The words *calibrated* and *flat* have only the algebraic meanings displayed above.

Remark 5.7 (A connection on a chart graph). One may specify Γ_e only on the oriented edges of a chart graph, together with inverse maps on reversed edges. Path transport and loop holonomy below still make sense. A direct cocycle defect for (a, b, c), however, requires a declared direct transport from a to c. The pairwise API in definition 5.5 keeps this comparison total.

5.3 Cocycle defect and triangle holonomy

The commutative ring hypothesis makes subtraction of linear maps available from this point through section 7.

Definition 5.8 (Cocycle defect)**.** For $a, b, c \in B$, define the additive cocycle defect

$$\Delta_{c,b,a} = \Gamma_{c\leftarrow b}\Gamma_{b\leftarrow a} - \Gamma_{c\leftarrow a} \in \mathrm{Hom}_R(C_a, C_c).$$

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.ReversibleConnection.cocycleDefect`

The manuscript subscript order is target, intermediate, source. The Lean arguments are supplied in traversal order `(a,b,c)`.

Definition 5.9 (Triangle holonomy)**.** The triangle holonomy based at a is

$$H_{c,b,a} = \Gamma_{a\leftarrow c}\Gamma_{c\leftarrow b}\Gamma_{b\leftarrow a} \in \mathrm{Aut}_R(C_a).$$

It traverses $a \to b \to c \to a$.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.ReversibleConnection.triangleHolonomy;Komorebi.PaperIII.Transport.ReversibleConnection.triangleObstruction`

Holonomy is attached to the chosen connection Γ, never to an unspecified coordinate change.

Theorem 5.10 (Identity between defect and holonomy)**.** *For every triple* (a, b, c)*,*

$$\boxed{H_{c,b,a} - \mathrm{id}_{C_a} = \Gamma_{a\leftarrow c}\Delta_{c,b,a}.}$$

Consequently,

$$\Delta_{c,b,a} = 0 \quad \Longleftrightarrow \quad H_{c,b,a} = \mathrm{id}_{C_a}.$$

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.ReversibleConnection.triangleHolonomy_sub_id;Komorebi.PaperIII.Transport.ReversibleConnection.cocycleDefect_eq_zero_iff_triangleHolonomy_eq_refl`

Proof. Using reversibility,

$$\begin{aligned}\Gamma_{a\leftarrow c}\Delta_{c,b,a} &= \Gamma_{a\leftarrow c}\Gamma_{c\leftarrow b}\Gamma_{b\leftarrow a} - \Gamma_{a\leftarrow c}\Gamma_{c\leftarrow a}\\ &= H_{c,b,a} - \mathrm{id}_{C_a}.\end{aligned}$$

Left composition by the equivalence $\Gamma_{a\leftarrow c}$ is injective, which gives the equivalence of the two vanishing statements. □

Corollary 5.11 (Canonical transport has zero connection defect)**.** *If* $\Gamma_{b\leftarrow a} = J_{b\leftarrow a}$*, then*

$$\Delta_{c,b,a} = 0, \qquad H_{c,b,a} = \mathrm{id}$$

for all triples.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.ReversibleConnection.canonicalTransport_zero_connectionDefect;Komorebi.PaperIII.Transport.ReversibleConnection.canonicalTransport_triangleHolonomy_eq_refl`

Proof. The defect vanishes by the canonical cocycle in theorem 4.4; apply theorem 5.10. □

5.4 Paths and endpoint independence

Definition 5.12 (Connection transport along a path)**.** For a vertex sequence $p = (a_0, a_1, \dots, a_n)$, define

$$\Gamma_p = \Gamma_{a_n\leftarrow a_{n-1}} \cdots \Gamma_{a_1\leftarrow a_0}, \qquad \Gamma_{(a_0)} = \mathrm{id}_{C_{a_0}}.$$

If $a_n = a_0$, then Γ_p is the holonomy of the loop p.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.ReversibleConnection.pathTransport;Komorebi.PaperIII.Transport.ReversibleConnection.VertexPath;Komorebi.PaperIII.Transport.ReversibleConnection.indexedPathTransport`

Products follow path order: the first traversed edge acts on the right.

Proposition 5.13 (Flat pair transport collapses to endpoints)**.** *If Γ is flat, then every path satisfies*

$$\Gamma_p = \Gamma_{a_n \leftarrow a_0}.$$

In particular, every loop has identity holonomy.

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.Transport.ReversibleConnection.pathTransport_eq_direct_of_flat;Komorebi.PaperIII.Transport.ReversibleConnection.indexedPathTransport_eq_direct_of_flat;Komorebi.PaperIII.Transport.ReversibleConnection.indexedLoopTransport_eq_refl_of_flat`

Proof. Induct on the path length. Appending a_{n+1} left-multiplies the inductive expression by $\Gamma_{a_{n+1}\leftarrow a_n}$, and flatness replaces the two factors by $\Gamma_{a_{n+1}\leftarrow a_0}$. For a loop, normalization gives $\Gamma_{a_0\leftarrow a_0} = \mathrm{id}$. □

Remark 5.14. The converse *trivial holonomy implies flatness* depends on which loops are present in an API defined on a chart graph. In the complete pair API, triviality of every triangle holonomy is equivalent to flatness by theorem 5.10. No theorem about topological simple connectivity is being invoked.

6 Gauge covariance and comparison with canonical transport

Definition 6.1 (Gauge change)**.** A gauge is a family of automorphisms $G_a \in \mathrm{Aut}_R(C_a)$. It acts on a connection by

$$\Gamma^G_{b\leftarrow a} = G_b \Gamma_{b\leftarrow a} G_a^{-1}.$$

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.Transport.ReversibleConnection.Gauge;Komorebi.PaperIII.Transport.ReversibleConnection.gauge`

A gauge changes coordinate representatives at each chart. The chart graph stays the same.

Theorem 6.2 (Gauge covariance)**.** *The defect and triangle holonomy transform as*

$$\Delta^G_{c,b,a} = G_c \Delta_{c,b,a} G_a^{-1}, \qquad H^G_{c,b,a} = G_a H_{c,b,a} G_a^{-1}.$$

Thus flatness and triviality of triangle holonomy are gauge invariant.

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.Transport.ReversibleConnection.gauge_cocycleDefect;Komorebi.PaperIII.Transport.ReversibleConnection.gauge_triangleHolonomy;Komorebi.PaperIII.Transport.ReversibleConnection.gauge_flat_iff;Komorebi.PaperIII.Transport.ReversibleConnection.gauge_triangleHolonomy_eq_refl_iff`

Proof. Substitute the definition of Γ^G. Adjacent factors $G_b^{-1}G_b$ and $G_c^{-1}G_c$ cancel. In the defect, both the composite and direct terms acquire the same outer factors G_c and G_a^{-1}. In the loop, all inner factors cancel and only conjugation at the base remains. □

Theorem 6.3 (Flat connections are gauges of the canonical pair transport)**.** *Suppose the C_a are exact packet coordinate modules for one ambient module, so the canonical $J_{b\leftarrow a}$ is available. If Γ is a flat normalized*

reversible pair connection, choose $o \in B$ and set

$$G_a = \Gamma_{a\leftarrow o} J_{o\leftarrow a}.$$

Then every pair transport satisfies

$$\boxed{\Gamma_{b\leftarrow a} = G_b J_{b\leftarrow a} G_a^{-1}.}$$

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.Transport.ReversibleConnection.flatGauge;Komorebi.PaperIII.Transport.ReversibleConnection.flat_isGaugeOfCanonical`

Proof. Reversibility gives $G_a^{-1} = J_{a\leftarrow o}\Gamma_{o\leftarrow a}$. Therefore

$$\begin{aligned} G_b J_{b\leftarrow a} G_a^{-1} &= \Gamma_{b\leftarrow o} J_{o\leftarrow b} J_{b\leftarrow a} J_{a\leftarrow o} \Gamma_{o\leftarrow a} \\ &= \Gamma_{b\leftarrow o}\Gamma_{o\leftarrow a} \\ &= \Gamma_{b\leftarrow a}. \end{aligned}$$

The first reduction uses the canonical cocycle twice; the second uses flatness of Γ. □

Boundary 6.4 (Flat does not mean calibrated). The theorem produces a gauge relating Γ to J. It does not prove $\Gamma = J$ before that gauge is chosen. Conversely, nontrivial triangle holonomy cannot be removed by a gauge, because conjugation cannot turn a nonidentity automorphism into the identity.

7 Obstruction packets: local certificates and holonomy blocks

7.1 The generic compiled certificate

Let Candidate, Packet, and Residual be arbitrary types, with a distinguished $0 \in$ Residual, and let

$$r : \mathsf{Candidate} \longrightarrow \mathsf{Packet} \longrightarrow \mathsf{Residual}$$

be a residual function.

Definition 7.1 (Solution and obstruction packet for a candidate)**.** A candidate x is a solution when

$$\mathrm{IsSolution}_r(x) \quad \Longleftrightarrow \quad \forall p,\ r(x,p) = 0.$$

An obstruction packet for x is a pair consisting of a named packet p and a certificate $r(x,p) \neq 0$.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.NonlinearObstructions.IsSolution;Komorebi.PaperIII.NonlinearObstructions.ObstructionPacket`

Theorem 7.2 (Exact logical force of a local obstruction)**.** *Every obstruction packet for x proves that x is not a solution, and*

$$\neg\,\mathrm{IsSolution}_r(x) \quad \Longleftrightarrow \quad \textit{there exists an obstruction packet for } x.$$

The reverse implication from failure to a packet witness is classical and does not require the packet type to be finite.

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.NonlinearObstructions.ObstructionPacket.not_isSolution;Komorebi.PaperIII.NonlinearObstructions.obstructionPacket_of_not_isSolution;Komorebi.PaperIII.NonlinearObstructions.not_isSolution_iff_nonempty_obstructionPacket`

Proof. A purported solution would make the residual vanish at the certified packet, contradicting its nonzero field. Conversely, if no packet carries a nonzero residual, then every residual is zero and the candidate satisfies the definition of solution. □

What one obstruction certifies

The certificate answers one question: does this specified candidate satisfy every declared packet condition? One nonzero residual settles that question for this candidate. A statement that no candidate can solve the problem would require an obstruction for every candidate.

Boundary 7.3 (Candidate-specific means candidate-specific). The theorem rules out the named x. It neither constructs a solution nor shows that every candidate is obstructed. It provides no minimality, stability, computability, or classification theorem for the packet witness.

7.2 Specialization to holonomy blocks

For a reversible connection, specialize the generic residual at a fixed triangle by taking a packet label to be a pair (μ, ν) and the residual to be the corresponding block of $H - \mathrm{id}$. A nonzero block, together with its label and a proof of nonvanishing, gives an obstruction packet certificate in the sense of definition 7.1. The reconstruction theorem below supplies the further step: vanishing of all these local residuals is equivalent to the cocycle equation at that triangle.

Fix a triangle (a, b, c) and resolve the base fiber C_a by a finite complete family of idempotents $(\Pi^a_\mu)_{\mu \in I_a}$. Orthogonality may also hold, as it does for the exact range coordinate projectors below, but the compiled reconstruction theorem does not require it. In exact range coordinates, Π^a_μ is simply the projector that retains component μ and zeros the other components.

Definition 7.4 (Holonomy obstruction and packet residuals)**.** Define

$$\mathrm{Ob}_{c,b,a} = H_{c,b,a} - \mathrm{id}_{C_a}, \qquad \mathrm{Ob}^{\mu\nu}_{c,b,a} = \Pi^a_\mu \, \mathrm{Ob}_{c,b,a} \, \Pi^a_\nu.$$

The map $\mathrm{Ob}^{\mu\nu}_{c,b,a}$ is the obstruction block, or packet residual, from input packet ν to output packet μ. If it is nonzero, then (μ, ν) and that nonvanishing proof form an obstruction packet certificate. Diagonal blocks detect holonomy within one packet. Off-diagonal blocks detect packet mixing induced by holonomy.

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution;Komorebi.PaperIII.HolonomyBlocks.PacketResolution.block`

The certificate records a specified holonomy defect, the label of an input and output packet, and a proof that the corresponding block is nonzero.

Theorem 7.5 (Exact reconstruction and detection by obstruction blocks)**.** *The total obstruction has the exact finite resolution*

$$\boxed{\mathrm{Ob}_{c,b,a} = \sum_{\mu,\nu \in I_a} \mathrm{Ob}^{\mu\nu}_{c,b,a}.}$$

Moreover, the following are equivalent:

(i) $\Delta_{c,b,a} = 0$;

(ii) $H_{c,b,a} = \mathrm{id}$;

(iii) $\mathrm{Ob}_{c,b,a} = 0$;

(iv) $\mathrm{Ob}^{\mu\nu}_{c,b,a} = 0$ *for every* μ, ν.

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.HolonomyBlocks.obstruction_sum_blocks;Komorebi.PaperIII.HolonomyBlocks.obstructionBlocks_zero_iff;Komorebi.PaperIII.HolonomyBlocks.obstructionDetection_equivalences`

Proof. Insert completeness on both sides:

$$\mathrm{Ob} = \left(\sum_{\mu} \Pi^a_\mu\right) \mathrm{Ob} \left(\sum_{\nu} \Pi^a_\nu\right) = \sum_{\mu,\nu} \Pi^a_\mu \,\mathrm{Ob}\, \Pi^a_\nu.$$

Thus (iv) implies (iii), and (iii) implies (iv) by composition with the projectors. Conditions (ii) and (iii) are equivalent by the definition of Ob and additive cancellation. Conditions (i) and (ii) are equivalent by theorem 5.10. □

Why both packet indices matter

For the declared finite complete resolution, reconstruction uses both packet indices. Inspecting only diagonal blocks can miss transport from one packet to another. The full family records that mixing and reconstructs the total obstruction exactly.

Figure 3 depicts the finite block resolution on both sides of the obstruction and the off-diagonal blocks that witness packet mixing.

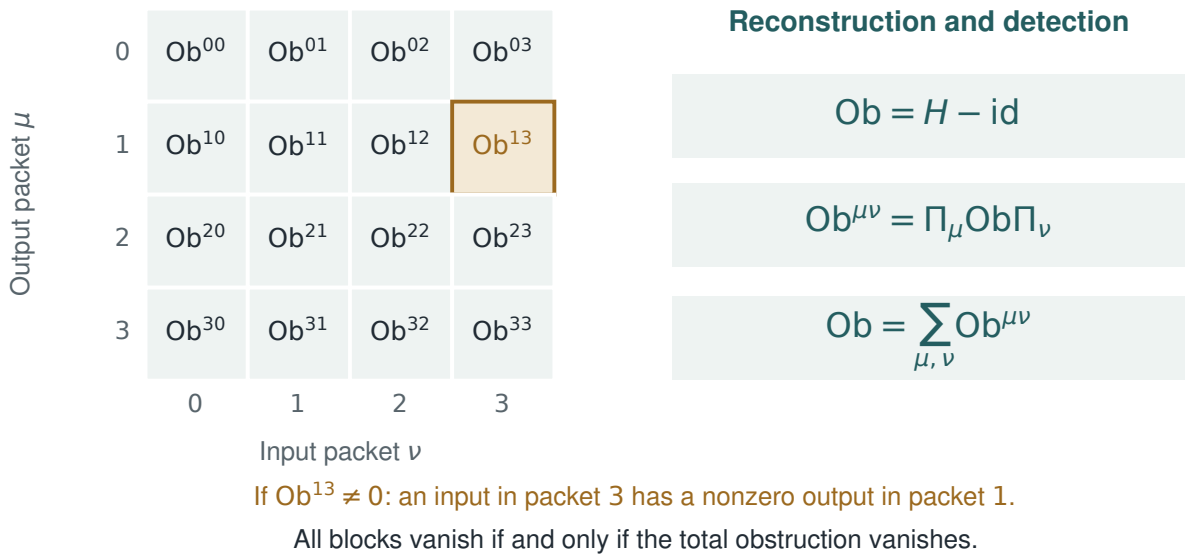


Figure 3. A finite complete packet resolution reconstructs the total holonomy obstruction from blocks indexed by input and output packets. A nonzero off-diagonal block is an exact witness of packet mixing.

Corollary 7.6 (Witness supported in one packet)**.** *If* $\mathrm{Ob}^{\mu\nu}_{c,b,a} \neq 0$*, then there is a vector* x *supported in packet* ν *such that*

$$\Pi^a_\mu \,\mathrm{Ob}_{c,b,a}\, x \neq 0.$$

Such an x *is an exact finite witness of nonflatness.*

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.HolonomyBlocks.exists_packetWitness_of_obstructionBlock_ne_zero`

Proof. Classically choose y on which the nonzero linear map $\Pi^a_\mu \,\mathrm{Ob}\, \Pi^a_\nu$ is nonzero and set $x = \Pi^a_\nu y$. Idempotence gives $\Pi^a_\nu x = x$, and the required output is the chosen nonzero value. □

7.3 Gauge behavior of packet blocks

Vanishing or nonvanishing of the total triangle obstruction is gauge invariant by theorem 6.2, but a fixed packet pattern need not be. A general G_a can mix the projectors Π^a_μ.

Proposition 7.7 (Gauge covariance when packet projectors are preserved)**.** *If* $G_a \Pi^a_\mu = \Pi^a_\mu G_a$ *for every* μ*, then*

$$\left(\mathrm{Ob}^G\right)^{\mu\nu} = G_a \,\mathrm{Ob}^{\mu\nu}\, G_a^{-1}.$$

Consequently the zero/nonzero status of each obstruction block is preserved by gauges that preserve the packet projectors.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.HolonomyBlocks.obstructionBlock_gauge_of_commutes;Komorebi.PaperIII.HolonomyBlocks.obstructionBlock_gauge_eq_zero_iff`

Proof. Use $\mathrm{Ob}^G = G_a \,\mathrm{Ob}\, G_a^{-1}$ and commute G_a and G_a^{-1} through the two packet projectors. Composition by an equivalence is injective in both directions. □

Boundary 7.8 (Packet patterns are not absolute). Without the commutation hypothesis in proposition 7.7, only the vanishing of the total obstruction is gauge invariant. The location of individual nonzero blocks may change under a gauge that mixes packets.

8 Products supported in packets and holonomy walks

The passage from words to walks also applies to products of different matrices. Let V be one finite packet type, let R be a semiring, and let $B_1, \dots, B_m \in R^{V\times V}$ be a finite sequence of possibly different packet matrices. Rows are targets and columns are sources as before.

Proposition 8.1 (Packet path expansion for different matrices)**.** *For $i, j \in V$ and $m \geq 1$, set $i_0 = i$ and $i_m = j$. Then*

$$(B_m \cdots B_1)_{ji} = \sum_{(i_1,\dots,i_{m-1})\in V^{m-1}} B_m[i_m, i_{m-1}] \cdots B_1[i_1, i_0].$$

When $m = 1$, the sum is over the unique empty tuple and its summand is $B_1[j, i]$. For $m = 0$, the product is the identity and the entry is the Kronecker delta δ_{ji}. If every $B_r[q, p]$ vanishes outside a declared relation $\mathcal{R}_r(p, q)$, then the endpoint entry is zero whenever there is no packet path compatible with the successive relations.

Formal contract: Paper III v1.0.1.
`Komorebi.PaperIII.EdgeOperators.heterogeneousProduct;Komorebi.PaperIII.EdgeOperators.heterogeneousPathSum;Komorebi.PaperIII.EdgeOperators.heterogeneousPacketPathExpansion;Komorebi.PaperIII.EdgeOperators.SupportedStep;Komorebi.PaperIII.EdgeOperators.SupportedPath;Komorebi.PaperIII.EdgeOperators.heterogeneousProduct_apply_eq_zero_of_no_supportedPath`

Proof. Expand each matrix product entry and distribute the finite sums. Induction on the factor list yields the displayed path sum, with the empty list evaluated as the identity matrix. If no supported intermediate packet exists, every summand contains an entry that is zero by its declared support relation. □

Boundary 8.2 (Changing packet types along a chain). The compiled theorems `heterogeneousPacketPathExpansion_two` and `heterogeneousPacketPathExpansion_three` allow different packet types at the successive stages of two- and three-factor products. The arbitrary-length theorem above permits different factors but uses one common finite packet type V. A fully dependent arbitrary-length chain of changing modules and packet types remains outside the version 1.0.1 contract.

Remark 8.3 (Checked specialization). When every B_r is one repeated endomorphism T, the recursive specialization to packet powers is inherited from Paper I.

Inherited formal contract: Papers I–II.
`Komorebi.PacketNaturalityCompletion.Packets.transferBlock_pow_eq_packetPathSum`

For a loop represented on one common finite scalar packet coordinate type, the proposition resolves its successive transport matrices into packet walks. If R is a ring, as in the reversible connection API, then

$(H - I)_{ji} = H_{ji} - \delta_{ji}$. In particular, off-diagonal entries are unchanged. Thus a nonzero off-diagonal holonomy obstruction in this model with a common index type requires a transport word along the closed chart loop whose packet path runs from its input packet to its output packet. This conclusion requires declared block supports for the connection matrices. An arbitrary connection need not respect the transfer graph from section 2.

9 A minimal exact example

Example 9.1 (A scalar triangle)**.** Let $B = \{0, 1, 2\}$ and $C_0 = C_1 = C_2 = \Bbbk$ for a field $\Bbbk$. Choose $u, v, w \in \Bbbk^\times$ and set

$$\Gamma_{1\leftarrow 0}(x) = ux, \qquad \Gamma_{2\leftarrow 1}(x) = vx, \qquad \Gamma_{2\leftarrow 0}(x) = wx,$$

with reverse transports given by inverse scalars. Then

$$\Delta_{2,1,0}(x) = (vu - w)x,$$

and the triangle holonomy based at 0 is

$$H_{2,1,0}(x) = w^{-1}vux.$$

Therefore

$$H_{2,1,0} - \mathrm{id} = w^{-1}\Delta_{2,1,0}.$$

The connection is flat on this triangle exactly when $vu = w$. If the three coordinate systems are literally the same and canonical transport is used, then $u = v = w = 1$ and the obstruction vanishes. Choosing arbitrary nonzero u, v, w is a connection choice, not a consequence of changing exact packet coordinates.

Example 9.2 (A transfer walk with two steps)**.** Let a packet graph have edges $e : 0 \to 1$ and $f : 1 \to 2$, with no other edges. Then

$$T = L_e + L_f, \qquad (T^2)_{20} = c(f)c(e).$$

The products L_eL_e, L_eL_f, and L_fL_f vanish by typed mismatch. Only L_fL_e, corresponding to the walk $0 \to 1 \to 2$, contributes to the $(2, 0)$ entry. This example also fixes the noncommutative coefficient order.

10 Canonical coordinates for arbitrary maps

The analysis and synthesis identities are identities of functions, not only of linear maps. Consequently exact coordinate change does not become curved merely because the represented operation is nonlinear.

Proposition 10.1 (Exact naturality for arbitrary functions)**.** *Let $\mathcal{A}^X_a : X \xrightarrow{\sim} X_a$ and $\mathcal{A}^X_b : X \xrightarrow{\sim} X_b$ be exact coordinate equivalences for X, with inverses $\mathcal{S}^X_a$ and $\mathcal{S}^X_b$. Likewise, let $\mathcal{A}^Y_r : Y \xrightarrow{\sim} Y_r$ and $\mathcal{A}^Y_s : Y \xrightarrow{\sim} Y_s$ be exact coordinate equivalences for Y, with inverses $\mathcal{S}^Y_r$ and $\mathcal{S}^Y_s$. Let $F : X \to Y$ be any function, with no linearity or differentiability hypothesis. Set*

$$J^X_{b\leftarrow a} = \mathcal{A}^X_b \circ \mathcal{S}^X_a, \qquad J^Y_{s\leftarrow r} = \mathcal{A}^Y_s \circ \mathcal{S}^Y_r,$$

and define

$$\widehat{F}_{r\leftarrow a} = \mathcal{A}^Y_r \circ F \circ \mathcal{S}^X_a.$$

Then

$$J^{Y}_{s\leftarrow r} \circ \widehat{F}_{r\leftarrow a} = \widehat{F}_{s\leftarrow b} \circ J^{X}_{b\leftarrow a}.$$

If $\mathcal{A}^{Z}_{t} : Z \xrightarrow{\sim} Z_t$ is another exact coordinate equivalence and $G : Y \to Z$ is any function, then exact coordinate representation also preserves composition:

$$\widehat{G \circ F}_{t\leftarrow a} = \widehat{G}_{t\leftarrow r} \circ \widehat{F}_{r\leftarrow a}.$$

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.Transport.exactCoordinateMap;Komorebi.PaperIII.Transport.exactCoordinateMap_comp;Komorebi.PaperIII.Transport.exactCoordinateMap_naturality`

Proof. Both sides of the naturality equation reduce to $\mathcal{A}^{Y}_{s} \circ F \circ \mathcal{S}^{X}_{a}$ using $\mathcal{S} \circ \mathcal{A} = \mathrm{id}$ in X and Y. For composition, the middle $\mathcal{S}^{Y} \circ \mathcal{A}^{Y}$ cancels in the same way. □

Boundary 10.2 (Where nonlinear defects can enter). A nonlinear *coordinate* defect cannot be blamed on the exact canonical J of definition 4.3; proposition 10.1 makes that defect identically zero. Nonzero defects require extra structure such as a noncanonical connection, truncation, approximation, restricted domains, or failure of an operation to respect a declared packet model.

11 Nonlinear operation trees: locality and the formalization frontier

11.1 The compiled foundation of unary and binary trees

Definition 11.1 (Finite grammar for operation trees)**.** For types of input labels I, constants K, unary symbols U, and binary symbols B, the released formal grammar has four constructors:

$$\begin{aligned} t ::= \ & \mathrm{input}(i) \mid \mathrm{constant}(k) \mid \mathrm{unary}(u, t) \\ & \mid \mathrm{binary}(b, t_1, t_2). \end{aligned}$$

Given an arbitrary value type Y, evaluation takes completely explicit maps

$$I \to Y, \qquad K \to Y, \qquad U \to Y \to Y, \qquad B \to Y \to Y \to Y.$$

No algebraic, continuity, or differentiability law is imposed on these semantics.

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.NonlinearObstructions.OperationTree;Komorebi.PaperIII.NonlinearObstructions.OperationTree.eval`

When I has decidable equality, let $\mathrm{inputs}(t)$ be the finite set of input labels actually occurring in t.

Theorem 11.2 (Input locality for operation trees)**.** *Let $\eta_1, \eta_2 : I \to Y$ be two input environments. If*

$$\eta_1(i) = \eta_2(i) \qquad \textit{for every } i \in \mathrm{inputs}(t),$$

then the two evaluations of t are equal under the same constant, unary, and binary semantics. In particular, if $\mathrm{inputs}(t) = \emptyset$, evaluation is independent of the input environment.

Formal contract: Paper III v1.0.1.

`Komorebi.PaperIII.NonlinearObstructions.OperationTree.eval_congr_on_inputs;Komorebi.PaperIII.NonlinearObstructions.OperationTree.eval_eq_of_inputs_eq_empty`

Proof. Structural induction on t. The input case is the agreement hypothesis; the constant case is reflexive; the unary case applies the induction hypothesis to its child; and the binary case applies it independently to the left and right subtrees, whose input sets form a union. □

Boundary 11.3 (Exact force of locality). The theorem says evaluation reads only syntactically named inputs. It does not prove that the operations are linear or nonlinear in any analytic sense, that the tree is stable, that two different trees are equivalent, or that a solution to a tree equation exists.

11.2 Typed colored extension beyond the finite contract

Definition 11.4 (Finitary colored operation signature)**.** A finitary colored signature consists of:

- a finite type V of colors;
- a type Σ of operation symbols;
- an arity $r(\sigma) \in \mathbb{N}$ for every $\sigma \in \Sigma$;
- input colors $\mathrm{in}_\sigma : \mathrm{Fin}(r(\sigma)) \to V$;

- an output color $\mathrm{out}(\sigma) \in V$.

A realization assigns a space X_v to each color and a map

$$\Phi_\sigma : \prod_{q \in \mathrm{Fin}(r(\sigma))} X_{\mathrm{in}_\sigma(q)} \longrightarrow X_{\mathrm{out}(\sigma)}.$$

Outside the finite formal contract. The intended Lean target is a dependent inductive typed tree; no such declaration is part of the version 1.0.1 completion contract.

Definition 11.5 (Typed operation tree and evaluation)**.** A typed operation tree is recursively either a leaf of declared color or a node labeled by σ, with one correctly colored child for each input slot of σ. Evaluation is recursive:

$$\mathrm{ev}(\mathrm{node}(\sigma; t_1, \dots, t_{r(\sigma)})) = \Phi_\sigma(\mathrm{ev}(t_1), \dots, \mathrm{ev}(t_{r(\sigma)})).$$

Outside the finite formal contract. Candidate API: `Komorebi.PaperIII.NonlinearObstructions.ColoredOperationTree` and `ColoredOperationTree.eval`. This is stronger than the compiled uncolored unary–binary grammar.

11.3 Packet labelings and the multilinear first target

Suppose every X_v has a finite complete packet family $(P_i^v)_{i \in I_v}$. A packet labeling assigns an admissible packet index to each edge or node of a typed tree. For multilinear Φ_σ, inserting

$$\mathrm{id}_{X_v} = \sum_{i \in I_v} P_i^v$$

at every input and output yields a finite sum over trees with packet labels. This is the branching analogue of inserting intermediate projectors in proposition 8.1.

Formalization target 11.6 (Finite multilinear tree expansion). The first milestone for the nonlinear module is an exact theorem saying that the evaluation of a finite tree of multilinear nodes equals the finite sum of its packet-resolved labeled evaluations. Required proof obligations include:

(a) a dependent well-typed tree syntax;

(b) finite packet-label types at every color;

(c) an explicit convention for nullary and unary nodes;

(d) multilinearity in every input slot;

(e) preservation of input-slot order without assuming commutativity of operations.

Outside the finite formal contract. Candidate target: `Komorebi.PaperIII.NonlinearObstructions.ColoredOperationTree.eval_eq_sum_packetLabellings`.

11.4 Connection defects at nonlinear nodes

For coordinate models of the source and target colors, an additive target coordinate space, and a chosen noncanonical connection, a pointwise node defect can be defined by

$$\mathfrak{D}^{\Gamma}_{b,a}(\Phi) = \Gamma^{Y}_{b\leftarrow a} \circ \widehat{\Phi}_a - \widehat{\Phi}_b \circ \left(\prod_q \Gamma^{X_q}_{b\leftarrow a} \right).$$

This expression compares two *chosen* routes. With canonical exact transport in every slot it vanishes by proposition 10.1.

Formalization target 11.7 (Operation-tree defect propagation). For multilinear nodes, one expects an exact finite expansion obtained by replacing one or more nodes by their defects and transporting the remaining branches. For smooth nonlinear nodes, the corresponding chain rule is indexed by rooted partitions and Faà di Bruno trees. A formal theorem will require a fixed differentiability category, base points, derivative conventions, symmetry factors, and remainder policy.

Outside the finite formal contract. Tree-level defect identities, higher chain rules, and nonlinear norm bounds are outside the finite Lean contract. Ordinary mathematical results in these directions appear in Part II under their stated hypotheses.

Boundary 11.8 (Analytic and geometric exclusions). The finite formal contract does not include convergence of an infinite tree series, existence of derivatives, a Banach-space Faà di Bruno theorem, a Baker–Campbell–Hausdorff expansion, a continuum limit, or an identification of algebraic holonomy with curvature of a smooth bundle. Part II supplies ordinary proofs for its explicit nonlinear and analytic statements. This edition does not derive a connection from data, optimization, physics, or probability.

12 Formalization ledger for version 1.0.1

Topic	Status	Declaration or roadmap target	Exact boundary
Packet matrix and edge operator	Lean-backed III	`PacketGraph.packetMatrix`; `EdgeSystem.edgeOp`	Finite scalar matrices over a semiring; one scalar coordinate per packet.
Edge support and pair dichotomy	Lean-backed III	`edgeOp_eq_zero_off_support`; `edgeOp_mul_edgeOp_eq_zero`; `edgeOp_mul_edgeOp_of_composable`	Broken interfaces vanish; matched pairs collapse to the endpoint block with coefficient in operator order.
Typed word evaluation	Lean-backed III	`Composable`; `wordOp`; `wordOp_append`; `wordOp_eq_zero_of_not_composable`	List order is execution order; concatenation reverses operator-product order; nonwalks vanish.

Topic	Status	Declaration or roadmap target	Exact boundary
Finite word and walk expansions	Lean-backed III	`explicitWordExpansion`; `walkExpansion`; `wordExpansion_eq_explicitWordExpansion`; `transferOp_pow_eq_walkExpansion`	Actual finite tuple sum and zero-filtered composable-walk sum; finite graph and packet types.
Endpoint coefficient walk sum	Lean-backed III	`reverseCoefficient`; `Connects`; `transferOp_pow_apply_eq_sum_walks`	Exact scalar endpoint entry, including the empty-walk case and noncommutative coefficient order.
Fixed reachability and orbit support	Lean-backed III; inherited hull bridge	`transferOp_pow_apply_eq_zero_of_no_walk`; `transferOp_pow_mulVec_apply_eq_zero_of_unreachable_support`	Finite powers and supported seeds; the orbit-hull span conclusion uses the frozen Paper I contract.
Exact frame transport	Lean-backed III	`exactTransport_cocycle`; `exactPathTransport_eq_endpoint`; `exactPathTransport_loop`	One frame family on a single module; arbitrary finite paths telescope and closed paths are trivial.
Exact coordinate cocycle	Inherited Lean-backed II	`NestedRefinement.refineCoordinates_comp`; `NestedRefinement.coordinateRefinementEquiv_trans`	Complete finite projector coordinates, not reduction.
Typed coordinate naturality	Inherited Lean-backed II	`NestedRefinement.coordinateOperator_comp`; `NestedRefinement.coordinateOperator_naturality`	Linear maps over semiring modules.
Minimal connection API	Lean-backed III	`Transport.Connection`; `cocycleDefect`; `triangleHolonomy`; `triangleHolonomy_eq_refl_of_flat`	Only self-normalization is assumed; reverse inversion and the cocycle are not structure fields.
Twisted three-point loop	Lean-backed III	`twistedThreePointConnection`; `twisted_triangleHolonomy_ne_refl`	Concrete rational connection; the selected loop sends 1 to 2.
Reversible additive defect	Lean-backed III	`ReversibleConnection`; `triangleHolonomy_sub_id`; `cocycleDefect_eq_zero_iff_triangleHolonomy_eq_refl`	Heterogeneous module fibers, reverse-edge inversion, and an additive defect over a commutative ring.
Reversible finite paths	Lean-backed III	`pathTransport_eq_direct_of_flat`; `indexedPathTransport_eq_direct_of_flat`; `indexedLoopTransport_eq_refl_of_flat`	Finite pair-connection paths collapse to endpoints under algebraic flatness.

Topic	Status	Declaration or roadmap target	Exact boundary
Gauge covariance	Lean-backed III	`gauge_cocycleDefect`; `gauge_triangleHolonomy`; `gauge_flat_iff`	Fiberwise algebraic conjugation; no smooth gauge theory.
Flat versus canonical transport	Lean-backed III	`flatGauge`; `flat_isGaugeOfCanonical`	A chosen base chart and ambient frames exhibit every flat pair connection as a gauge transform of canonical transport.
Operation-tree locality	Lean-backed III	`OperationTree`; `OperationTree.eval`; `OperationTree.eval_congr_on_inputs`	Arbitrary unary and binary semantics; only dependence on named inputs is proved.
Generic obstruction packet	Lean-backed III	`ObstructionPacket.not_isSolution`; `not_isSolution_iff_nonempty_obstructionPacket`	Exact candidate-specific logical certificate; no global classification.
Holonomy block obstruction	Lean-backed III	`obstruction_sum_blocks`; `obstructionBlocks_zero_iff`; `obstructionDetection_equivalences`	Finite complete idempotent packet family on the base fiber; orthogonality is not required by the compiled reconstruction.
Packet witness and block gauge	Lean-backed III	`exists_packetWitness_of_obstructionBlock_ne_zero`; `obstructionBlock_gauge_of_commutes`; `obstructionBlock_gauge_eq_zero_iff`	Witness extraction is classical; blockwise covariance requires the gauge to commute with every packet projector.
Heterogeneous matrix paths	Lean-backed III	`heterogeneousPacketPathExpansion`; `heterogeneousProduct_apply_eq_zero_of_no_supportedPath`	Arbitrarily many different matrices over one common finite packet type; fully typed changing-index chains compile only at lengths two and three.
Arbitrary-function coordinates	Lean-backed III	`exactCoordinateMap_comp`; `exactCoordinateMap_naturality`	Exact set-level equivalences and arbitrary functions; no linearity or differentiability assumption.

Topic	Status	Declaration or roadmap target	Exact boundary
Colored and smooth trees	Roadmap only	roadmap `ColoredOperationTree.eval_eq_sum_packetLabellings`	Dependent colors, multilinear packet expansion, derivatives, and Faà di Bruno identities remain outside the contract.

12.1 Kernel-backed Paper III surface

The authoritative declaration-level contract is `formalization/lean/LEAN_THEOREM_MAP_v1.0.1.md`. It groups the compiled surface as follows.

- **EdgeOperators and EdgeExtensions:** scalar packet matrices, typed edge and word support, finite word/walk expansions, endpoint coefficient sums, fixed-length reachability, orbit support, and common-index heterogeneous products with declared supports.
- **Transport and ReversibleConnections:** exact frame transport, the minimal connection, heterogeneous reversible pair transport, additive defect and triangle holonomy, finite paths, canonical connections, and the fiberwise gauge action.
- **CoordinateNaturality and HolonomyBlocks:** composition and naturality for arbitrary functions between exact coordinate systems, finite two-sided block reconstruction, equivalent obstruction criteria, packet-supported witnesses, and packet-preserving block gauges.
- **NonlinearObstructions:** finite unary–binary operation trees, input locality, and candidate-specific obstruction certificates.

The release verification report records the checker-generated declaration and audit totals. The parity gate requires every fully qualified Paper III target printed beside a statement to occur in the theorem map, rejects stale or duplicate targets, and checks it against the compiled environment. `KernelAudit.lean` reports dependencies for the mapped and supporting theorems, and its fail-closed gate scans every compiled `Komorebi.PaperIII` declaration. The four inherited statement contracts remain identified separately by their Papers I–II names.

Boundary 12.1 (No scope promotion by prose). The displayed endpoint coefficient formula, reachability results, reversible connection theorems, additive defect–holonomy identity, gauge covariance, holonomy-block reconstruction, packet witnesses, common-index heterogeneous path expansion, and arbitrary-function coordinate naturality are all part of the version 1.0.1 kernel-backed surface. What remains outside is the dependent fiber-valued edge-system generalization, a fully dependent arbitrary-length chain whose module and packet type change at every stage, dependent colored operation trees, packet-labeled multilinear tree expansions, nonlinear tree-defect and Faà di Bruno identities, and the analytic and geometric extensions listed above.

13 The finite contract and the nonlinear transition

Typed packet support turns the expansion of a transfer power into a walk calculus. Incompatible words vanish because their interfaces do not match. Canonical packet coordinate transport is the exact equivalence obtained from analysis and synthesis. It satisfies the cocycle law, so every finite canonical loop telescopes to the identity. Holonomy enters after a separate connection is supplied. The compiled minimal API proves that flatness makes both the relative defect and holonomy trivial, and it exhibits a nontrivial rational loop. Under the compiled reversible API, a nonzero additive triangle defect is equivalent to nonidentity holonomy, transforms covariantly under gauge, and is detected exactly by a finite family of packet blocks. Finite flat paths collapse to their endpoints. Every flat reversible pair connection is explicitly related to canonical transport by a gauge change through a chosen base chart.

These facts define the boundary of the finite formal contract. At that boundary, exact coordinates preserve composition and satisfy naturality for arbitrary functions in Paper III, while the typed linear packet coordinate counterpart remains inherited from Paper II. Nontrivial nonlinear defects must therefore be attached to additional modeling choices or to packet compatibility of the particular operation. Colored operation trees, multilinear packet labelings, and Faà di Bruno expansions remain outside this finite Lean contract. Part II develops the corresponding ordinary mathematics, including packet jets, rooted tree expansions, and explicit nonlinear obstruction theorems.

Part II Nonlinear packet calculus, obstruction complexes, and holonomy

This part moves from exact coordinates for arbitrary functions to differentiable interactions, specified correction problems, and finite or smooth transport. Each ordinary proof retains its stated hypotheses. These results are not asserted to belong to the finite Lean contract of Part I.

The notational change from $\mathcal{A}, \mathcal{S}$ to $\mathcal{A}, \mathcal{S}$ is typographical: both denote analysis and synthesis for the declared packet system. A connection remains additional data. Smooth canonical transport is constructed from a moving projector family, whereas the algebraic exact coordinate transport in Part I compares fixed exact descriptions of one ambient object.

14 Introduction

A finite family of pairwise annihilating idempotents gives exact coordinates. If

$$P_\alpha P_\beta = 0 \quad \text{for } \alpha \neq \beta, \qquad \sum_\alpha P_\alpha = I,$$

then every vector is the sum of its packet components. Linear operators become block matrices in these coordinates. A nonlinear map cannot be reconstructed from its diagonal restrictions, since mixed interactions between packet components are usually present. It can still be represented exactly if its input is the complete packet tuple.

This part develops nonlinear packet representation and the obstruction theory that follows from it. Exact coordinates preserve a nonlinear problem, but they do not solve it. Once the equation is written in packet coordinates, its derivatives, compositions, correction maps, refinement failures, and gluing defects can be separated without discarding mixed terms.

The first stage of Part II treats the local nonlinear calculus. Packet analysis and synthesis conjugate arbitrary maps. Fréchet derivatives split into multilinear packet jets. The higher chain rule then becomes a Faà di Bruno formula whose terms carry intermediate packet labels. Standard background on Fréchet calculus and the higher chain rule may be found in [1, 2, 4].

The next stage studies nonlinear iteration. Repeated substitution is indexed by rooted trees rather than operator words. A scalar majorant gives an absolute convergence theorem for polynomial fixed point equations. The quadratic case recovers the Catalan series. Rooted tree combinatorics are discussed in [16, 11, 6].

A linear correction map

$$D : U \longrightarrow V$$

places the obstruction to solving $Du = -r$ in the canonical quotient coker D. This quotient has a universal property and behaves naturally under commuting squares. Mapping cones extend the same construction to cochain complexes. In a Hilbert space with closed range, the quotient is represented by the orthogonal projection onto Ker D^*. The Moore Penrose inverse then gives the least norm correction. Relevant background appears in [18, 15].

A vanishing linear obstruction is only the first step. We give a contraction theorem for nonlinear correction, followed by a local Lyapunov Schmidt reduction. In a complete filtered problem, every lifting step has its own cokernel class. Vanishing at all depths and compatible choices of corrections produce an exact inverse limit solution. A filtered correction complex carries a spectral sequence. Its first page records graded obstruction candidates, while later differentials record interactions that can cancel them. We use the standard spectral sequence of a filtered complex from [18, 12].

The final stage studies elimination and gluing. The implicit function theorem gives an exact Schur derivative after hidden variables are removed. Finite transport systems reduce, after a spanning tree gauge, to one holonomy element for each fundamental cycle. The same idea extends to smooth projector families through Kato transport. We use standard material from [4, 5, 7, 8, 10].

The article does not claim that every nonlinear problem has a finite obstruction space. The cokernel is canonical only after a correction map has been specified. It may be infinite dimensional. A first order obstruction may vanish while a higher order obstruction remains. The purpose of the calculus is to identify the exact object in which each failure lives and to show how those objects change under packets, refinement, filtration, and transport.

15 A map of the nonlinear calculus

Four questions organize the nonlinear calculus: representation, interaction, correction, and gluing.

The first question is representation. Given a nonlinear map F, can its packet form be written without approximation? The answer is yes. Analysis and synthesis are inverse coordinate maps, so conjugation preserves the full equation.

The second question is interaction. Which input packets influence a given output packet, and at what derivative order? Packet jets answer this question. They form a directed hypergraph rather than an ordinary block matrix because one nonlinear interaction can have several inputs.

The third question is correction. Suppose an approximate solution leaves a defect r. A declared linearized correction map D removes the part of r lying in $\operatorname{im} D$. The quotient class

$$[r] \in \operatorname{coker} D$$

is the part that no allowed first order correction can remove. This quotient class is the obstruction. Its vanishing must be supplemented by a nonlinear theorem.

The fourth question is gluing. Local coordinate changes can solve every edge problem and still fail around a loop. The remaining cycle product is holonomy. It measures the failure of a global frame even when each local equation has been solved.

These four layers fit into the sequence

$$\begin{aligned}\text{exact coordinates} &\longrightarrow \text{packet interactions} \longrightarrow \text{correction classes}\\ &\longrightarrow \text{global gluing.}\end{aligned}$$

Each arrow needs hypotheses. Exact coordinates need a complete packet system. Differential interactions need regularity. A cokernel obstruction needs a specified correction map. Holonomy needs invertible local transports.

A second distinction concerns scale. A refinement defect compares a fine nonlinear model with a coarse one. It is a point in the coarse output space. An obstruction class is the image of such a defect, or of another residual, in a quotient by allowed corrections. The correction map must be declared before

the defect determines an obstruction class. A small defect can have a nonzero obstruction, and a large defect can have zero obstruction if it lies entirely in the correction range.

The paper uses finite packet systems in the coordinate and graph sections. Banach space hypotheses enter only where norms, contractions, or Fréchet derivatives are used. Module and complex language is used for cokernels, filtered lifting, and cohomological obstructions. This separation keeps each result at the level of generality supported by its proof.

Standing conventions

All packet systems are finite unless a statement says otherwise. In a normed setting every packet projector and every coarsening operator is bounded. Differentiability of operator valued maps is taken in operator norm. A Banach space isomorphism is a bounded linear bijection with bounded inverse. For a decreasing filtration $(F^m X)$, completeness means that the canonical map

$$X \longrightarrow \varprojlim_m X/F^m X$$

is an isomorphism and separatedness means $\bigcap_m F^m X = 0$. Statements about congruence classes in a filtered nonlinear problem also assume that the nonlinear map preserves filtration congruences at the stated depth. These assumptions are repeated where they are essential to a proof.

16 Exact nonlinear packet coordinates

Let X be a module over a unital ring. A finite packet system on X is a family

$$\mathcal{P} = (P_\alpha)_{\alpha \in I} \subseteq \mathrm{End}(X)$$

of nonzero idempotents such that

$$P_\alpha P_\beta = 0 \quad \text{when } \alpha \neq \beta, \qquad \sum_{\alpha \in I} P_\alpha = I_X.$$

Write

$$X_\alpha = P_\alpha X, \qquad X^{\mathcal{P}} = \bigoplus_{\alpha \in I} X_\alpha.$$

The analysis and synthesis maps are

$$\mathcal{A}_{\mathcal{P}} x = (P_\alpha x)_\alpha, \qquad \mathcal{S}_{\mathcal{P}} (x_\alpha)_\alpha = \sum_\alpha x_\alpha.$$

They are inverse isomorphisms.

Definition 16.1 (Nonlinear packet coordinate map)**.** Let $\mathcal{P}$ be a packet system on X and let $\mathcal{Q}$ be a packet system on Y. For any map $F : X \to Y$, define

$$F_{\mathcal{Q} \leftarrow \mathcal{P}} = \mathcal{A}_{\mathcal{Q}} F \mathcal{S}_{\mathcal{P}}.$$

Thus

$$F_{\mathcal{Q}\leftarrow\mathcal{P}}((x_\alpha)_\alpha) = \left(Q_\gamma F\left(\sum_\alpha x_\alpha\right)\right)_\gamma.$$

Theorem 16.2 (Exact nonlinear coordinate theorem)**.** *For every map $F : X \to Y$,*

$$\mathcal{S}_{\mathcal{Q}} F_{\mathcal{Q}\leftarrow\mathcal{P}} \mathcal{A}_{\mathcal{P}} = F.$$

If $G : Y \to Z$ and $\mathcal{R}$ is a packet system on Z, then

$$(G \circ F)_{\mathcal{R}\leftarrow\mathcal{P}} = G_{\mathcal{R}\leftarrow\mathcal{Q}} \circ F_{\mathcal{Q}\leftarrow\mathcal{P}}.$$

No linearity, continuity, or differentiability assumption is needed.

Proof. The identities $\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}} = I_X$ and $\mathcal{S}_{\mathcal{Q}}\mathcal{A}_{\mathcal{Q}} = I_Y$ give

$$\mathcal{S}_{\mathcal{Q}} F_{\mathcal{Q}\leftarrow\mathcal{P}} \mathcal{A}_{\mathcal{P}} = \mathcal{S}_{\mathcal{Q}}\mathcal{A}_{\mathcal{Q}} F \mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}} = F.$$

For composition,

$$G_{\mathcal{R}\leftarrow\mathcal{Q}} F_{\mathcal{Q}\leftarrow\mathcal{P}} = \mathcal{A}_{\mathcal{R}} G \mathcal{S}_{\mathcal{Q}}\mathcal{A}_{\mathcal{Q}} F \mathcal{S}_{\mathcal{P}} = \mathcal{A}_{\mathcal{R}}(G \circ F)\mathcal{S}_{\mathcal{P}}.$$

□

Corollary 16.3 (Dynamical equivalence)**.** *Suppose $X = Y$ and the same packet system is used on both sides. Packet coordinates preserve fixed points, periodic points, injectivity, surjectivity, and invertibility. In particular,*

$$F(x) = x \quad \Longleftrightarrow \quad F_{\mathcal{P}\leftarrow\mathcal{P}}(\mathcal{A}_{\mathcal{P}} x) = \mathcal{A}_{\mathcal{P}} x.$$

Proof. The packet map is conjugate to F through the isomorphism $\mathcal{A}_{\mathcal{P}}$. Every listed property is invariant under conjugacy. □

Proposition 16.4 (Change of packet coordinates)**.** *Let $\mathcal{P}$ and $\mathcal{P}'$ be packet systems on X. Let $\mathcal{Q}$ and $\mathcal{Q}'$ be packet systems on Y. For any two packet systems $\mathcal{U}$ and $\mathcal{V}$ on the same space, write*

$$J_{\mathcal{U}\leftarrow\mathcal{V}} = \mathcal{A}_{\mathcal{U}}\mathcal{S}_{\mathcal{V}}.$$

Then

$$F_{\mathcal{Q}'\leftarrow\mathcal{P}'} = J_{\mathcal{Q}'\leftarrow\mathcal{Q}} F_{\mathcal{Q}\leftarrow\mathcal{P}} J_{\mathcal{P}\leftarrow\mathcal{P}'}.$$

Proof. Insert $\mathcal{S}_{\mathcal{Q}}\mathcal{A}_{\mathcal{Q}} = I_Y$ and $\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}} = I_X$ into the definition of $F_{\mathcal{Q}'\leftarrow\mathcal{P}'}$. □

Exactness by cancellation

Analysis and synthesis undo each other on the complete packet tuple. In a composition of packet descriptions, the intermediate synthesis and analysis maps cancel. This compares exact descriptions of the same ambient object. A connection adds data for transport along parameters or paths, with its own hypotheses.

Example 16.5 (A mixed quadratic map)**.** Let $X = Y = \Bbbk^2$ with coordinate packets. For

$$F(x_1, x_2) = (x_1 x_2, x_1^2 + x_2),$$

the first output packet depends on both input packets. The second output packet contains a quadratic contribution from the first input packet and a linear contribution from the second. Packet coordinates record these interactions exactly. Replacing F by the two diagonal restrictions would lose the product $x_1 x_2$.

Remark 16.6. Exact packet coordinates do not imply packet independence. They retain the complete input tuple. Every output packet may depend on every input packet.

17 Packet jets and higher composition

Let X and Y be Banach spaces and let the packet projectors on both spaces be bounded. Assume that $F : X \to Y$ is m times continuously Fréchet differentiable. The derivative

$$D^r F(x) : X^r \longrightarrow Y$$

is a continuous symmetric multilinear map.

Definition 17.1 (Packet jet)**.** For $1 \le r \le m$, define

$$J^r_{\gamma|\alpha_1,\dots,\alpha_r} F(x) = Q_\gamma D^r F(x)\big|_{X_{\alpha_1}\times\cdots\times X_{\alpha_r}}.$$

This is a continuous multilinear map

$$X_{\alpha_1} \times \cdots \times X_{\alpha_r} \longrightarrow Y_\gamma.$$

Theorem 17.2 (Exact reconstruction of derivatives)**.** *For $h_1, \dots, h_r \in X$,*

$$D^r F(x)[h_1, \dots, h_r] = \sum_\gamma \sum_{\alpha_1,\dots,\alpha_r} J^r_{\gamma|\alpha_1,\dots,\alpha_r} F(x)[P_{\alpha_1} h_1, \dots, P_{\alpha_r} h_r].$$

Proof. Insert $I_Y = \sum_\gamma Q_\gamma$ on the output and $I_X = \sum_\alpha P_\alpha$ in every input slot. Multilinearity expands the finite sums. □

Proposition 17.3 (Jet norm bound)**.** *The operator norm of each packet jet satisfies*

$$\left\| J^r_{\gamma|\alpha_1,\dots,\alpha_r} F(x) \right\| \le \|Q_\gamma\| \, \|D^r F(x)\| \prod_{j=1}^{r} \|P_{\alpha_j}\|.$$

When all packets are orthogonal projections on Hilbert spaces, every nonzero projector has norm one and the bound reduces to the norm of the full derivative.

Proof. Apply the definition of the operator norm to the composition of $D^r F(x)$ with the input and output projectors. □

An exact jet model with two packets

$F(x, y) = (x + xy,\ y + x^2)$ at $(0, 0)$. Coordinate packets in both input and output.

$$DF(0) = I: \quad DF(0)e_x = (1, 0), \qquad DF(0)e_y = (0, 1).$$

Output 1

H_1

	x	y
x	0	**1**
y	**1**	0

$D^2F_1(0)[e_x, e_y] = D^2F_1(0)[e_y, e_x] = 1$

Output 2

H_2

	x	y
x	**2**	0
y	0	0

$D^2F_2(0)[e_x, e_x] = 2$

$$D^2F(0)[h, k] = (h_xk_y + h_yk_x,\ 2h_xk_x)$$

$$\tfrac{1}{2}D^2F(0)[h, h] = (h_xh_y,\ h_x^2)$$

$F(h) = h + \frac{1}{2}D^2F(0)[h, h]$ exactly. All higher jets vanish.

Cells are ordered Hessian responses: the two mixed slots and the Taylor factor 1/2 are both retained.

Figure 4. Second packet jets for $F(x, y) = (x + xy, y + x^2)$ at the origin. Coordinate axis projectors give $DF(0) = I$. The first output has mixed second derivatives equal to one, while the second has a repeated first input derivative equal to two. Each grid records the ordered pair of input packets. The mixed entries retain an interaction that diagonal restrictions would lose.

Corollary 17.4 (Packet Taylor formula). *Assume F is $m+1$ times continuously Fréchet differentiable on an open set containing the segment $x+th$ for $0 \le t \le 1$. Then*

$$Q_\gamma F(x+h) = Q_\gamma F(x) + \sum_{r=1}^{m} \frac{1}{r!} \sum_{\alpha_1,\dots,\alpha_r} J^r_{\gamma|\alpha_1,\dots,\alpha_r} F(x)[P_{\alpha_1}h, \dots, P_{\alpha_r}h] + Q_\gamma R_{m+1}(x,h),$$

where the Bochner integral remainder is

$$R_{m+1}(x,h) = \frac{1}{m!}\int_0^1 (1-t)^m D^{m+1}F(x+th)[h,\dots,h]\,dt.$$

Proof. The Banach space Taylor formula with integral remainder gives the unprojected identity. Apply Q_γ to every term and use theorem 17.2 on each derivative. Boundedness of Q_γ allows it to pass through the Bochner integral. □

At a fixed base point, the nonzero packet jets form a directed hypergraph. An order r interaction has r input packet labels and one output packet label. The support therefore records interactions relative to the chosen packet system.

For a block $B = \{i_1 < \cdots < i_s\}$ and an intermediate packet γ, write

$$G_{\gamma,B}(x \mid h) = J^s_{\gamma|\alpha_{i_1},\dots,\alpha_{i_s}} G(x)[h_{i_1},\dots,h_{i_s}].$$

Arity of packet jets

A packet jet has one output slot and as many input slots as its derivative order. In the composition formula, each partition block collects inputs for one derivative of the inner map. The number of blocks determines the arity of the derivative of the outer map. Intermediate packet labels record the outputs passed between them.

Theorem 17.5 (Packet Faà di Bruno formula). *Let $G : X \to Y$ and $F : Y \to Z$ be n times continuously Fréchet differentiable. Let $\mathcal{P}$, $\mathcal{Q}$, and $\mathcal{R}$ be packet systems on X, Y, and Z. Fix input packets $\alpha_1,\dots,\alpha_n$, an output packet δ, and directions $h_i \in X_{\alpha_i}$. Then*

$$J^n_{\delta|\alpha_1,\dots,\alpha_n}(F\circ G)(x)[h_1,\dots,h_n] = \sum_{\pi=\{B_1,\dots,B_k\}} \sum_{\gamma_1,\dots,\gamma_k} J^k_{\delta|\gamma_1,\dots,\gamma_k} F(Gx)[G_{\gamma_1,B_1}(x \mid h),\dots,G_{\gamma_k,B_k}(x \mid h)].$$

The first sum runs over all set partitions of $\{1,\dots,n\}$.

Proof. The higher chain rule writes $D^n(F\circ G)(x)$ as a sum over set partitions. Insert $I_Y = \sum_\gamma Q_\gamma$ in every argument of the derivative of F. Insert the input and output packet identities into each derivative of G and into the final output. Group the terms by the partition and by the intermediate packet labels. □

Proposition 17.6 (Number of labeled composition terms)**.** *If Y has M packets, then the number of packet labeled terms at order n before zero jets are removed is*

$$N_{n,M} = \sum_{k=1}^{n} S(n,k) M^k,$$

where $S(n,k)$ is a Stirling number of the second kind. Thus $N_{n,M}$ is the nth Touchard polynomial evaluated at M.

Proof. A set partition with k blocks receives one independent intermediate packet label on each block. It therefore gives M^k labeled terms. There are $S(n,k)$ such partitions. □

Corollary 17.7 (Structural pruning)**.** *A term in theorem 17.5 vanishes whenever one of its constituent packet jets vanishes. The support of the composed jet is therefore contained in the set of decorated partitions built from the supports of the jets of F and G.*

Proof. Every summand is a composition of the displayed packet jets. A zero factor makes the entire summand zero. □

Example 17.8 (The second order composition law)**.** At order two, the partition formula has two shapes. For packet directions h_1 and h_2,

$$\begin{aligned} Q_\delta D^2(F \circ G)(x)[h_1,h_2] = &\sum_{\gamma_1,\gamma_2} J^2_{\delta|\gamma_1,\gamma_2} F(Gx)[J^1_{\gamma_1|\alpha_1} G(x)h_1, J^1_{\gamma_2|\alpha_2} G(x)h_2] \\ &+ \sum_{\gamma} J^1_{\delta|\gamma} F(Gx)[J^2_{\gamma|\alpha_1,\alpha_2} G(x)[h_1,h_2]]. \end{aligned}$$

The first line records two first order changes entering a quadratic interaction of F. The second records a genuine second order interaction created inside G before F is applied.

Remark 17.9. Jet sparsity is not invariant under an arbitrary change of coordinates. A useful packet system normally comes from symmetry, scale, support, grading, valuation, or another structure that is already present in the problem. Every sparsity claim should therefore name the packet system with respect to which it is made.

18 Rooted tree expansions

Repeated nonlinear substitution is indexed by rooted trees. We begin with a polynomial fixed point equation on a Banach space.

Let X be a Banach space. Fix $s \in X$ and continuous multilinear maps

$$B_r : X^r \longrightarrow X, \qquad 2 \le r \le d.$$

Set

$$a = \|s\|, \qquad b_r = \|B_r\|, \qquad \phi(t) = a + \sum_{r=2}^{d} b_r t^r.$$

An ordered rooted tree has leaves labeled by s and every internal vertex of arity r labeled by B_r. Its value is obtained by evaluating from the leaves toward the root.

Theorem 18.1 (Polynomial tree expansion). *Suppose there is $R > 0$ such that*

$$\phi(R) \le R$$

and

$$q = \sum_{r=2}^{d} r b_r R^{r-1} < 1.$$

Then the map

$$\Phi(x) = s + \sum_{r=2}^{d} B_r(x, \dots, x)$$

has a unique fixed point in the closed ball $\overline{B}_R(0)$. The sum of the values of all finite ordered rooted trees converges absolutely to that fixed point.

Proof. For $x, y \in \overline{B}_R(0)$, multilinearity and telescoping in the r input positions give

$$\|B_r(x, \dots, x) - B_r(y, \dots, y)\| \le r b_r R^{r-1} \|x - y\|.$$

Hence Φ is q Lipschitz on the ball. The inequality $\phi(R) \le R$ shows that Φ preserves the ball. The contraction mapping theorem gives a unique fixed point u in that ball.

Let $\mathfrak{T}_m$ be the finite set of ordered rooted trees of height less than m. A leaf has value s. If the root of τ has arity r and ordered subtrees $\tau_1, \dots, \tau_r$, define

$$B_\tau(s) = B_r(B_{\tau_1}(s), \dots, B_{\tau_r}(s)).$$

Induction on m shows that the Picard iterate $u_0 = 0$ and $u_{m+1} = \Phi(u_m)$ satisfies

$$u_m = \sum_{\tau \in \mathfrak{T}_m} B_\tau(s).$$

Define a nonnegative scalar weight by assigning weight a to a leaf and applying the scalar map $(t_1, \dots, t_r) \mapsto b_r t_1 \cdots t_r$ at a vertex of arity r. A second induction gives

$$\sum_{\tau \in \mathfrak{T}_m} \|B_\tau(s)\| \le \sum_{\tau \in \mathfrak{T}_m} w(\tau) = t_m,$$

where $t_0 = 0$ and $t_{m+1} = \phi(t_m)$. The sequence (t_m) is increasing because ϕ has nonnegative coefficients. It is bounded by R because $t_0 \le R$ and $\phi(R) \le R$. The sum of the norms of all tree values is therefore finite. The tree series converges absolutely. Its partial sums are the Picard iterates, so its sum is u. □

Corollary 18.2 (Quadratic Catalan expansion). *Let $B : X \times X \to X$ be bounded and consider*

$$u = s + B(u, u).$$

Put $a = \|s\|$ and $b = \|B\|$. If $4ab < 1$, then

$$u = \sum_{n \ge 1} \sum_{\substack{\tau \text{ full ordered binary tree} \\ \text{with } n \text{ leaves}}} B_\tau(s)$$

converges absolutely. It also satisfies

$$\|u\| \le \frac{1-\sqrt{1-4ab}}{2b}$$

when $b > 0$.

Proof. If $b = 0$, the equation is $u = s$ and the tree series contains only its one leaf. If $a = 0$, every tree value is zero. Choose any R with $0 < R < 1/(2b)$ and apply theorem 18.1 to obtain uniqueness of the zero fixed point in that ball.

Assume now that $a > 0$ and $b > 0$. Take

$$R = \frac{1-\sqrt{1-4ab}}{2b}.$$

Then $a + bR^2 = R$ and $2bR < 1$. Apply theorem 18.1. The number of full ordered binary trees with n leaves is the Catalan number C_{n-1}, which gives the displayed majorant. □

Theorem 18.3 (Packet labeled tree reconstruction)**.** *Let X carry a packet system $\mathcal{P}$. At each internal vertex of a tree, insert the output identity and one input identity for every child. Summing over all internal packet labels reconstructs the original tree value exactly.*

Proof. At a vertex labeled by an r multilinear map B_r, insert

$$I_X = \sum_{\beta} P_\beta$$

on the output and in every input slot. Multilinearity gives an exact finite sum of packet blocks at that vertex. Apply this identity successively from the leaves toward the root. Since the tree and the packet set are finite, all sums are finite and their total reconstructs the original tree value. □

For an infinite tree expansion, the finite sum of packet decorations for each underlying tree is kept together. The unlabelled tree majorant does not imply absolute convergence when all decorations are summed separately for arbitrary oblique projectors. Finite tree reconstruction and the grouped interpretation remain exact.

Arity in a tree

At each internal vertex, the number of children is the number of input slots of its multilinear map. Their order is retained because symmetry is not assumed. Packet labels resolve each finite tree into a finite sum. In an infinite expansion these sums stay grouped by tree, as required by the convergence statement above.

Corollary 18.4 (Tree pruning)**.** *A packet labeled tree vanishes if one internal vertex carries a zero packet block of the corresponding multilinear map. Removing all such zero terms leaves every finite tree sum unchanged and therefore leaves the grouped infinite series unchanged.*

Proof. A zero vertex map makes the value of the full tree zero. □

The tree theorem separates combinatorial growth from analytic convergence. The number of trees may grow rapidly, while multilinear bounds control their values. The scalar map ϕ controls the total weight

of the expansion. Packet pruning changes the combinatorics but not the proof. It removes trees whose values are already zero before any norm estimate is used.

Example 18.5 (Catalan parity packets)**.** Over a field of characteristic zero, the formal equation

$$u = z + u^2$$

has the branch

$$u(z) = \frac{1 - \sqrt{1 - 4z}}{2}.$$

Write $u = u_0 + u_1$, where u_0 contains the even powers of z and u_1 contains the odd powers. Exact packet projection gives

$$u_0 = u_0^2 + u_1^2, \qquad u_1 = z + 2u_0u_1.$$

The two equations remain coupled. Their sum is the original fixed point equation.

19 Graded nonlinear operations

Let G be a finite abelian group and let

$$A = \bigoplus_{g \in G} A_g$$

be a G graded module.

Definition 19.1 (Grade respecting operation)**.** An r multilinear map $\mu : A^r \to A$ respects the grading when

$$\mu(A_{g_1}, \dots, A_{g_r}) \subseteq A_{g_1 + \cdots + g_r}.$$

Theorem 19.2 (Graded packet convolution)**.** *If $a_j = \sum_{g \in G} a_{j,g}$ and μ respects the grading, then*

$$P_k \mu(a_1, \dots, a_r) = \sum_{g_1 + \cdots + g_r = k} \mu(a_{1,g_1}, \dots, a_{r,g_r}).$$

Proof. Expand every input into its homogeneous components. Each term lands in the component whose label is the sum of its input labels. Projection onto A_k retains exactly the displayed terms. □

Corollary 19.3 (Cyclic product formula)**.** *For a cyclically graded algebra*

$$A = \bigoplus_{j \in \mathbb{Z}/N\mathbb{Z}} A_j, \qquad A_i A_j \subseteq A_{i+j},$$

one has

$$P_k(ab) = \sum_{i+j=k} a_i b_j.$$

Proof. Apply theorem 19.2 to multiplication. □

Theorem 19.4 (Packet Maurer Cartan equations)**.** *Let*

$$\mathcal{L} = \bigoplus_{g \in G} \mathcal{L}_g$$

be a differential graded Lie algebra over a ring in which two is invertible. Assume

$$d\mathcal{L}_g \subseteq \mathcal{L}_g, \qquad [\mathcal{L}_g, \mathcal{L}_h] \subseteq \mathcal{L}_{g+h}.$$

For a degree one element $\eta = \sum_g \eta_g$, the Maurer Cartan equation

$$d\eta + \frac{1}{2}[\eta, \eta] = 0$$

is equivalent to

$$d\eta_k + \frac{1}{2} \sum_{g+h=k} [\eta_g, \eta_h] = 0$$

for every $k \in G$.

Proof. The differential preserves the group label and the bracket adds labels. Apply the projector onto $\mathcal{L}_k$ and use exact reconstruction. □

The same argument applies to higher brackets.

Proposition 19.5 (Packet equations for L_∞ operations)**.** *Let $\mathcal{L}$ be complete and carry operations ℓ_r that respect a finite abelian group grading. Assume the packet projectors are continuous for the topology in which the series below converges. Assume the Maurer Cartan series*

$$\sum_{r \geq 1} \frac{1}{r!} \ell_r(\eta, \dots, \eta)$$

converges and the required factorials are invertible. Then its packet of label k is

$$\sum_{r \geq 1} \frac{1}{r!} \sum_{g_1 + \cdots + g_r = k} \ell_r(\eta_{g_1}, \dots, \eta_{g_r}).$$

The full Maurer Cartan equation holds if and only if every packet equation vanishes.

Proof. Apply theorem 19.2 to each operation ℓ_r. Continuity of the packet projectors permits the convergent sums to be projected term by term. □

Definition 19.6 (Divided square)**.** A divided square on degree one elements is a map q satisfying

$$q(x + y) = q(x) + q(y) + [x, y], \qquad q(ax) = a^2 q(x),$$

whenever the expressions are defined.

Proposition 19.7 (Divided square packet equations)**.** *Assume $q(\mathcal{L}_g) \subseteq \mathcal{L}_{2g}$. Choose an ordering of the finite label set. The equation*

$$d\eta + q(\eta) = 0$$

is equivalent to

$$d\eta_k + \sum_{2g=k} q(\eta_g) + \sum_{\substack{g<h \\ g+h=k}} [\eta_g, \eta_h] = 0$$

for every k.

Proof. Repeated use of the polarization identity for q gives

$$q\left(\sum_g \eta_g\right) = \sum_g q(\eta_g) + \sum_{g<h} [\eta_g, \eta_h].$$

Project onto the component of label k. □

Example 19.8 (Matrix difference grading)**.** Let $A = M_N(\Bbbk)$ and assign the matrix unit E_{rs} the label $r-s$ modulo N. Since

$$E_{rs}E_{uv} = \delta_{su}E_{rv},$$

the output label is

$$(r-s) + (u-v) = r-v$$

whenever the product is nonzero. Matrix multiplication therefore obeys the cyclic convolution law exactly. The same grading resolves commutators and polynomial matrix equations.

Remark 19.9. These results begin with a graded differential or higher algebra. They do not assert that every deformation problem has such a controlling object. Differential graded and infinity algebra methods are developed in [11].

20 Nonlinear refinement defects

Let

$$F_f : X_f \longrightarrow Y_f, \qquad F_c : X_c \longrightarrow Y_c$$

be fine and coarse maps. Let

$$C_X : X_f \longrightarrow X_c, \qquad C_Y : Y_f \longrightarrow Y_c$$

be linear coarsening maps. Whenever a norm, a Lipschitz constant, or a Fréchet derivative is used, the spaces are Banach spaces and the coarsening maps are bounded.

Definition 20.1 (Nonlinear refinement defect)**.** The pointwise defect of F is

$$\Delta_F(x) = C_Y F_f(x) - F_c(C_X x).$$

The two models are exactly compatible when Δ_F vanishes everywhere.

The defect is a vector in Y_c. It determines an obstruction class after a correction map into Y_c has been chosen and the defect has been passed to the corresponding cokernel. Norm estimates measure the size of the defect. The quotient determines whether an allowed linear correction can remove it.

Theorem 20.2 (Exact defect under composition). *Let $G_f : Y_f \to Z_f$ and $G_c : Y_c \to Z_c$, with coarsening $C_Z : Z_f \to Z_c$. Then*

$$\Delta_{G\circ F}(x) = \Delta_G(F_f(x)) + G_c(F_c(C_X x) + \Delta_F(x)) - G_c(F_c(C_X x)).$$

If G_c is L Lipschitz on the relevant set, then

$$\|\Delta_{G\circ F}(x)\| \le \|\Delta_G(F_f(x))\| + L\|\Delta_F(x)\|.$$

Proof. Add and subtract $G_c(C_Y F_f(x))$. Use

$$C_Y F_f(x) = F_c(C_X x) + \Delta_F(x).$$

The norm estimate follows from the Lipschitz bound. □

Corollary 20.3 (Linear product rule). *If all maps in theorem 20.2 are linear, then*

$$\Delta_{GF} = \Delta_G F_f + G_c \Delta_F.$$

Proof. Linearity turns the last difference in theorem 20.2 into $G_c\Delta_F(x)$. □

Theorem 20.4 (Iterated orbit defect). *Let $F_f : X_f \to X_f$ and $F_c : X_c \to X_c$, with coarsening $C : X_f \to X_c$. Suppose F_c is q Lipschitz. For a fine orbit starting at x, assume*

$$\|\Delta_F(F_f^j x)\| \le \varepsilon_j.$$

Then

$$\left\|CF_f^n(x) - F_c^n(Cx)\right\| \le \sum_{j=0}^{n-1} q^{n-1-j}\varepsilon_j.$$

If $0 \le q < 1$ and every $\varepsilon_j \le \varepsilon$, then

$$\left\|CF_f^n(x) - F_c^n(Cx)\right\| \le \varepsilon\frac{1-q^n}{1-q}.$$

Proof. Let

$$e_n = \left\|CF_f^n(x) - F_c^n(Cx)\right\|.$$

The definition of the defect and the Lipschitz property give

$$e_{n+1} \le \varepsilon_n + qe_n.$$

Iterating this recurrence proves the first estimate. The second is the geometric sum. □

Theorem 20.5 (Fixed point stability). *Assume F_c is a contraction with constant $q < 1$. Let x_f and x_c be fixed points of F_f and F_c. If*

$$\|\Delta_F(x_f)\| \le \varepsilon,$$

then

$$\|Cx_f - x_c\| \le \frac{\varepsilon}{1-q}.$$

When the defect at x_f is zero, the fixed points are compatible.

Proof. Using the fixed point equations,

$$\begin{aligned}\|Cx_f - x_c\| &\le \|CF_f(x_f) - F_c(Cx_f)\| + \|F_c(Cx_f) - F_c(x_c)\| \\ &\le \varepsilon + q\|Cx_f - x_c\|.\end{aligned}$$

Rearrange. □

Corollary 20.6 (Several refinement levels)**.** *Let $C_\ell : X_{\ell+1} \to X_\ell$ for $0 \le \ell < m$. Suppose x_ℓ is a fixed point at level ℓ, the coarse map at level ℓ has contraction constant $q_\ell < 1$, and the adjacent defect at $x_{\ell+1}$ is at most ε_ℓ. Write*

$$C_{a,b} = C_a C_{a+1} \cdots C_{b-1} \quad \text{for } a < b, \qquad C_{a,a} = I.$$

Then

$$\|C_{0,m}x_m - x_0\| \le \sum_{\ell=0}^{m-1} \|C_{0,\ell}\| \frac{\varepsilon_\ell}{1-q_\ell}.$$

Proof. Insert the intermediate vectors $C_{0,\ell}x_\ell$ and telescope. Apply theorem 20.5 to every adjacent pair. □

Proposition 20.7 (Derivative defects)**.** *Assume F_f and F_c are r times Fréchet differentiable and the coarsening maps are bounded and linear. Then*

$$D\Delta_F(x) = C_Y DF_f(x) - DF_c(C_X x)C_X.$$

For $r \ge 2$,

$$D^r\Delta_F(x)[h_1, \dots, h_r] = C_Y D^r F_f(x)[h_1, \dots, h_r] - D^r F_c(C_X x)[C_X h_1, \dots, C_X h_r].$$

Proof. Differentiate the defining identity for Δ_F. The coarsening maps are linear, so their higher derivatives vanish. □

Example 20.8 (Sharp affine bound)**.** Let $F_c(y) = qy + b$ with $0 < q < 1$. Choose a fine fixed point whose coarsening has constant defect ε in the direction of the scalar state. The fixed point mismatch is exactly $\varepsilon/(1-q)$. Thus the constant in theorem 20.5 cannot be improved without more information.

21 Canonical obstruction objects

We begin by isolating the part of a defect that cannot be removed by an allowed linear correction.

Definition 21.1 (Linear correction datum)**.** A linear correction datum consists of a homomorphism

$$D : U \longrightarrow V$$

and a defect $r \in V$. A correction is an element $u \in U$ satisfying

$$Du = -r.$$

The obstruction class is

$$\mathrm{Ob}_D(r) = [r] \in \operatorname{coker} D = V / \operatorname{im} D.$$

Theorem 21.2 (Cokernel obstruction criterion)**.** *A linear correction exists if and only if*

$$\mathrm{Ob}_D(r) = 0.$$

The quotient map

$$q_D : V \longrightarrow \operatorname{coker} D$$

has the following universal property. If $\varphi : V \to W$ is linear and $\varphi D = 0$, then there is a unique linear map

$$\overline{\varphi} : \operatorname{coker} D \longrightarrow W$$

with $\varphi = \overline{\varphi} q_D$.

Proof. The class of r is zero exactly when $r \in \operatorname{im} D$, which is equivalent to solvability of $Du = -r$. If $\varphi D = 0$, then φ vanishes on $\operatorname{im} D$. It therefore descends uniquely to the quotient. □

The universal property explains why the cokernel is the canonical obstruction object relative to D. Any linear obstruction test that vanishes on correctable defects factors through it.

Canonical obstruction data

The residual, the correction map, and the obstruction class are distinct data. The residual is a vector. The correction map specifies which changes are allowed, and its cokernel records the remaining class. Canonicality is relative to that declared correction map. A scalar or orthogonal representative of the class may require further choices or hypotheses.

Theorem 21.3 (Naturality of obstruction classes)**.** *Suppose*

$$\begin{array}{ccc} U & \xrightarrow{D} & V \\ a \downarrow & & \downarrow b \\ U' & \xrightarrow{D'} & V' \end{array}$$

is a commuting square. Then there is a unique induced map

$$\overline{b} : \operatorname{coker} D \longrightarrow \operatorname{coker} D'$$

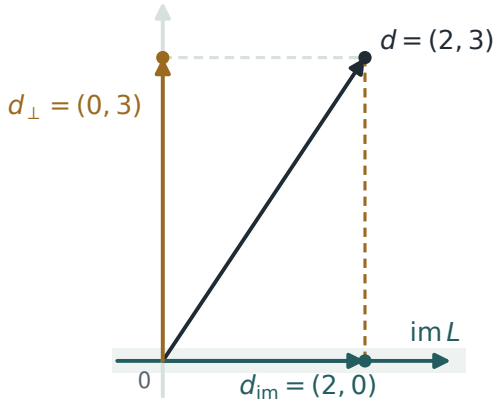
satisfying

$$\overline{b}\ \mathrm{Ob}_D(r) = \mathrm{Ob}_{D'}(br).$$

Proof. The relation $bD = D'a$ shows that the quotient map $q_{D'} b$ vanishes on $\operatorname{im} D$. Apply the universal property in theorem 21.2. □

Correctable defect and cokernel residual

$L : \mathbb{R}^2 \to \mathbb{R}^2$, $L(a, b) = (a, 0)$. Defect $d = (2, 3)$.



A chosen scalar detector

$$\ell(x, y) = y$$

$$\ell L = 0$$

$$\ell(d) = 3 \neq 0$$

$$u = (-2, 0): \quad d + Lu = (0, 3) \neq 0.$$

The class $[d] \in \mathbb{R}^2/\operatorname{im} L$ is represented by $(0, 3)$.

Standard Euclidean splitting is used. The scalar 3 depends on the displayed quotient identification.

Figure 5. A specified correction range in $\mathbb{R}^2$. For $L(a, b) = (a, 0)$, the image is the horizontal axis. The defect $(2, 3)$ splits into $(2, 0)$ in that image and the remaining representative $(0, 3)$. Subtracting the image component leaves the same cokernel class. The functional $(x, y) \mapsto y$ vanishes on the correction range and detects the value three. The correction equation $Lu = -(2, 3)$ therefore has no solution.

Theorem 21.4 (Packet decomposition of an obstruction)**.** *Suppose*

$$U = \bigoplus_{\alpha \in I} U_\alpha, \qquad V = \bigoplus_{\alpha \in I} V_\alpha,$$

and $D(U_\alpha) \subseteq V_\alpha$ for every α. Let

$$D_\alpha : U_\alpha \longrightarrow V_\alpha$$

be the restriction. Then

$$\operatorname{coker} D \cong \bigoplus_{\alpha \in I} \operatorname{coker} D_\alpha.$$

Under this isomorphism, the obstruction class of $r = \sum_\alpha r_\alpha$ is the tuple

$$\big([r_\alpha]\big)_\alpha.$$

Hence r is correctable if and only if every packet obstruction vanishes.

Proof. The grading assumption gives

$$\operatorname{im} D = \bigoplus_{\alpha} \operatorname{im} D_\alpha.$$

Taking the quotient of the direct sum by this direct sum of images gives the displayed isomorphism. □

Remark 21.5. The conclusion of theorem 21.4 requires the correction map to preserve the packets. Off diagonal correction blocks can couple packet defects. In that case the full cokernel remains canonical, but it need not split into the separate packet cokernels.

Proposition 21.6 (Projectors on the obstruction quotient)**.** *Let $(Q_\alpha)_{\alpha \in I}$ be a complete family of pairwise annihilating idempotents on V. Assume*

$$Q_\alpha(\operatorname{im} D) \subseteq \operatorname{im} D$$

for every α. Then each Q_α induces an idempotent $\overline{Q}_\alpha$ on $\operatorname{coker} D$, and

$$\operatorname{coker} D = \bigoplus_{\alpha \in I} \overline{Q}_\alpha(\operatorname{coker} D).$$

For every defect $r \in V$,

$$\overline{Q}_\alpha \operatorname{Ob}_D(r) = [Q_\alpha r].$$

No packet decomposition of the correction space U is required.

Proof. If $v - v' \in \operatorname{im} D$, then

$$Q_\alpha(v - v') \in \operatorname{im} D.$$

Hence $[v] \mapsto [Q_\alpha v]$ is well defined on the quotient. The idempotent, annihilation, and reconstruction identities descend from V to $V/\operatorname{im} D$. The formula for an obstruction class follows from the definition. □

21.1 Relative obstruction under coarsening

Coarsening can erase an obstruction that remains present in the fine problem. The next exact sequence identifies the part removed by coarsening.

Theorem 21.7 (Relative obstruction exact sequence). *Suppose the following diagram of modules commutes and has exact rows:*

$$\begin{array}{ccccccccc} 0 & \longrightarrow & U_0 & \xrightarrow{i_U} & U_f & \xrightarrow{p_U} & U_c & \longrightarrow & 0 \\ & & D_0\downarrow & & D_f\downarrow & & D_c\downarrow & & \\ 0 & \longrightarrow & V_0 & \xrightarrow{i_V} & V_f & \xrightarrow{p_V} & V_c & \longrightarrow & 0. \end{array}$$

There is a natural exact sequence

$$\begin{aligned} 0 \longrightarrow \operatorname{Ker} D_0 \longrightarrow \operatorname{Ker} D_f \longrightarrow \operatorname{Ker} D_c \xrightarrow{\delta} \operatorname{coker} D_0 \\ \longrightarrow \operatorname{coker} D_f \longrightarrow \operatorname{coker} D_c \longrightarrow 0. \end{aligned}$$

For $u_c \in \operatorname{Ker} D_c$, choose $u_f \in U_f$ with $p_U u_f = u_c$. There is a unique $v_0 \in V_0$ such that

$$D_f u_f = i_V v_0,$$

and the connecting map is

$$\delta(u_c) = [v_0].$$

Proof. Since $D_c u_c = 0$, commutativity gives

$$p_V D_f u_f = D_c p_U u_f = 0.$$

Exactness of the lower row gives a unique v_0 with $D_f u_f = i_V v_0$. A different lift has the form $u_f + i_U u_0$. The corresponding element of V_0 changes by $D_0 u_0$, so $[v_0]$ is independent of the lift.

The map $\operatorname{Ker} D_0 \to \operatorname{Ker} D_f$ is injective because i_U is injective. If $u_f \in \operatorname{Ker} D_f$ maps to zero in U_c, then $u_f = i_U u_0$ for a unique u_0. The identity

$$i_V D_0 u_0 = D_f i_U u_0 = 0$$

and injectivity of i_V show that $u_0 \in \operatorname{Ker} D_0$. This proves exactness at $\operatorname{Ker} D_f$.

If u_c comes from an element of $\operatorname{Ker} D_f$, one may choose that element as its lift and obtain $\delta(u_c) = 0$. Conversely, if $\delta(u_c) = 0$, then $v_0 = D_0 u_0$ for some u_0. Replacing u_f by $u_f - i_U u_0$ gives an element of $\operatorname{Ker} D_f$ that maps to u_c. This proves exactness at $\operatorname{Ker} D_c$.

The class $[v_0] \in \operatorname{coker} D_0$ maps to zero in $\operatorname{coker} D_f$ whenever it equals $\delta(u_c)$, since the chosen lift satisfies $i_V v_0 = D_f u_f$. Conversely, if $i_V v_0 = D_f u_f$, then $u_c = p_U u_f$ lies in $\operatorname{Ker} D_c$ and $\delta(u_c) = [v_0]$. This proves exactness at $\operatorname{coker} D_0$.

If a class in $\operatorname{coker} D_f$ comes from $\operatorname{coker} D_0$, its image in $\operatorname{coker} D_c$ is zero. Conversely, suppose $[v_f]$ maps to zero in $\operatorname{coker} D_c$. Choose u_c with $D_c u_c = p_V v_f$ and lift it to $u_f \in U_f$. Then

$$p_V(v_f - D_f u_f) = 0,$$

so $v_f - D_f u_f = i_V v_0$ for some $v_0 \in V_0$. Thus $[v_f]$ comes from $[v_0]$. Surjectivity at the last term follows from surjectivity of p_V. □

Corollary 21.8 (Coarse vanishing and fine vanishing)**.** *Let $[r_f] \in \operatorname{coker} D_f$. If its image in $\operatorname{coker} D_c$ is zero, then $[r_f]$ comes from $\operatorname{coker} D_0$. Coarse vanishing therefore means that the fine obstruction is supported in information removed by coarsening. It does not imply $[r_f] = 0$.*

Proof. This is exactness of the sequence in theorem 21.7 at $\operatorname{coker} D_f$. □

Example 21.9 (A fine obstruction erased by coarsening)**.** Take all three correction spaces to be zero. Let

$$V_f = \Bbbk^2, \qquad V_c = \Bbbk, \qquad p_V(x, y) = x + y,$$

and let

$$V_0 = \operatorname{Ker} p_V = \operatorname{span}\{(1, -1)\}.$$

The fine defect $(1, -1)$ is a nonzero class in $\operatorname{coker} D_f = V_f$, while its coarse image is zero. The class comes from $\operatorname{coker} D_0 = V_0$, exactly as corollary 21.8 predicts.

The same construction has a cohomological form.

Definition 21.10 (Mapping cone)**.** Let $D : C^\bullet \to E^\bullet$ be a cochain map. Its mapping cone is

$$\operatorname{Cone}(D)^n = E^n \oplus C^{n+1}$$

with differential

$$d_{\operatorname{Cone}}(e, c) = (d_E e + Dc, -d_C c).$$

Theorem 21.11 (Cohomological obstruction through the cone)**.** *Let $e \in E^n$ satisfy $d_E e = 0$. The class of $(e, 0)$ in*

$$H^n(\operatorname{Cone}(D))$$

vanishes if and only if the class $[e] \in H^n(E)$ lies in the image of

$$H^n(D) : H^n(C) \longrightarrow H^n(E).$$

When C and E are concentrated in degree zero,

$$H^0(\operatorname{Cone}(D)) \cong \operatorname{coker} D.$$

Proof. The class of $(e, 0)$ vanishes exactly when there are $e' \in E^{n-1}$ and $c \in C^n$ with

$$(e, 0) = d_{\operatorname{Cone}}(e', c).$$

This means

$$e = d_E e' + Dc, \qquad d_C c = 0.$$

Equivalently, $[e] = H^n(D)[c]$. The degree zero statement follows directly from the definition of the cone differential. □

Corollary 21.12 (Packet obstruction complexes)**.** *If $C^\bullet$ and $E^\bullet$ split into finite packet subcomplexes and D preserves those packets, then $\operatorname{Cone}(D)$ splits into packet subcomplexes and*

$$H^n(\operatorname{Cone}(D)) \cong \bigoplus_\alpha H^n(\operatorname{Cone}(D_\alpha)).$$

Proof. Both the cone and its cohomology commute with finite direct sums. □

Example 21.13 (A two component obstruction)**.** Let

$$D : \Bbbk \longrightarrow \Bbbk^2, \qquad D(t) = (t, t).$$

The cokernel is one dimensional. The defect (a, b) is correctable exactly when $a = b$. The obstruction can be represented by $a - b$, but this scalar representative depends on a choice of isomorphism from the cokernel to $\Bbbk$. The quotient class itself is canonical.

22 Split Hilbert obstructions and nonlinear correction

The quotient obstruction has a concrete orthogonal representative when the correction map acts between Hilbert spaces and has closed range. The Hilbert and Banach space results below are analytic extensions of the finite calculus. They use the stated analytic hypotheses and lie outside the finite formal contract.

Theorem 22.1 (Orthogonal obstruction projector)**.** *Let $D : H_1 \to H_2$ be a bounded operator between Hilbert spaces with closed range. Let $D^\dagger$ be its Moore Penrose inverse. Then*

$$P_{\text{ob}} = I_{H_2} - DD^\dagger$$

is the orthogonal projector onto

$$(\operatorname{im} D)^\perp = \operatorname{Ker} D^*.$$

For $r \in H_2$, the following are equivalent:

1. *$Du = -r$ is solvable*
2. *$[r] = 0$ in $\operatorname{coker} D$*
3. *$P_{\text{ob}} r = 0$.*

When these conditions hold, $-D^\dagger r$ is the least norm correction. For every r,

$$\operatorname{dist}(r, \operatorname{im} D) = \|P_{\text{ob}} r\|.$$

Proof. Closed range gives the orthogonal decomposition

$$H_2 = \operatorname{im} D \oplus \operatorname{Ker} D^*.$$

The Moore Penrose identities show that $DD^\dagger$ is the orthogonal projector onto $\operatorname{im} D$. The stated equivalences and distance formula follow from this decomposition. The least norm property is the standard characterization of $D^\dagger r$. □

Once the defect lies in the range, nonlinear correction still requires control of the nonlinear term. The next theorem gives that control through a bounded right inverse and a contraction estimate.

Theorem 22.2 (Right inverse correction theorem)**.** *Let $D : X \to Y$ be a bounded linear map between Banach spaces. Assume $Z = \operatorname{im} D$ is closed and there is a bounded linear right inverse*

$$R : Z \longrightarrow X, \qquad DR = I_Z.$$

Put $X_R = R(Z)$. Let

$$N : \overline{B}_\rho(0) \cap X_R \longrightarrow Z$$

be L Lipschitz. Let $y \in Z$ and put

$$q = \|R\|L.$$

Assume $q < 1$ and

$$\|R(y - N(0))\| \leq (1 - q)\rho.$$

Then the equation

$$Dx + N(x) = y$$

has a unique solution in $\overline{B}_\rho(0) \cap X_R$.

Proof. The space X_R is closed. Indeed, if $Rz_n \to x$, then $z_n = DRz_n \to Dx$, so $x = R(Dx)$. Define

$$\Psi(x) = R(y - N(x)).$$

This map takes X_R to itself. For $x \in \overline{B}_\rho(0) \cap X_R$,

$$\begin{aligned}\|\Psi(x)\| &\leq \|R(y - N(0))\| + \|R\|\,\|N(x) - N(0)\| \\ &\leq (1 - q)\rho + q\rho = \rho.\end{aligned}$$

Also,

$$\|\Psi(x) - \Psi(x')\| \leq q\|x - x'\|.$$

The contraction mapping theorem gives a unique fixed point in the stated set. Applying D to the fixed point identity gives the equation. Conversely, a solution in X_R equals $R(Dx)$ and is therefore a fixed point of Ψ. □

The hypotheses behind correction

Closed range gives the obstruction an orthogonal representative in the Hilbert setting. A bounded right inverse chooses a correction slice. In the nonlinear theorem, the target and the nonlinear term both lie in the range, and the Lipschitz bound makes the correction map a contraction on the chosen ball. The uniqueness statement concerns the chosen ball within that correction slice.

Corollary 22.3 (Stability with respect to the target)**.** *Under the hypotheses of theorem 22.2, let x and x' be the fixed points associated with targets y and y'. If both fixed point maps preserve the same ball, then*

$$\|x - x'\| \leq \frac{\|R\|}{1 - q}\|y - y'\|.$$

Proof. Subtract the two fixed point equations and use the Lipschitz bound for N:

$$\|x - x'\| \leq \|R\|\|y - y'\| + q\|x - x'\|.$$

□

Lyapunov Schmidt reduction separates the equation into two parts. The implicit function theorem solves the range component, leaving an equation in the obstruction space. Standard accounts of this reduction appear in [9, 19].

Theorem 22.4 (Local Lyapunov Schmidt reduction)**.** *Let $\mathcal{U} \subseteq X$ be an open neighborhood of zero and let $F : \mathcal{U} \to Y$ be a C^r map between Banach spaces with $r \geq 1$, $F(0) = 0$, and $L = DF(0)$. Assume there are closed decompositions*

$$X = K \oplus X_1, \qquad Y = R \oplus O,$$

where $K = \operatorname{Ker} L$, $R = \operatorname{im} L$, and

$$L_1 = L|_{X_1} : X_1 \longrightarrow R$$

is a Banach space isomorphism. Let π_R and π_O be the two projections on Y. Then there are neighborhoods $U_K \subseteq K$ and $U_1 \subseteq X_1$ of zero and a unique C^r map

$$\psi : U_K \longrightarrow U_1$$

with $\psi(0) = 0$ such that

$$\pi_R F(k + \psi(k)) = 0$$

for every $k \in U_K$. After the neighborhoods are made smaller if needed, the solutions of $F(x) = 0$ near zero are exactly the points

$$x = k + \psi(k)$$

for which the reduced obstruction map

$$\kappa(k) = \pi_O F(k + \psi(k))$$

vanishes. At the base point,

$$D\psi(0) = 0, \qquad D\kappa(0) = 0.$$

Proof. Define

$$G(k, x_1) = \pi_R F(k + x_1).$$

The derivative of G in the X_1 variable at $(0, 0)$ is L_1, which is invertible. The implicit function theorem gives the unique map ψ. Every nearby x has a unique decomposition $x = k + x_1$. The range equation forces $x_1 = \psi(k)$, and the remaining equation is precisely $\kappa(k) = 0$.

Differentiate $G(k, \psi(k)) = 0$ at zero. Since L vanishes on K, one obtains $L_1 D\psi(0) = 0$, hence $D\psi(0) = 0$. Finally,

$$D\kappa(0) = \pi_O L|_K = 0.$$

□

Corollary 22.5 (Quadratic reduced obstruction)**.** *Assume the hypotheses of theorem 22.4 with $r \geq 2$. For $k_1, k_2 \in K$,*

$$D^2\kappa(0)[k_1, k_2] = \pi_O D^2 F(0)[k_1, k_2].$$

Proof. Differentiate $\kappa(k) = \pi_O F(k+\psi(k))$ twice. The terms containing $D\psi(0)$ vanish. The term containing $D^2\psi(0)$ is multiplied by $\pi_O L$, which is zero because $L(X) = R$. □

Example 22.6 (A purely quadratic obstruction)**.** Let

$$F(x, y) = (y, x^2)$$

on $\mathbb{R}^2$. At the origin,

$$L(h, k) = (k, 0).$$

The kernel is the x axis, the range is the first output axis, and the cokernel is the second output axis. The range equation forces $y = 0$. The reduced obstruction map is

$$\kappa(x) = x^2.$$

The linear obstruction vanishes for every kernel direction, but the nonlinear equation still has only the solution $x = 0$ near the origin.

Remark 22.7. The reduced map κ begins at quadratic order. This is why vanishing of the first order cokernel class need not settle a nonlinear problem. Higher terms can produce a genuine obstruction on the kernel directions.

23 Filtered lifting

Let X and Y be modules with decreasing filtrations

$$X = F^0X \supseteq F^1X \supseteq \cdots, \qquad Y = F^0Y \supseteq F^1Y \supseteq \cdots.$$

Assume both are separated and complete for their filtration topologies. Let $\Phi : X \to Y$ be continuous and filtration congruence preserving in the sense that

$$h \in F^mX \quad \Longrightarrow \quad \Phi(x + h) - \Phi(x) \in F^mY$$

for every $x \in X$ and every m. This condition makes the statement $\Phi(x) \in F^mY$ depend only on the class of x modulo F^mX.

Suppose x_m satisfies

$$\Phi(x_m) \in F^mY.$$

Assume that corrections $h \in F^mX$ obey a first order congruence

$$\Phi(x_m + h) \equiv \Phi(x_m) + L_mh \pmod{F^{m+1}Y},$$

where $L_m : F^mX \to F^mY$ is a module homomorphism satisfying

$$L_m(F^{m+1}X) \subseteq F^{m+1}Y$$

and therefore induces

$$\overline{L}_m : F^mX/F^{m+1}X \longrightarrow F^mY/F^{m+1}Y.$$

At depth m, only the associated graded correction is visible. The cokernel below has finite length whenever the target graded piece has finite length, even if the completed problem is infinite. Vanishing of the depth obstruction decides whether the current approximation can be improved by one level. When the graded pieces are finite, the branching count below applies. Finite length alone does not imply finite cardinality.

Definition 23.1 (Depth obstruction)**.** The depth m obstruction of x_m is

$$\mathrm{Ob}_m(x_m) = [\Phi(x_m)] \in \operatorname{coker} \overline{L}_m,$$

where $\Phi(x_m)$ is first viewed in $F^mY/F^{m+1}Y$.

Theorem 23.2 (One step lifting criterion)**.** *The approximation x_m has a correction $h_m \in F^m X$ such that*

$$\Phi(x_m + h_m) \in F^{m+1}Y$$

if and only if

$$\mathrm{Ob}_m(x_m) = 0.$$

When a correction exists, the set of its classes in

$$F^m X / F^{m+1} X$$

is a coset of $\operatorname{Ker} \overline{L}_m$*. If the graded pieces are vector spaces, this coset is an affine space.*

Proof. The next depth condition is equivalent to

$$\overline{L}_m[h_m] = -[\Phi(x_m)]$$

in $F^m Y / F^{m+1} Y$. This is solvable exactly when the displayed defect has zero cokernel class. If h_m and h'_m are two solutions, their difference lies in $\operatorname{Ker} \overline{L}_m$. Adding an element of the kernel to one solution gives another, so the solution set is a coset of the kernel. □

Corollary 23.3 (Finite branching count)**.** *Suppose the graded pieces are finite vector spaces over a finite field. If the depth obstruction vanishes, then the number of correction classes at depth m is*

$$|\operatorname{Ker} \overline{L}_m|.$$

If the obstruction does not vanish, the number is zero.

Proof. A nonempty affine space over a finite vector space has the same number of elements as that vector space. □

Lemma 23.4 (Well defined lift classes)**.** *Under the filtration congruence assumption, the condition*

$$\Phi(x) \in F^m Y$$

depends only on the class of x in $X/F^m X$. Fix a representative x_m of a depth m class. The map

$$[h] \longmapsto [x_m + h]_{m+1}$$

identifies correction classes $[h] \in F^m X / F^{m+1} X$ with the classes at depth $m+1$ that reduce to $[x_m]_m$. Under this identification, the solutions of the associated graded correction equation in theorem 23.2 are exactly the children whose defect lies in $F^{m+1}Y$. A different representative of the same parent gives a translated parametrization of the same set of children.

Proof. If $x' - x \in F^m X$, then filtration congruence preservation gives

$$\Phi(x') - \Phi(x) \in F^m Y.$$

Thus one defect lies in $F^m Y$ exactly when the other does. If $h - h' \in F^{m+1} X$, then $x_m + h$ and $x_m + h'$ define the same depth $m+1$ class. Conversely, two depth $m+1$ classes reducing to $[x_m]_m$ differ from x_m by elements of $F^m X$, and they are equal precisely when those elements agree modulo $F^{m+1} X$. The

first order congruence shows that such a child has defect in $F^{m+1}Y$ exactly when its correction class solves the associated graded equation. Replacing x_m by another representative translates the correction coordinate by an element of $F^mX/F^{m+1}X$ and leaves the child classes themselves unchanged. □

Definition 23.5 (Lift tree)**.** Fix an initial approximation. A vertex at depth m is a class $[x]_m \in X/F^mX$ with $\Phi(x) \in F^mY$ that reduces to the fixed initial class. A depth $m+1$ vertex is joined to its reduction in X/F^mX. By lemma 23.4, this rooted graph is independent of the chosen representatives. It is called the lift tree.

Theorem 23.6 (Finitely branching lift tree)**.** *Assume every level of the lift tree is nonempty and every vertex has only finitely many children. Then the tree contains an infinite branch. If X and Y are separated and complete and Φ is continuous, that branch determines an exact solution of $\Phi(x) = 0$.*

Proof. The first assertion is König's infinity argument. The root has finitely many children. Since vertices occur at arbitrarily large depths, at least one child has descendants at arbitrarily large depths. Choose such a child and repeat the argument at every chosen vertex. This gives one compatible vertex at each depth.

Choose representatives recursively. Once x_m represents the selected depth m vertex, choose a representative x_{m+1} of its selected child with

$$x_{m+1} - x_m \in F^mX.$$

The resulting sequence is Cauchy for the filtration topology. Each representative satisfies $\Phi(x_m) \in F^mY$. The conclusion now follows from theorem 23.8. □

Corollary 23.7 (Unique lift branch)**.** *Suppose an initial approximation is fixed, every depth obstruction vanishes along its descendants, and every map $\overline{L}_m$ has zero kernel. Then at most one compatible lift branch extends the initial approximation.*

Proof. By theorem 23.2, the correction classes at each depth form a coset of $\operatorname{Ker}\overline{L}_m$. A zero kernel leaves at most one correction class, so induction gives at most one branch. □

Theorem 23.8 (Complete filtered lifting)**.** *Assume there is a compatible sequence*

$$x_{m+1} \equiv x_m \pmod{F^mX}$$

with

$$\Phi(x_m) \in F^mY$$

for every m. Then x_m converges to an element $x_\infty \in X$ and

$$\Phi(x_\infty) = 0.$$

In particular, if the obstruction vanishes at every depth and compatible corrections are chosen, an exact solution exists.

Proof. For every fixed r, the congruences imply

$$x_n - x_m \in F^r X$$

whenever $n \geq m \geq r$. Thus (x_m) is Cauchy for the filtration topology. Completeness gives a limit x_∞.

Fix r. For every $m \geq r$, one has $\Phi(x_m) \in F^m Y \subseteq F^r Y$. The submodule $F^r Y$ is open and therefore closed in the filtration topology. Continuity of Φ gives $\Phi(x_m) \to \Phi(x_\infty)$, so $\Phi(x_\infty) \in F^r Y$. Since this holds for every r and Y is separated,

$$\Phi(x_\infty) \in \bigcap_r F^r Y = 0.$$

□

Theorem 23.9 (Naturality of filtered obstructions)**.** *Suppose a morphism of filtered lifting problems sends x_m to x'_m, intertwines L_m with L'_m, and sends $\Phi(x_m)$ to $\Phi'(x'_m)$ on the associated graded pieces. Then the induced cokernel map sends*

$$\mathrm{Ob}_m(x_m) \quad \textit{to} \quad \mathrm{Ob}'_m(x'_m).$$

Proof. Apply theorem 21.3 to the induced commuting square on the depth m graded pieces. □

Example 23.10 (Three lifting outcomes)**.** The scalar equation $x^2 - 2 = 0$ at $x = 3$ modulo 7 has invertible derivative. It has one correction modulo 49 and gives $x = 10$ modulo 49.

For $x^2 - 3 = 0$ at $x = 0$ modulo 3, the derivative is zero and the defect class is nonzero. No correction exists.

For $x^2 = 0$ at $x = 0$ modulo 3, the derivative and defect both vanish. Every correction digit lies in the kernel, so the lift branches.

Proposition 23.11 (Maurer Cartan lifting obstruction)**.** *Let $\mathcal{L}$ be a complete filtered differential graded Lie algebra over a ring in which two is invertible. Assume*

$$dF^i\mathcal{L} \subseteq F^i\mathcal{L}, \qquad [F^i\mathcal{L}, F^j\mathcal{L}] \subseteq F^{i+j}\mathcal{L}.$$

For a degree one element η, put

$$\Omega(\eta) = d\eta + \frac{1}{2}[\eta, \eta].$$

Suppose $\Omega(\eta) \in F^m\mathcal{L}^2$ with $m \geq 1$. For every $h \in F^m\mathcal{L}^1$,

$$\Omega(\eta + h) \equiv \Omega(\eta) + d_\eta h \pmod{F^{m+1}\mathcal{L}^2}, \qquad d_\eta = d + [\eta, \cdot].$$

The obstruction to improving η by one depth is therefore the class of $\Omega(\eta)$ in the cokernel of

$$\mathrm{gr}^m(d_\eta) : \mathrm{gr}^m \mathcal{L}^1 \longrightarrow \mathrm{gr}^m \mathcal{L}^2.$$

Proof. Direct expansion gives

$$\Omega(\eta + h) = \Omega(\eta) + dh + [\eta, h] + \frac{1}{2}[h, h].$$

The filtration rule places $[h, h]$ in $F^{2m}\mathcal{L}^2$, which is contained in $F^{m+1}\mathcal{L}^2$ because $m \geq 1$. The congruence follows. The last statement is theorem 23.2 applied to the induced graded correction map. □

24 Filtered obstruction complexes

A direct packet decomposition splits a complex into independent pieces, so cohomology splits packet by packet. A filtration retains relations between depths. Its associated graded cohomology can contain classes that disappear on later pages of a spectral sequence.

There are therefore two ways for fine obstruction data to give zero total obstruction. Each fine class may already vanish, or later differentials may remove nonzero classes through compatibility between depths. Only the first mechanism occurs in a direct sum of subcomplexes. A filtration can exhibit both.

Standard Fact 24.1 (Spectral sequence of a finite filtered complex)**.** Let $C^\bullet$ be a cochain complex with a finite decreasing filtration by subcomplexes

$$C^\bullet = F^0C^\bullet \supseteq F^1C^\bullet \supseteq \cdots \supseteq F^{N+1}C^\bullet = 0.$$

There is a natural spectral sequence with

$$E_0^{p,q} = \mathrm{gr}^p\, C^{p+q}, \qquad E_1^{p,q} = H^{p+q}(\mathrm{gr}^p\, C^\bullet),$$

which converges to the associated graded object of $H^{p+q}(C^\bullet)$ with its induced filtration. A filtered cochain map induces a morphism of spectral sequences.

This is the standard construction for a filtered complex, in the form given in [18, 12]. When $C^\bullet$ is a correction or deformation complex, the groups on its pages can be read as graded obstruction candidates. For an arbitrary filtered complex, they are cohomology classes of the associated graded problem.

> **Lifting and survival answer different questions**
>
> A depth obstruction tests whether a chosen approximation admits one further correction. A spectral sequence follows cohomology classes of a filtered complex through later differentials. Turning successive corrections into an exact solution also uses compatible choices, completeness, continuity, and separatedness.

Proposition 24.2 (Fine obstruction survival)**.** *A class on the first page contributes to the total obstruction only if it is represented by a permanent cycle, so it is neither killed nor identified as a boundary on any later page. If the spectral sequence degenerates at the first page, then $H^n(C^\bullet) = 0$ exactly when $E_1^{p,n-p} = 0$ for every p.*

Proof. Convergence identifies the last page with the associated graded object of the total cohomology. A first page class contributes to that object exactly when it determines a nonzero class on the last page. If the sequence degenerates at the first page, the associated graded pieces of $H^n(C^\bullet)$ are the groups $E_1^{p,n-p}$. A finite filtered module is zero exactly when all its associated graded pieces are zero. □

Corollary 24.3 (Direct packet complexes)**.** *Suppose*

$$C^\bullet = \bigoplus_{\alpha \in I} C_\alpha^\bullet$$

is a finite direct sum of subcomplexes. Then

$$H^n(C^\bullet) \cong \bigoplus_{\alpha \in I} H^n(C^\bullet_\alpha).$$

A total cohomology class vanishes if and only if every packet class vanishes.

Proof. The differential acts independently on the summands. Kernels and images therefore split as finite direct sums, and so does their quotient. □

Example 24.4 (A fine class killed at the next page)**.** Let $C^0 = \Bbbk x$ and $C^1 = \Bbbk y$, with differential $d(x) = y$. Give C^0 the filtration

$$F^0C^0 = \Bbbk x, \qquad F^1C^0 = 0,$$

and give C^1 the filtration

$$F^0C^1 = F^1C^1 = \Bbbk y, \qquad F^2C^1 = 0.$$

The associated graded differential is zero in the relevant positions, so both x and y give nonzero classes on the next page. The next differential sends the class of x to the class of y. The total complex is acyclic. Thus nonzero fine obstruction classes can cancel through a later refinement differential.

Remark 24.5. The spectral sequence does not guarantee that a total obstruction vanishes. It records which fine classes survive and which are removed by compatibility across depths.

25 Nonlinear elimination

Let U, V, Y_P, Y_Q be Banach spaces. On an open neighborhood of $(u_0, v_0) \in U \times V$, split the variables and equations as

$$x = (u, v), \qquad F(u, v) = \begin{pmatrix} F_P(u, v) \\ F_Q(u, v) \end{pmatrix}.$$

Assume F takes values in $Y_P \times Y_Q$, is twice continuously Fréchet differentiable near (u_0, v_0), satisfies $F_Q(u_0, v_0) = 0$, and

$$A = D_v F_Q(u_0, v_0)$$

is a Banach space isomorphism from V to Y_Q. The implicit function theorem gives a local map $v = \varphi(u)$, with $\varphi(u_0) = v_0$, satisfying

$$F_Q(u, \varphi(u)) = 0.$$

Define the reduced equation

$$R(u) = F_P(u, \varphi(u)).$$

Theorem 25.1 (Exact local elimination)**.** *After the neighborhoods are chosen small enough, the equation*

$$F(u, v) = 0$$

is equivalent to

$$v = \varphi(u) \qquad \text{and} \qquad R(u) = 0.$$

Thus eliminating the hidden variable loses no local solutions.

Proof. The hidden equation $F_Q(u, v) = 0$ has the unique local solution $v = \varphi(u)$ by the implicit function theorem. After this identity is imposed, the remaining component of F is exactly $R(u)$. □

Theorem 25.2 (Nonlinear Schur derivative)**.** *At* (u_0, v_0)*,*

$$DR = D_u F_P - D_v F_P A^{-1} D_u F_Q.$$

Proof. Differentiate $F_Q(u, \varphi(u)) = 0$:

$$D_u F_Q + AD\varphi = 0.$$

Thus

$$D\varphi = -A^{-1} D_u F_Q.$$

Substitute this into

$$DR = D_u F_P + D_v F_P D\varphi.$$

□

The hidden variable still contributes

The reduced derivative includes the response of the hidden variable to a retained variation. That response travels through the inverse hidden derivative and then back into the retained equation. The Schur term records this contribution. Its use depends on the hidden derivative being invertible, as required in the local elimination setup.

The second derivative records how this response enters the first nonlinear feedback from the hidden variables.

Theorem 25.3 (Second Schur derivative)**.** *For retained directions* h, k*, set*

$$\hat{h} = (h, D\varphi h), \qquad \hat{k} = (k, D\varphi k).$$

Then

$$D^2\varphi[h, k] = -A^{-1} D^2 F_Q[\hat{h}, \hat{k}]$$

and

$$D^2 R[h, k] = D^2 F_P[\hat{h}, \hat{k}] - D_v F_P A^{-1} D^2 F_Q[\hat{h}, \hat{k}].$$

All derivatives on the right are evaluated at (u_0, v_0)*.*

Proof. Differentiate the identity $F_Q(u, \varphi(u)) = 0$ twice. The chain rule gives

$$D^2 F_Q[\hat{h}, \hat{k}] + AD^2\varphi[h, k] = 0.$$

This proves the first formula. Differentiate $R(u) = F_P(u, \varphi(u))$ twice and substitute the first formula.

□

Corollary 25.4 (Packet jets of the reduced equation)**.** *If the retained and hidden spaces carry packet systems, then the first and second packet jets of* R *are obtained by projecting the formulas in theorems 25.2 and 25.3. Hidden packet feedback is carried by the factors involving* A^{-1}*.*

Proof. Apply the packet projectors to the input and output slots of the two derivative formulas. □

Example 25.5 (Two retained variables and one hidden variable)**.** Suppose the derivative of F at a solution has block form

$$DF = \begin{pmatrix} A & B \\ C & D \end{pmatrix},$$

where D is invertible. Eliminating the hidden variable replaces the retained derivative A by

$$A - BD^{-1}C.$$

The feedback term is exact. Keeping only A would ignore the path from retained variables into the hidden variable and back.

Remark 25.6. Elimination and obstruction reduction address different situations. The Schur formula assumes that the hidden derivative is invertible. Lyapunov Schmidt reduction allows a nontrivial kernel and cokernel, then leaves a reduced obstruction equation on the kernel directions.

26 Finite transport and holonomy

Let $\Gamma = (V, E)$ be a finite connected graph and fix a vector space X. At every vertex take one labeled copy of X. Every oriented edge $u \to v$ carries an invertible transport

$$T_{vu} \in \mathrm{GL}(X), \qquad T_{uv} = T_{vu}^{-1}.$$

Equivalently, one may begin with separate but isomorphic vertex fibers and choose one frame in each fiber. The matrices below are then the transports in those frames. For a path $\gamma = (v_0, \dots, v_r)$, set

$$T_\gamma = T_{v_r v_{r-1}} \cdots T_{v_1 v_0}.$$

For a loop, this product is its holonomy. In the based loop group, $\gamma_2\gamma_1$ means first traverse γ_1 and then γ_2, so $T_{\gamma_2\gamma_1} = T_{\gamma_2} T_{\gamma_1}$. Standard graph terminology may be found in [5].

An invertible map on each edge supplies the local transport data. A global frame must make these maps agree around every cycle. A spanning tree has no cycles, so a gauge can make every tree transport the identity. Each remaining edge closes a loop and retains that loop's holonomy.

Definition 26.1 (Gauge change)**.** A gauge is a family $G_v \in \mathrm{GL}(X)$. It changes the edge transports to

$$T'_{vu} = G_v T_{vu} G_u^{-1}.$$

Proposition 26.2 (Gauge covariance)**.** *For a loop γ based at v_0,*

$$\mathrm{Hol}'(\gamma) = G_{v_0} \mathrm{Hol}(\gamma) G_{v_0}^{-1}.$$

Thus the conjugacy class of each loop holonomy is gauge invariant.

Proof. Every intermediate gauge factor cancels with its inverse. The initial and terminal factors remain at the base vertex. □

Theorem 26.3 (Spanning tree normalization)**.** *Choose a root and a spanning tree $\mathcal{T}$ of Γ. There is a gauge in which every tree edge transport is the identity. In that gauge, the transport on each edge outside the tree is the holonomy of its fundamental cycle, with the orientation used to define that cycle.*

Proof. Set the gauge at the root to the identity. Propagate gauges along the unique tree paths so that every tree edge becomes the identity. An edge outside the tree closes one fundamental cycle. Since every tree factor is now the identity, the remaining edge transport is the cycle product. □

Theorem 26.4 (Cycle gluing criterion)**.** *There are vertex coordinates in which every edge transport is the identity if and only if every fundamental cycle holonomy is the identity.*

Proof. If all edge transports are identity maps, every loop product is the identity. Conversely, use theorem 26.3. Identity fundamental holonomy forces every edge outside the tree to become the identity as well. □

Corollary 26.5 (Number of independent cycle tests)**.** *For a connected graph with $|V|$ vertices and $|E|$ unoriented edges, a spanning tree leaves*

$$|E| - |V| + 1$$

fundamental cycle holonomies.

Proof. A spanning tree has $|V| - 1$ edges. Every remaining edge creates one fundamental cycle. □

26.1 Intertwining two transport systems

Let A be a transport system on copies of a fixed vector space V_0, and let B be a transport system on copies of a fixed vector space W_0. Fix a root r and an isomorphism

$$U_r : V_0 \longrightarrow W_0.$$

Theorem 26.6 (Fundamental holonomy intertwining criterion)**.** *The following conditions are equivalent.*

1. *There is a family of isomorphisms $U_v : V_0 \to W_0$ with the prescribed root value U_r and satisfying*

$$U_v A_{vu} = B_{vu} U_u$$

   for every oriented edge $u \to v$.

2. *For every fundamental loop γ based at r,*

$$U_r \operatorname{Hol}_A(\gamma) = \operatorname{Hol}_B(\gamma) U_r.$$

3. *The same identity holds for every loop based at r.*

When these conditions hold, the family (U_v) is uniquely determined by U_r.

Proof. If the edge intertwiners exist, multiply their equations around a loop. Every intermediate vertex map cancels. This proves that the first condition implies the third, and the third implies the second.

Conversely, choose a spanning tree. Starting from U_r, propagate U_v along the unique tree path by the rule

$$U_v = B_{vu} U_u A_{vu}^{-1}$$

on every tree edge. This is the only possible extension satisfying the tree edge equations. An edge outside the tree closes one fundamental loop. Its intertwining equation holds exactly when U_r intertwines the two corresponding fundamental holonomies. The second condition therefore supplies every remaining edge equation. Uniqueness follows from propagation along the tree. □

Corollary 26.7 (Scalar holonomy bridge)**.** *Suppose every vertex space is one dimensional. Two transport systems are gauge equivalent if and only if their holonomies agree on a fundamental cycle basis.*

Proof. Choose a nonzero root isomorphism. Scalar loop maps commute with it, so theorem 26.6 reduces to equality of the cycle scalars. □

Theorem 26.8 (Gauge classification of transports)**.** *Let $G \subseteq \mathrm{GL}(X)$ be the chosen structure group and fix a root r. Restrict both edge transports and gauge values to G. Gauge equivalence classes of G valued transport systems are naturally identified with*

$$\mathrm{Hom}(\pi_1(\Gamma, r), G)/G,$$

where G acts by conjugation. After a spanning tree and orientations of the remaining edges are chosen, this set is identified with simultaneous conjugacy classes of

$$b_1 = |E| - |V| + 1$$

tuples in G.

Proof. A transport system assigns a group element to every oriented edge, with inversion under reversal. The product along a based loop gives a homomorphism from the free group $\pi_1(\Gamma, r)$ to G. A gauge change conjugates this homomorphism by the gauge value at the root. Conversely, normalize a transport system on a spanning tree. The remaining edge values prescribe the images of a free basis of $\pi_1(\Gamma, r)$, and every homomorphism arises in this way. A further gauge preserving the tree normalization is constant at all vertices and acts by simultaneous conjugation. The number of non tree edges is b_1. □

Corollary 26.9 (Abelian cohomology class)**.** *When the structure group A is abelian, gauge classes of transports are naturally*

$$H^1(\Gamma, A) \cong \mathrm{Hom}(H_1(\Gamma, \mathbb{Z}), A).$$

After a basis of $H_1(\Gamma, \mathbb{Z})$ is chosen, this group is identified with A^{b_1}.

Proof. An edge assignment is an A valued one cochain. A gauge change multiplies it by a coboundary. A graph has no two cells, so every one cochain is a cocycle. The usual computation of the first cohomology of a connected graph gives A^{b_1}. □

Corollary 26.10 (Projective transport)**.** *The same normalization and gluing statements hold for transports valued in a projective linear group. A projective transport is globally trivial exactly when all fundamental holonomies are identity elements in that projective group.*

Proof. Apply theorems 26.3 and 26.4 with the projective linear group as the structure group. □

Example 26.11 (A square loop)**.** A square graph has four vertices and four edges, so it has one fundamental cycle. After a tree gauge, three edges are identities and the fourth is the full loop holonomy. The system glues to one global frame exactly when that remaining matrix is the identity.

27 Smooth projector transport

Finite graph transport has a continuous counterpart. Let H be a Hilbert space and let

$$\mathcal{P}(t) = (P_\alpha(t))_{\alpha \in I}$$

be a complete orthogonal packet system with finitely many labels. Assume that it is continuously differentiable in operator norm for $t \in [0, T]$, where $T \geq 0$. Here every $P_\alpha(t)$ is a bounded self adjoint projection, so $P_\alpha(t)^* = P_\alpha(t) = P_\alpha(t)^2$.

Zero projections are permitted in this section and contribute zero to all generator sums. When $T = 0$, every partition product is the empty product I. Completeness and orthogonality give

$$P_\alpha(t)P_\beta(t) = 0 \quad \text{for } \alpha \neq \beta, \qquad \sum_\alpha P_\alpha(t) = I.$$

Define the canonical generator

$$K(t) = \sum_\alpha \dot{P}_\alpha(t) P_\alpha(t).$$

Lemma 27.1 (Complete packet commutator identity)**.** *For every packet label β,*

$$\dot{P}_\beta(t) = [K(t), P_\beta(t)].$$

The generator is also skew adjoint:

$$K(t)^* = -K(t).$$

Proof. Differentiating $P_\alpha^2 = P_\alpha$ gives

$$P_\alpha \dot{P}_\alpha P_\alpha = 0.$$

Differentiating $P_\beta P_\alpha = 0$ for $\alpha \neq \beta$ gives

$$P_\beta \dot{P}_\alpha = -\dot{P}_\beta P_\alpha.$$

Hence

$$K P_\beta = \dot{P}_\beta P_\beta$$

and

$$P_\beta K = -\dot{P}_\beta (I - P_\beta).$$

Their difference is $\dot{P}_\beta$. Also,

$$K + K^* = \sum_\alpha (\dot{P}_\alpha P_\alpha + P_\alpha \dot{P}_\alpha) = \sum_\alpha \dot{P}_\alpha = 0.$$

□

Theorem 27.2 (Canonical packet transport)**.** *Let $U(t)$ solve*

$$\dot{U}(t) = K(t)U(t), \qquad U(0) = I.$$

Then $U(t)$ is unitary and transports every packet exactly:

$$P_\alpha(t)U(t) = U(t)P_\alpha(0)$$

for all α.

Proof. The continuous bounded generator has an evolution inverse. Indeed, solve $\dot{V} = -VK$ with $V(0) = I$. Differentiation gives $(VU)' = 0$, hence $VU = I$. The product $W = UV$ satisfies $\dot{W} = KW - WK$ and $W(0) = I$; uniqueness of this bounded linear equation gives $UV = I$. Thus U is invertible.

Skew adjointness of $K(t)$ gives

$$\frac{d}{dt}(U(t)^*U(t)) = 0,$$

so $U(t)$ is unitary. By lemma 27.1,

$$\frac{d}{dt}(U(t)^{-1}P_\alpha(t)U(t)) = U(t)^{-1}(\dot{P}_\alpha - [K, P_\alpha])U(t) = 0.$$

The expression equals $P_\alpha(0)$ at time zero. □

Corollary 27.3 (One moving subspace)**.** *For one orthogonal projection $P(t)$ and its complement, the canonical generator is*

$$K(t) = [\dot{P}(t), P(t)].$$

The transport in theorem 27.2 maps $\operatorname{im} P(0)$ *unitarily onto* $\operatorname{im} P(t)$.

Proof. Apply the complete packet formula to $P(t)$ and $I - P(t)$, then simplify the sum. □

Definition 27.4 (Direct rotation of nearby packet systems)**.** Let $(P_\alpha)_{\alpha\in I}$ and $(Q_\alpha)_{\alpha\in I}$ be complete orthogonal packet systems on the same Hilbert space. Put

$$S(\mathcal{Q}, \mathcal{P}) = \sum_{\alpha\in I} Q_\alpha P_\alpha.$$

When $S(\mathcal{Q}, \mathcal{P})$ is invertible, define

$$\mathcal{U}(\mathcal{Q}, \mathcal{P}) = S(\mathcal{Q}, \mathcal{P})\left(S(\mathcal{Q}, \mathcal{P})^*S(\mathcal{Q}, \mathcal{P})\right)^{-1/2}.$$

Proposition 27.5 (Exact direct packet rotation)**.** *If*

$$\max(\{0\} \cup \{\|P_\alpha - Q_\alpha\| : \alpha \in I\}) < 1,$$

then $S(\mathcal{Q}, \mathcal{P})$ is invertible. The operator $\mathcal{U}(\mathcal{Q}, \mathcal{P})$ is unitary and satisfies

$$Q_\alpha\mathcal{U}(\mathcal{Q}, \mathcal{P}) = \mathcal{U}(\mathcal{Q}, \mathcal{P})P_\alpha$$

for every α.

Proof. The identities

$$Q_\alpha S(\mathcal{Q},\mathcal{P}) = S(\mathcal{Q},\mathcal{P})P_\alpha = Q_\alpha P_\alpha$$

follow from pairwise orthogonality. Also

$$S^*S = \sum_\alpha P_\alpha Q_\alpha P_\alpha, \qquad SS^* = \sum_\alpha Q_\alpha P_\alpha Q_\alpha.$$

Put $\delta = \max(\{0\} \cup \{\|P_\alpha - Q_\alpha\| : \alpha \in I\})$. On $P_\alpha H$,

$$P_\alpha Q_\alpha P_\alpha = P_\alpha - P_\alpha(P_\alpha - Q_\alpha)P_\alpha,$$

so this block is bounded below by $1-\delta > 0$. The same argument on $Q_\alpha H$ applies to $Q_\alpha P_\alpha Q_\alpha$. Thus both S^*S and SS^* are invertible. It follows that S is injective and surjective. The polar factor $\mathcal{U} = S(S^*S)^{-1/2}$ is unitary. Since S^*S commutes with every P_α, the intertwining identity for S passes to $\mathcal{U}$. □

Lemma 27.6 (One step direct rotation estimate)**.** *Assume $\mathcal{P}(t)$ is twice continuously differentiable in operator norm on $[0,T]$. There are constants $h_0 > 0$ and $C > 0$ such that, whenever $t, t+h \in [0,T]$ and $|h| \le h_0$,*

$$\|\mathcal{U}(\mathcal{P}(t+h),\mathcal{P}(t)) - I - hK(t)\| \le Ch^2.$$

The constants can be chosen uniformly in t.

Proof. Taylor's theorem in the Banach space of bounded operators gives

$$P_\alpha(t+h) = P_\alpha(t) + h\dot{P}_\alpha(t) + E_\alpha(t,h), \qquad \|E_\alpha(t,h)\| \le C_1h^2,$$

uniformly in t and α. Hence

$$S(\mathcal{P}(t+h),\mathcal{P}(t)) = I + hK(t) + E(t,h), \qquad \|E(t,h)\| \le C_2h^2.$$

Since $K(t)^* = -K(t)$,

$$S^*S = I + E_2(t,h), \qquad \|E_2(t,h)\| \le C_3h^2.$$

For small h, the spectrum of S^*S lies in a fixed compact subinterval of $(0,\infty)$. The functional calculus for $z^{-1/2}$ gives

$$(S^*S)^{-1/2} = I + E_3(t,h), \qquad \|E_3(t,h)\| \le C_4h^2.$$

Multiplying the last two expansions proves the estimate. □

Theorem 27.7 (Direct rotations converge to packet transport)**.** *Assume the complete packet family $\mathcal{P}(t)$ is twice continuously differentiable in operator norm on $[0,T]$. For a partition*

$$0 = t_0 < t_1 < \cdots < t_N = T$$

with sufficiently small mesh, define

$$V_\pi = \mathcal{U}(\mathcal{P}(t_N),\mathcal{P}(t_{N-1}))\cdots\mathcal{U}(\mathcal{P}(t_1),\mathcal{P}(t_0)).$$

Then

$$V_\pi \longrightarrow U(T)$$

in operator norm, where U is the canonical transport from theorem 27.2. On a fixed compact interval the error is bounded by a constant times the mesh.

Proof. Let $U(t,s)$ be the exact propagator generated by K. Since K is continuously differentiable, the integral equation for $U(t+h,t)$ gives a uniform estimate

$$\|U(t+h,t) - I - hK(t)\| \le C_1 h^2.$$

By lemma 27.6, the direct rotation V_j from t_j to t_{j+1} therefore satisfies

$$\|V_j - U(t_{j+1},t_j)\| \le C_2 h_j^2, \qquad h_j = t_{j+1} - t_j.$$

Both V_j and the exact propagator are unitary. A telescoping product comparison gives

$$\|V_\pi - U(T,0)\| \le \sum_j \|V_j - U(t_{j+1},t_j)\| \le C_2 \sum_j h_j^2 \le C_2 T|\pi|.$$

This proves norm convergence and the stated first order global bound. □

Exact packet matching at each sample

Each direct rotation maps a sampled packet exactly to the corresponding packet at the next sample. The product therefore preserves packet labels throughout the partition. Convergence compares this discrete product with the canonical continuous transport. The uniform estimate uses twice continuous differentiability in operator norm, as stated in the theorem.

Theorem 27.8 (Convergence of sampled transports)**.** *Assume K is Lipschitz on $[0,T]$. For a partition*

$$0 = t_0 < t_1 < \cdots < t_N = T,$$

define

$$U_\pi = \exp\left((t_N - t_{N-1})K(t_{N-1})\right) \cdots \exp\left((t_1 - t_0)K(t_0)\right).$$

As the mesh $|\pi|$ tends to zero,

$$U_\pi \longrightarrow U(T)$$

in operator norm. There is a constant C, depending only on T, the supremum norm of K, and its Lipschitz constant, such that

$$\|U_\pi - U(T)\| \le C|\pi|.$$

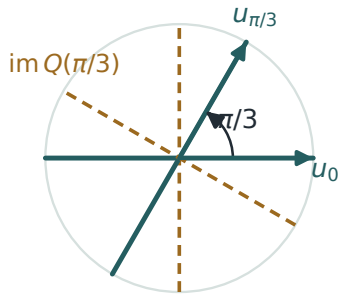
Proof. On each interval, compare the exact propagator with the frozen exponential. Variation of constants and the Lipschitz bound give a local error bounded by a constant times the square of the interval length. Every factor is unitary, so multiplication does not amplify norms. Summing the local errors gives a bound by a constant times

$$\sum_j (t_{j+1} - t_j)^2 \le T|\pi|.$$

□

A rotating line and its complementary packet

$u_\theta = (\cos\theta, \sin\theta)$, $P(\theta) = u_\theta u_\theta^\top$, $Q(\theta) = I - P(\theta)$.



Solid: P packet Dashed: Q packet

Canonical continuous transport

$$K = [P', P] = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$U'(\theta) = KU(\theta), \quad U(0) = I$$

$$U(\theta) = R(\theta)$$

$$S = \cos\delta\, R(\delta), \qquad S^\top S = \cos^2\delta\, I$$

For $|\delta| < \pi/2$: polar$(S) = R(\delta)$ exactly.

Steps N	Each increment	Ordered product
2	$\pi/6$	$R(\pi/3)$
4	$\pi/12$	$R(\pi/3)$
8	$\pi/24$	$R(\pi/3)$

Every mesh product is exact in this finite model.
The general theorem concerns convergence for broader projector paths.

Figure 6. A rotating rank one packet and its orthogonal complement. With $u(\theta) = (\cos\theta, \sin\theta)$ and $P(\theta) = u(\theta)u(\theta)^\top$, the canonical generator is $[P', P] = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$. For a positive step $\delta < \pi/2$, the polar unitary is the usual rotation $R(\delta)$. On each displayed mesh, the direct rotation product equals $R(\pi/3)$ exactly. This special planar example illustrates the construction. The general theorem supplies convergence for families whose step products need not already equal the continuous transport.

Corollary 27.9 (Continuous holonomy from graph products)**.** *Assume the complete packet family is twice continuously differentiable in operator norm on $[0, T]$. If $P_\alpha(0) = P_\alpha(T)$ for every α, then $U(T)$ preserves every initial packet. Its restriction to each packet is the holonomy of the closed packet path. The products in theorems 27.7 and 27.8 converge to these holonomies. The direct rotation product has the added property that every factor maps each sampled packet exactly to its successor.*

Proof. The transport identity in theorem 27.2 gives packet preservation at the endpoint. The two convergence statements follow from theorems 27.7 and 27.8. □

Example 27.10 (A rotating line)**.** In $\mathbb{R}^2$, let

$$v(t) = (\cos t, \sin t)$$

and let $P(t)$ be the orthogonal projection onto the line spanned by $v(t)$. The canonical transport is the ordinary rotation matrix

$$U(t) = \begin{pmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{pmatrix}.$$

It carries the initial line and its orthogonal complement to the two moving packets. Products of short rotations converge to the same transport.

Standard Fact 27.11 (Curvature of a projected connection)**.** Let P be a smooth orthogonal projector valued map on a parameter manifold. The connection

$$\nabla = P\, d$$

on the range bundle has curvature

$$\mathcal{F} = P(dP \wedge dP)P.$$

Proof. For a section s satisfying $Ps = s$,

$$\nabla^2 s = P\, d(P\, ds) = P(dP) \wedge ds.$$

Since $ds = (dP)s + P ds$ and $P(dP)P = 0$, the surviving term is

$$P(dP \wedge dP)Ps.$$

□

28 Realization data and limits of the calculus

The smooth transport theorem starts with a declared projector family. Applying it to a physical model requires a construction of that family and, where relevant, a separate continuum limit. Triviality of holonomy is a further geometric question. The curvature formula in standard fact 27.11 is classical. Relating it to global holonomy requires geometric information as in [7, 8, 10].

An application begins by specifying the following objects:

1. an ambient nonlinear equation or continuation problem
2. the allowed correction space
3. a linear correction map or a correction complex

4. the defect whose class is to be tested
5. a packet decomposition or filtration compatible with the correction data
6. transition maps between refinement levels
7. transport maps when local solutions must be glued.

Once these objects are fixed, the cokernel and cone constructions are functorial by theorems 21.3 and 21.11. Packet splitting and descent follow from theorem 21.4, proposition 21.6, and corollary 21.12. Relative loss under coarsening is measured by theorem 21.7.

Filtered lifting follows from theorems 23.2, 23.6 and 23.8. Gluing and comparison of transport systems follow from theorems 26.4 and 26.6. Smooth packet transport follows from theorems 27.2 and 27.7.

Exact packet coordinates give a representation of the data. Solving the associated problem still requires the correction and lifting hypotheses. The correction map belongs to the problem data, and its cokernel is canonical relative to that choice. The cokernel may be infinite dimensional. In the Hilbert setting, the orthogonal obstruction projector requires closed range.

Coarse vanishing can erase a nonzero fine obstruction, and higher nonlinear terms can obstruct a problem whose linear class vanishes. A spectral sequence records which classes survive the later differentials. An application of finite graph holonomy or smooth projector transport to a continuum field theory requires the additional realization data just described.

Whenever a nonlinear continuation problem supplies a correction map, its residual can be recorded in an exact quotient or cohomology object. Under the stated hypotheses, these objects admit packet decomposition, filtered refinement, natural transport, and holonomy tests. They identify the obstruction that a correction or gluing step must address.

A Visual atlas

The plates give exact finite models for the algebraic constructions in this volume. Their stable labels run from V1 to V24. Each plate names the related statements in the paper. These models make the hypotheses and map directions visible. They do not extend the finite Lean contract to the analytic results of Part II.

> **Three routes through the atlas**
>
> For typed transfer, read V3 to V8. For coordinates, connections, and gauge, read V9 to V18, followed by V23. For candidate witnesses and trees, read V19 to V22, followed by V24. V1 and V2 introduce the objects and the questions they answer.

Products act in traversal order. The first edge traversed is the rightmost operator. Rows represent targets and columns represent sources.

V1 / PAPER III / A WORKED VISUAL COMPANION

Three objects, three different questions

Traditional paper: Section 1 (p. 1), Section 4 (p. 7), Section 5 (p. 9)

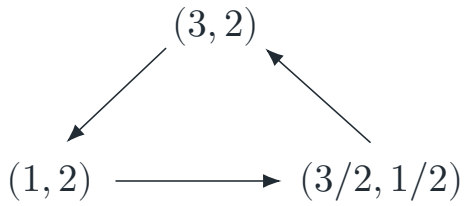
An edge operator moves data between named packets. An exact coordinate change describes the same data in another complete chart. A connection supplies an additional choice of transport. The three can use similar arrows while making different mathematical assertions.

EDGE ACTION

0	0	0
2	0	0
0	0	0

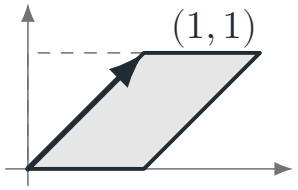
$e_0 \longmapsto 2e_1$

EXACT COORDINATES



one unchanged ambient vector

CHOSEN CONNECTION



$H = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right) \neq I$

Three actual models used below: a single supported matrix entry, one ambient vector in three coordinate systems, and a shear obtained as chosen loop transport. The coordinates in the middle are not three different ambient vectors.

One paper, three tests

For an edge word, ask whether the target of each step matches the source of the next. For exact coordinates, ask whether analysis and synthesis are inverses. For a connection, ask whether its transports through two steps agree with its declared direct transports. None of these tests can replace another.

This guide follows explicit integer and rational matrices through these tests. Each example has an inspectable input, calculation, and conclusion. Vector drawings display those calculations rather than replace them. A graph with two paths can have zero net transfer. A loop with trace two and determinant one can still differ from the identity.

The edition used here

The worked models originate in the finite algebraic v1.0.1 release. The final traditional paper also develops rooted tree expansions, nonlinear defects, obstruction objects, filtered lifting, and analytic transport under their stated hypotheses. Those ordinary mathematical developments have a wider scope than the finite Lean contract. The present models explain the finite mechanisms and provide a route into the broader paper.

> **Read this as:** Follow the object, not the arrow shape. A typed transfer can vanish, a complete coordinate loop must telescope, and an independently chosen connection can have genuine algebraic holonomy.

V2 / ORIENTATION / KEEP THE TYPES VISIBLE

An atlas of the objects

Traditional paper: Definition 2.1 (p. 2), Definition 5.5 (p. 11), Definition 7.4 (p. 16)

Rows are targets and columns are sources throughout. If E_{ji} is a matrix unit, then $E_{ji}e_i = e_j$. The notation e_i means a basis vector. A named graph edge is written a, b, c, d or e, f when its role is explicit.

Object	What it records	First worked page
$\mathrm{src}, \mathrm{tgt} : E \to V$	The ordered endpoints of each distinct edge, including parallel edges.	V – 3 to V – 4
$L_e = c(e)E_{\mathrm{tgt}(e),\mathrm{src}(e)}$	One operator supported on a packet.	V – 3
$L_w = L_{e_n} \cdots L_{e_1}$	Execute the written word from left to right.	V – 4 to V – 5
$T = \sum_e L_e$	Add all edge operators before taking powers.	V – 4 to V – 7
F_a	A synthesis frame from chart a into the ambient space.	V – 9
$\mathcal{A}_a, \mathcal{S}_a$	Inverse analysis and synthesis maps, which may be maps of sets.	V – 10
$J_{b \leftarrow a}$	Canonical coordinate transport, forced by the exact charts.	V – 9 to V – 10
$\Gamma_{b \leftarrow a}$	Additional connection data, with reversibility stated explicitly.	V – 11 to V – 12
$\Delta_{c,b,a}$	The difference between transport through two steps and direct transport, a map from C_a to C_c.	V – 12
$H_{c,b,a}$	Loop transport based at a, an automorphism of C_a.	V – 13
G_a	An invertible gauge change at each chart.	V – 14 to V – 15
$\Pi_\mu \,\mathrm{Ob}\, \Pi_\nu$	Block from input to output of $\mathrm{Ob} = H - I$.	V – 16 to V – 18
$r(x, p)$	A residual at packet p for the named candidate x.	V – 19
$t, \mathrm{inputs}(t)$	A finite operation tree and its syntactically used input labels.	V – 20

Reading routes

For transfer, read V3 to V8. For the finite geometry models, read V9 to V18. For candidate certificates and nonlinear scope, read V19 to V22. V23 puts the matrix mechanisms together. V24 gives the formal boundary and replay guide.

> **Read this as:** A $C_a \to C_c$ defect and a $C_a \to C_a$ holonomy are not the same typed map. Returning the defect to the base fiber requires a specified inverse transport.

V3 / MODEL A / ONE EDGE IS ONE MATRIX ENTRY

Read the product in execution order

Traditional paper: Lemma 2.2 (p. 2), Theorem 2.3 (p. 2), Theorem 2.4 (p. 3)

Work over $\mathbb{Z}$ with packets $0, 1, 2$. Let $e : 0 \to 1$ have coefficient 2, and let $f : 1 \to 2$ have coefficient 3. The elementary operators are

$$L_e = 2E_{10}, \qquad L_f = 3E_{21}.$$

The location of the nonzero entry records the type of the transfer, not an arbitrary matrix convention.

L_e

	0	1	2
0	0	0	0
1	2	0	0
2	0	0	0

L_f

	0	1	2
0	0	0	0
1	0	0	0
2	0	3	0

$L_f L_e$

	0	1	2
0	0	0	0
1	0	0	0
2	6	0	0

row = target / column = source



e first, f second; $\qquad L_e L_f = 0$

Each grid is an actual 3×3 matrix. The rightmost matrix factor acts first. In $L_f L_e$, packet 1 is the shared intermediate index. In $L_e L_f$ the interface is broken.

On a vector $(x_0, x_1, x_2)^{\mathsf{T}}$, the first step produces $(0, 2x_0, 0)^{\mathsf{T}}$, and the second produces $(0, 0, 6x_0)^{\mathsf{T}}$. Thus

$$L_f L_e = 6E_{20}, \qquad L_e L_f = 0.$$

For general edges, a summand in the entrywise product can survive only when the same intermediate packet is both $\mathrm{tgt}(e)$ and $\mathrm{src}(f)$:

$$(L_f L_e)_{yx} = \sum_z (L_f)_{yz} (L_e)_{zx}.$$

This is why a broken interface annihilates the entire product, even if both edge coefficients are nonzero.

What the zero says

It says that this ordered composition does not pass data through the declared packet types. It does not say that either edge is individually zero. For a longer word, finding one broken adjacent interface is sufficient to prove that the whole word vanishes.

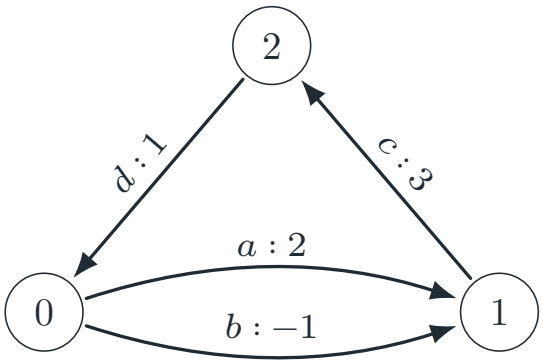
> **Read this as:** Write the traversed edges first. Then reverse their order when writing the operator product: e followed by f is $L_f L_e$.

V4 / MODEL A / PARALLEL EDGES REMAIN DISTINCT

All sixteen words, only five walks

Traditional paper: Theorem 3.8 (p. 5), Corollary 3.9 (p. 6)

Add four edges on three packets: $a : 0 \to 1$ with coefficient 2, $b : 0 \to 1$ with coefficient -1, $c : 1 \to 2$ with coefficient 3, and $d : 2 \to 0$ with coefficient 1. The parallel edges a and b are separate labels even though their source and target agree.



$$(T^2)_{20} = 6 - 3 = 3$$

first executed edge = column

second executed edge = row	a	b	c	d
a	×	×	×	2
b	×	×	×	-1
c	6	-3	×	×
d	×	×	3	×

×: broken packet interface

The board tests all ordered pairs. Columns are the first executed edge and rows are the second. A shaded cell gives the ordered coefficient of a valid walk. A cross means the product is zero by type mismatch.

The surviving words are ac, bc, cd, da, db. Their coefficients are $6, -3, 3, 2, -1$, respectively. Their endpoints determine where these numbers enter the matrix:

$$T = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix}, \qquad T^2 = \begin{pmatrix} 0 & 3 & 0 \\ 0 & 0 & 1 \\ 3 & 0 & 0 \end{pmatrix}.$$

At row 2, column 0, the two parallel choices give $6 - 3 = 3$. Deduplicating the edge labels before summing would alter the operator. On the other hand, adding the parallel coefficients in the already assembled matrix is legitimate: both act in the same supported matrix position.

The same calculation extends from pairs to all finite words by distributivity. The included check enumerates every word through length six and independently accumulates the endpoint coefficients. The empty word is treated separately as the identity.

> **Read this as:** The matrix power is a signed, ordered walk sum. It is not simply the number of paths, and a graph edge is not identified solely by its endpoints.

V5 / MODEL A / A MATRIX RING AS THE COEFFICIENT SEMIRING

Coefficients need not commute

Traditional paper: Lemma 3.7 (p. 5), Section 9 (p. 20)

The edge theorem allows a noncommutative coefficient semiring. Use $R = M_2(\mathbb{Q})$ and let the two coefficients themselves be matrices:

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Attach A to $0 \to 1$ and B to $1 \to 2$. The single surviving endpoint coefficient is BA, not AB.

A

1	1
0	1

B

1	0
1	1

BA

1	1
1	2

AB

2	1
1	1

$e : 0 \to 1$ carries A; $\quad f : 1 \to 2$ carries B

$$BA(1,0)^{\mathsf{T}} = (1,1)^{\mathsf{T}}, \qquad AB(1,0)^{\mathsf{T}} = (2,1)^{\mathsf{T}}$$

All entries are exact integers. The two products differ on the diagonal and act differently on the same input vector. This example is within the scalar model with semiring coefficients by taking the scalar ring to be $M_2(\mathbb{Q})$.

The distinction is visible before any abstraction:

$$BA \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad AB \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

For a written walk $(e_1, \dots, e_n)$, its coefficient must therefore be

$$c^{\leftarrow}(w) = c(e_n)c(e_{n-1}) \cdots c(e_1).$$

A commutative example can hide an orientation mistake because reversing the coefficients changes nothing. This one cannot.

One ring is not a dependent family of fibers

The 3×3 matrix over $M_2(\mathbb{Q})$ can be displayed as an ordinary 6×6 matrix with 2×2 blocks. This is a concrete representation of the example over one ring. It does not establish an arbitrary family of differently typed edge fibers, which is outside the finite formal release's scalar interface for edge systems.

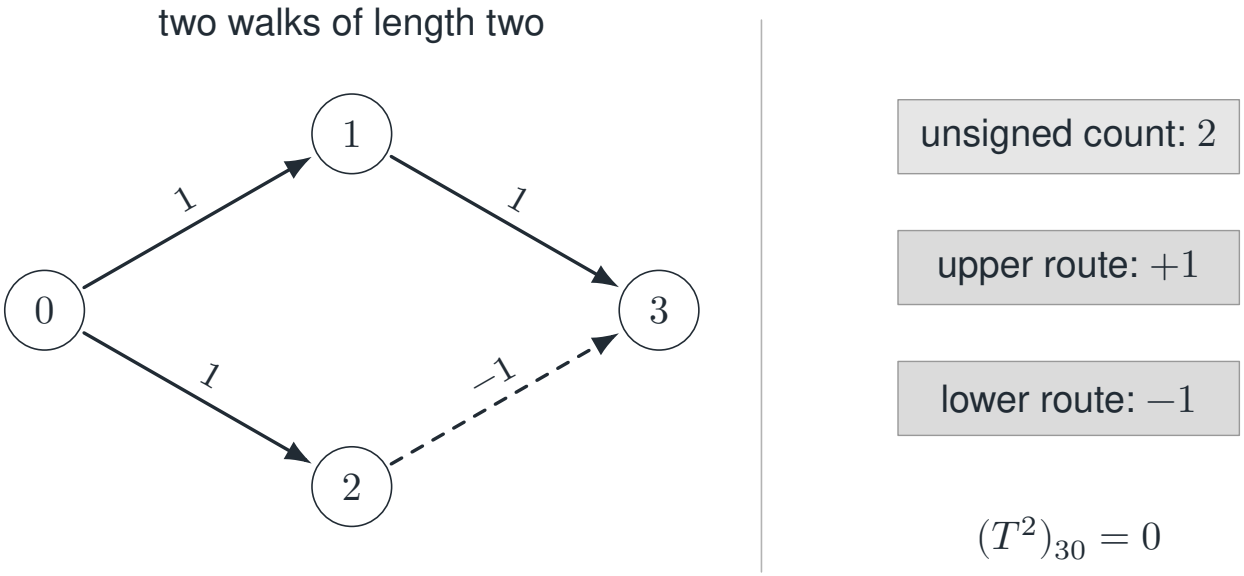
> **Read this as:** The order rule carries mathematical information. It survives exact tests over a noncommutative ring and must not be inferred from pictures drawn over scalar real numbers alone.

V6 / MODEL A / CANCELLATION IS NOT A BROKEN INTERFACE

A reachable packet can receive zero

Traditional paper: Corollary 3.9 (p. 6), Corollary 3.10 (p. 6)

Consider a diamond: two routes of length two from packet 0 to packet 3. Three edge coefficients are 1 and the lower final coefficient is -1. Both routes are correctly typed. Neither is annihilated by an interface mismatch.



The dashed edge carries the coefficient -1. It is an actual edge, not an uncertain or missing one. Both paths exist. The unsigned count is two, while their exact operator contributions sum to zero.

The endpoint identity gives

$$(T^2)_{30} = (1)(1) + (-1)(1) = 0.$$

Thus the paper's reachability implication has a definite direction:

$$\text{no compatible walk} \implies (T^n)_{ji} = 0.$$

Its converse is false. A zero entry may come from absent paths, zero coefficients, zero divisors in the coefficient ring, or cancellation among nonzero terms. The diamond exhibits the last possibility over $\mathbb{Z}$, where no zero divisor is needed.

Three different support records

The declared graph records permitted routes. The support of an individual word records the endpoints of that word when its coefficient is nonzero. The support of the sum records what remains after all contributions have been added. Passing from either of the first two records to the last requires the coefficients, not just adjacency.

A deliberately false test in the companion script asserts that reachability forces nonzero transfer. This diamond makes the test fail. That is a check against overreading the theorem, not a defect in the theorem.

> **Read this as:** Do not interpret a visible route as an occupied output. Graph support is a useful upper bound. Signed operator support can be strictly smaller.

V7 / MODEL A / AN ORBIT THAT STAYS IN TWO DIMENSIONS

Reachability is only an upper bound

Traditional paper: Corollary 3.11 (p. 6)

Keep the diamond and add an isolated packet 4. Start with the single seed $e_0 \in \mathbb{Q}^5$. Direct multiplication gives

$$Te_0 = e_1 + e_2, \qquad T^2 e_0 = 0.$$

Every later orbit vector is zero. The orbit hull generated by T is therefore exactly the span of the first two displayed columns.

$[e_0 \;\; Te_0 \;\; T^2 e_0]$

0	1	0	0
1	0	1	0
2	0	1	0
3	0	0	0
4	0	0	0

a basis of the orbit hull

1	0
0	1
0	1
0	0
0	0

$\text{rank} = 2$

$$\text{Reach}(0) = \{0, 1, 2, 3\}$$

$$\text{Hull}_T(e_0) = \{(a, b, b, 0, 0)\}$$

e_3 : reachable but absent

e_4 : unreachable and absent

The first matrix contains the actual orbit columns. The second supplies a basis of their span. The orbit hull lies inside the reachable subspace on four coordinates, but it has dimension two, not four.

A vector belongs to this hull precisely when it has the form

$$(a, b, b, 0, 0)^{\mathsf{T}}, \qquad a, b \in \mathbb{Q}.$$

Packet 4 is excluded because it is unreachable. Packet 3 is also absent, but for a different reason: the two contributions cancel. Even the reachable packets 1 and 2 remain linked. Powers of the single total operator do not separate them.

The admissible operator grammar matters

Allowing each individual edge operator is a different generation problem. For instance, applying the edge $0 \to 1$ and then $1 \to 3$ individually produces e_3. It does not follow that e_3 belongs to the orbit of the summed operator T. The paper's theorem on orbit support concerns the explicitly stated operator action.

> **Read this as:** Unreachable means impossible under the declared action. Reachable means only that graph geometry has not ruled it out. Linear dependence and cancellation still have to be checked.

V8 / MODEL B / RECTANGULAR PRODUCTS WITH TWO OR THREE STEPS

Paths through changing packet lists

Traditional paper: Proposition 8.1 (p. 19)

Here the packet lists have sizes $2, 3, 2, 1$ in succession. Let the actual maps be

$$\mathbb{Q}^2 \xrightarrow{A} \mathbb{Q}^3 \xrightarrow{B} \mathbb{Q}^2 \xrightarrow{C} \mathbb{Q}.$$

Unlike a power of one matrix, each step has its own entries and its own source and target list.

$A : \mathbb{Q}^2 \to \mathbb{Q}^3$

1	0
2	1
0	3

$B : \mathbb{Q}^3 \to \mathbb{Q}^2$

1	−1	2
0	1	1

$C : \mathbb{Q}^2 \to \mathbb{Q}$

2	−1

CBA

−4	6

	$(j,k)=(0,0)$	$(0,1)$	$(0,2)$	$(1,0)$	$(1,1)$	$(1,2)$	
$i=0$	2	−4	0	0	−2	0	$=-4$
$i=1$	0	−2	12	0	−1	−3	$=6$

The lower board expands one final row into all six choices of intermediate indices for each input packet. The entries are $C_{0j}B_{jk}A_{ki}$, with $k \in \{0,1,2\}$ and $j \in \{0,1\}$. Zero terms remain visible.

The intermediate and total products are

$$BA = \begin{pmatrix} -1 & 5 \\ 2 & 4 \end{pmatrix}, \qquad CBA = \begin{pmatrix} -4 & 6 \end{pmatrix}.$$

For input packet 0, the six contributions sum to $2 - 4 + 0 + 0 - 2 + 0 = -4$. For input packet 1, they sum to $0 - 2 + 12 + 0 - 1 - 3 = 6$. These are literal matrix entries, not probabilities or path counts.

The formal scope is deliberately precise

The finite formal release's forms with two and three steps allow changing packet types. Its heterogeneous product theorem for arbitrary lengths allows changing *factors* over one common finite packet index. That formal statement does not assert a fully dependent chain of changing index types at arbitrary lengths. The finite rectangular illustration here uses exactly three factors.

For a single factor, the list of intermediate indices is empty and contributes that matrix entry once. For zero factors, the result is the identity on the common space. These conventions must not be replaced by an empty sum equal to zero.

> **Read this as:** Heterogeneous can mean different matrices or different spaces. State which one is allowed before using a path formula.

V9 / MODEL C / CANONICAL TRANSPORT HAS NO HOLONOMY

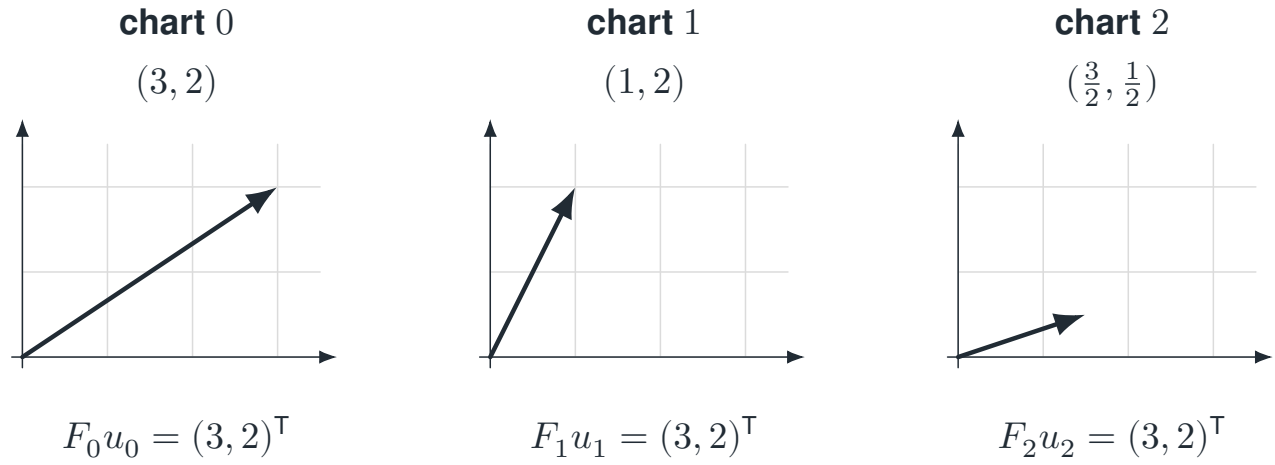
One vector, three exact frames

Traditional paper: Theorem 4.1 (p. 7), Theorem 4.2 (p. 7)

Let F_a synthesize chart coordinates into one ambient $\mathbb{Q}^2$. Choose

$$F_0 = I, \qquad F_1 = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \qquad F_2 = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}.$$

The same ambient vector $x = (3,2)^{\mathsf{T}}$ has chart coordinates $u_a = F_a^{-1}x$.



Each panel is a coordinate plane, not a different physical location. The numerical arrows differ because the frames differ. Multiplying each coordinate vector by its corresponding F_a recovers the same x.

Transport from chart a to chart b is forced:

$$J_{b \leftarrow a} = F_b^{-1} F_a.$$

The route $0 \to 1 \to 2$ agrees with the direct route because the middle frame cancels:

$$(F_2^{-1}F_1)(F_1^{-1}F_0) = F_2^{-1}F_0.$$

Returning to chart 0 cancels every remaining frame. The final coordinate vector is $(3,2)^{\mathsf{T}}$, not a sheared or rotated variant.

No metric is required

The frames need not be orthogonal. The second one is a shear, and the third has determinant two. Exact invertibility, rather than preservation of angles or lengths, gives the cocycle and loop identities. The same telescoping argument works for every finite chart path.

> **Read this as:** A canonical coordinate loop is a sequence of changes in description of one object. Nonidentity holonomy requires additional transport data, not merely nonorthogonal charts.

V10 / MODEL C / NATURALITY DOES NOT REQUIRE DERIVATIVES

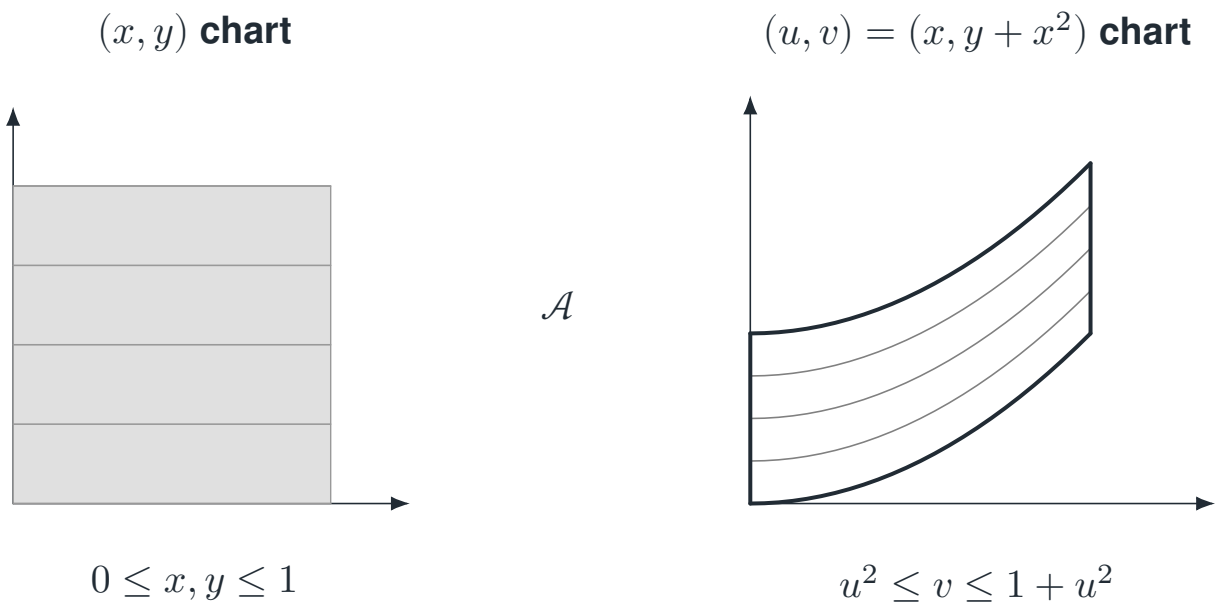
Nonlinear charts still cancel exactly

Traditional paper: Proposition 10.1 (p. 20)

Exact coordinate equivalences can be nonlinear. On $\mathbb{Q}^2$, use

$$\mathcal{A}(x, y) = (x, y + x^2), \qquad \mathcal{S}(u, v) = (u, v - u^2).$$

Both composites are the identity by direct substitution. The real drawing below is a geometric view of these same polynomial formulas.



Horizontal segments become parabolas, but no information is discarded. The drawing samples the displayed polynomial boundaries for rendering. Inverse identities and route equality are checked symbolically, not inferred from the curves.

Now represent the nonlinear operation $F(x, y) = (x + y, xy)$ in the new chart. Set $w = u + v - u^2$. Its correct coordinate formula is

$$\widehat{F}_b(u, v) = \mathcal{A}F\mathcal{S}(u, v) = (w,\ u(v - u^2) + w^2).$$

For $(x, y) = (1, 2)$, applying F first gives $(3, 2)$ and then $(3, 11)$ in the new chart. Changing the input first gives $(u, v) = (1, 3)$, and $\widehat{F}_b(1, 3) = (3, 11)$ as well.

The equality holds as a polynomial identity for all inputs. It is the naturality theorem for maps of sets: only the inverse laws for the coordinate maps are needed. No Taylor expansion, derivative, or higher chain rule is involved.

> **Read this as:** Nonlinearity alone does not create a coordinate defect. Reusing the old formula after changing coordinates would be an extra modeling choice, not exact conjugation.

V11 / MODEL D / THE MINIMAL AND REVERSIBLE INTERFACES

Reversibility is a real hypothesis

Traditional paper: Definition 5.1 (p. 9), Definition 5.5 (p. 11)

Choose the forward scalar transports $\Gamma_{1\leftarrow 0} = 2$, $\Gamma_{2\leftarrow 1} = 3$, and $\Gamma_{2\leftarrow 0} = 5$. Their multiplicative comparison between transport through two steps and direct transport is $D^{\times} = 6/5$. A loop also needs a declared return map $\Gamma_{0\leftarrow 2}$.

connection data	$\Gamma_{0\leftarrow 2}$	H	$D^{\times}$	$\Gamma_{0\leftarrow 2}\Gamma_{2\leftarrow 0}$
self normalization only	$\frac{1}{6}$	1	$6/5$	$\frac{5}{6}$
reversible	$\frac{1}{5}$	$\frac{6}{5}$	$6/5$	1

$$\Gamma_{1\leftarrow 0} = 2, \quad \Gamma_{2\leftarrow 1} = 3, \quad \Gamma_{2\leftarrow 0} = 5$$

The first row has H = identity at this triangle, but fails its cocycle.

All transports are invertible rational scalars and transport from each chart to itself is one. The first row is permitted by the minimal interface. The second uses the inverse of the direct forward edge and therefore satisfies that part of reversibility.

In the first row, choose the return scalar $1/6$. Then

$$H = (1/6) \cdot 3 \cdot 2 = 1, \qquad \Gamma_{2\leftarrow 1}\Gamma_{1\leftarrow 0} - \Gamma_{2\leftarrow 0} = 1.$$

This triangle loop is the identity even though its cocycle equation fails. The return is not the inverse of the direct transport: $(1/6)5 \neq 1$. Thus one cannot transfer the reversible equivalence between defect and holonomy to the minimal interface.

In the reversible row, the return is $1/5$. Now the loop is $6/5$, and the additive defect of one is converted into the loop obstruction $1/5$ by that inverse return map.

Two safeguards

Global flatness in the minimal interface implies trivial loops by using its cocycle on the whole route, including the return. Separately, in the reversible interface, triviality of each triangle loop is equivalent to that triangle's cocycle. Neither assertion says that one isolated loop certifies all triangles.

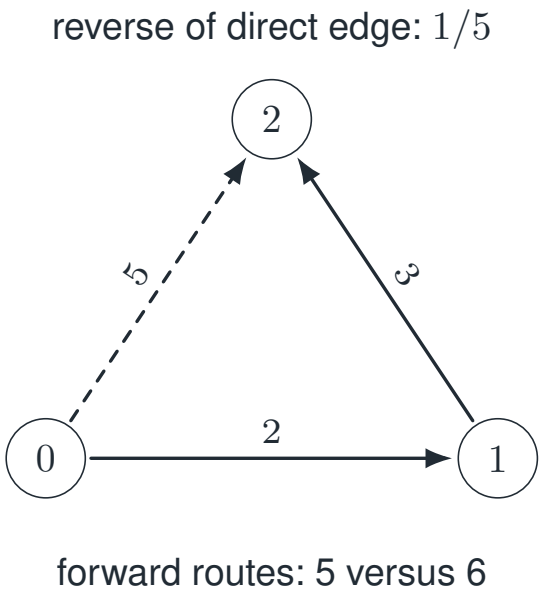
> **Read this as:** Self normalization means $\Gamma_{a\leftarrow a} = I$. Reversible additionally means $\Gamma_{a\leftarrow b} = \Gamma_{b\leftarrow a}^{-1}$. These are different assumptions.

V12 / MODEL D / AN EXACT REVERSIBLE TRIANGLE

Measure the mismatch at its base

Traditional paper: Theorem 5.10 (p. 13), Section 9 (p. 20)

Retain the reversible scalar model with forward values $2, 3, 5$ and all reverse values given by inverses. Two routes from chart 0 to chart 2 multiply the input by 6 and 5, respectively. The additive discrepancy equals the original input.



loop input 1 returns $6/5$

loops	returned value
0	1
1	$\frac{6}{5}$
2	$\frac{36}{25}$
3	$\frac{216}{125}$
4	$\frac{1296}{625}$

The dashed direct edge is a specified comparison map, not a missing edge. The table shows the exact effect of traversing the same loop repeatedly. These are rational values of this example, not an analytic convergence claim.

The defect lives at the target of the forward routes:

$$\Delta_{2,1,0} = 3 \cdot 2 - 5 = 1.$$

The holonomy lives back at chart 0:

$$H_{2,1,0} = 5^{-1} \cdot 3 \cdot 2 = \frac{6}{5}, \qquad H - I = \frac{1}{5} = \Gamma_{0\leftarrow 2}\Delta.$$

For input 5, the direct output and the output after two steps are 25 and 30. Returning by $1/5$ gives 5 and 6. The target discrepancy of 5 has become a base discrepancy of 1. The types and locations of these differences matter even in an example in one dimension.

Because the return map is invertible, it cannot turn a nonzero defect into zero. This is the mechanism behind $\Delta = 0 \iff H = I$ in the reversible interface.

> **Read this as:** Holonomy compares paths after both have been returned to the same base fiber. It is the transported mismatch, not an untyped difference between outputs in different charts.

V13 / MODEL E / A SHEAR WITH NONTRIVIAL HOLONOMY

The identity can hide in the spectrum

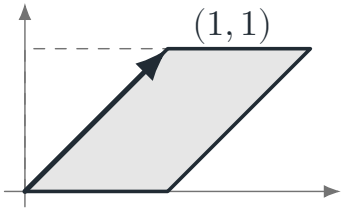
Traditional paper: Definition 5.9 (p. 12), Definition 7.4 (p. 16)

Put $\Gamma_{1\leftarrow 0} = H$, $\Gamma_{2\leftarrow 1} = I$, and $\Gamma_{2\leftarrow 0} = I$, with reverse maps given by inverses, where

$$H = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

The loop $0 \to 1 \to 2 \to 0$ then has exactly this shear as its holonomy.

The unit square under H



H

1	1
0	1

$H - I$

0	1
0	0

$$\operatorname{tr} H = 2, \quad \det H = 1, \quad (H - I)e_1 = e_0$$

The dashed square is the original set. The solid parallelogram is its exact image. The second coordinate basis vector moves to $(1, 1)^{\mathsf{T}}$. Its nonzero displacement is the first basis vector.

The usual scalar invariants can be misleading:

$$\operatorname{tr} H = 2, \qquad \det H = 1, \qquad \chi_H(t) = (t-1)^2.$$

These are also the invariants of the 2×2 identity matrix. Nevertheless,

$$\mathrm{Ob} = H - I = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \neq 0, \qquad \mathrm{Ob}\, e_1 = e_0.$$

A complete obstruction test must therefore retain the operator or a certificate strong enough to detect its nonzero action. Trace, determinant, and eigenvalue lists alone do not distinguish this loop from the identity.

What the picture certifies, and what it does not

The plotted corners are the images of the four corners of the unit square under the displayed matrix. The equation $\mathrm{Ob}\, e_1 = e_0$ is the exact witness. No smooth connection, curvature tensor, or continuum limit is used in this model. Its holonomy is the finite algebraic loop product. The traditional paper treats smooth transport separately under additional hypotheses.

> **Read this as:** Flatness is the equality $H = I$, not agreement of a short list of spectral summaries. A single explicit moved vector can disprove that equality.

V14 / MODEL E / CONJUGATION AND ITS LIMITS

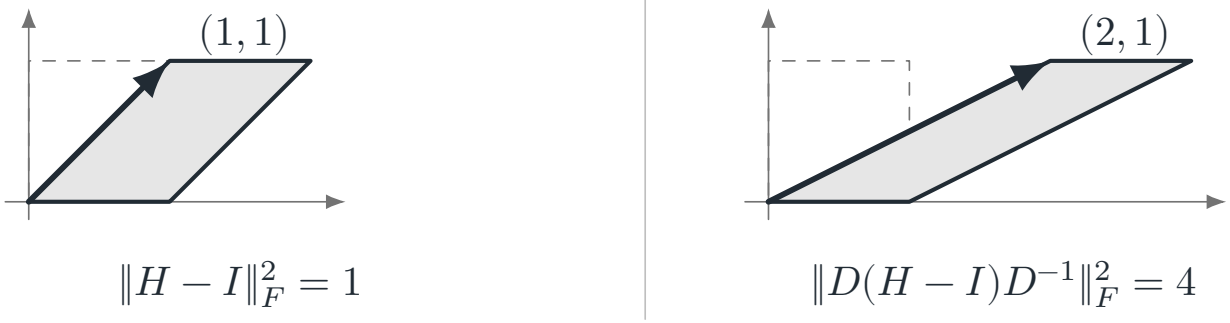
Gauge changes the picture, not flatness

Traditional paper: Theorem 6.2 (p. 14)

At the base chart, change coordinates by $D = \operatorname{diag}(2, 1)$. Transform every connection edge by the prescribed endpoint gauges. The loop transforms by conjugation, giving

$$H^D = DHD^{-1} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}.$$

The shear looks stronger in these fixed numerical axes, but the verdict on flatness is unchanged by gauge.



Both figures are exact images of a unit square under their displayed matrices. The Frobenius norms are computed in the indicated fixed coordinate bases. A nonorthogonal gauge need not preserve such a norm.

The rank of the obstruction is one on both sides. Trace, determinant, and characteristic polynomial are preserved under this conjugation. Most importantly,

$$D(H - I)D^{-1} = 0 \quad \Longleftrightarrow \quad H - I = 0.$$

An invertible coordinate change cannot turn a nonidentity loop into the identity. The fact that a particular residual norm changes does not contradict the invariance of exact vanishing.

The base chart determines the outer gauge

In a triangle product, adjacent intermediate gauges cancel:

$$(G_a \Gamma_{a \leftarrow c} G_c^{-1})(G_c \Gamma_{c \leftarrow b} G_b^{-1})(G_b \Gamma_{b \leftarrow a} G_a^{-1}) = G_a H G_a^{-1}.$$

Only the base gauge survives around the loop. For the additive defect, the surviving outer gauges instead belong to its different source and target fibers.

> **Read this as:** Exact nonvanishing is gauge invariant. The numerical size of a matrix entry or a chosen norm generally is not.

V15 / MODEL D / A FLAT CONNECTION WITH NONIDENTITY EDGES

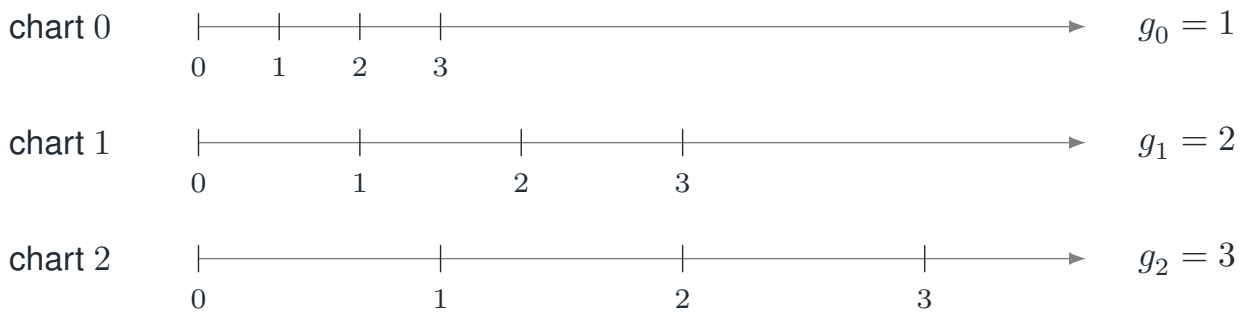
Flat is not the same as calibrated

Traditional paper: Theorem 6.3 (p. 14)

Make all three canonical charts literally identical, so $J_{b\leftarrow a} = 1$ over $\mathbb{Q}$. Choose scalar gauges $g_0 = 1$, $g_1 = 2$, and $g_2 = 3$, and define

$$\Gamma_{b\leftarrow a} = g_b g_a^{-1}.$$

The edge $0 \to 1$ is multiplication by two, although its canonical counterpart is the identity.



$$\Gamma_{1\leftarrow 0} = 2, \quad \Gamma_{2\leftarrow 1} = 3/2, \quad \Gamma_{2\leftarrow 0} = 3$$

The rulers display the chosen gauge scales: the marks are $g_a n$ for integer n. They visualize the supplied scalar factors, not an intrinsic metric. Relative transport is the ratio of the target and source scales.

The route through two steps $0 \to 1 \to 2$ has multiplier $(3/2)2 = 3$, equal to the direct route $0 \to 2$. More generally,

$$(g_c g_b^{-1})(g_b g_a^{-1}) = g_c g_a^{-1}$$

for every triple. Thus this connection is flat and reversible, but it is not calibrated to the original J, whose values are identities.

Recovering the gauge from one root

Choose a base chart o. The theorem constructs

$$G_a = \Gamma_{a\leftarrow o} J_{o\leftarrow a}, \qquad \Gamma_{b\leftarrow a} = G_b J_{b\leftarrow a} G_a^{-1}.$$

With $o = 0$ and $J = 1$, the recovered G_a are exactly $1, 2, 3$. The statement concerns the complete pairwise interface and its cocycle. It is not a topological claim about an arbitrary graph with missing direct comparisons.

> **Read this as:** Flatness makes the chosen transports independent of path. Calibration identifies them with one particular canonical transport before any gauge change.

V16 / MODEL F / PACKET BLOCKS FROM INPUT TO OUTPUT

Resolve the entire obstruction

Traditional paper: Theorem 7.5 (p. 17), Corollary 7.6 (p. 18)

Use three coordinate packets with projectors P_0, P_1, P_2, and set

$$H = I + 2E_{01} + 3E_{12}, \qquad \mathrm{Ob} = H - I.$$

Here H is invertible, so it can be the loop product of a reversible connection on three charts. Its entries outside the diagonal encode two effects that mix packets.

H

1	2	0
0	1	3
0	0	1

$\mathrm{Ob} = H - I$

0	2	0
0	0	3
0	0	0

	$\nu = 0$	$\nu = 1$	$\nu = 2$
$\mu = 0$	0	$2E_{01}$	0
$\mu = 1$	0	0	$3E_{12}$
$\mu = 2$	0	0	0

$$\mathrm{Ob}(4,5,6)^{\mathsf{T}} = (10,18,0)^{\mathsf{T}}$$

sum all nine blocks

The board on the right contains all nine actual embedded blocks $P_\mu \,\mathrm{Ob}\, P_\nu$. Row labels are output packets and column labels are input packets. Only the addresses $(0,1)$ and $(1,2)$ carry nonzero maps.

Completeness on both sides supplies the reconstruction:

$$\mathrm{Ob} = (P_0 + P_1 + P_2)\,\mathrm{Ob}(P_0 + P_1 + P_2) = \sum_{\mu,\nu} P_\mu \,\mathrm{Ob}\, P_\nu.$$

For $x = (4,5,6)^{\mathsf{T}}$, the input in packet 1 contributes $10e_0$ and the input in packet 2 contributes $18e_1$. Their sum is the full displacement $\mathrm{Ob}\, x$.

A block becomes a witness when it acts

The nonzero block $2E_{01}$ has the explicit source vector e_1, supported entirely in packet 1, and $P_0 \,\mathrm{Ob}\, e_1 = 2e_0 \neq 0$. Similarly, $P_1 \,\mathrm{Ob}\, e_2 = 3e_1 \neq 0$. The general theorem supplies such a witness classically when a nonzero block is known. The example displays it concretely.

> **Read this as:** An obstruction packet is not just a colored matrix cell. It consists of a specific label for a pair of packets and an exact nonvanishing statement about the corresponding map.

V17 / MODEL E / COMPLETENESS, TWO SIDES, AND OBLIQUE PACKETS

Why diagonal checks are insufficient

Traditional paper: Theorem 7.5 (p. 17)

Return to $\mathrm{Ob} = E_{01}$. With the two standard coordinate projectors, both diagonal blocks vanish:

$$P_0 \,\mathrm{Ob}\, P_0 = P_1 \,\mathrm{Ob}\, P_1 = 0.$$

The obstruction nevertheless survives in the block outside the diagonal $P_0 \,\mathrm{Ob}\, P_1 = E_{01}$.

Ob

0	1
0	0

nonzero

$\sum_i P_i \,\mathrm{Ob}\, P_i$

0	0
0	0

diagonal test misses it

Q_0

1	0
1	0

Q_1

0	0
−1	1

$$Q_0 + Q_1 = I, \quad Q_i^2 = Q_i$$

$$\sum_{i,j} Q_i \,\mathrm{Ob}\, Q_j = \mathrm{Ob}; \qquad Q_0^{\mathsf{T}} \neq Q_0$$

The test of diagonal blocks alone on the left misses the whole obstruction. On the right, the displayed oblique projectors still form a complete exact resolution. They need not be orthogonal projections for the Euclidean inner product.

There is a second failure mode. Supplying only P_0 gives a family of idempotents but not a complete one. The single tested block is zero, while the omitted packet carries the missing input. The implication “all tested blocks vanish, therefore the operator vanishes” requires the completeness hypothesis.

Exact reconstruction does not require right angles

For the displayed Q_0, Q_1,

$$Q_0 + Q_1 = I, \qquad Q_i^2 = Q_i, \qquad Q_0 Q_1 = Q_1 Q_0 = 0,$$

but $Q_0^{\mathsf{T}} \neq Q_0$. Expanding both copies of I still proves

$$\sum_{i,j} Q_i \,\mathrm{Ob}\, Q_j = \mathrm{Ob}\,.$$

The block reconstruction identity itself uses the condition that the projectors sum to the identity. Idempotence is additionally used when constructing a witness that remains supported in one packet after projection. The release does not require pairwise annihilation for its general reconstruction statement.

> **Read this as:** Test every block from input to output of a complete resolution. Testing only the diagonal, or only the packets supplied so far, answers a weaker question.

V18 / MODEL E / FIXED VERSUS TRANSPORTED PROJECTORS

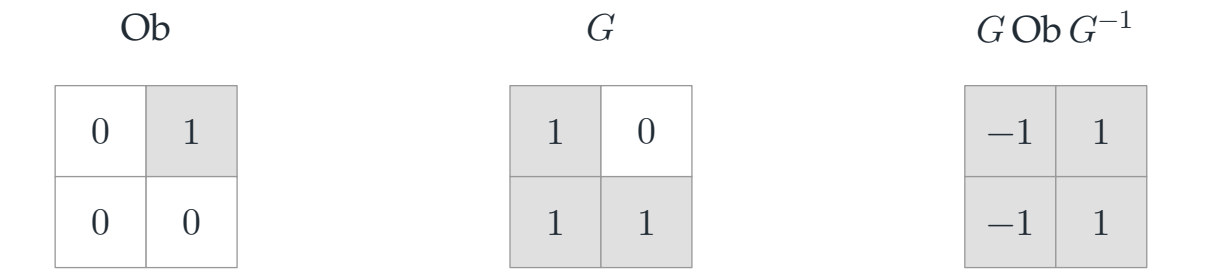
A gauge can move every packet label

Traditional paper: Proposition 7.7 (p. 18)

The diagonal gauge on V14 commuted with each standard projector. Now use the gauge that mixes packets

$$G = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad \mathrm{Ob} = E_{01}.$$

The total obstruction transforms by conjugation, but its pattern in the *fixed* standard packets changes.



fixed axes: different packet pattern / moving axes: conjugate labeled blocks

The first obstruction has one nonzero entry. Its conjugate has four. Both have rank one and are nonzero. The change in fixed block pattern is not a change in whether the connection is flat.

Direct multiplication gives

$$G\,\mathrm{Ob}\,G^{-1} = \begin{pmatrix} -1 & 1 \\ -1 & 1 \end{pmatrix}.$$

For example, the original $(0,0)$ block vanishes, while the new $(0,0)$ block relative to the old coordinate projectors does not. This is why the theorem about preservation of each block's zero status requires $GP_i = P_iG$.

Transporting the packet frame as well

Define $Q_i = GP_iG^{-1}$. Then the identities cancel exactly:

$$Q_i(G\,\mathrm{Ob}\,G^{-1})Q_j = G(P_i\,\mathrm{Ob}\,P_j)G^{-1}.$$

Relative to these transported packets, the corresponding labeled blocks again have matching vanishing status. These are the same oblique projectors displayed on V17. One must distinguish moving the operator alone in a fixed resolution from moving the entire resolved description.

> **Read this as:** The total obstruction is intrinsic under gauge. Its location in a fixed packet grid is not, unless the gauge preserves that grid.

V19 / MODEL G / THE QUANTIFIERS ARE THE CONTENT

An obstruction rejects one candidate

Traditional paper: Theorem 7.2 (p. 15)

Let the candidates be rational numbers and the packet labels be $0, 1$. Define

$$r(x, 0) = x^2 - 1, \qquad r(x, 1) = x - 1.$$

A candidate is a solution when both residuals vanish. The five candidate columns below are a display of this explicit residual function, not a search over an unspecified class.

candidate x	-2	-1	0	1	2
$r(x,0) = x^2 - 1$	3	0	-1	0	3
$r(x,1) = x - 1$	-3	-2	-1	0	1
	rejected	rejected	rejected	solution	rejected

$$x = -1 : \quad r(-1, 0) = 0, \quad r(-1, 1) = -2$$

A shaded cell is a nonzero exact residual. At $x = -1$, the first equation passes but the second supplies an obstruction. At $x = 1$, both pass. The rejected candidate does not obstruct the successful one.

The statement proved by the generic certificate has the form

$$\neg(\forall p,\ r(x, p) = 0) \quad \Longleftrightarrow \quad \exists p,\ r(x, p) \neq 0.$$

The same candidate x appears on both sides. A certificate consists of the packet label and its nonvanishing proof. The general converse uses classical reasoning and does not itself provide an algorithm to find the label.

What would be required to rule out all solutions?

One would need the stronger assertion $\forall x\, \exists p,\ r(x, p) \neq 0$. The value $x = 1$ disproves that assertion in this example. A witness for $x = -1$ supplies no evidence for it. Likewise, finding an obstruction block for one triangle of one chosen connection does not classify every possible connection.

The term *obstruction* in this generic interface does not automatically mean a cokernel class, a cohomology group, a norm bound, or a minimal certificate. Those require additional constructions and theorems.

> **Read this as:** Keep the candidate attached to the witness. The theorem's strength is its exact logical scope, not a universal existence or nonexistence claim.

V20 / MODEL H / ACTUAL SYNTAX, ARBITRARY SEMANTICS

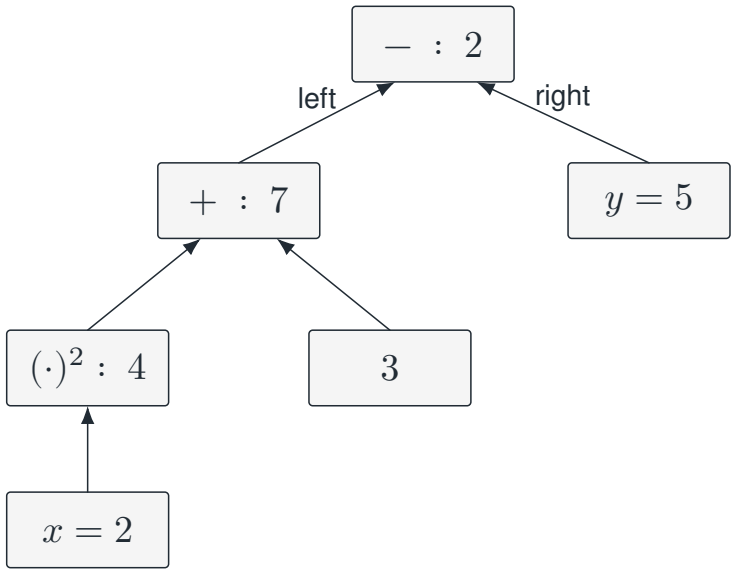
A tree reads its named leaves

Traditional paper: Definition 11.1 (p. 22), Theorem 11.2 (p. 22)

Use the grammar's constant, input, unary, and binary constructors to build

$$t = (x^2 + 3) - y.$$

Here "square," "add," and "subtract" are explicitly supplied meanings of operation symbols. At $x = 2, y = 5$, every node value can be evaluated exactly.



$\text{inputs}(t) = \{x, y\}$

unused input: -100 or 100

same value: 2

$x - x : \{x\}$ syntactically

but always zero

The rooted tree records the order of input slots. Arrows indicate evaluation from children to parent, not chart transport. An environment may contain additional variables, but this tree does not read them.

The input set is $\text{inputs}(t) = \{x, y\}$. If two environments agree at x and y, structural induction proves that the two evaluations agree: input leaves agree, constants agree, and every parent applies the same supplied operation to equal child values.

Syntactic locality is not minimal dependence

The tree $x - x$ has input set $\{x\}$, although its value is always zero under ordinary subtraction. Locality provides a sufficient agreement condition. It does not compute the smallest set of semantically relevant variables. Nor does it prove that two different trees are equivalent.

The semantics may act on arbitrary values. For example, a binary concatenation symbol applied to strings "ab" and "cd" gives "abcd." No linearity or differentiability is required. Conversely, swapping the ordered inputs of the subtraction root above changes the value from 2 to -2.

> **Read this as:** The proved tree theorem controls which input labels are read. It does not supply a nonlinear chain rule, a stability estimate, or a solution of a tree equation.

V21 / MODEL H / EXACT COORDINATES VERSUS A CHOSEN RULE

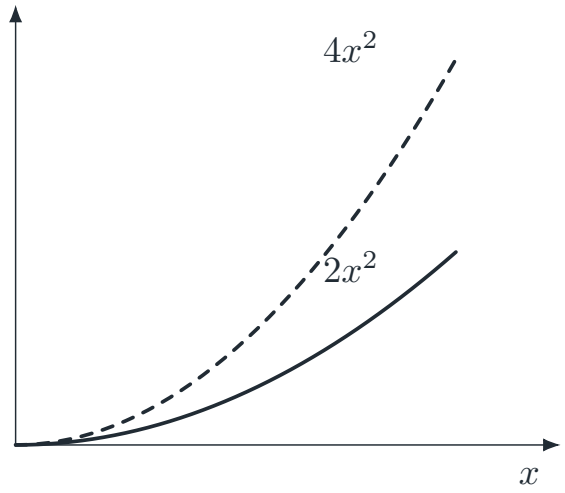
Where a nonlinear defect can enter

Traditional paper: Proposition 10.1 (p. 20), Formalization target 11.7 (p. 24), Theorem 20.2 (p. 42)

Take the unary operation $f(x) = x^2$ over $\mathbb{Q}$. Suppose both input and output transport are chosen as multiplication by two, while the same numerical formula f is reused at the target. The two routes give

$$2f(x) = 2x^2, \qquad f(2x) = 4x^2.$$

Their difference is $-2x^2$, which is generally nonzero.



x	0	1	2
$2f(x)$	0	2	8
$f(2x)$	0	4	16
$2f(x) - f(2x)$	0	-2	-8

correct chart formula: $f_b(v) = v^2/2$

Solid: $2x^2$. Dashed: $4x^2$. The curves are a real visualization of exact polynomial formulas. The table contains exact evaluations over the rationals. The mismatch is caused by the specified combination of transport and formula.

For a canonical coordinate change with input and output chart $v = 2x$, the target formula is not still v^2. It is

$$\hat{f}_b(v) = 2f(v/2) = \frac{v^2}{2}.$$

Consequently $\hat{f}_b(2x) = 2x^2$, and the exact coordinate defect vanishes. This is the same distinction illustrated in the example in two dimensions on V10.

An example is not a general propagation theorem

The polynomial calculation here is exact and checked symbolically. It explains a chosen node defect, while the traditional paper develops composition identities, iterated defects, fixed point stability, and derivative defects under explicit hypotheses. Those broader ordinary proofs must be read on their own terms. This one example and the finite Lean locality theorem do not certify them.

> **Read this as:** A nonzero nonlinear route difference must come from identifiable extra data: a changed transport, an approximation, a restricted model, or a reused formula that was not conjugated correctly.

V22 / FINITE EXAMPLES / WHY MULTILINEARITY MATTERS

Mixed terms are not optional

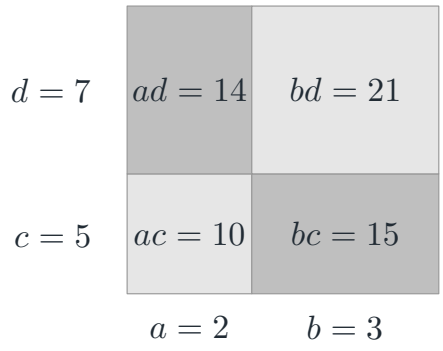
Traditional paper: Formalization target 11.6 (p. 23), Theorem 18.3 (p. 38)

A bilinear node distributes over two independently resolved inputs:

$$(a+b)(c+d) = ac + ad + bc + bd.$$

The four terms correspond to all ordered choices of one input packet from each slot. Keeping only matching packet labels loses the mixed terms.

$$(2+3)(5+7) = 60$$



Diagonal pieces are not the whole product

$$ac + bd = 10 + 21 = 31$$

$$ad + bc = 14 + 15 = 29$$

$$(2+3)^2 = 25 \neq 2^2 + 3^2 = 13$$

missing square term: $2 \cdot 2 \cdot 3 = 12$

The rectangle has width $2 + 3$ and height $5 + 7$, with differently scaled horizontal and vertical display units. Each cell is labeled by its actual product. The lighter diagonal pair contributes only 31 of the total 60.

For a unary square, inserting $a + b$ does not give $a^2 + b^2$. The extra $2ab$ arises because the operation is nonlinear in its single argument. It can be analyzed by representing the square using a binary multiplication node with a repeated input, but that is an additional choice of syntax and semantics.

The broader tree theorem needs its hypotheses

General packet tree expansion needs correctly typed input slots, finite sets of packet labels at each node, and multilinearity in every slot used for distributive expansion. Empty inputs and unary nodes need explicit conventions. Slot order must survive even when values or operations do not commute. The traditional paper supplies rooted tree reconstruction and pruning results in its ordinary mathematical development.

This page supplies elementary exact examples of the issue. The finite v1.0.1 Lean contract remains narrower: it contains syntactic input locality, not the general dependent tree expansion or the traditional paper's analytic convergence results.

> **Read this as:** A branching picture does not justify throwing away mixed interactions. Exact packet expansion requires the appropriate multilinearity and typing, not merely a diagram that appears finite.

V23 / CAPSTONE / A COMPLETE FINITE DIAGNOSIS

From loop data to a packet witness

Traditional paper: Theorem 5.10 (p. 13), Theorem 6.2 (p. 14), Corollary 7.6 (p. 18)

Supply a reversible connection on three copies of $\mathbb{Q}^2$ by choosing

$$\Gamma_{1\leftarrow 0} = A, \qquad \Gamma_{2\leftarrow 1} = B, \qquad \Gamma_{2\leftarrow 0} = C = I,$$

and declaring every reverse edge to be the inverse. This time both nontrivial forward matrices are retained.

A

1	1
0	1

B

1	0
1	1

$H = BA$

1	1
1	2

$H - I$

0	1
1	1

$$e_0 = (1,0) \;\mapsto\; (1,0) \;\mapsto\; (1,1) \;\mapsto\; (1,1)$$

$$P_1(H - I)e_0 = e_1 \neq 0$$

---

replace direct edge $C = I$ by $C = BA$: then $C^{-1}BA = I$

The vector row starts with the source witness e_0 and follows it through A, then B, then C^{-1}. Returning to the base produces $(1,1)^{\mathsf{T}}$, whose displacement from the input is e_1.

The calculation gives

$$\Delta = BA - C = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}, \qquad H = C^{-1}BA = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}.$$

Choose the standard packet projectors. The $(1,0)$ block is nonzero, and the explicit candidate vector satisfies $P_1(H - I)e_0 = e_1$. This proves nonflatness of the supplied connection at the supplied triangle.

Repairing data is not the same as changing gauge

Replacing the direct map $C = I$ by $C = BA$, and updating its reverse, makes the new loop equal to I. That changes the connection. It cannot be a gauge of the original nonflat connection, because gauge preserves whether holonomy is the identity. For this complete example on three charts all cocycles then hold. Changing one edge in a larger network would require checking the other affected loops as well.

> **Read this as:** Declare the data, multiply in traversal order, form the defect, resolve all blocks, and exhibit an input on which one block acts nontrivially. Each step has a checkable mathematical object.

V24 / EVIDENCE / EXAMPLES, PROOFS, AND FORMAL SCOPE

Read the evidence with the pictures

Traditional paper: Section 12 (p. 24)

This guide preserves the 24 worked pages and 22 exact figures from the finite visual appendix. It updates their references to the final traditional paper and explains the relation to that paper's broader nonlinear and analytic material.

Read each evidence record in its scope

Evidence	What it supports
Finite Lean release	The frozen declarations under their original hypotheses and pinned foundations. The Lean guide records the precise theorem map, source identity checks, and inherited build evidence. No fresh kernel compilation is claimed here.
Ordinary mathematics	The traditional paper's expanded proofs, including nonlinear, obstruction, lifting, and smooth transport results. Inclusion in that paper does not add these results to the finite Lean contract.
Exact model checks	Fresh replay on 8 October 2026 passed all 124 exact checks and rejected all four deliberately false controls. The scope is the displayed finite matrices, paths, polynomials, candidate tables, and trees.
Figure regeneration	All 22 editable TikZ figures regenerated from the preserved exact model data. Labels and spacing were refined for this edition. Their current sources match the regenerated output. The original figure sources remain separately preserved.

Separately, eleven mixed exact integer and numerical verification groups passed in a fresh run of the preserved strengthened source and checker, not the newly edited final TeX. Those checks have their stated finite ranges and numerical tolerances. They do not certify every statement in the monograph.

Boundaries that remain in force

The formal edge system uses a single scalar coefficient semiring. The heterogeneous path theorem for arbitrary lengths uses a common packet index, with forms allowing changing types for two and three factors. General dependent tree expansion and analytic convergence lie outside this finite formal contract. The traditional paper states and proves its broader results with their own hypotheses.

The generic obstruction theorem rejects its named candidate. A nonzero holonomy block rejects flatness for its named loop data. Neither conclusion supplies a universal classification, a new arithmetic realization, or an algorithmic search guarantee.

Replay the worked layer

From the editable visual guide directory, compile the PDF with `python build.py`.

Check the exact examples with `python checks/verify_examples.py`.

Regenerate the figures with `python source/generate_graphics.py`.

Use a temporary copy to retain the delivered evidence unchanged. The full package preserves the finite release and its original formal sources separately.

> **Read this as:** Use the figures to understand the mechanisms, the traditional paper to read the mathematical proofs, and the Lean guide to identify the exact formal contract. These are related records with distinct scopes.

References

[1] Ralph Abraham, Jerrold E. Marsden, and Tudor Ratiu. *Manifolds, Tensor Analysis, and Applications*, volume 75 of *Applied Mathematical Sciences*. Springer, New York, second edition, 1988.

[2] Gregory M. Constantine and Thomas H. Savits. A multivariate Faà di Bruno formula with applications. *Transactions of the American Mathematical Society*, 348(2):503–520, 1996.

[3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[4] Klaus Deimling. *Nonlinear Functional Analysis*. Springer, Berlin, 1985.

[5] Jonathan L. Gross and Thomas W. Tucker. *Topological Graph Theory*. John Wiley and Sons, New York, 1987.

[6] Ernst Hairer, Christian Lubich, and Gerhard Wanner. *Geometric Numerical Integration: Structure Preserving Algorithms for Ordinary Differential Equations*. Springer, Berlin, second edition, 2006.

[7] Tosio Kato. On the adiabatic theorem of quantum mechanics. *Journal of the Physical Society of Japan*, 5:435–439, 1950.

[8] Tosio Kato. *Perturbation Theory for Linear Operators*. Springer, Berlin, 1995.

[9] Hansjörg Kielhöfer. *Bifurcation Theory: An Introduction with Applications to Partial Differential Equations*. Springer, New York, second edition, 2012.

[10] Shoshichi Kobayashi and Katsumi Nomizu. *Foundations of Differential Geometry. Volume I*. John Wiley and Sons, New York, 1963.

[11] Jean Louis Loday and Bruno Vallette. *Algebraic Operads*. Springer, Berlin, 2012.

[12] John McCleary. *A User's Guide to Spectral Sequences*. Cambridge University Press, Cambridge, second edition, 2001.

[13] Komorebi P. Projector-resolved seed calculus i: Typed seeds, operator grammars, and exact reconstruction, 2026. Version 1.2.1.

[14] Komorebi P. Projector-resolved seed calculus ii: Polyphase refinement, weighted transfer, and dyadic–odd resolution, 2026. Version 1.2.1.

[15] Roger Penrose. A generalized inverse for matrices. *Proceedings of the Cambridge Philosophical Society*, 51:406–413, 1955.

[16] Richard P. Stanley. *Enumerative Combinatorics. Volume 1*, volume 49 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, Cambridge, second edition, 2012.

[17] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. Association for Computing Machinery, 2020.

[18] Charles A. Weibel. *An Introduction to Homological Algebra*. Cambridge University Press, Cambridge, 1994.

[19] Eberhard Zeidler. *Nonlinear Functional Analysis and Its Applications. I: Fixed Point Theorems*. Springer, New York, 1986.

OPERATION KOMOREBI

Projector Resolved Seed Calculus III

Analytic Packet Jets and Exact Composition

Mathematical and verification supplement

Komorebi P. 20 October 2026

Private scoped publication edition · Version 1.0.0

This supplement gives full ordinary mathematical arguments for the analytic packet claims of Section 17, identifies a stronger norm bound, and records how the calculation relates to the actual projector ranges. It does *not* assert new Lean kernel verification. The formal algebraic development through Section 16 retains its recorded evidence. The newer Section 17 Lean files are candidates, not accepted proofs.

1 Two distinct contracts

Let $\mathsf{X}, \mathsf{Y}, \mathsf{Z}$ be real Banach spaces. A *bounded packet system* on X is a finite family $P_i \in \mathcal{L}(\mathsf{X}, \mathsf{X})$ such that

$$\sum_{i \in I} P_i = \mathrm{id}_{\mathsf{X}}, \qquad P_i^2 = P_i, \qquad P_i P_j = 0 \quad (i \neq j). \tag{1}$$

Write $\mathsf{X}_i = \operatorname{ran} P_i$ with the subspace norm. Analogously, let $(Q_\gamma)_{\gamma \in J}$ and $(R_\delta)_{\delta \in K}$ be bounded packet systems on Y and Z.

A weaker family of operators that only satisfies $\sum_i P_i = \mathrm{id}$ yields a useful algebraic expansion, but it need not produce independent coordinates. We keep these statements separate.

Lemma 1.1 (Closed ranges and bounded coordinates)**.** *Each X_i is closed. The maps*

$$A_{\mathsf{X}}(x) = (P_i x)_{i \in I} : \mathsf{X} \longrightarrow \prod_{i \in I} \mathsf{X}_i, \qquad S_{\mathsf{X}}((z_i)_{i \in I}) = \sum_{i \in I} z_i$$

are inverse bounded linear maps when the finite product carries the maximum norm.

Proof. The identity $\operatorname{ran} P_i = \ker(\mathrm{id} - P_i)$ follows from idempotence. The kernel of a continuous linear map is closed. Both coordinate maps are bounded because the family is finite. Completeness gives $S_{\mathsf{X}} A_{\mathsf{X}} = \mathrm{id}$. If $z_i \in \operatorname{ran} P_i$, then $P_i z_i = z_i$ and $P_j z_i = 0$ for $j \neq i$. Thus $A_{\mathsf{X}} S_{\mathsf{X}} = \mathrm{id}$. Moreover, $\|A_{\mathsf{X}}\| \leq \max_i \|P_i\|$ and $\|S_{\mathsf{X}}\| \leq |I|$ whenever I is nonempty. The zero-space case follows from the identities directly. □

2 Multilinear reconstruction and packet jets

Theorem 2.1 (Completeness is enough for multilinear expansion)**.** *Let I, J be finite. Suppose $\sum_i P_i = \mathrm{id}_{\mathsf{X}}$ and $\sum_\gamma Q_\gamma = \mathrm{id}_{\mathsf{Y}}$, without assuming idempotence or annihilation. Let $B : \mathsf{X}^r \to \mathsf{Y}$ be multilinear. Then, for all $h_1, \ldots, h_r \in \mathsf{X}$,*

$$B(h_1, \ldots, h_r) = \sum_{\gamma \in J} \sum_{\alpha \in I^r} Q_\gamma B(P_{\alpha_1} h_1, \ldots, P_{\alpha_r} h_r). \tag{2}$$

The statement includes $r = 0$ under the usual empty-product convention.

Proof. For every input, $h_a = \sum_{i\in I} P_i h_a$. Additivity in each slot expands B into the sum over all choices $\alpha \in I^r$. The finite output identity expresses each resulting value as its sum of Q_γ components. All exchanges of sums are finite. No property beyond completeness and multilinearity enters the calculation. For $r = 0$, the input-index sum has exactly one term, and output completeness still gives the identity. □

Proposition 2.2 (Literal continuous range jet)**.** *Under* (1) *for the input and output families, let* $B : \mathsf{X}^r \to \mathsf{Y}$ *be continuous multilinear,* $r \geq 1$*. The formula*

$$J_{\gamma|\alpha}B(z_1, \ldots, z_r) = Q_\gamma B(z_1, \ldots, z_r), \qquad z_a \in \mathsf{X}_{\alpha_a},$$

defines a continuous multilinear map $\prod_a \mathsf{X}_{\alpha_a} \to \mathsf{Y}_\gamma$ *with the actual range as codomain. For* $z_a = P_{\alpha_a} h_a$*, its image in* Y *is the corresponding summand in* (2)*.*

Proof. The output lies in $\operatorname{ran} Q_\gamma$ by construction. The subspace inclusions are continuous and isometric. The output projector Q_γ and B are continuous. Their composition therefore gives a continuous multilinear map into Y. Since Y_γ carries the induced norm and is closed, viewing this value in Y_γ does not change its norm or continuity. On the projected arguments $P_{\alpha_a} h_a$, the defining expression is precisely the summand in (2). □

Proposition 2.3 (A stronger range norm bound)**.** *The literal range jet satisfies*

$$\left\| J_{\gamma|\alpha}B \right\| \leq \|Q_\gamma\| \, \|B\| \,. \tag{3}$$

Its ambient extension $\widetilde{J} = Q_\gamma \circ B \circ (P_{\alpha_1}, \ldots, P_{\alpha_r})$ *satisfies*

$$\left\| \widetilde{J} \right\| \leq \|Q_\gamma\| \, \|B\| \prod_{a=1}^{r} \|P_{\alpha_a}\| \,. \tag{4}$$

In particular, the published bound for the range jet with the projector norm product on the right follows from (3)*.*

Proof. For inputs $z_a \in \mathsf{X}_{\alpha_a}$, the induced norms yield $\|Q_\gamma B(z_1, \ldots, z_r)\| \leq \|Q_\gamma\| \, \|B\| \prod_a \|z_a\|$, proving (3). Apply additionally $\|P_{\alpha_a} h_a\| \leq \|P_{\alpha_a}\| \, \|h_a\|$ to obtain (4). If every selected input projector is nonzero, idempotence implies $\|P_{\alpha_a}\| \geq 1$ by evaluating at a unit vector in its range. Then (3) implies the weaker published inequality. If any selected input projector is zero, the corresponding input range is the zero space and the positive-order jet is zero. Both sides of the published inequality then vanish. This completes the comparison, including the degenerate case. □

3 Derivatives, coordinates, and Taylor remainder

Let $U \subset \mathsf{X}$ be open and $F : U \to \mathsf{Y}$ be C^m. For $1 \leq r \leq m$, the Fréchet derivative $\mathrm{D}^r F(x)$ is continuous multilinear. Apply Theorem 2.1 to $B = \mathrm{D}^r F(x)$. Thus the actual packet jets recover the entire derivative, without presupposing a separate packet derivative object.

Corollary 3.1 (Conjugated derivatives in packet coordinates)**.** *Let* $\widehat{F} = A_{\mathsf{Y}} \circ F \circ S_{\mathsf{X}}$ *on the open set* $A_{\mathsf{X}}(U)$*. Then* $\widehat{F}$ *is* C^m *and*

$$\mathrm{D}^r \widehat{F}(u)[v_1, \ldots, v_r] = A_{\mathsf{Y}}\big(\mathrm{D}^r F(S_{\mathsf{X}} u)[S_{\mathsf{X}} v_1, \ldots, S_{\mathsf{X}} v_r]\big).$$

If v_a *is supported only in packet slot* α_a*, let* $z_a \in \mathsf{X}_{\alpha_a}$ *denote its nonzero coordinate. Then output component* γ *is*

$$J_{\gamma|\alpha}(\mathrm{D}^r F(S_{\mathsf{X}} u))[z_1, \ldots, z_r].$$

Proof. The operators A_{Y} and S_{X} are bounded and linear by Lemma 1.1. Applying the ordinary Fréchet chain rule repeatedly, their higher derivatives vanish. Hence the stated formula. The last assertion follows because S_{X} includes a single supported slot into X and A_{Y} applies Q_γ in the chosen output slot. □

Theorem 3.2 (Packet Taylor identity with integral remainder)**.** *Let $F : U \to \mathsf{Y}$ be C^{m+1} on an open set containing the closed segment $[x, x+h]$. For $m \geq 0$,*

$$Q_\gamma F(x+h) = Q_\gamma F(x) + \sum_{r=1}^{m} \frac{1}{r!} \sum_{\alpha \in I^r} J_{\gamma|\alpha}(\mathrm{D}^r F(x))[P_{\alpha_1} h, \ldots, P_{\alpha_r} h] + Q_\gamma R_{m+1},$$

where

$$R_{m+1} = \frac{1}{m!} \int_0^1 (1-t)^m \mathrm{D}^{m+1} F(x+th)[h^{m+1}]\, dt.$$

If $\|\mathrm{D}^{m+1} F(x+th)\| \leq M$ on the segment, then

$$\|Q_\gamma R_{m+1}\| \leq \frac{\|Q_\gamma\|\ M\, \|h\|^{m+1}}{(m+1)!}.$$

Proof. The function $g(t) = F(x+th)$ is a C^{m+1} Banach-valued function on a neighborhood of $[0,1]$. Repeated chain differentiation gives $g^{(r)}(t) = \mathrm{D}^r F(x+th)[h^r]$. Apply the one-dimensional Taylor formula with the Bochner integral remainder. Each Q_γ is bounded, so it commutes with the Bochner integral. Expand the finite derivative sum by Theorem 2.1 and take its γ component. For the bound use $\|Q_\gamma v\| \leq \|Q_\gamma\|\, \|v\|$, the operator norm of the derivative, and $\int_0^1 (1-t)^m\, dt = 1/(m+1)$. □

4 The partition chain rule and packet decorations

Let $G : \mathsf{X} \to \mathsf{Y}$ and $F : \mathsf{Y} \to \mathsf{Z}$ be C^n on the relevant open sets, with G mapping its domain into the domain of F. Write Π_n for set partitions of $\{1, \ldots, n\}$.

Theorem 4.1 (Faà di Bruno in Banach spaces)**.** *For $n \geq 1$,*

$$\mathrm{D}^n (F \circ G)(x)[h_1, \ldots, h_n] = \sum_{\pi \in \Pi_n} \mathrm{D}^{|\pi|} F(Gx) \left[\mathrm{D}^{|B|} G(x)[h_i : i \in B] : B \in \pi \right].$$

The order of blocks is immaterial because Fréchet higher derivatives are symmetric.

Proof. For $n = 1$ this is the chain rule. Suppose the identity holds at order n and the maps are C^{n+1}. Differentiate a summand in direction h_{n+1}. Differentiating the outer $\mathrm{D}^{|\pi|} F(Gx)$ produces the same partition with the new singleton block $\{n+1\}$. Differentiating the argument $\mathrm{D}^{|B|} G(x)[h_i : i \in B]$ produces the partition obtained by adjoining $n+1$ to the existing block B. Every partition of $\{1, \ldots, n+1\}$ arises in exactly one of these ways, depending on whether $\{n+1\}$ is a singleton. The product and chain rules apply to the bounded multilinear evaluations, and all sums are finite. Induction completes the proof. □

Corollary 4.2 (Decorated packet chain rule)**.** *Fix input labels α_i, directions $h_i \in \mathsf{X}_{\alpha_i}$, and an output label δ. For each partition π and intermediate label function $\gamma : \pi \to J$, replace the Bth argument of the outer derivative by $Q_{\gamma(B)} \mathrm{D}^{|B|} G(x)[h_i : i \in B]$. The sum over all such decorations, followed by R_δ, equals the δ output packet of $\mathrm{D}^n (F \circ G)(x)[h_1, \ldots, h_n]$.*

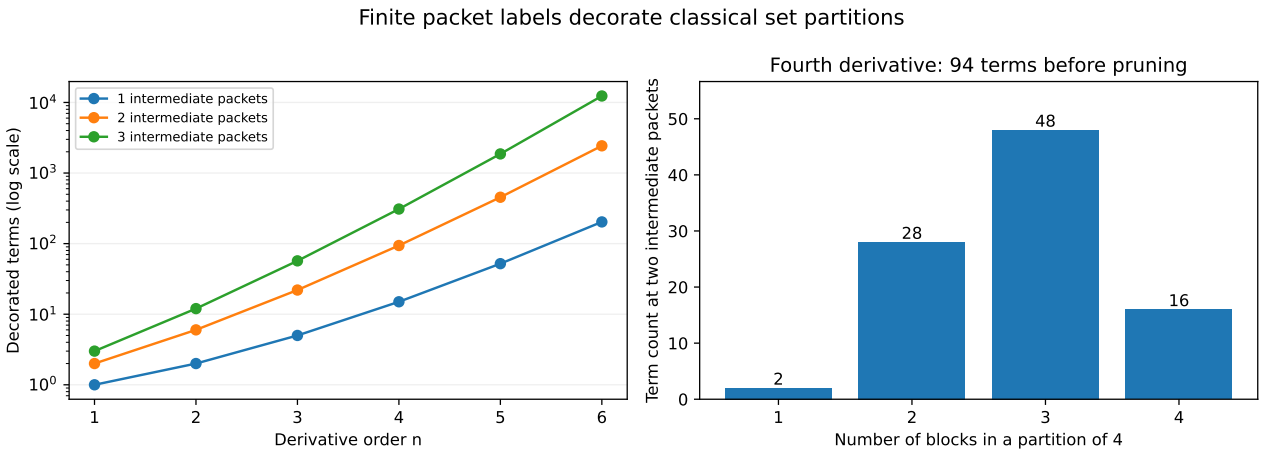


Figure 1: The growth of labeled partition terms. At fourth order with two intermediate packet labels, the four block-count contributions are 2, 28, 48, and 16, totaling 94. These are exact combinatorial counts, before structural pruning or cancellations.

Proof. In every outer argument of Theorem 4.1, insert the identity on Y as $\sum_{\gamma \in J} Q_\gamma$. Expand by multilinearity. The output projection R_δ distributes over the finite sum. Because the directions are already in their actual packet ranges, this is precisely the literal packet-jet expression. No statement assumes that mixed input labels disappear. □

Proposition 4.3 (Decorated term count and pruning)**.** *If the intermediate family contains M packet labels, the number of formal decorated terms before pruning is*

$$N_{n,M} = \sum_{k=1}^{n} S(n,k) M^k,$$

where $S(n,k)$ is a Stirling number of the second kind. A decorated summand vanishes if any of its constituent packet jets vanishes. The support of the composed jet is contained in the combinatorially admissible decorated support. Equality of supports need not hold because nonzero terms may cancel.

Proof. There are $S(n,k)$ partitions with k blocks and M^k functions assigning labels to those blocks. The support assertion follows by multilinearity, since a zero argument or a zero outer restricted jet yields a zero summand. The formula is a sum, not a disjoint union of nonzero vectors, so cancellation is allowed. These are statements about formal summands, not claims of computational sparsity for arbitrary coordinate systems. □

5 Reproducibility and limitations

Layer	Acceptable inference
Section 16 Lean	The recorded 12-module production closure, 558 logical declarations, and 14 separately inventoried runtime helpers remain inherited accepted evidence, not a fresh replay.

Layer	Acceptable inference
Section 17 ordinary proofs	The Banach-space identities in this supplement are mathematical arguments, without a current compiler certificate. The range bound (3) strengthens the printed proposition.
Independent exact checks	Original Section 16 and 17 suites were replayed. New exact polynomial examples verify decorated chain rules through order four and Taylor remainder identities. These do not establish a general theorem.
Unaccepted Lean sources	Two Section 17 candidate files are preserved outside the accepted import graph. Neither is counted as compiled.
Expanded monograph beyond Section 17	Written proofs and analyses are preserved. Their full correspondence to the formal development has not been demonstrated. The separate October 16 larger declaration count has not been authenticated.
Public redistribution	A project-wide license has not been selected. This is a private release package and grants no new redistribution permission.

Reading rule. A valid ordinary proof, an exact computation, a Lean theorem under explicit hypotheses, and an open research construction are separate kinds of evidence. Paper III's release records identify each rather than silently promoting one into another.

PAPER III

Projector Resolved Seed Calculus III

Typed transfer, nonlinear packets, and obstruction theory

Twenty-four worked plates make the direction of each map, the role of every hypothesis, and the limits of finite evidence visible.

Komorebi P.

Visual guide • 8 October 2026

How to use this volume

Twenty-four worked plates and twenty-two exact model figures accompany the monograph and its Lean guide. Integer, rational, and symbolic calculations determine the figures. The source package includes the model generators and their explicit finite verification ranges.

A route through the examples

The labels V1 through V24 identify worked pages independently of PDF pagination. Each page names its corresponding statement in the monograph edition. The Lean guide provides the separate formal contract.

V1 Three objects, three different questions
V2 An atlas of the objects
V3 Read the product in execution order
V4 All sixteen words, only five walks
V5 Coefficients need not commute
V6 A reachable packet can receive zero
V7 Reachability is only an upper bound
V8 Paths through changing packet lists
V9 One vector, three exact frames
V10 Nonlinear charts still cancel exactly
V11 Reversibility is a real hypothesis
V12 Measure the mismatch at its base
V13 The identity can hide in the spectrum
V14 Gauge changes the picture, not flatness
V15 Flat is not the same as calibrated
V16 Resolve the entire obstruction
V17 Why diagonal checks are insufficient
V18 A gauge can move every packet label
V19 An obstruction rejects one candidate
V20 A tree reads its named leaves
V21 Where a nonlinear defect can enter
V22 Mixed terms are not optional
V23 From loop data to a packet witness
V24 Read the evidence with the pictures

Three reading routes

Typed transfer: V3 to V8. **Coordinates, connection, and gauge:** V9 to V18, then V23. **Candidate witnesses and trees:** V19 to V22, then V24.

From the finite models to the broader paper

The monograph continues with rooted tree expansions, nonlinear refinement defects, obstruction objects, filtered lifting, elimination, and smooth projector transport. Read the finite examples as concrete preparation. The broader ordinary proofs keep their own hypotheses and are distinct from the finite Lean contract. Rooted trees: Section 18 (p. 36). Nonlinear defects: Section 20 (p. 41). Obstruction objects: Section 21 (p. 43). Filtered lifting: Section 23 (p. 52). Smooth transport: Section 27 (p. 62).

Read products in traversal order

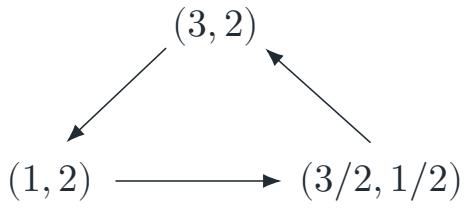
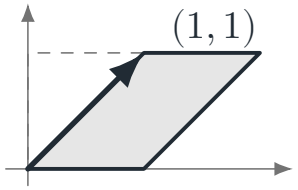
A path $e_1, e_2, \dots, e_n$ acts as $L_{e_n} \cdots L_{e_2} L_{e_1}$: the first traversed edge is the rightmost operator. Row indices are targets and column indices are sources throughout the guide.

V1 / PAPER III / A WORKED VISUAL COMPANION

Three objects, three different questions

Traditional paper: Section 1 (p. 1), Section 4 (p. 7), Section 5 (p. 9)

An edge operator moves data between named packets. An exact coordinate change describes the same data in another complete chart. A connection supplies an additional choice of transport. The three can use similar arrows while making different mathematical assertions.

EDGE ACTION	EXACT COORDINATES	CHOSEN CONNECTION
$\begin{pmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$		
$e_0 \longmapsto 2e_1$	one unchanged ambient vector	$H = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right) \neq I$

Three actual models used below: a single supported matrix entry, one ambient vector in three coordinate systems, and a shear obtained as chosen loop transport. The coordinates in the middle are not three different ambient vectors.

One paper, three tests

For an edge word, ask whether the target of each step matches the source of the next. For exact coordinates, ask whether analysis and synthesis are inverses. For a connection, ask whether its transports through two steps agree with its declared direct transports. None of these tests can replace another.

This guide follows explicit integer and rational matrices through these tests. Each example has an inspectable input, calculation, and conclusion. Vector drawings display those calculations rather than replace them. A graph with two paths can have zero net transfer. A loop with trace two and determinant one can still differ from the identity.

The edition used here

The worked models originate in the finite algebraic v1.0.1 release. The final traditional paper also develops rooted tree expansions, nonlinear defects, obstruction objects, filtered lifting, and analytic transport under their stated hypotheses. Those ordinary mathematical developments have a wider scope than the finite Lean contract. The present models explain the finite mechanisms and provide a route into the broader paper.

> **Read this as:** Follow the object, not the arrow shape. A typed transfer can vanish, a complete coordinate loop must telescope, and an independently chosen connection can have genuine algebraic holonomy.

V2 / ORIENTATION / KEEP THE TYPES VISIBLE

An atlas of the objects

Traditional paper: Definition 2.1 (p. 2), Definition 5.5 (p. 11), Definition 7.4 (p. 16)

Rows are targets and columns are sources throughout. If E_{ji} is a matrix unit, then $E_{ji}e_i = e_j$. The notation e_i means a basis vector. A named graph edge is written a, b, c, d or e, f when its role is explicit.

Object	What it records	First worked page
$\mathrm{src}, \mathrm{tgt} : E \to V$	The ordered endpoints of each distinct edge, including parallel edges.	V – 3 to V – 4
$L_e = c(e)E_{\mathrm{tgt}(e),\mathrm{src}(e)}$	One operator supported on a packet.	V – 3
$L_w = L_{e_n} \cdots L_{e_1}$	Execute the written word from left to right.	V – 4 to V – 5
$T = \sum_e L_e$	Add all edge operators before taking powers.	V – 4 to V – 7
F_a	A synthesis frame from chart a into the ambient space.	V – 9
$\mathcal{A}_a, \mathcal{S}_a$	Inverse analysis and synthesis maps, which may be maps of sets.	V – 10
$J_{b\leftarrow a}$	Canonical coordinate transport, forced by the exact charts.	V – 9 to V – 10
$\Gamma_{b\leftarrow a}$	Additional connection data, with reversibility stated explicitly.	V – 11 to V – 12
$\Delta_{c,b,a}$	The difference between transport through two steps and direct transport, a map from C_a to C_c.	V – 12
$H_{c,b,a}$	Loop transport based at a, an automorphism of C_a.	V – 13
G_a	An invertible gauge change at each chart.	V – 14 to V – 15
$\Pi_\mu \,\mathrm{Ob}\, \Pi_\nu$	Block from input to output of $\mathrm{Ob} = H - I$.	V – 16 to V – 18
$r(x, p)$	A residual at packet p for the named candidate x.	V – 19
$t, \mathrm{inputs}(t)$	A finite operation tree and its syntactically used input labels.	V – 20

Reading routes

For transfer, read V3 to V8. For the finite geometry models, read V9 to V18. For candidate certificates and nonlinear scope, read V19 to V22. V23 puts the matrix mechanisms together. V24 gives the formal boundary and replay guide.

> **Read this as:** A $C_a \to C_c$ defect and a $C_a \to C_a$ holonomy are not the same typed map. Returning the defect to the base fiber requires a specified inverse transport.

V3 / MODEL A / ONE EDGE IS ONE MATRIX ENTRY

Read the product in execution order

Traditional paper: Lemma 2.2 (p. 2), Theorem 2.3 (p. 2), Theorem 2.4 (p. 3)

Work over $\mathbb{Z}$ with packets $0, 1, 2$. Let $e : 0 \to 1$ have coefficient 2, and let $f : 1 \to 2$ have coefficient 3. The elementary operators are

$$L_e = 2E_{10}, \qquad L_f = 3E_{21}.$$

The location of the nonzero entry records the type of the transfer, not an arbitrary matrix convention.

L_e

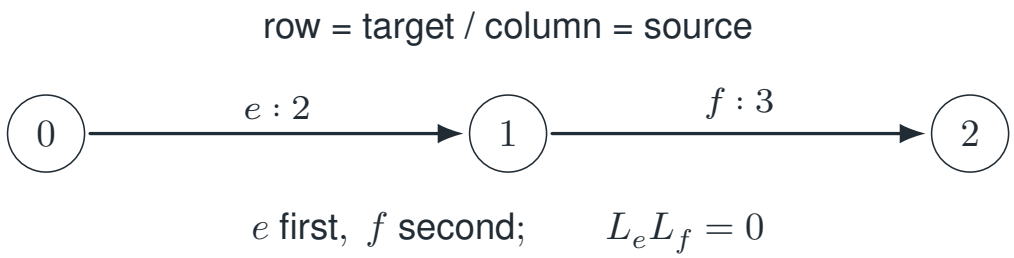
	0	1	2
0	0	0	0
1	2	0	0
2	0	0	0

L_f

	0	1	2
0	0	0	0
1	0	0	0
2	0	3	0

$L_f L_e$

	0	1	2
0	0	0	0
1	0	0	0
2	6	0	0



Each grid is an actual 3×3 matrix. The rightmost matrix factor acts first. In $L_f L_e$, packet 1 is the shared intermediate index. In $L_e L_f$ the interface is broken.

On a vector $(x_0, x_1, x_2)^{\mathsf{T}}$, the first step produces $(0, 2x_0, 0)^{\mathsf{T}}$, and the second produces $(0, 0, 6x_0)^{\mathsf{T}}$. Thus

$$L_f L_e = 6E_{20}, \qquad L_e L_f = 0.$$

For general edges, a summand in the entrywise product can survive only when the same intermediate packet is both $\mathrm{tgt}(e)$ and $\mathrm{src}(f)$:

$$(L_f L_e)_{yx} = \sum_z (L_f)_{yz} (L_e)_{zx}.$$

This is why a broken interface annihilates the entire product, even if both edge coefficients are nonzero.

What the zero says

It says that this ordered composition does not pass data through the declared packet types. It does not say that either edge is individually zero. For a longer word, finding one broken adjacent interface is sufficient to prove that the whole word vanishes.

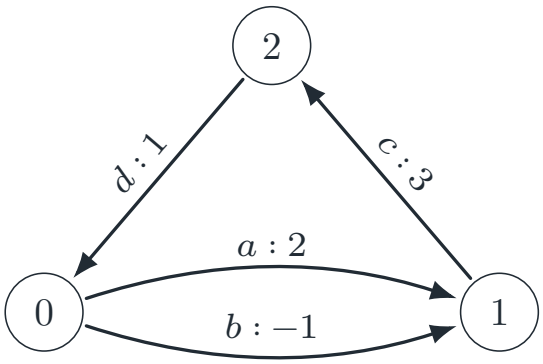
> **Read this as:** Write the traversed edges first. Then reverse their order when writing the operator product: e followed by f is $L_f L_e$.

V4 / MODEL A / PARALLEL EDGES REMAIN DISTINCT

All sixteen words, only five walks

Traditional paper: Theorem 3.8 (p. 5), Corollary 3.9 (p. 6)

Add four edges on three packets: $a : 0 \to 1$ with coefficient 2, $b : 0 \to 1$ with coefficient -1, $c : 1 \to 2$ with coefficient 3, and $d : 2 \to 0$ with coefficient 1. The parallel edges a and b are separate labels even though their source and target agree.



$$(T^2)_{20} = 6 - 3 = 3$$

first executed edge = column

	a	b	c	d
a	×	×	×	2
b	×	×	×	-1
c	6	-3	×	×
d	×	×	3	×

second executed edge = row

×: broken packet interface

The board tests all ordered pairs. Columns are the first executed edge and rows are the second. A shaded cell gives the ordered coefficient of a valid walk. A cross means the product is zero by type mismatch.

The surviving words are ac, bc, cd, da, db. Their coefficients are $6, -3, 3, 2, -1$, respectively. Their endpoints determine where these numbers enter the matrix:

$$T = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix}, \qquad T^2 = \begin{pmatrix} 0 & 3 & 0 \\ 0 & 0 & 1 \\ 3 & 0 & 0 \end{pmatrix}.$$

At row 2, column 0, the two parallel choices give $6 - 3 = 3$. Deduplicating the edge labels before summing would alter the operator. On the other hand, adding the parallel coefficients in the already assembled matrix is legitimate: both act in the same supported matrix position.

The same calculation extends from pairs to all finite words by distributivity. The included check enumerates every word through length six and independently accumulates the endpoint coefficients. The empty word is treated separately as the identity.

> **Read this as:** The matrix power is a signed, ordered walk sum. It is not simply the number of paths, and a graph edge is not identified solely by its endpoints.

V5 / MODEL A / A MATRIX RING AS THE COEFFICIENT SEMIRING

Coefficients need not commute

Traditional paper: Lemma 3.7 (p. 5), Section 9 (p. 20)

The edge theorem allows a noncommutative coefficient semiring. Use $R = M_2(\mathbb{Q})$ and let the two coefficients themselves be matrices:

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Attach A to $0 \to 1$ and B to $1 \to 2$. The single surviving endpoint coefficient is BA, not AB.

A

1	1
0	1

B

1	0
1	1

BA

1	1
1	2

AB

2	1
1	1

$e : 0 \to 1$ carries A; $\quad f : 1 \to 2$ carries B

$$BA(1,0)^{\mathsf{T}} = (1,1)^{\mathsf{T}}, \qquad AB(1,0)^{\mathsf{T}} = (2,1)^{\mathsf{T}}$$

All entries are exact integers. The two products differ on the diagonal and act differently on the same input vector. This example is within the scalar model with semiring coefficients by taking the scalar ring to be $M_2(\mathbb{Q})$.

The distinction is visible before any abstraction:

$$BA \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad AB \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

For a written walk $(e_1, \dots, e_n)$, its coefficient must therefore be

$$c^{\leftarrow}(w) = c(e_n)c(e_{n-1}) \cdots c(e_1).$$

A commutative example can hide an orientation mistake because reversing the coefficients changes nothing. This one cannot.

One ring is not a dependent family of fibers

The 3×3 matrix over $M_2(\mathbb{Q})$ can be displayed as an ordinary 6×6 matrix with 2×2 blocks. This is a concrete representation of the example over one ring. It does not establish an arbitrary family of differently typed edge fibers, which is outside the finite formal release's scalar interface for edge systems.

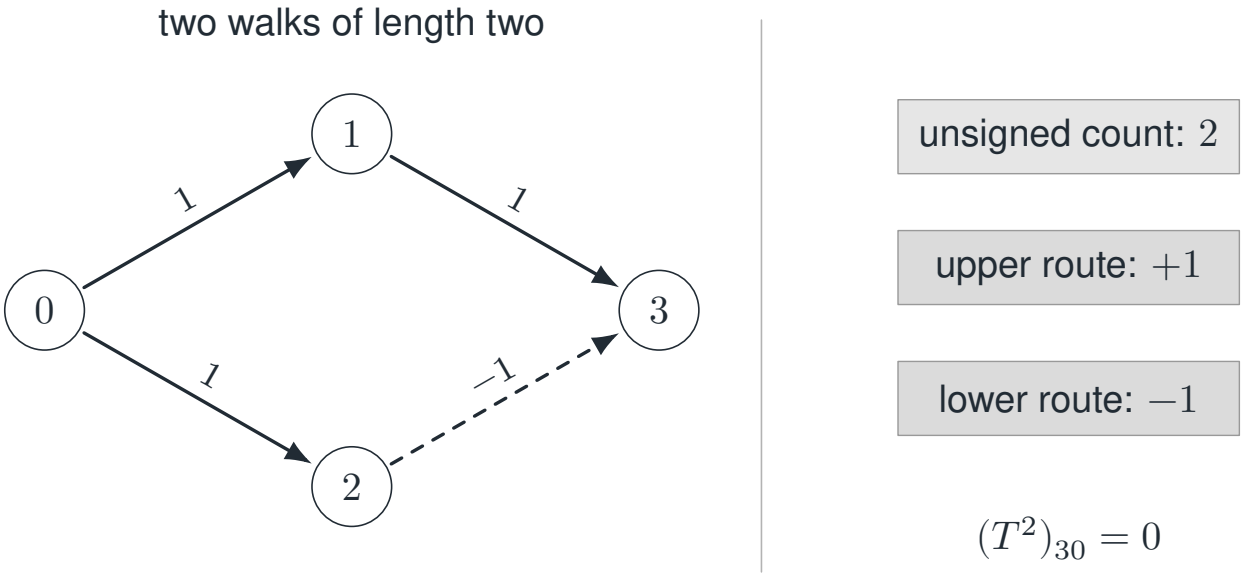
> **Read this as:** The order rule carries mathematical information. It survives exact tests over a noncommutative ring and must not be inferred from pictures drawn over scalar real numbers alone.

V6 / MODEL A / CANCELLATION IS NOT A BROKEN INTERFACE

A reachable packet can receive zero

Traditional paper: Corollary 3.9 (p. 6), Corollary 3.10 (p. 6)

Consider a diamond: two routes of length two from packet 0 to packet 3. Three edge coefficients are 1 and the lower final coefficient is -1. Both routes are correctly typed. Neither is annihilated by an interface mismatch.



The dashed edge carries the coefficient -1. It is an actual edge, not an uncertain or missing one. Both paths exist. The unsigned count is two, while their exact operator contributions sum to zero.

The endpoint identity gives

$$(T^2)_{30} = (1)(1) + (-1)(1) = 0.$$

Thus the paper's reachability implication has a definite direction:

$$\text{no compatible walk} \implies (T^n)_{ji} = 0.$$

Its converse is false. A zero entry may come from absent paths, zero coefficients, zero divisors in the coefficient ring, or cancellation among nonzero terms. The diamond exhibits the last possibility over $\mathbb{Z}$, where no zero divisor is needed.

Three different support records

The declared graph records permitted routes. The support of an individual word records the endpoints of that word when its coefficient is nonzero. The support of the sum records what remains after all contributions have been added. Passing from either of the first two records to the last requires the coefficients, not just adjacency.

A deliberately false test in the companion script asserts that reachability forces nonzero transfer. This diamond makes the test fail. That is a check against overreading the theorem, not a defect in the theorem.

> **Read this as:** Do not interpret a visible route as an occupied output. Graph support is a useful upper bound. Signed operator support can be strictly smaller.

V7 / MODEL A / AN ORBIT THAT STAYS IN TWO DIMENSIONS

Reachability is only an upper bound

Traditional paper: Corollary 3.11 (p. 6)

Keep the diamond and add an isolated packet 4. Start with the single seed $e_0 \in \mathbb{Q}^5$. Direct multiplication gives

$$Te_0 = e_1 + e_2, \qquad T^2 e_0 = 0.$$

Every later orbit vector is zero. The orbit hull generated by T is therefore exactly the span of the first two displayed columns.

$[e_0 \;\; Te_0 \;\; T^2 e_0]$

0	1	0	0
1	0	1	0
2	0	1	0
3	0	0	0
4	0	0	0

a basis of the orbit hull

1	0
0	1
0	1
0	0
0	0

$\mathrm{Reach}(0) = \{0, 1, 2, 3\}$

$\mathrm{Hull}_T(e_0) = \{(a, b, b, 0, 0)\}$

e_3 : reachable but absent

e_4 : unreachable and absent

rank $= 2$

The first matrix contains the actual orbit columns. The second supplies a basis of their span. The orbit hull lies inside the reachable subspace on four coordinates, but it has dimension two, not four.

A vector belongs to this hull precisely when it has the form

$$(a, b, b, 0, 0)^{\mathsf{T}}, \qquad a, b \in \mathbb{Q}.$$

Packet 4 is excluded because it is unreachable. Packet 3 is also absent, but for a different reason: the two contributions cancel. Even the reachable packets 1 and 2 remain linked. Powers of the single total operator do not separate them.

The admissible operator grammar matters

Allowing each individual edge operator is a different generation problem. For instance, applying the edge $0 \to 1$ and then $1 \to 3$ individually produces e_3. It does not follow that e_3 belongs to the orbit of the summed operator T. The paper's theorem on orbit support concerns the explicitly stated operator action.

> **Read this as:** Unreachable means impossible under the declared action. Reachable means only that graph geometry has not ruled it out. Linear dependence and cancellation still have to be checked.

V8 / MODEL B / RECTANGULAR PRODUCTS WITH TWO OR THREE STEPS

Paths through changing packet lists

Traditional paper: Proposition 8.1 (p. 19)

Here the packet lists have sizes $2, 3, 2, 1$ in succession. Let the actual maps be

$$\mathbb{Q}^2 \xrightarrow{A} \mathbb{Q}^3 \xrightarrow{B} \mathbb{Q}^2 \xrightarrow{C} \mathbb{Q}.$$

Unlike a power of one matrix, each step has its own entries and its own source and target list.

$A : \mathbb{Q}^2 \to \mathbb{Q}^3$

1	0
2	1
0	3

$B : \mathbb{Q}^3 \to \mathbb{Q}^2$

1	−1	2
0	1	1

$C : \mathbb{Q}^2 \to \mathbb{Q}$

2	−1

CBA

−4	6

	$(j,k)=(0,0)$	$(0,1)$	$(0,2)$	$(1,0)$	$(1,1)$	$(1,2)$	
$i=0$	2	−4	0	0	−2	0	$=-4$
$i=1$	0	−2	12	0	−1	−3	$=6$

The lower board expands one final row into all six choices of intermediate indices for each input packet. The entries are $C_{0j}B_{jk}A_{ki}$, with $k \in \{0,1,2\}$ and $j \in \{0,1\}$. Zero terms remain visible.

The intermediate and total products are

$$BA = \begin{pmatrix} -1 & 5 \\ 2 & 4 \end{pmatrix}, \qquad CBA = \begin{pmatrix} -4 & 6 \end{pmatrix}.$$

For input packet 0, the six contributions sum to $2 - 4 + 0 + 0 - 2 + 0 = -4$. For input packet 1, they sum to $0 - 2 + 12 + 0 - 1 - 3 = 6$. These are literal matrix entries, not probabilities or path counts.

The formal scope is deliberately precise

The finite formal release's forms with two and three steps allow changing packet types. Its heterogeneous product theorem for arbitrary lengths allows changing *factors* over one common finite packet index. That formal statement does not assert a fully dependent chain of changing index types at arbitrary lengths. The finite rectangular illustration here uses exactly three factors.

For a single factor, the list of intermediate indices is empty and contributes that matrix entry once. For zero factors, the result is the identity on the common space. These conventions must not be replaced by an empty sum equal to zero.

> **Read this as:** Heterogeneous can mean different matrices or different spaces. State which one is allowed before using a path formula.

V9 / MODEL C / CANONICAL TRANSPORT HAS NO HOLONOMY

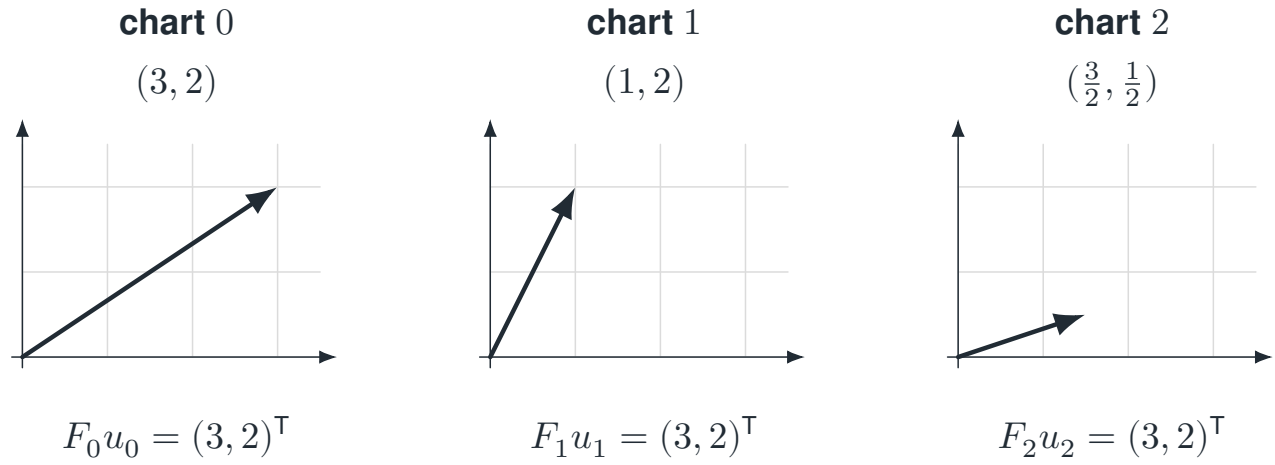
One vector, three exact frames

Traditional paper: Theorem 4.1 (p. 7), Theorem 4.2 (p. 7)

Let F_a synthesize chart coordinates into one ambient $\mathbb{Q}^2$. Choose

$$F_0 = I, \qquad F_1 = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \qquad F_2 = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}.$$

The same ambient vector $x = (3,2)^{\mathsf{T}}$ has chart coordinates $u_a = F_a^{-1}x$.



Each panel is a coordinate plane, not a different physical location. The numerical arrows differ because the frames differ. Multiplying each coordinate vector by its corresponding F_a recovers the same x.

Transport from chart a to chart b is forced:

$$J_{b \leftarrow a} = F_b^{-1} F_a.$$

The route $0 \to 1 \to 2$ agrees with the direct route because the middle frame cancels:

$$(F_2^{-1}F_1)(F_1^{-1}F_0) = F_2^{-1}F_0.$$

Returning to chart 0 cancels every remaining frame. The final coordinate vector is $(3,2)^{\mathsf{T}}$, not a sheared or rotated variant.

No metric is required

The frames need not be orthogonal. The second one is a shear, and the third has determinant two. Exact invertibility, rather than preservation of angles or lengths, gives the cocycle and loop identities. The same telescoping argument works for every finite chart path.

> **Read this as:** A canonical coordinate loop is a sequence of changes in description of one object. Nonidentity holonomy requires additional transport data, not merely nonorthogonal charts.

V10 / MODEL C / NATURALITY DOES NOT REQUIRE DERIVATIVES

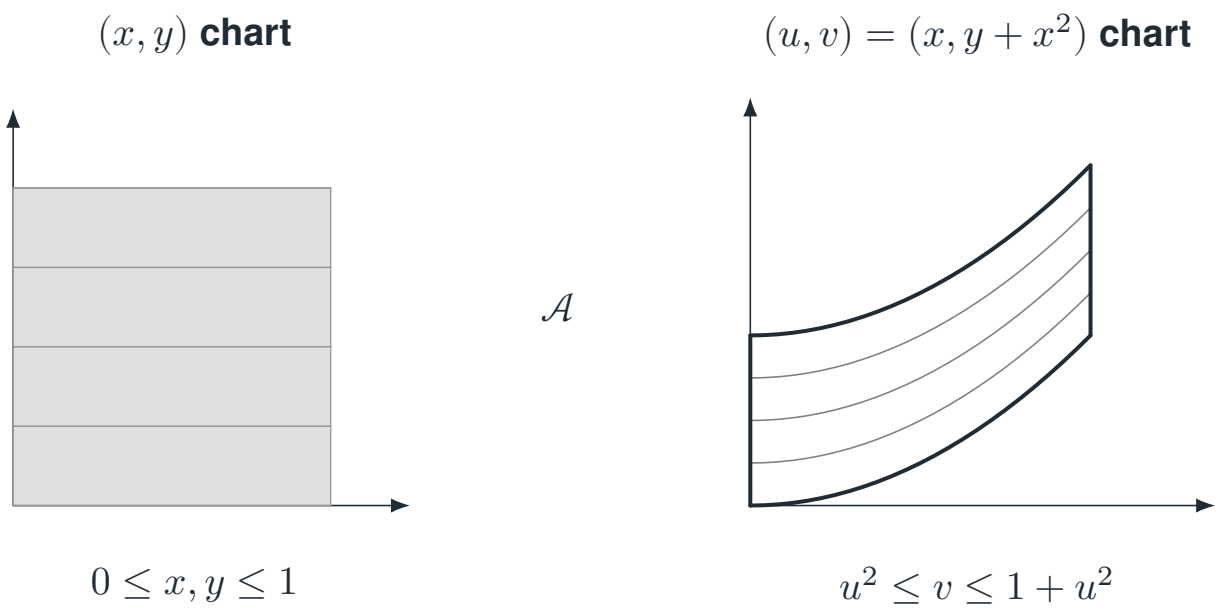
Nonlinear charts still cancel exactly

Traditional paper: Proposition 10.1 (p. 20)

Exact coordinate equivalences can be nonlinear. On $\mathbb{Q}^2$, use

$$\mathcal{A}(x, y) = (x, y + x^2), \qquad \mathcal{S}(u, v) = (u, v - u^2).$$

Both composites are the identity by direct substitution. The real drawing below is a geometric view of these same polynomial formulas.



Horizontal segments become parabolas, but no information is discarded. The drawing samples the displayed polynomial boundaries for rendering. Inverse identities and route equality are checked symbolically, not inferred from the curves.

Now represent the nonlinear operation $F(x, y) = (x + y, xy)$ in the new chart. Set $w = u + v - u^2$. Its correct coordinate formula is

$$\widehat{F}_b(u, v) = \mathcal{A}F\mathcal{S}(u, v) = (w,\ u(v - u^2) + w^2).$$

For $(x, y) = (1, 2)$, applying F first gives $(3, 2)$ and then $(3, 11)$ in the new chart. Changing the input first gives $(u, v) = (1, 3)$, and $\widehat{F}_b(1, 3) = (3, 11)$ as well.

The equality holds as a polynomial identity for all inputs. It is the naturality theorem for maps of sets: only the inverse laws for the coordinate maps are needed. No Taylor expansion, derivative, or higher chain rule is involved.

> **Read this as:** Nonlinearity alone does not create a coordinate defect. Reusing the old formula after changing coordinates would be an extra modeling choice, not exact conjugation.

V11 / MODEL D / THE MINIMAL AND REVERSIBLE INTERFACES

Reversibility is a real hypothesis

Traditional paper: Definition 5.1 (p. 9), Definition 5.5 (p. 11)

Choose the forward scalar transports $\Gamma_{1\leftarrow 0} = 2$, $\Gamma_{2\leftarrow 1} = 3$, and $\Gamma_{2\leftarrow 0} = 5$. Their multiplicative comparison between transport through two steps and direct transport is $D^{\times} = 6/5$. A loop also needs a declared return map $\Gamma_{0\leftarrow 2}$.

connection data	$\Gamma_{0\leftarrow 2}$	H	$D^{\times}$	$\Gamma_{0\leftarrow 2}\Gamma_{2\leftarrow 0}$
self normalization only	$\frac{1}{6}$	1	$6/5$	$\frac{5}{6}$
reversible	$\frac{1}{5}$	$\frac{6}{5}$	$6/5$	1

$$\Gamma_{1\leftarrow 0} = 2, \quad \Gamma_{2\leftarrow 1} = 3, \quad \Gamma_{2\leftarrow 0} = 5$$

The first row has H = identity at this triangle, but fails its cocycle.

All transports are invertible rational scalars and transport from each chart to itself is one. The first row is permitted by the minimal interface. The second uses the inverse of the direct forward edge and therefore satisfies that part of reversibility.

In the first row, choose the return scalar $1/6$. Then

$$H = (1/6) \cdot 3 \cdot 2 = 1, \qquad \Gamma_{2\leftarrow 1}\Gamma_{1\leftarrow 0} - \Gamma_{2\leftarrow 0} = 1.$$

This triangle loop is the identity even though its cocycle equation fails. The return is not the inverse of the direct transport: $(1/6)5 \neq 1$. Thus one cannot transfer the reversible equivalence between defect and holonomy to the minimal interface.

In the reversible row, the return is $1/5$. Now the loop is $6/5$, and the additive defect of one is converted into the loop obstruction $1/5$ by that inverse return map.

Two safeguards

Global flatness in the minimal interface implies trivial loops by using its cocycle on the whole route, including the return. Separately, in the reversible interface, triviality of each triangle loop is equivalent to that triangle's cocycle. Neither assertion says that one isolated loop certifies all triangles.

> **Read this as:** Self normalization means $\Gamma_{a\leftarrow a} = I$. Reversible additionally means $\Gamma_{a\leftarrow b} = \Gamma_{b\leftarrow a}^{-1}$. These are different assumptions.

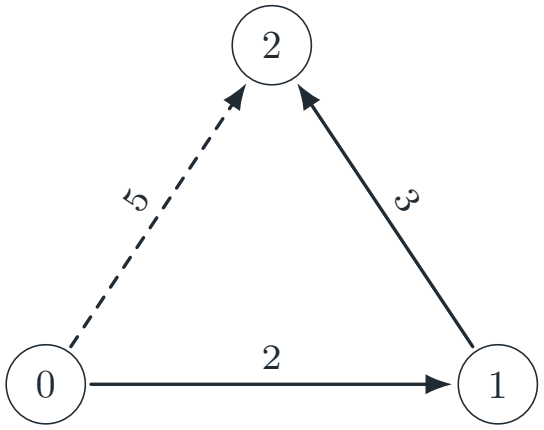
V12 / MODEL D / AN EXACT REVERSIBLE TRIANGLE

Measure the mismatch at its base

Traditional paper: Theorem 5.10 (p. 13), Section 9 (p. 20)

Retain the reversible scalar model with forward values $2, 3, 5$ and all reverse values given by inverses. Two routes from chart 0 to chart 2 multiply the input by 6 and 5, respectively. The additive discrepancy equals the original input.

reverse of direct edge: $1/5$



forward routes: 5 versus 6

loop input 1 returns $6/5$

loops	returned value
0	1
1	$\frac{6}{5}$
2	$\frac{36}{25}$
3	$\frac{216}{125}$
4	$\frac{1296}{625}$

The dashed direct edge is a specified comparison map, not a missing edge. The table shows the exact effect of traversing the same loop repeatedly. These are rational values of this example, not an analytic convergence claim.

The defect lives at the target of the forward routes:

$$\Delta_{2,1,0} = 3 \cdot 2 - 5 = 1.$$

The holonomy lives back at chart 0:

$$H_{2,1,0} = 5^{-1} \cdot 3 \cdot 2 = \frac{6}{5}, \qquad H - I = \frac{1}{5} = \Gamma_{0\leftarrow 2}\Delta.$$

For input 5, the direct output and the output after two steps are 25 and 30. Returning by $1/5$ gives 5 and 6. The target discrepancy of 5 has become a base discrepancy of 1. The types and locations of these differences matter even in an example in one dimension.

Because the return map is invertible, it cannot turn a nonzero defect into zero. This is the mechanism behind $\Delta = 0 \iff H = I$ in the reversible interface.

> **Read this as:** Holonomy compares paths after both have been returned to the same base fiber. It is the transported mismatch, not an untyped difference between outputs in different charts.

V13 / MODEL E / A SHEAR WITH NONTRIVIAL HOLONOMY

The identity can hide in the spectrum

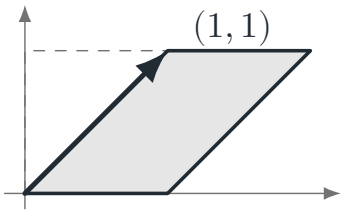
Traditional paper: Definition 5.9 (p. 12), Definition 7.4 (p. 16)

Put $\Gamma_{1\leftarrow 0} = H$, $\Gamma_{2\leftarrow 1} = I$, and $\Gamma_{2\leftarrow 0} = I$, with reverse maps given by inverses, where

$$H = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

The loop $0 \to 1 \to 2 \to 0$ then has exactly this shear as its holonomy.

The unit square under H



H

1	1
0	1

$H - I$

0	1
0	0

$$\operatorname{tr} H = 2, \quad \det H = 1, \quad (H - I)e_1 = e_0$$

The dashed square is the original set. The solid parallelogram is its exact image. The second coordinate basis vector moves to $(1, 1)^{\mathsf{T}}$. Its nonzero displacement is the first basis vector.

The usual scalar invariants can be misleading:

$$\operatorname{tr} H = 2, \qquad \det H = 1, \qquad \chi_H(t) = (t-1)^2.$$

These are also the invariants of the 2×2 identity matrix. Nevertheless,

$$\mathrm{Ob} = H - I = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \neq 0, \qquad \mathrm{Ob}\, e_1 = e_0.$$

A complete obstruction test must therefore retain the operator or a certificate strong enough to detect its nonzero action. Trace, determinant, and eigenvalue lists alone do not distinguish this loop from the identity.

What the picture certifies, and what it does not

The plotted corners are the images of the four corners of the unit square under the displayed matrix. The equation $\mathrm{Ob}\, e_1 = e_0$ is the exact witness. No smooth connection, curvature tensor, or continuum limit is used in this model. Its holonomy is the finite algebraic loop product. The traditional paper treats smooth transport separately under additional hypotheses.

> **Read this as:** Flatness is the equality $H = I$, not agreement of a short list of spectral summaries. A single explicit moved vector can disprove that equality.

V14 / MODEL E / CONJUGATION AND ITS LIMITS

Gauge changes the picture, not flatness

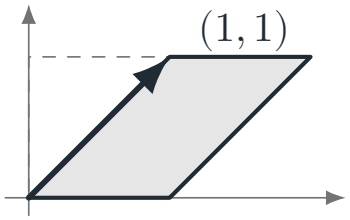
Traditional paper: Theorem 6.2 (p. 14)

At the base chart, change coordinates by $D = \operatorname{diag}(2, 1)$. Transform every connection edge by the prescribed endpoint gauges. The loop transforms by conjugation, giving

$$H^D = DHD^{-1} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}.$$

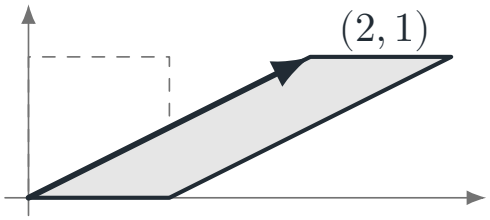
The shear looks stronger in these fixed numerical axes, but the verdict on flatness is unchanged by gauge.

H



$\|H - I\|_F^2 = 1$

$DHD^{-1}, \quad D = \operatorname{diag}(2, 1)$



$\|D(H - I)D^{-1}\|_F^2 = 4$

Both figures are exact images of a unit square under their displayed matrices. The Frobenius norms are computed in the indicated fixed coordinate bases. A nonorthogonal gauge need not preserve such a norm.

The rank of the obstruction is one on both sides. Trace, determinant, and characteristic polynomial are preserved under this conjugation. Most importantly,

$$D(H - I)D^{-1} = 0 \quad \Longleftrightarrow \quad H - I = 0.$$

An invertible coordinate change cannot turn a nonidentity loop into the identity. The fact that a particular residual norm changes does not contradict the invariance of exact vanishing.

The base chart determines the outer gauge

In a triangle product, adjacent intermediate gauges cancel:

$$(G_a \Gamma_{a \leftarrow c} G_c^{-1})(G_c \Gamma_{c \leftarrow b} G_b^{-1})(G_b \Gamma_{b \leftarrow a} G_a^{-1}) = G_a H G_a^{-1}.$$

Only the base gauge survives around the loop. For the additive defect, the surviving outer gauges instead belong to its different source and target fibers.

> **Read this as:** Exact nonvanishing is gauge invariant. The numerical size of a matrix entry or a chosen norm generally is not.

V15 / MODEL D / A FLAT CONNECTION WITH NONIDENTITY EDGES

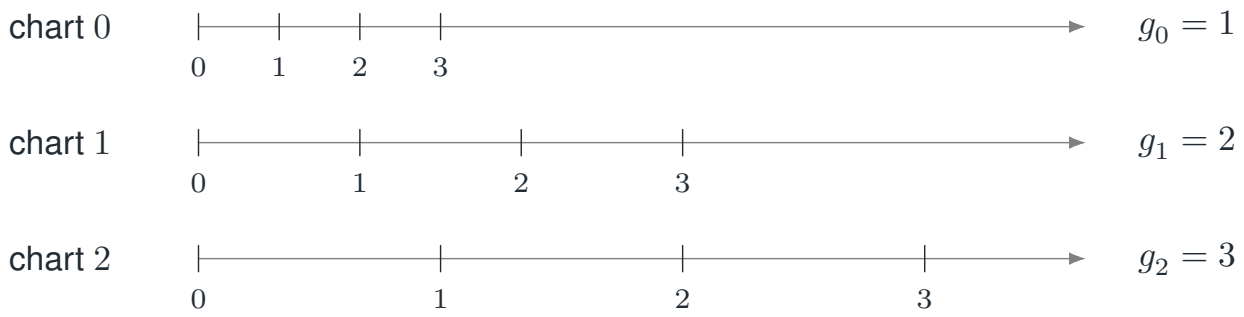
Flat is not the same as calibrated

Traditional paper: Theorem 6.3 (p. 14)

Make all three canonical charts literally identical, so $J_{b\leftarrow a} = 1$ over $\mathbb{Q}$. Choose scalar gauges $g_0 = 1$, $g_1 = 2$, and $g_2 = 3$, and define

$$\Gamma_{b\leftarrow a} = g_b g_a^{-1}.$$

The edge $0 \to 1$ is multiplication by two, although its canonical counterpart is the identity.



$$\Gamma_{1\leftarrow 0} = 2, \quad \Gamma_{2\leftarrow 1} = 3/2, \quad \Gamma_{2\leftarrow 0} = 3$$

The rulers display the chosen gauge scales: the marks are $g_a n$ for integer n. They visualize the supplied scalar factors, not an intrinsic metric. Relative transport is the ratio of the target and source scales.

The route through two steps $0 \to 1 \to 2$ has multiplier $(3/2)2 = 3$, equal to the direct route $0 \to 2$. More generally,

$$(g_c g_b^{-1})(g_b g_a^{-1}) = g_c g_a^{-1}$$

for every triple. Thus this connection is flat and reversible, but it is not calibrated to the original J, whose values are identities.

Recovering the gauge from one root

Choose a base chart o. The theorem constructs

$$G_a = \Gamma_{a\leftarrow o} J_{o\leftarrow a}, \qquad \Gamma_{b\leftarrow a} = G_b J_{b\leftarrow a} G_a^{-1}.$$

With $o = 0$ and $J = 1$, the recovered G_a are exactly $1, 2, 3$. The statement concerns the complete pairwise interface and its cocycle. It is not a topological claim about an arbitrary graph with missing direct comparisons.

> **Read this as:** Flatness makes the chosen transports independent of path. Calibration identifies them with one particular canonical transport before any gauge change.

V16 / MODEL F / PACKET BLOCKS FROM INPUT TO OUTPUT

Resolve the entire obstruction

Traditional paper: Theorem 7.5 (p. 17), Corollary 7.6 (p. 18)

Use three coordinate packets with projectors P_0, P_1, P_2, and set

$$H = I + 2E_{01} + 3E_{12}, \qquad \mathrm{Ob} = H - I.$$

Here H is invertible, so it can be the loop product of a reversible connection on three charts. Its entries outside the diagonal encode two effects that mix packets.

H

1	2	0
0	1	3
0	0	1

$\mathrm{Ob} = H - I$

0	2	0
0	0	3
0	0	0

	$\nu = 0$	$\nu = 1$	$\nu = 2$
$\mu = 0$	0	$2E_{01}$	0
$\mu = 1$	0	0	$3E_{12}$
$\mu = 2$	0	0	0

$$\mathrm{Ob}(4,5,6)^{\mathsf{T}} = (10,18,0)^{\mathsf{T}}$$

sum all nine blocks

The board on the right contains all nine actual embedded blocks $P_\mu \,\mathrm{Ob}\, P_\nu$. Row labels are output packets and column labels are input packets. Only the addresses $(0,1)$ and $(1,2)$ carry nonzero maps.

Completeness on both sides supplies the reconstruction:

$$\mathrm{Ob} = (P_0 + P_1 + P_2)\,\mathrm{Ob}(P_0 + P_1 + P_2) = \sum_{\mu,\nu} P_\mu \,\mathrm{Ob}\, P_\nu.$$

For $x = (4,5,6)^{\mathsf{T}}$, the input in packet 1 contributes $10e_0$ and the input in packet 2 contributes $18e_1$. Their sum is the full displacement $\mathrm{Ob}\, x$.

A block becomes a witness when it acts

The nonzero block $2E_{01}$ has the explicit source vector e_1, supported entirely in packet 1, and $P_0 \,\mathrm{Ob}\, e_1 = 2e_0 \neq 0$. Similarly, $P_1 \,\mathrm{Ob}\, e_2 = 3e_1 \neq 0$. The general theorem supplies such a witness classically when a nonzero block is known. The example displays it concretely.

> **Read this as:** An obstruction packet is not just a colored matrix cell. It consists of a specific label for a pair of packets and an exact nonvanishing statement about the corresponding map.

V17 / MODEL E / COMPLETENESS, TWO SIDES, AND OBLIQUE PACKETS

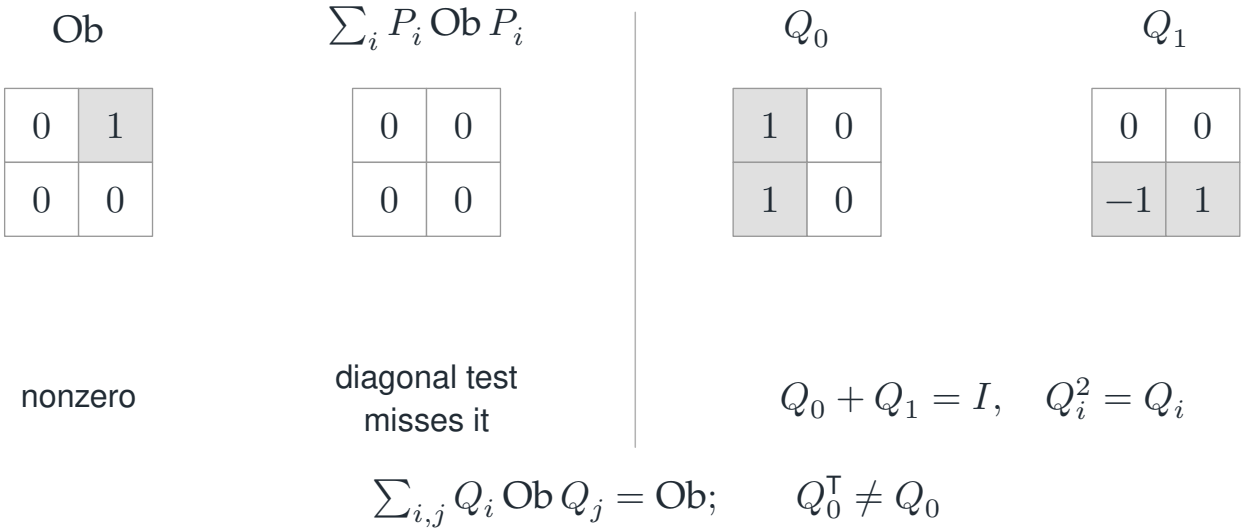
Why diagonal checks are insufficient

Traditional paper: Theorem 7.5 (p. 17)

Return to $\mathrm{Ob} = E_{01}$. With the two standard coordinate projectors, both diagonal blocks vanish:

$$P_0 \,\mathrm{Ob}\, P_0 = P_1 \,\mathrm{Ob}\, P_1 = 0.$$

The obstruction nevertheless survives in the block outside the diagonal $P_0 \,\mathrm{Ob}\, P_1 = E_{01}$.



The test of diagonal blocks alone on the left misses the whole obstruction. On the right, the displayed oblique projectors still form a complete exact resolution. They need not be orthogonal projections for the Euclidean inner product.

There is a second failure mode. Supplying only P_0 gives a family of idempotents but not a complete one. The single tested block is zero, while the omitted packet carries the missing input. The implication "all tested blocks vanish, therefore the operator vanishes" requires the completeness hypothesis.

Exact reconstruction does not require right angles

For the displayed Q_0, Q_1,

$$Q_0 + Q_1 = I, \qquad Q_i^2 = Q_i, \qquad Q_0 Q_1 = Q_1 Q_0 = 0,$$

but $Q_0^{\mathsf{T}} \neq Q_0$. Expanding both copies of I still proves

$$\sum_{i,j} Q_i \,\mathrm{Ob}\, Q_j = \mathrm{Ob}\,.$$

The block reconstruction identity itself uses the condition that the projectors sum to the identity. Idempotence is additionally used when constructing a witness that remains supported in one packet after projection. The release does not require pairwise annihilation for its general reconstruction statement.

> **Read this as:** Test every block from input to output of a complete resolution. Testing only the diagonal, or only the packets supplied so far, answers a weaker question.

V18 / MODEL E / FIXED VERSUS TRANSPORTED PROJECTORS

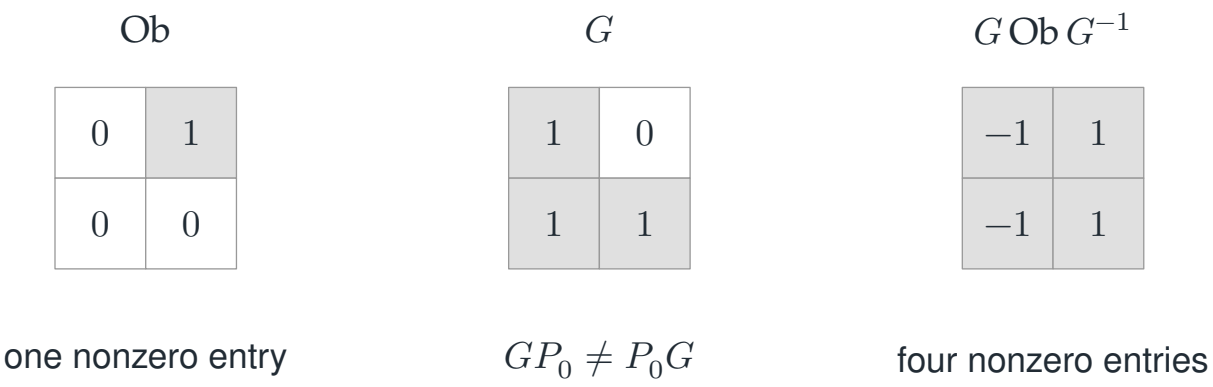
A gauge can move every packet label

Traditional paper: Proposition 7.7 (p. 18)

The diagonal gauge on V14 commuted with each standard projector. Now use the gauge that mixes packets

$$G = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad \mathrm{Ob} = E_{01}.$$

The total obstruction transforms by conjugation, but its pattern in the *fixed* standard packets changes.



The first obstruction has one nonzero entry. Its conjugate has four. Both have rank one and are nonzero. The change in fixed block pattern is not a change in whether the connection is flat.

Direct multiplication gives

$$G\,\mathrm{Ob}\,G^{-1} = \begin{pmatrix} -1 & 1 \\ -1 & 1 \end{pmatrix}.$$

For example, the original $(0,0)$ block vanishes, while the new $(0,0)$ block relative to the old coordinate projectors does not. This is why the theorem about preservation of each block's zero status requires $GP_i = P_iG$.

Transporting the packet frame as well

Define $Q_i = GP_iG^{-1}$. Then the identities cancel exactly:

$$Q_i(G\,\mathrm{Ob}\,G^{-1})Q_j = G(P_i\,\mathrm{Ob}\,P_j)G^{-1}.$$

Relative to these transported packets, the corresponding labeled blocks again have matching vanishing status. These are the same oblique projectors displayed on V17. One must distinguish moving the operator alone in a fixed resolution from moving the entire resolved description.

> **Read this as:** The total obstruction is intrinsic under gauge. Its location in a fixed packet grid is not, unless the gauge preserves that grid.

V19 / MODEL G / THE QUANTIFIERS ARE THE CONTENT

An obstruction rejects one candidate

Traditional paper: Theorem 7.2 (p. 15)

Let the candidates be rational numbers and the packet labels be $0, 1$. Define

$$r(x, 0) = x^2 - 1, \qquad r(x, 1) = x - 1.$$

A candidate is a solution when both residuals vanish. The five candidate columns below are a display of this explicit residual function, not a search over an unspecified class.

candidate x	-2	-1	0	1	2
$r(x, 0) = x^2 - 1$	3	0	-1	0	3
$r(x, 1) = x - 1$	-3	-2	-1	0	1
	rejected	rejected	rejected	solution	rejected

$$x = -1 : \quad r(-1, 0) = 0, \quad r(-1, 1) = -2$$

A shaded cell is a nonzero exact residual. At $x = -1$, the first equation passes but the second supplies an obstruction. At $x = 1$, both pass. The rejected candidate does not obstruct the successful one.

The statement proved by the generic certificate has the form

$$\neg(\forall p,\ r(x, p) = 0) \quad \Longleftrightarrow \quad \exists p,\ r(x, p) \neq 0.$$

The same candidate x appears on both sides. A certificate consists of the packet label and its nonvanishing proof. The general converse uses classical reasoning and does not itself provide an algorithm to find the label.

What would be required to rule out all solutions?

One would need the stronger assertion $\forall x \exists p,\ r(x, p) \neq 0$. The value $x = 1$ disproves that assertion in this example. A witness for $x = -1$ supplies no evidence for it. Likewise, finding an obstruction block for one triangle of one chosen connection does not classify every possible connection.

The term *obstruction* in this generic interface does not automatically mean a cokernel class, a cohomology group, a norm bound, or a minimal certificate. Those require additional constructions and theorems.

> **Read this as:** Keep the candidate attached to the witness. The theorem's strength is its exact logical scope, not a universal existence or nonexistence claim.

V20 / MODEL H / ACTUAL SYNTAX, ARBITRARY SEMANTICS

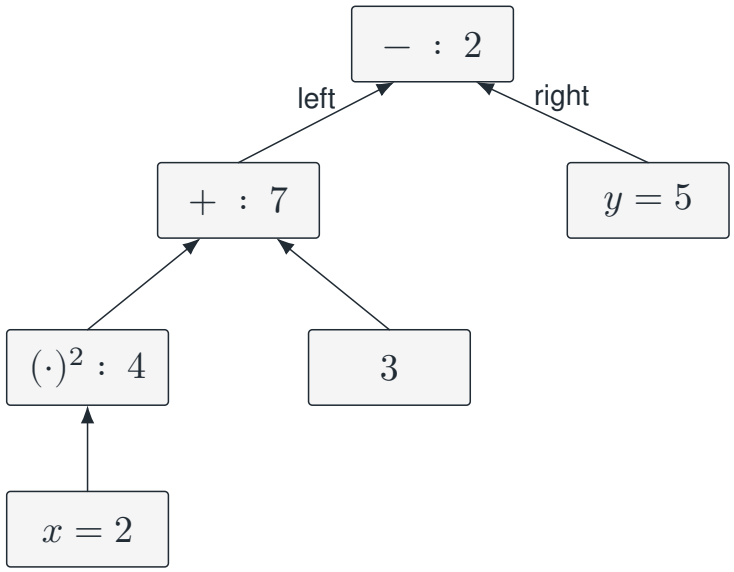
A tree reads its named leaves

Traditional paper: Definition 11.1 (p. 22), Theorem 11.2 (p. 22)

Use the grammar's constant, input, unary, and binary constructors to build

$$t = (x^2 + 3) - y.$$

Here "square," "add," and "subtract" are explicitly supplied meanings of operation symbols. At $x = 2, y = 5$, every node value can be evaluated exactly.



$\text{inputs}(t) = \{x, y\}$

unused input: -100 or 100

same value: 2

$x - x : \{x\}$ syntactically

but always zero

The rooted tree records the order of input slots. Arrows indicate evaluation from children to parent, not chart transport. An environment may contain additional variables, but this tree does not read them.

The input set is $\text{inputs}(t) = \{x, y\}$. If two environments agree at x and y, structural induction proves that the two evaluations agree: input leaves agree, constants agree, and every parent applies the same supplied operation to equal child values.

Syntactic locality is not minimal dependence

The tree $x - x$ has input set $\{x\}$, although its value is always zero under ordinary subtraction. Locality provides a sufficient agreement condition. It does not compute the smallest set of semantically relevant variables. Nor does it prove that two different trees are equivalent.

The semantics may act on arbitrary values. For example, a binary concatenation symbol applied to strings "ab" and "cd" gives "abcd." No linearity or differentiability is required. Conversely, swapping the ordered inputs of the subtraction root above changes the value from 2 to -2.

> **Read this as:** The proved tree theorem controls which input labels are read. It does not supply a nonlinear chain rule, a stability estimate, or a solution of a tree equation.

V21 / MODEL H / EXACT COORDINATES VERSUS A CHOSEN RULE

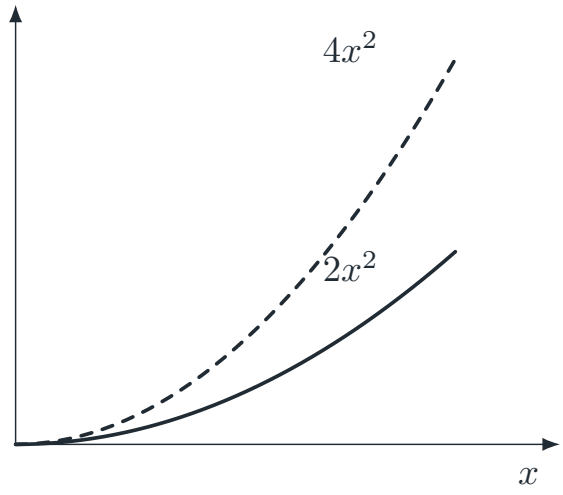
Where a nonlinear defect can enter

Traditional paper: Proposition 10.1 (p. 20), Formalization target 11.7 (p. 24), Theorem 20.2 (p. 42)

Take the unary operation $f(x) = x^2$ over $\mathbb{Q}$. Suppose both input and output transport are chosen as multiplication by two, while the same numerical formula f is reused at the target. The two routes give

$$2f(x) = 2x^2, \qquad f(2x) = 4x^2.$$

Their difference is $-2x^2$, which is generally nonzero.



x	0	1	2
$2f(x)$	0	2	8
$f(2x)$	0	4	16
$2f(x) - f(2x)$	0	-2	-8

correct chart formula: $f_b(v) = v^2/2$

Solid: $2x^2$. Dashed: $4x^2$. The curves are a real visualization of exact polynomial formulas. The table contains exact evaluations over the rationals. The mismatch is caused by the specified combination of transport and formula.

For a canonical coordinate change with input and output chart $v = 2x$, the target formula is not still v^2. It is

$$\hat{f}_b(v) = 2f(v/2) = \frac{v^2}{2}.$$

Consequently $\hat{f}_b(2x) = 2x^2$, and the exact coordinate defect vanishes. This is the same distinction illustrated in the example in two dimensions on V10.

An example is not a general propagation theorem

The polynomial calculation here is exact and checked symbolically. It explains a chosen node defect, while the traditional paper develops composition identities, iterated defects, fixed point stability, and derivative defects under explicit hypotheses. Those broader ordinary proofs must be read on their own terms. This one example and the finite Lean locality theorem do not certify them.

> **Read this as:** A nonzero nonlinear route difference must come from identifiable extra data: a changed transport, an approximation, a restricted model, or a reused formula that was not conjugated correctly.

V22 / FINITE EXAMPLES / WHY MULTILINEARITY MATTERS

Mixed terms are not optional

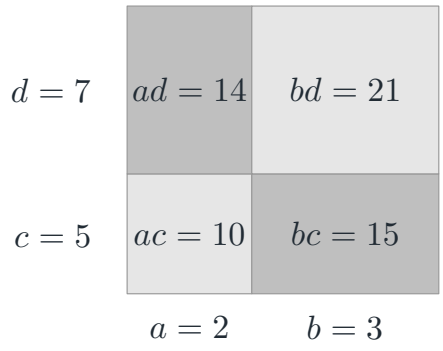
Traditional paper: Formalization target 11.6 (p. 23), Theorem 18.3 (p. 38)

A bilinear node distributes over two independently resolved inputs:

$$(a+b)(c+d) = ac + ad + bc + bd.$$

The four terms correspond to all ordered choices of one input packet from each slot. Keeping only matching packet labels loses the mixed terms.

$$(2+3)(5+7) = 60$$



Diagonal pieces are not the whole product

$$ac + bd = 10 + 21 = 31$$

$$ad + bc = 14 + 15 = 29$$

$$(2+3)^2 = 25 \neq 2^2 + 3^2 = 13$$

missing square term: $2 \cdot 2 \cdot 3 = 12$

The rectangle has width $2 + 3$ and height $5 + 7$, with differently scaled horizontal and vertical display units. Each cell is labeled by its actual product. The lighter diagonal pair contributes only 31 of the total 60.

For a unary square, inserting $a + b$ does not give $a^2 + b^2$. The extra $2ab$ arises because the operation is nonlinear in its single argument. It can be analyzed by representing the square using a binary multiplication node with a repeated input, but that is an additional choice of syntax and semantics.

The broader tree theorem needs its hypotheses

General packet tree expansion needs correctly typed input slots, finite sets of packet labels at each node, and multilinearity in every slot used for distributive expansion. Empty inputs and unary nodes need explicit conventions. Slot order must survive even when values or operations do not commute. The traditional paper supplies rooted tree reconstruction and pruning results in its ordinary mathematical development.

This page supplies elementary exact examples of the issue. The finite v1.0.1 Lean contract remains narrower: it contains syntactic input locality, not the general dependent tree expansion or the traditional paper's analytic convergence results.

> **Read this as:** A branching picture does not justify throwing away mixed interactions. Exact packet expansion requires the appropriate multilinearity and typing, not merely a diagram that appears finite.

V23 / CAPSTONE / A COMPLETE FINITE DIAGNOSIS

From loop data to a packet witness

Traditional paper: Theorem 5.10 (p. 13), Theorem 6.2 (p. 14), Corollary 7.6 (p. 18)

Supply a reversible connection on three copies of $\mathbb{Q}^2$ by choosing

$$\Gamma_{1\leftarrow 0} = A, \qquad \Gamma_{2\leftarrow 1} = B, \qquad \Gamma_{2\leftarrow 0} = C = I,$$

and declaring every reverse edge to be the inverse. This time both nontrivial forward matrices are retained.

A

1	1
0	1

B

1	0
1	1

$H = BA$

1	1
1	2

$H - I$

0	1
1	1

$$e_0 = (1,0) \;\mapsto\; (1,0) \;\mapsto\; (1,1) \;\mapsto\; (1,1)$$

$$P_1(H - I)e_0 = e_1 \neq 0$$

---

replace direct edge $C = I$ by $C = BA$: then $C^{-1}BA = I$

The vector row starts with the source witness e_0 and follows it through A, then B, then C^{-1}. Returning to the base produces $(1,1)^{\mathsf{T}}$, whose displacement from the input is e_1.

The calculation gives

$$\Delta = BA - C = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}, \qquad H = C^{-1}BA = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}.$$

Choose the standard packet projectors. The $(1,0)$ block is nonzero, and the explicit candidate vector satisfies $P_1(H - I)e_0 = e_1$. This proves nonflatness of the supplied connection at the supplied triangle.

Repairing data is not the same as changing gauge

Replacing the direct map $C = I$ by $C = BA$, and updating its reverse, makes the new loop equal to I. That changes the connection. It cannot be a gauge of the original nonflat connection, because gauge preserves whether holonomy is the identity. For this complete example on three charts all cocycles then hold. Changing one edge in a larger network would require checking the other affected loops as well.

> **Read this as:** Declare the data, multiply in traversal order, form the defect, resolve all blocks, and exhibit an input on which one block acts nontrivially. Each step has a checkable mathematical object.

V24 / EVIDENCE / EXAMPLES, PROOFS, AND FORMAL SCOPE

Read the evidence with the pictures

Traditional paper: Section 12 (p. 24)

This guide preserves the 24 worked pages and 22 exact figures from the finite visual appendix. It updates their references to the final traditional paper and explains the relation to that paper's broader nonlinear and analytic material.

Read each evidence record in its scope

Evidence	What it supports
Finite Lean release	The frozen declarations under their original hypotheses and pinned foundations. The Lean guide records the precise theorem map, source identity checks, and inherited build evidence. No fresh kernel compilation is claimed here.
Ordinary mathematics	The traditional paper's expanded proofs, including nonlinear, obstruction, lifting, and smooth transport results. Inclusion in that paper does not add these results to the finite Lean contract.
Exact model checks	Fresh replay on 8 October 2026 passed all 124 exact checks and rejected all four deliberately false controls. The scope is the displayed finite matrices, paths, polynomials, candidate tables, and trees.
Figure regeneration	All 22 editable TikZ figures regenerated from the preserved exact model data. Labels and spacing were refined for this edition. Their current sources match the regenerated output. The original figure sources remain separately preserved.

Separately, eleven mixed exact integer and numerical verification groups passed in a fresh run of the preserved strengthened source and checker, not the newly edited final TeX. Those checks have their stated finite ranges and numerical tolerances. They do not certify every statement in the monograph.

Boundaries that remain in force

The formal edge system uses a single scalar coefficient semiring. The heterogeneous path theorem for arbitrary lengths uses a common packet index, with forms allowing changing types for two and three factors. General dependent tree expansion and analytic convergence lie outside this finite formal contract. The traditional paper states and proves its broader results with their own hypotheses.

The generic obstruction theorem rejects its named candidate. A nonzero holonomy block rejects flatness for its named loop data. Neither conclusion supplies a universal classification, a new arithmetic realization, or an algorithmic search guarantee.

Replay the worked layer

From the editable visual guide directory, compile the PDF with `python build.py`.

Check the exact examples with `python checks/verify_examples.py`.

Regenerate the figures with `python source/generate_graphics.py`.

Use a temporary copy to retain the delivered evidence unchanged. The full package preserves the finite release and its original formal sources separately.

> **Read this as:** Use the figures to understand the mechanisms, the traditional paper to read the mathematical proofs, and the Lean guide to identify the exact formal contract. These are related records with distinct scopes.

PAPER III

Projector Resolved Seed Calculus III

Typed transfer, nonlinear packets, and obstruction theory

Read the exact source declarations, follow the map directions, and distinguish the preserved kernel evidence from current static checks.

Komorebi P.

Lean and evidence guide • 8 October 2026

How to use this volume

A practical guide to the complete 41-row finite algebraic contract. Read source-exact proof excerpts, follow the direction of each map, distinguish exact coordinate transport from independent connection data, and locate every formal target.

Evidence at this edition. The preserved 3 September release reports 143 mapped declarations and a namespace audit of 468 entries. Its kernel transcript is included and reconciled with the sources. Fresh checks here establish static consistency and source identity. No new Lean compiler execution is claimed.

Contents

1 Reading the formal paper 1
- 1.1 Three levels of evidence . . . 1
- 1.2 A route through the source . . . 1
- 1.3 Notation that carries mathematical content . . . 2
- 1.4 What a short proof is doing . . . 2

2 Edges, words, and reachable packets 3
- 2.1 Graph orientation and finite enumeration . . . 3
- 2.2 Lists record chronological traversal . . . 4
- 2.3 From a transfer power to a finite walk sum . . . 5
- 2.4 Endpoint coefficients retain the empty path . . . 6
- 2.5 What the support theorem implies . . . 7

3 Heterogeneous products and exact coordinates 8
- 3.1 What varies in a heterogeneous path . . . 8
- 3.2 Arbitrary finite length on one packet type . . . 9
- 3.3 Declared support is a certificate . . . 9
- 3.4 Exact frames point from coordinates to ambient space . . . 10
- 3.5 Exact coordinate systems point the other way . . . 11
- 3.6 The naturality square is a typed equality . . . 12

4 Connections, defects, and holonomy 13
- 4.1 The minimal interface separates a choice from a theorem . . . 13
- 4.2 A counterexample to automatic flatness in the source . . . 13
- 4.3 Reversible connections use dependent fibers . . . 14
- 4.4 The identity between defect and holonomy . . . 15
- 4.5 Finite paths and their endpoint types . . . 16

5 Gauge reconstruction and packet obstruction blocks 17
- 5.1 The gauge action fixes its own convention . . . 17
- 5.2 Reconstructing a flat connection from canonical transport . . . 18
- 5.3 A packet resolution need not be orthogonal . . . 19
- 5.4 Extracting a witness supported in a packet . . . 20
- 5.5 When a fixed packet pattern survives a gauge . . . 21

6 Operation trees and certificates for individual candidates 22

6.1 The grammar does not assume algebraic laws . . . 22
6.2 The locality proof follows the syntax . . . 22
6.3 What the nonlinear extension adds outside this contract . . . 23
6.4 Residuals and a fixed proposed solution . . . 23
6.5 The quantifiers determine the conclusion . . . 24

7 Evidence, trust, and replay 25
7.1 The pinned environment is part of the claim . . . 25
7.2 A map gate and a namespace gate answer different questions . . . 25
7.3 Reading the inherited transcript . . . 26
7.4 The inherited Papers I–II references . . . 26
7.5 Replay from a preserved source copy . . . 27
7.6 Rebuilding this guide . . . 27
7.7 The edition's verification boundary . . . 27

8 The complete 41-row formal contract 28
8.1 Row 1: Packet graph and scalar packet matrix . . . 28
8.2 Row 2: Typed edge system, elementary edge operator, and finite transfer operator . . . 28
8.3 Row 3: Edge support outside the target-source block vanishes . . . 28
8.4 Row 4: Scalar packet multiplication and the two-edge composition dichotomy . . . 29
8.5 Row 5: Composable edge words and their operators . . . 29
8.6 Row 6: Broken and noncomposable words vanish . . . 29
8.7 Row 7: Recursive, tuple, explicit-word, and walk expansions . . . 29
8.8 Row 8: Successor laws for explicit words . . . 30
8.9 Row 9: Transfer powers equal recursive, explicit-word, and composable-walk expansions . . . 30
8.10 Row 10: Endpoint block and coefficient of a composable word . . . 30
8.11 Row 11: Fixed-endpoint coefficient sum . . . 31
8.12 Row 12: Fixed-length and global reachability obstructions . . . 31
8.13 Row 13: Fully typed two- and three-step packet expansions . . . 31
8.14 Row 14: Arbitrary-length heterogeneous factor expansion . . . 32
8.15 Row 15: Supported heterogeneous paths and vanishing . . . 32
8.16 Row 16: Canonical homogeneous linear coordinate transport . . . 32
8.17 Row 17: Exact self, cocycle, round-trip, and triangle laws . . . 32
8.18 Row 18: Exact finite path transport telescopes . . . 33
8.19 Row 19: Set-level exact coordinate transport . . . 33
8.20 Row 20: Representation, composition, and naturality of arbitrary functions . . . 34
8.21 Row 21: Minimal homogeneous noncanonical connection and its defects . . . 34
8.22 Row 22: Flat minimal connections have trivial defect and triangle holonomy . . . 34
8.23 Row 23: Explicit three-point connection with nontrivial loop . . . 35
8.24 Row 24: Heterogeneous reversible connection and gauges . . . 35
8.25 Row 25: Reversible cocycle, flatness, additive defect, holonomy, and obstruction . . . 35
8.26 Row 26: Defect-holonomy identity and equivalent flatness criteria . . . 36
8.27 Row 27: Canonical reversible frame connection is flat . . . 36
8.28 Row 28: Every flat reversible connection is gauge-equivalent to canonical transport . . . 36
8.29 Row 29: List-based reversible path transport is endpoint-only when flat . . . 37
8.30 Row 30: Indexed finite path transport and flat loops . . . 37
8.31 Row 31: Gauge action, covariance, and invariance . . . 37
8.32 Row 32: Finite complete idempotent packet resolution and blocks . . . 38

8.33 Row 33: Exact block reconstruction, zero detection, and packet witnesses 38
8.34 Row 34: Triangle-obstruction block reconstruction and four-way detection 38
8.35 Row 35: Gauge conjugation and packet preservation 38
8.36 Row 36: Packet-preserving conjugation acts blockwise and preserves vanishing 39
8.37 Row 37: Gauge covariance of total and packetwise triangle obstructions 39
8.38 Row 38: Nonlinear operation-tree grammar, evaluation, and input support 39
8.39 Row 39: Operation-tree packet locality 40
8.40 Row 40: Candidate-specific solution and obstruction packet 40
8.41 Row 41: Candidate failure is equivalent to a local obstruction 40

A Monograph navigation 41

B Inherited declaration names 43

C Imports and exact dependency revisions 44

D Source and reproduction inventory 45

1 Reading the formal paper

Paper III turns three kinds of data into precise finite statements: directed packet operators, coordinate and connection transport, and local obstruction certificates. Its Lean component is small enough to read as a connected development. Seven mathematical modules contain the authors' constructions and proofs. An aggregate import exposes them together. A separate audit module checks the compiled namespace.

This guide follows the definitions before using their theorem names. A declaration name is a precise destination, but its type determines what has actually been proved. The authoritative map has 41 rows and 143 distinct targets. Of these, 88 are theorem declarations and 55 are constructions. A construction can be a structure, inductive type, definition, or abbreviation. The count does not mean 143 independent mathematical theorems.

1.1 Three levels of evidence

The preserved report, dated 3 September 2026, records a successful aggregate build of 3,076 jobs, the gate that checks parity with the theorem map, and a namespace audit of 468 declarations including 229 entries of kind `theoremInfo`. Paper III also preserves the kernel audit transcript. That transcript contains one axiom report for each of the 143 mapped targets and the final namespace PASS line.

For this edition, a Python reader reconciled all 41 map rows with their lexical source locations, all 143 explicit `#print axioms` directives, and all 143 transcript records. It checked the 16 files in the preserved formal component against the original checksum manifest. The theorem map and transcript hashes also match the values printed in the inherited verification report. These are fresh static checks of the preserved evidence. They do not replace a new Lean execution, and no fresh compiler run is claimed by this guide.

The expanded traditional companion includes additional ordinary mathematical proofs. The finite v1.0.1 formal contract stays fixed. A section of the final paper may combine a formally covered finite result with a stronger ordinary theorem. The presence of a related Lean name does not make every statement in that section part of the contract.

1.2 A route through the source

Module	Reading objective
`EdgeOperators`	Orientation, broken-word vanishing, and finite walk expansion.
`EdgeExtensions`	Endpoint blocks, reachability, and heterogeneous factor types.
`Transport`	Exact frames, the minimal connection, and a nontrivial loop.
`CoordinateNaturality`	Exact coordinate representation of arbitrary functions.
`ReversibleConnections`	Dependent fibers, defect, holonomy, gauges, paths, and blocks.
`HolonomyBlocks`	The commutation hypothesis for packetwise gauge covariance.
`NonlinearObstructions`	Tree locality and candidate-specific obstruction certificates.

The name of a file and the namespace of its declarations need not coincide. For example, the basic packet resolution declarations occur at the end of `ReversibleConnections.lean` under `Komorebi.PaperIII.HolonomyBlocks`. The complete map gives both the fully qualified name and the file and line. Follow that location when searching a source checkout.

1.3 Notation that carries mathematical content

Lean writes a linear map as `M →ₗ[R] N` and a linear equivalence as `M ≃ₗ[R] N`. The latter includes an inverse and its inverse laws. An arbitrary equivalence is `X ≃ Y`. A function `X → Y` carries no linear structure. Replacing one of these types with another changes the claim.

Bracketed arguments such as `[Semiring R]` are typeclass hypotheses. They supply operations and laws, not merely notation. Braced arguments such as `\{R : Type u\}` are implicit parameters inferred from use. `Type u` allows the object to live in a universe. Different universe names allow different input, fiber, and coefficient types. `autoImplicit false` requires undeclared identifiers to be rejected rather than silently generalized.

`Fintype V` supplies finite enumeration. `DecidableEq V` supplies a decision procedure for equality. Some definitions use `classical` so that a finite sum can be filtered by a proposition without demanding a decision procedure supplied by the user. This logical choice does not turn an abstract graph theorem into an efficient graph algorithm.

Three composition conventions must remain distinct:

Written expression	Execution order
`B * A`	Matrix A acts first, then B.
`g.comp f`	Linear map f acts first, then g.
`e.trans f`	Equivalence e acts first, then f.

A calculation translated from matrices to `LinearEquiv.trans` therefore reverses the written composition pattern. The underlying maps still execute in the same order.

1.4 What a short proof is doing

`rfl` closes a definitional equality: the two sides reduce to the same expression. `ext x` or `LinearEquiv.ext` reduces equality of maps to equality at an arbitrary input. `simp` uses definitions and registered simplification rules, including inverse laws and zero laws. `rw` rewrites by an explicit equality. `induction` exposes a finite recursive object, such as a list or tree, and proves the conclusion from the smaller objects.

The printed excerpts are exact source selections, not standalone files. Implicit variables, local instances, namespaces, and earlier lemmas may be outside a selection. Every exact UTF-8 excerpt has a source path, inclusive line range, and hashes in `generated/excerpts.json`. Copy the adjacent `.lean.txt` file when exact tokens matter. The PDF display uses font fallbacks for otherwise unsupported characters.

2 Edges, words, and reachable packets

2.1 Graph orientation and finite enumeration

`PacketGraph V E` has two fields, `source : E → V` and `target : E → V`. Distinct values of `E` may share endpoints, so parallel edges are allowed. The structure itself does not carry a finite enumeration. The theorems that sum over vertices or edges request the appropriate `Fintype` instances separately. This makes the boundary visible in each declaration.

The elementary scalar block is the matrix whose only potentially nonzero entry is in target row and source column.

Source: `KomorebiPaperIII/EdgeOperators.lean`.

```
def packetMatrix [Zero R] [DecidableEq V]
    (source target : V) (coefficient : R) : Matrix V V R :=
  fun row column ↦ if row = target ∧ column = source then coefficient else 0

@[simp] theorem packetMatrix_apply [Zero R] [DecidableEq V]
    (source target row column : V) (coefficient : R) :
    packetMatrix source target coefficient row column =
      if row = target ∧ column = source then coefficient else 0 :=
  rfl
```

For a column vector x, this block sends the source coordinate to the target coordinate and multiplies it by the stored coefficient. `EdgeSystem G R` is a coefficient function `E → R`. It adds weights to graph edges without changing their endpoints.

If $e : a \to b$ has coefficient r and $f : b \to c$ has coefficient s, the product for the traversal e followed by f is $E_f E_e$, and its coefficient is sr. A semiring need not be commutative. That coefficient cannot in general be replaced by rs.

Source: `KomorebiPaperIII/EdgeOperators.lean`.

```
theorem edgeOp_mul_edgeOp_of_composable
    [Fintype V] [DecidableEq V] [Semiring R]
    (e f : E) (h : G.target e = G.source f) :
    edgeOp G S f * edgeOp G S e =
      packetMatrix (G.source e) (G.target f)
        (S.coefficient f * S.coefficient e) := by
  ext row column
  by_cases hrow : row = G.target f
  · by_cases hcolumn : column = G.source e
    · simp [edgeOp, packetMatrix, Matrix.mul_apply, h, hrow, hcolumn]
    · simp [edgeOp, packetMatrix, Matrix.mul_apply, h, hcolumn]
  · simp [edgeOp, packetMatrix, Matrix.mul_apply, hrow]
```

The proof examines one matrix entry. The sum over intermediate vertices has at most one surviving term after support conditions are imposed. If the interface does not match, the companion theorem `edgeOp_mul_edgeOp_eq_zero` shows every intermediate term is zero. The zero conclusion depends on support, not on an assumed numerical cancellation.

2.2 Lists record chronological traversal

`Composable G` accepts the empty list and every singleton. A longer list must satisfy equality between the target of one edge and the source of the next at each step. The evaluator then places the first listed edge on the right:

Source: `KomorebiPaperIII/EdgeOperators.lean`.

```
def wordOp [Fintype V] [DecidableEq V] [Semiring R]
    (G : PacketGraph V E) (S : EdgeSystem G R) : List E → Matrix V V R
  | [] => 1
  | e :: rest => wordOp G S rest * edgeOp G S e

@[simp] theorem wordOp_nil [Fintype V] [DecidableEq V] [Semiring R] :
    wordOp G S [] = 1 :=
  rfl

@[simp] theorem wordOp_singleton [Fintype V] [DecidableEq V] [Semiring R]
    (e : E) : wordOp G S [e] = edgeOp G S e := by
  simp [wordOp]
```

Thus `[e,f,g]` evaluates to $E_g E_f E_e$. The empty list evaluates to the identity matrix, including when the finite vertex type is empty. The formal definitions cover the empty case directly.

The concatenation law gives a quick way to check the orientation:

Source: `KomorebiPaperIII/EdgeOperators.lean`.

```
theorem wordOp_append [Fintype V] [DecidableEq V] [Semiring R]
    (first second : List E) :
    wordOp G S (first ++ second) = wordOp G S second * wordOp G S first := by
  induction first with
  | nil => simp
  | cons e rest ih =>
      simp [wordOp, ih, Matrix.mul_assoc]
```

The operator for `first ++ second` is the operator for `second` multiplied by the operator for `first`. The proof is list induction plus associativity. Its order is essential in a noncommutative coefficient system.

The next theorem removes every word with a broken interface, not just a break at the start.

Source: `KomorebiPaperIII/EdgeOperators.lean`.

```
theorem wordOp_eq_zero_of_not_composable
    [Fintype V] [DecidableEq V] [Semiring R]
    (word : List E) (h : ¬ Composable G word) :
    wordOp G S word = 0 := by
  induction word with
  | nil => simp [Composable] at h
  | cons e rest ih =>
      cases rest with
      | nil => simp [Composable] at h
      | cons f tail =>
```

```
          simp only [Composable, not_and_or] at h
          rcases h with hbreak | htail
          · exact wordOp_eq_zero_of_first_break G S e f tail hbreak
          · change wordOp G S (f :: tail) * edgeOp G S e = 0
            rw [ih htail]
            simp
```

The induction distinguishes a mismatch in the first pair from a mismatch in the tail. The former invokes the support result for two edges. The latter uses the induction hypothesis to make the tail operator zero. Empty and singleton words cannot satisfy the noncomposability hypothesis, so those branches eliminate a contradiction.

2.3 From a transfer power to a finite walk sum

The transfer operator is $T = \sum_{e \in E} E_e$. Four related objects are kept separate:

1. `wordExpansion` recursively multiplies on the left by T.
2. `tupleWordOp` converts a tuple `Fin n → E` to an execution list.
3. `explicitWordExpansion` sums over every tuple of length n.
4. `walkExpansion` keeps precisely the composable tuples.

At length zero there is one empty tuple, not zero tuples. Its contribution is the identity. This is why the explicit sum agrees with T^0.

To pass from length n to length $n+1$, the proof separates a tuple into its first n entries and its final entry. `Fin.snocEquiv` supplies the finite indexing equivalence. Appending the final edge places its operator to the left of the prefix operator.

Source: `KomorebiPaperIII/EdgeOperators.lean`.

```
theorem explicitWordExpansion_succ
    [Fintype E] [Fintype V] [DecidableEq V] [Semiring R]
    (n : ℕ) :
    explicitWordExpansion G S (n + 1) =
      transferOp G S * explicitWordExpansion G S n := by
  unfold explicitWordExpansion
  rw [← (Fin.snocEquiv (fun _ : Fin (n + 1) ↦ E)).sum_comp]
  rw [Fintype.sum_prod_type]
  change (∑ final : E, ∑ prior : Fin n → E,
    tupleWordOp G S (n + 1) (Fin.snoc prior final)) = _
  simp only [tupleWordOp_snoc]
  unfold transferOp
  rw [Finset.sum_mul]
  simp_rw [Finset.mul_sum]
```

This is a finite distributivity argument. It uses neither an infinite geometric series nor a convergence limit. The proof then identifies the recursive expansion with the explicit tuple sum by induction on n.

Pruning the tuple sum uses the theorem for words with a broken interface, term by term:

Source: `KomorebiPaperIII/EdgeOperators.lean`.

```
theorem explicitWordExpansion_eq_walkExpansion
    [Fintype E] [Fintype V] [DecidableEq V] [Semiring R]
    (n : ℕ) :
    explicitWordExpansion G S n = walkExpansion G S n := by
  classical
  unfold explicitWordExpansion walkExpansion
  apply Finset.sum_congr rfl
  intro word _
  by_cases hwalk : Composable G (List.ofFn word)
  · simp [hwalk]
  · have hzero : tupleWordOp G S n word = 0 :=
      wordOp_eq_zero_of_not_composable G S (List.ofFn word) hwalk
    simp [hwalk, hzero]
```

When a tuple is composable, the filtered term is unchanged. When it is not, its unfiltered operator was already zero. Combining this equality with the recursive power identity gives `transferOp_pow_eq_walkExpansion`.

2.4 Endpoint coefficients retain the empty path

`reverseCoefficient` recursively multiplies coefficients in operator order. `terminalTarget` records the last endpoint of a nonempty word. `Connects G source target word` combines the endpoint conditions with composability. For an empty word it means `source = target`.

Source: `KomorebiPaperIII/EdgeExtensions.lean`.

```
theorem wordOp_apply_eq_if_connects
    [Fintype V] [DecidableEq V] [Semiring R]
    (word : List E) (source target : V) :
    wordOp G S word target source =
      if Connects G source target word then reverseCoefficient G S word else 0 := by
  cases word with
  | nil =>
      by_cases h : source = target
      · subst target
        simp [wordOp, Connects, reverseCoefficient]
      · have hreverse : target ≠ source := fun h' => h h'.symm
        simp [wordOp, Connects, h, hreverse]
  | cons first rest =>
      by_cases hwalk : Composable G (first :: rest)
      · rw [wordOp_eq_packetMatrix_of_composable G S first rest hwalk]
        simp [packetMatrix, Connects, hwalk, and_comm]
      · rw [wordOp_eq_zero_of_not_composable G S (first :: rest) hwalk]
        simp [Connects, hwalk]
```

This theorem includes both a nonempty composable word and the diagonal entry of the empty identity word. The endpoint sum is consequently valid for every natural length, including zero. The entry of T^n at the chosen endpoints is the sum of the reverse coefficients of exactly those words of length n connecting them.

2.5 What the support theorem implies

`Reachable G source target` means that some finite list connects the vertices. The empty list makes reachability reflexive. Absence of a walk of length n forces the corresponding entry of T^n to vanish. Absence of any walk gives this conclusion at every length.

The source also handles a seed supported on a specified set:

Source: `KomorebiPaperIII/EdgeExtensions.lean`.

```
theorem transferOp_pow_mulVec_apply_eq_zero_of_unreachable_support
    [Fintype E] [Fintype V] [DecidableEq V] [Semiring R]
    (n : ℕ) (x : V → R) (support : Set V) (target : V)
    (hx : ∀ source, source ∉ support → x source = 0)
    (hunreachable : ∀ source, source ∈ support → ¬ Reachable G source target) :
    ((transferOp G S ^ n) *ᵥ x) target = 0 := by
  rw [Matrix.mulVec_apply_eq_sum]
  apply Finset.sum_eq_zero
  intro source _
  by_cases hsource : source ∈ support
  · rw [transferOp_pow_apply_eq_zero_of_not_reachable G S n source target
      (hunreachable source hsource)]
    simp
  · rw [hx source hsource]
    simp
```

For each source vertex, the proof splits on membership in the support. Inside the support, the relevant transfer entry vanishes by unreachability. Outside it, the seed coordinate vanishes. Every term in the sum defining the matrix action is therefore zero.

The direction cannot be reversed without extra assumptions. A graph may contain a path whose coefficient is zero. Distinct paths may cancel over a ring. Thus a zero orbit coordinate does not by itself prove that the target is unreachable. The Lean theorem uses unreachability as a hypothesis and returns zero as a conclusion.

3 Heterogeneous products and exact coordinates

3.1 What varies in a heterogeneous path

There are two different kinds of heterogeneity. The matrix factor may change at every step while the packet index type stays fixed. Alternatively, the source, intermediate, and target packet types themselves may differ. The formal component includes a theorem for arbitrary finite length of the first kind and separate theorems for two and three steps of the second kind.

The statement for two steps displays all three packet types:

Source: `KomorebiPaperIII/EdgeExtensions.lean`.

```
theorem heterogeneousPacketPathExpansion_two
    {I0 : Type u0} {I1 : Type u1} {I2 : Type u2}
    [Fintype I1] [NonUnitalNonAssocSemiring R]
    (first : Matrix I1 I0 R) (second : Matrix I2 I1 R)
    (source : I0) (target : I2) :
    (second * first) target source =
      ∑ middle : I1, second target middle * first middle source := by
  rw [Matrix.mul_apply]
```

Only the intermediate type `I1` needs a finite enumeration. The source and target types are not enumerated in this entrywise identity. The coefficient hypothesis is `NonUnitalNonAssocSemiring R`. No multiplicative unit or associativity is needed for a single entry sum with two factors.

The statement for three steps introduces two finite intermediate types and a semiring:

Source: `KomorebiPaperIII/EdgeExtensions.lean`.

```
theorem heterogeneousPacketPathExpansion_three
    {I0 : Type u0} {I1 : Type u1} {I2 : Type u2} {I3 : Type u3}
    [Fintype I1] [Fintype I2] [Semiring R]
    (first : Matrix I1 I0 R) (second : Matrix I2 I1 R)
    (third : Matrix I3 I2 R) (source : I0) (target : I3) :
    ((third * second) * first) target source =
      ∑ middle2 : I2, ∑ middle1 : I1,
        (third target middle2 * second middle2 middle1) * first middle1 source := by
  rw [Matrix.mul_apply]
  simp_rw [Matrix.mul_apply, Finset.sum_mul]
  rw [Finset.sum_comm]
```

The parentheses on the coefficient product match `(third * second) * first`. The proof expands matrix multiplication twice, distributes multiplication across a finite sum, and exchanges the two finite sums. The packet index types remain separate throughout the calculation.

3.2 Arbitrary finite length on one packet type

`heterogeneousProduct` evaluates a list of square matrices in execution order. The head acts first and appears on the right. `heterogeneousPathSum` independently sums the intermediate packet at each recursive step. Its empty case is the identity coefficient, equal to one when source and target agree and zero otherwise.

Source: `KomorebiPaperIII/EdgeExtensions.lean`.

```
theorem heterogeneousPacketPathExpansion
    [Fintype V] [DecidableEq V] [Semiring R]
    (steps : List (Matrix V V R)) (source target : V) :
    heterogeneousProduct steps target source =
      heterogeneousPathSum steps source target := by
  induction steps generalizing source target with
  | nil =>
      by_cases h : source = target
      · subst target
        simp [heterogeneousProduct, heterogeneousPathSum]
      · have hreverse : target ≠ source := fun h' => h h'.symm
        simp [heterogeneousProduct, heterogeneousPathSum, hreverse]
  | cons step rest ih =>
      simp [heterogeneousProduct, heterogeneousPathSum, Matrix.mul_apply, ih]
```

The induction is on the list of factors. The base case is the matrix identity. The successor case is exactly the entry formula for matrix multiplication, with the induction hypothesis substituted into the remaining product. The theorem quantifies over every finite list, but all factors are `Matrix V V R`.

The traditional companion may treat a fully dependent chain with a new packet index type at every stage. That theorem lies outside the fixed Lean contract of 41 rows. Rows 13 and 14 make the distinction explicit: differing types at lengths two and three, varying factors on one common type at arbitrary finite length.

3.3 Declared support is a certificate

A `SupportedStep` stores an operator, a relation on source and target packets, and a proof that an entry is zero whenever the relation does not hold. The relation need not be the exact nonzero support. It can be a conservative relation of allowed packet transitions. `SupportedPath` says each successive step respects the corresponding relation.

Source: `KomorebiPaperIII/EdgeExtensions.lean`.

```
theorem heterogeneousPathSum_eq_zero_of_no_supportedPath
    [Fintype V] [DecidableEq V] [Semiring R]
    (steps : List (SupportedStep (V := V) (R := R))) (source target : V)
    (h : ¬ SupportedPath steps source target) :
    heterogeneousPathSum (steps.map SupportedStep.operator) source target = 0 := by
  induction steps generalizing source target with
  | nil =>
      have hreverse : target ≠ source := fun h' => h h'.symm
      simp [heterogeneousPathSum, hreverse]
  | cons step rest ih =>
```

```
      simp only [List.map_cons, heterogeneousPathSum]
      apply Finset.sum_eq_zero
      intro middle _
      by_cases hedge : step.relation source middle
      · have hrest : ¬ SupportedPath rest middle target := by
          intro hpath
          exact h ⟨middle, hedge, hpath⟩
        rw [ih middle target hrest]
        simp
      · rw [step.supported source middle hedge]
        simp
```

The proof splits at every candidate intermediate packet. If the first relation fails, the first matrix entry is zero by the stored certificate. If it holds, absence of an entire supported path forces absence of a supported tail, and the induction hypothesis makes the tail sum zero. This is useful even when exact nonzero support would be expensive to decide.

3.4 Exact frames point from coordinates to ambient space

In `Transport.lean`, a frame synthesizes an ambient vector from its coordinate representation. The homogeneous model uses an automorphism of one module, so source and target types look alike, but the direction still matters:

Source: `KomorebiPaperIII/Transport.lean`.

```
def exactTransport (frame : X → M ≃ₗ[R] M) (a b : X) : M ≃ₗ[R] M :=
  (frame a).trans (frame b).symm

@[simp] theorem exactTransport_apply (frame : X → M ≃ₗ[R] M)
    (a b : X) (x : M) :
    exactTransport frame a b x = (frame b).symm (frame a x) :=
  rfl
```

If Ψ_a is the frame, this is $\Psi_b^{-1}\Psi_a$. It first synthesizes through frame a, then analyzes through frame b. Cancellation of the intervening frame proves the cocycle law:

Source: `KomorebiPaperIII/Transport.lean`.

```
theorem exactTransport_cocycle (frame : X → M ≃ₗ[R] M) (a b c : X) :
    (exactTransport frame a b).trans (exactTransport frame b c) =
      exactTransport frame a c := by
  apply LinearEquiv.ext
  intro x
  simp [exactTransport]
```

The proof applies extensionality and simplifies an equivalence followed by its inverse. The results for self transport, return trips, triangles, and finite paths follow from the same cancellation. Exact transport obtained from one frame family cannot acquire a nontrivial loop merely by changing the path.

3.5 Exact coordinate systems point the other way

`CoordinateNaturality.lean` uses an equivalence `X ≃ Source` from an ambient type to a coordinate type. This is an analysis map, inverse in direction to the frame convention above.

Source: `KomorebiPaperIII/CoordinateNaturality.lean`.

```
def exactCoordinateTransport
    {X : Type u} {Source : Type v} {Target : Type w}
    (source : X ≃ Source) (target : X ≃ Target) : Source ≃ Target :=
  source.symm.trans target

@[simp] theorem exactCoordinateTransport_apply
    {X : Type u} {Source : Type v} {Target : Type w}
    (source : X ≃ Source) (target : X ≃ Target) (value : Source) :
    exactCoordinateTransport source target value = target (source.symm value) :=
  rfl
```

For analysis equivalences $\Phi_a : X \to A$ and $\Phi_b : X \to B$, the transport is $\Phi_b \Phi_a^{-1}$. Both conventions express the same operation of synthesizing and then analyzing. Check the direction of the supplied equivalence before comparing the formulas. This avoids an apparent inverse error.

> **Two directions for exact coordinates**
>
> A frame takes coordinates to ambient space. An analysis equivalence takes ambient data to coordinates. In both conventions, transport first returns to the ambient object and then enters the destination coordinates. The different written formulas reflect the different directions of the supplied equivalences.

An arbitrary function $f : X \to Y$ has coordinate representative

$$f_{a,r} = \Phi_r f \Phi_a^{-1}.$$

The type is an ordinary function `Source → Target`. It need not be linear, invertible, continuous, or differentiable. Composition is preserved exactly:

Source: `KomorebiPaperIII/CoordinateNaturality.lean`.

```
theorem exactCoordinateMap_comp
    {X : Type u} {Y : Type v} {Z : Type w}
    {CX : Type x} {CY : Type y} {CZ : Type z}
    (xCoordinate : X ≃ CX) (yCoordinate : Y ≃ CY)
    (zCoordinate : Z ≃ CZ) (first : X → Y) (second : Y → Z) :
    exactCoordinateMap xCoordinate zCoordinate (second ∘ first) =
      exactCoordinateMap yCoordinate zCoordinate second ∘
        exactCoordinateMap xCoordinate yCoordinate first := by
  funext value
  simp [exactCoordinateMap]
```

The middle coordinate analysis and synthesis cancel. The source proves equality of functions with `funext`, then simplifies at one value. No approximation is involved.

3.6 The naturality square is a typed equality

Source: `KomorebiPaperIII/CoordinateNaturality.lean`.

```
theorem exactCoordinateMap_naturality
    {X : Type u} {Y : Type v}
    {XA : Type w} {XB : Type x} {YR : Type y} {YS : Type z}
    (sourceA : X ≃ XA) (sourceB : X ≃ XB)
    (targetR : Y ≃ YR) (targetS : Y ≃ YS) (map : X → Y) :
    exactCoordinateTransport targetR targetS ∘
        exactCoordinateMap sourceA targetR map =
      exactCoordinateMap sourceB targetS map ∘
        exactCoordinateTransport sourceA sourceB := by
  funext value
  simp [exactCoordinateTransport, exactCoordinateMap, Function.comp_def]
```

On the left, represent the map in the first coordinate systems and then change target coordinates. On the right, change source coordinates first and then use the second representative. Both sides map `XA` to `YS`. Expanding them gives the same function.

This exact statement about sets already applies to a nonlinear function. It does not imply differentiability, a chain rule for jets, or convergence of a formal series. Those stronger properties require further structure and separate theorems. The formal contribution here is exact representation and naturality for arbitrary functions under exact equivalences.

4 Connections, defects, and holonomy

4.1 The minimal interface separates a choice from a theorem

The minimal `Connection R M X` stores an equivalence for each ordered pair of vertices, together with the identity law at equal endpoints.

Source: `KomorebiPaperIII/Transport.lean`.

```
structure Connection (R : Type u) (M : Type v) (X : Type w)
    [Semiring R] [AddCommMonoid M] [Module R M] where
  transport : X → X → M ≃ₗ[R] M
  transport_self : ∀ x, transport x x = LinearEquiv.refl R M
```

Neither reversal by inversion nor the cocycle equation is a field. Both could fail for an instance. This makes the distinction from exact frame transport substantive: exact transport has a prescribed construction that forces the cocycle law, while a connection carries separately chosen pair maps.

The minimal connection's defect is an equivalence. It compares the direct map with the route through an intermediate vertex by composing with an inverse:

Source: `KomorebiPaperIII/Transport.lean`.

```
def cocycleDefect (a b c : X) : M ≃ₗ[R] M :=
  (C.transport a c).symm.trans
    ((C.transport a b).trans (C.transport b c))
```

With ordinary function composition, this is

$$D_{a,b,c} = (\Gamma_{b,c}\Gamma_{a,b})\Gamma_{a,c}^{-1}.$$

It is based at the target coordinate space. In this multiplicative model, absence of a defect means that the comparison is the *identity*, rather than an additive zero map. Since the fibers here are homogeneous, the Lean type does not itself display which endpoint is the base. The definition fixes that interpretation.

4.2 A counterexample to automatic flatness in the source

`doubleEquiv` multiplies rational coordinates by two and divides by two for its inverse. The connection on three points chooses it for the directed edge $0 \to 1$ and chooses the identity for every other unequal pair.

Source: `KomorebiPaperIII/Transport.lean`.

```
def twistedThreePointConnection : Connection ℚ ℚ (Fin 3) where
  transport a b :=
    if a = b then LinearEquiv.refl ℚ ℚ
    else if a = (0 : Fin 3) ∧ b = (1 : Fin 3) then doubleEquiv
    else LinearEquiv.refl ℚ ℚ
  transport_self a := by simp
```

The loop $0 \to 1 \to 2 \to 0$ doubles the input one. Equality with the identity would therefore force $2 = 1$. The source proves nonidentity by applying the proposed equality of equivalences to that input:

Source: KomorebiPaperIII/Transport.lean.

```
theorem twisted_triangleHolonomy_ne_refl :
    twistedThreePointConnection.triangleHolonomy
        (0 : Fin 3) (1 : Fin 3) (2 : Fin 3) ≠ LinearEquiv.refl ℚ ℚ := by
  intro h
  have happly := congrArg (fun e : ℚ ≃ₗ[ℚ] ℚ ↦ e (1 : ℚ)) h
  norm_num [Connection.triangleHolonomy, twistedThreePointConnection,
    doubleEquiv] at happly
```

This is an example for the minimal connection interface. The reverse edge $1 \to 0$ is the identity, so it is not the inverse of doubling. The example must not be advertised as an instance of the stronger reversible interface.

4.3 Reversible connections use dependent fibers

The second interface assigns a module $F(a)$ to each vertex. Pair transport is a linear equivalence between the actual endpoint modules, and reverse transport is required to be its inverse.

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
structure ReversibleConnection
    (R : Type u) (X : Type v) (F : X → Type w)
    [CommRing R] [∀ a, AddCommGroup (F a)] [∀ a, Module R (F a)] where
  transport : ∀ a b, F a ≃ₗ[R] F b
  transport_self : ∀ a, transport a a = LinearEquiv.refl R (F a)
  transport_reverse : ∀ a b, transport b a = (transport a b).symm
```

The hypotheses include a commutative ring, additive commutative groups on all fibers, and module structures. These permit subtraction of parallel linear maps. The cocycle law is still not a structural field. Thus reversible connections may have nontrivial triangle holonomy.

The additive defect has a source and target visible in its type:

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
def cocycleDefect (a b c : X) : F a →ₗ[R] F c :=
  (C.transport b c).toLinearMap.comp (C.transport a b).toLinearMap -
    (C.transport a c).toLinearMap
```

Write

$$\Delta_{a,b,c} = \Gamma_{b,c}\Gamma_{a,b} - \Gamma_{a,c} \quad : F(a) \longrightarrow F(c).$$

The triangle holonomy is

$$H_{a,b,c} = \Gamma_{c,a}\Gamma_{b,c}\Gamma_{a,b} \quad : F(a) \longrightarrow F(a),$$

and the triangle obstruction is $O_{a,b,c} = H_{a,b,c} - I$. Unlike the minimal multiplicative defect, this Δ is a linear map that need not be invertible. Both APIs use the short name `cocycleDefect`, but they live in different namespaces and have different types.

4.4 The identity between defect and holonomy

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
theorem triangleHolonomy_sub_id (a b c : X) :
    C.triangleObstruction a b c =
      (C.transport c a).toLinearMap.comp (C.cocycleDefect a b c) := by
  ext x
  simp only [triangleObstruction, triangleHolonomy, cocycleDefect,
    LinearMap.sub_apply, LinearMap.id_apply, LinearMap.comp_apply,
    LinearEquiv.coe_toLinearMap, LinearEquiv.trans_apply, map_sub]
  rw [C.transport_reverse a c]
  simp
```

Pointwise, the identity reads

$$(H - I)x = \Gamma_{c,a}(\Gamma_{b,c}\Gamma_{a,b}x - \Gamma_{a,c}x).$$

Linearity moves the subtraction through $\Gamma_{c,a}$. The reverse law makes $\Gamma_{c,a}\Gamma_{a,c}$ the identity. The source proof uses exactly these two steps after unfolding the definitions.

Because $\Gamma_{c,a}$ is injective, the obstruction is zero exactly when the defect is zero. Vanishing of the additive defect is the cocycle equation. Vanishing of holonomy minus identity means the holonomy is the identity. The theorem chain establishes these equivalences at a fixed ordered triple before quantifying over every triple.

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
theorem flat_iff_triangleHolonomy_eq_refl :
    C.Flat ↔ ∀ a b c,
      C.triangleHolonomy a b c = LinearEquiv.refl R (F a) := by
  constructor
  · intro hflat a b c
    rw [← C.cocycleDefect_eq_zero_iff_triangleHolonomy_eq_refl a b c,
      C.cocycleDefect_eq_zero_iff_cocycleAt]
    exact hflat a b c
  · intro hhol a b c
    rw [← C.cocycleDefect_eq_zero_iff_cocycleAt,
      C.cocycleDefect_eq_zero_iff_triangleHolonomy_eq_refl]
    exact hhol a b c
```

Here `Flat` means the pair-cocycle law at every triple of vertices. It is a global algebraic condition on all pair transports. It is not a differential curvature condition obtained from a limiting process or a local chart cover.

4.5 Finite paths and their endpoint types

The path function on lists returns `F start ≃₁[R] F (pathEndpoint start vertices)`. The return type depends on the endpoint computed from the list. Flatness lets induction replace every remaining segment by direct transport, then collapse the next triangle by the cocycle law.

The indexed alternative packages the endpoints in the path's type. `VertexPath X a b` is a finite path from a to b. An empty path has equal endpoints. A constructor prefixes a step to a path from the intermediate vertex. Its transport theorem is:

Source: `KomorebiPaperIII/ReversibleConnections.lean`.

```
theorem indexedPathTransport_eq_direct_of_flat (hflat : C.Flat) :
    ∀ {a b : X} (path : VertexPath X a b),
      indexedPathTransport C path = C.transport a b := by
  intro a b path
  induction path with
  | nil a => exact (C.transport_self a).symm
  | cons a b c rest ih =>
      simp only [indexedPathTransport]
      rw [ih]
      exact hflat a b c
```

The loop corollary takes `path : VertexPath X a a`. Its output is then already an automorphism of `F a`, and the self law identifies it with the identity. This avoids an extra coercion between a computed endpoint and the initial vertex.

The paths are finite syntax. Nothing here states that an infinite composition exists or that approximating finite paths converge. The finite endpoint theorem is exact on its stated domain.

5 Gauge reconstruction and packet obstruction blocks

5.1 The gauge action fixes its own convention

A gauge is a family of fiber automorphisms $G_a : F(a) \to F(a)$. The source convention is

$$\Gamma^G_{a,b} = G_b \Gamma_{a,b} G_a^{-1}.$$

The definition has to prove that the new pair maps satisfy the self and reverse laws, because its return value is again a reversible connection.

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
def gauge (G : Gauge (R := R) (F := F)) : ReversibleConnection R X F where
  transport a b := ((G a).symm.trans (C.transport a b)).trans (G b)
  transport_self a := by
    apply LinearEquiv.ext
    intro x
    rw [C.transport_self]
    simp
  transport_reverse a b := by
    rw [C.transport_reverse a b]
    apply LinearEquiv.ext
    intro x
    simp
```

The proof of each structural law is pointwise cancellation. Flatness is not needed to define the gauge action.

The defect has different source and target fibers, so it transforms with a gauge at each end:

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
theorem gauge_cocycleDefect
    (G : Gauge (R := R) (F := F)) (a b c : X) :
    (C.gauge G).cocycleDefect a b c =
      (G c).toLinearMap.comp
        ((C.cocycleDefect a b c).comp (G a).symm.toLinearMap) := by
  ext x
  simp [gauge, cocycleDefect]
```

Thus $\Delta^G_{a,b,c} = G_c \Delta_{a,b,c} G_a^{-1}$. Holonomy is based at a, so both endpoint gauges are at that vertex:

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
theorem gauge_triangleHolonomy
    (G : Gauge (R := R) (F := F)) (a b c : X) :
    (C.gauge G).triangleHolonomy a b c =
      ((G a).symm.trans (C.triangleHolonomy a b c)).trans (G a) := by
  apply LinearEquiv.ext
  intro x
  simp [gauge, triangleHolonomy]
```

The transformed matrix is conjugate to the original. Whether the holonomy is the identity is invariant, and the source also proves that the cocycle condition and flatness are invariant. A coordinate change may alter entries while preserving these exact statements.

5.2 Reconstructing a flat connection from canonical transport

Suppose a family of frames $\Psi_a : F(a) \simeq M$ and a base vertex o are supplied. The canonical connection is $K_{a,b} = \Psi_b^{-1}\Psi_a$. The source defines

$$G_a = \Gamma_{o,a} K_{a,o}.$$

Its type is a gauge on each fiber:

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
def flatGauge (frame : ∀ a, F a ≃ₗ[R] M) (o : X) :
    Gauge (R := R) (F := F) :=
  fun a ↦ (canonicalTransport frame a o).trans (C.transport o a)
```

The reconstruction theorem proves $\Gamma_{a,b} = G_b K_{a,b} G_a^{-1}$ under flatness.

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
theorem flat_isGaugeOfCanonical
    (hflat : C.Flat) (frame : ∀ a, F a ≃ₗ[R] M) (o a b : X) :
    C.transport a b =
      ((C.flatGauge frame o a).symm.trans
        ((canonicalConnection frame).transport a b)).trans
          (C.flatGauge frame o b) := by
  apply LinearEquiv.ext
  intro x
  have hg : (C.flatGauge frame o a).symm =
      (C.transport o a).symm.trans (canonicalTransport frame a o).symm := rfl
  rw [hg]
  have hcanonical_symm (p q : X) :
      (canonicalTransport frame p q).symm = canonicalTransport frame q p := rfl
  rw [hcanonical_symm]
  simp only [flatGauge, canonicalConnection, LinearEquiv.trans_apply]
  rw [show (C.transport o a).symm = C.transport a o by
    rw [C.transport_reverse o a]]
  simp only [canonicalTransport, LinearEquiv.trans_apply,
    LinearEquiv.symm_apply_apply, LinearEquiv.apply_symm_apply]
  have hao : (C.transport a o).trans (C.transport o b) = C.transport a b :=
    hflat a o b
  exact (congrArg (fun e : F a ≃ₗ[R] F b ↦ e x) hao).symm
```

The proof first expands the inverse of the defined gauge, then cancels the canonical frame maps. What remains is $\Gamma_{o,b}\Gamma_{a,o} = \Gamma_{a,b}$, the flatness equation at a, o, b.

The theorem is relative to supplied frames and a supplied vertex. It does not silently choose a vertex from an empty type. Nor does this declaration prove a classification theorem for topological bundles. Its result is an explicit algebraic formula for a flat normalized reversible pair connection on the supplied data.

5.3 A packet resolution need not be orthogonal

The formal interface for packet resolution asks for a finite complete family of idempotent endomorphisms.

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
structure PacketResolution (R : Type u) (M : Type v) (I : Type w)
    [CommRing R] [AddCommGroup M] [Module R M] [Fintype I] where
  projector : I → M →ₗ[R] M
  complete : ∑ i, projector i = LinearMap.id
  idempotent : ∀ i, (projector i).comp (projector i) = projector i
```

Completeness is $\sum_i P_i = I$. Idempotence is $P_i^2 = P_i$. The structure does not assert pairwise orthogonality. The block from input packet ν to output packet μ is

$$A_{\mu\nu} = P_\mu A P_\nu.$$

Reconstruction only needs completeness on both sides:

Source: KomorebiPaperIII/ReversibleConnections.lean.

```
theorem sum_blocks (A : M →ₗ[R] M) :
    ∑ mu, ∑ nu, P.block A mu nu = A := by
  have hcomplete (x : M) : ∑ i, P.projector i x = x := by
    have hx := congrArg (fun L : M →ₗ[R] M ↦ L x) P.complete
    simpa using hx
  ext x
  simp only [block, LinearMap.sum_apply, LinearMap.comp_apply]
  calc
    (∑ mu, ∑ nu, P.projector mu (A (P.projector nu x))) =
        ∑ mu, P.projector mu (A (∑ nu, P.projector nu x)) := by
          apply Finset.sum_congr rfl
          intro mu _
          calc
            (∑ nu, P.projector mu (A (P.projector nu x))) =
                P.projector mu (∑ nu, A (P.projector nu x)) := by
                  exact (map_sum (P.projector mu)
                    (fun nu ↦ A (P.projector nu x)) Finset.univ).symm
            _ = P.projector mu (A (∑ nu, P.projector nu x)) := by
                  congr 1
                  exact (map_sum A
                    (fun nu ↦ P.projector nu x) Finset.univ).symm
    _ = ∑ mu, P.projector mu (A x) := by rw [hcomplete x]
    _ = A x := hcomplete (A x)
```

The proof applies the completeness equation to a vector. It combines the finite inner sum using linearity, applies completeness at the input, and then uses completeness at the output. No orthogonality cancellation appears.

All blocks vanish if and only if the map vanishes:

Source: `KomorebiPaperIII/ReversibleConnections.lean`.

```
theorem blocks_zero_iff (A : M →ₗ[R] M) :
    (∀ mu nu, P.block A mu nu = 0) ↔ A = 0 := by
  constructor
  · intro h
    rw [← P.sum_blocks A]
    simp [h]
  · intro h mu nu
    simp [block, h]
```

The forward direction rewrites the map as its double block sum. The reverse direction substitutes the zero map into the definition of a block. This result is a detector for the whole map even when the projectors overlap.

5.4 Extracting a witness supported in a packet

Idempotence is needed for the stronger witness statement:

Source: `KomorebiPaperIII/ReversibleConnections.lean`.

```
theorem exists_packetWitness_of_block_ne_zero
    (A : M →ₗ[R] M) (mu nu : I) (hne : P.block A mu nu ≠ 0) :
    ∃ x, P.projector nu x = x ∧ P.projector mu (A x) ≠ 0 := by
  classical
  have hexists : ∃ y, P.block A mu nu y ≠ 0 := by
    by_contra h
    apply hne
    ext y
    by_contra hy
    exact h ⟨y, hy⟩
  rcases hexists with ⟨y, hy⟩
  refine ⟨P.projector nu y, ?_, ?_⟩
  · have hid := congrArg (fun L : M →ₗ[R] M ↦ L y) (P.idempotent nu)
    simpa using hid
  · simpa [block] using hy
```

A nonzero block supplies some vector y on which that block is nonzero. The proof then takes $x = P_\nu y$. Idempotence ensures $P_\nu x = x$, and the original nonzero block value becomes $P_\mu A x \neq 0$.

The extraction is classical. It proves existence and does not supply a numerical search routine for an arbitrary module. Its input is a proof that a block is nonzero. It does not decide that predicate for the user.

Apply these results to $O = H - I$ on the base fiber of a fixed triangle. The declarations in rows 33 and 34 show

$$\Delta = 0 \quad \Longleftrightarrow \quad H = I \quad \Longleftrightarrow \quad O = 0 \quad \Longleftrightarrow \quad (\forall \mu, \nu)\ P_\mu O P_\nu = 0.$$

The theorem for the whole map, `obstructionDetection_equivalences`, records the three equivalences joining these four criteria. Nonzero obstruction blocks inherit the theorem giving a witness supported in a packet.

5.5 When a fixed packet pattern survives a gauge

Total obstruction always transforms by conjugation at the base vertex. For a *fixed* packet family to transform block by block, the gauge must commute with every projector.

Source: `KomorebiPaperIII/HolonomyBlocks.lean`.

```
def PacketPreserving (gauge : M ≃ₗ[R] M) : Prop :=
  ∀ packet, gauge.toLinearMap.comp (P.projector packet) =
    (P.projector packet).comp gauge.toLinearMap
```

The source first proves that the inverse gauge also commutes with the projectors. This lets the proof move both the gauge and its inverse past the output and input projectors:

Source: `KomorebiPaperIII/HolonomyBlocks.lean`.

```
theorem block_gauge_of_commutes
    (gauge : M ≃ₗ[R] M) (map : M →ₗ[R] M)
    (hpreserves : P.PacketPreserving gauge) (target source : I) :
    P.block (gaugeConjugate gauge map) target source =
      gaugeConjugate gauge (P.block map target source) := by
  ext value
  simp only [block, gaugeConjugate, LinearMap.comp_apply,
    LinearEquiv.coe_toLinearMap]
  have htarget (y : M) :
      P.projector target (gauge y) = gauge (P.projector target y) :=
    (LinearMap.congr_fun (hpreserves target) y).symm
  have hsource :
      gauge.symm (P.projector source value) =
        P.projector source (gauge.symm value) :=
    LinearMap.congr_fun
      (P.symm_commutes_of_packetPreserving gauge hpreserves source) value
  calc
    P.projector target
        (gauge (map (gauge.symm (P.projector source value)))) =
        gauge
          (P.projector target
            (map (gauge.symm (P.projector source value)))) :=
      htarget _
    _ = gauge
          (P.projector target
            (map (P.projector source (gauge.symm value)))) := by
      rw [hsource]
```

Consequently each block is conjugated separately, and each block's zero or nonzero status is invariant. Row 37 specializes this statement to a triangle obstruction and requires `P.PacketPreserving (G a)` at the base vertex.

An arbitrary gauge need not preserve the blocks of a fixed resolution. The unconditional fact is conjugation of the total obstruction. The conditional fact is conjugation of each fixed block. Keeping those hypotheses distinct is essential when interpreting a packet obstruction diagram.

6 Operation trees and certificates for individual candidates

6.1 The grammar does not assume algebraic laws

The nonlinear module starts with a finite syntax tree. Inputs, constants, unary labels, and binary labels can each have their own type.

Source: `KomorebiPaperIII/NonlinearObstructions.lean`.

```
inductive OperationTree
    (Input : Type u) (Constant : Type v) (Unary : Type w) (Binary : Type x)
    where
  | input : Input → OperationTree Input Constant Unary Binary
  | constant : Constant → OperationTree Input Constant Unary Binary
  | unary : Unary → OperationTree Input Constant Unary Binary →
      OperationTree Input Constant Unary Binary
  | binary : Binary → OperationTree Input Constant Unary Binary →
      OperationTree Input Constant Unary Binary →
      OperationTree Input Constant Unary Binary
```

To evaluate a tree, supply four functions: input values, constant values, unary semantics, and binary semantics. Unary and binary semantics are arbitrary functions on one common value type. They are not assumed additive, multilinear, differentiable, associative, or commutative. This generality is what makes the locality theorem applicable to any chosen semantics.

Every tree has finitely many nodes. The input type itself need not be finite. The function `inputs` computes a finite set of names appearing in the tree:

Source: `KomorebiPaperIII/NonlinearObstructions.lean`.

```
def inputs [DecidableEq Input] :
    OperationTree Input Constant Unary Binary → Finset Input
  | .input i => {i}
  | .constant _ => ∅
  | .unary _ child => inputs child
  | .binary _ left right => inputs left ∪ inputs right
```

A constant reads no input. A unary node reads exactly the names read by its child. A binary node reads the union of its two children's names. Because this is a set, repeated uses of one input are recorded once. The semantics may still depend nonlinearly on repeated occurrences.

6.2 The locality proof follows the syntax

Source: `KomorebiPaperIII/NonlinearObstructions.lean`.

```
theorem eval_congr_on_inputs [DecidableEq Input]
    (tree : OperationTree Input Constant Unary Binary)
    (first second : Input → Value)
    (constantValue : Constant → Value)
    (unaryValue : Unary → Value → Value)
    (binaryValue : Binary → Value → Value → Value)
```

```
    (hagree : ∀ i ∈ tree.inputs, first i = second i) :
    tree.eval first constantValue unaryValue binaryValue =
      tree.eval second constantValue unaryValue binaryValue := by
  induction tree with
  | input i =>
      exact hagree i (by simp [inputs])
  | constant c =>
      rfl
  | unary op child ih =>
      simp only [eval]
      congr 1
      exact ih fun i hi ↦ hagree i (by simpa [inputs] using hi)
  | binary op left right ihLeft ihRight =>
      simp only [eval]
      congr 1
      · exact ihLeft fun i hi ↦ hagree i (by simp [inputs, hi])
      · exact ihRight fun i hi ↦ hagree i (by simp [inputs, hi])
```

In an input case, the agreement hypothesis applies to that input's singleton support. In a constant case, both evaluations are definitionally equal. At a unary node, the induction hypothesis identifies the child values, and applying the same unary function preserves equality. At a binary node, the two child induction hypotheses apply because each child's support lies in the union.

The result states one implication: agreement on the syntactically named inputs gives equal evaluation. It does not claim every named input is semantically influential. For example, an operation may ignore one argument. The syntactic support remains a safe dependency set.

If `tree.inputs = ∅`, agreement on the empty set is vacuous, so the value is independent of the input environment. The source proves this as `eval_eq_of_inputs_eq_empty`. Constants and semantics are held fixed between the two evaluations. Changing the semantics would require a different comparison theorem.

6.3 What the nonlinear extension adds outside this contract

The traditional companion develops stronger ordinary statements about typed or colored trees, multilinear packet expansions, composition defects, and higher-order structures. The finite Lean contract contains the common-value unary/binary grammar and its input-locality theorem. It does not contain the dependent colored-tree development or a tree-level Faà di Bruno theorem.

This boundary is mathematical, not stylistic. A colored tree needs compatibility between node input and output types. A multilinear packet expansion needs algebraic hypotheses absent from arbitrary unary and binary semantics. The existing locality proof cannot certify these statements by a change of terminology.

6.4 Residuals and a fixed proposed solution

The obstruction API asks for a function `residual : Candidate → Packet → Residual`, with only `Zero Residual` assumed. A candidate is a solution when every packet residual is zero. A certificate stores one packet at which its residual is nonzero:

Source: `KomorebiPaperIII/NonlinearObstructions.lean`.

```
structure ObstructionPacket
    (residual : Candidate → Packet → Residual) (candidate : Candidate) where
  packet : Packet
  nonzero : residual candidate packet ≠ 0
```

The candidate appears in the certificate's type. A certificate for one candidate cannot directly be passed as a certificate for a different one. The theorem `ObstructionPacket.not_isSolution` uses the stored packet to contradict the condition that every packet residual is zero.

Classically, failure of the condition that every packet residual is zero supplies a nonempty certificate type. The combined equivalence is:

Source: `KomorebiPaperIII/NonlinearObstructions.lean`.

```
theorem not_isSolution_iff_nonempty_obstructionPacket
    {residual : Candidate → Packet → Residual} {candidate : Candidate} :
    ¬ IsSolution residual candidate ↔
      Nonempty (ObstructionPacket residual candidate) := by
  constructor
  · exact obstructionPacket_of_not_isSolution
  · rintro ⟨certificate⟩
    exact certificate.not_isSolution
```

The output is `Nonempty (ObstructionPacket residual candidate)`. This is a proposition asserting that the certificate type has an inhabitant. It is not a computable function that enumerates packets or finds one within a time bound.

Neither `Packet` nor `Candidate` needs a finite enumeration for this logical equivalence. The adjective "finite" in the overall contract describes the walk sums, paths, packet resolutions, and operation-tree syntax where finiteness is imposed. It should not be misread as an extra `Fintype` assumption on this candidate theorem.

6.5 The quantifiers determine the conclusion

For one proposed candidate c, the theorem says

$$\neg(\forall p,\ r(c,p) = 0) \quad \Longleftrightarrow \quad \text{a certificate of } r(c,p) \neq 0 \text{ exists.}$$

To show that a global solution does not exist, one needs the additional assertion that *every* candidate fails, or an argument producing an obstruction for each candidate. To prove existence of a global solution, one must produce a candidate with every residual zero. Neither conclusion follows merely from the certificate interface.

Once a witness is supplied, the interface makes local failure portable and checkable. The formal type records the residual, the candidate, the packet, and the exact nonzero assertion. A later theorem can build on these data without confusing rejection of one candidate with a universal impossibility result.

7 Evidence, trust, and replay

7.1 The pinned environment is part of the claim

The formal project names Lean `leanprover/lean4:v4.34.0-rc2`, with reported Lean commit `6a10ac8c22beadecabdbb0919c2b50214762f91d`. Its Lake manifest pins Mathlib at `aeb72f1e9981b6d35b426b16dca7759be3e7ba06`. The project version is 1.0.1. Warnings are promoted to errors through `moreLeanArgs = ["-E", "warning"]`.

The preserved release deliberately omits compiled outputs and dependency caches. A fresh environment therefore needs the pinned toolchain and the dependencies named by the manifest. Rebuilding the PDF guide does not need Lean. Replaying the mathematical formalization does.

Descriptive dependency branches such as `master` or `main` are not substitutes for exact manifest revisions. Updating a dependency and obtaining a successful build may be useful, but it is a new verification event. The original event remains tied to the recorded environment.

7.2 A map gate and a namespace gate answer different questions

The authoritative `scripts/verify_theorem_map.py` checks the map row sequence, uniqueness, compiled declaration kinds, manuscript status annotations, inherited references, and explicit audit coverage. In the pinned compiled environment, it distinguishes the 88 theorem targets from the 55 construction targets. A lexical scan can locate these declarations in source, but does not independently elaborate their types.

The kernel audit file asks Lean to print transitive axioms for all mapped targets. It then inspects the compiled `Komorebi.PaperIII` namespace, including generated declarations and implementation auxiliaries. This second step protects the boundary beyond just the manually listed names.

Source: `KomorebiPaperIII/KernelAudit.lean`.

```
run_cmd do
  let env ← getEnv
  let mut audited : Nat := 0
  let mut theorems : Nat := 0
  for (name, info) in env.checked.get.constants.toList do
    if Name.isPrefixOf `Komorebi.PaperIII name then
      audited := audited + 1
      match info with
      | .thmInfo _ => theorems := theorems + 1
      | _ => pure ()
      if info.isUnsafe then
        throwError "unsafe Paper III declaration: {name}"
      match info with
      | .axiomInfo _ =>
          throwError "Paper III axiom declaration: {name}"
      | .opaqueInfo _ =>
          throwError "opaque Paper III declaration: {name}"
      | _ => pure ()
      let axioms ← Lean.collectAxioms name
      for axiomName in axioms do
        unless axiomName == ``propext || axiomName == ``Classical.choice ||
            axiomName == ``Quot.sound do
```

```
        throwError "unexpected axiom {axiomName} in Paper III declaration {name}"
if audited != 468 then
  throwError "Paper III namespace declaration count changed: expected 468, found {audited}"
if theorems != 229 then
  throwError "Paper III namespace theorem count changed: expected 229, found {theorems}"
logInfo m!"FAIL-CLOSED PAPER III v1.0.1 KERNEL AUDIT PASS: {audited} declarations ({theorems}
 ↪ theoremInfo); 143 explicit mapped targets; allowed axioms only: propext, Classical.choice,
 ↪ Quot.sound"
```

The gate rejects unsafe declarations, project axioms, opaque declarations, and any collected axiom outside `propext`, `Classical.choice`, and `Quot.sound`. It also checks the exact namespace and theorem-kind counts. These expected counts are version-specific: a legitimate addition requires revisiting them, not deleting the checks to make the build pass.

The source policy check additionally rejects proof placeholders and specified shortcuts. Text scanning alone cannot establish a theorem's correctness. It is one gate among the build, compiled map checks, axiom traversal, and checks of source integrity.

7.3 Reading the inherited transcript

The saved transcript has 143 named records. Some say that the target depends on no axioms. Others list a subset of the allowed standard basis. The final line records 468 audited namespace declarations, including 229 `theoremInfo` entries, and 143 explicit mapped targets.

The difference between 88 and 229 is not a discrepancy. Eighty-eight counts the theorem declarations deliberately mapped to the paper. The compiled namespace count includes generated theorems and other supporting theorem entries. Similarly, 143 and 468 refer to different denominators: the explicitly mapped targets versus every compiled entry under the audited namespace.

This guide checked that the transcript's named records agree with the map and the audit directives. It also checked the transcript hash against the inherited report. That establishes consistency and byte identity of the preserved materials. It does not authenticate an external compiler session by itself. A new execution is the direct way to regenerate compiler evidence from those sources.

7.4 The inherited Papers I–II references

Paper III records twelve unique declarations from the separately verified Papers I–II v1.2.1 bundle. They appear in fourteen manuscript references because two refinement declarations are used twice. The bundle is pinned by SHA-256:

```
a20cf9353f0e882834d31e665dfd819a04e43fa606d402309be63e3b94447052
```

The standalone Paper III project does not import or recompile those declarations. Its gate validates inherited annotations against `INHERITED_DEPENDENCY_MANIFEST_v1.2.1.md`. These references must therefore be described as inherited evidence. They are not extra entries in the 143-target Paper III namespace audit. Appendix B prints the exact dependency names and roles.

7.5 Replay from a preserved source copy

The source release offers an offline integrity mode. From its root, run:

```
bash VERIFY.sh
```

This mode checks release membership, checksums, map and audit consistency, manuscript status references, and the saved transcript. It does not run Lean. It is suitable for checking that a copied release still matches its own preserved manifest.

For a new formal run, work in a copy of the preserved release. From `formalization/lean`, run the release's documented commands in order:

```
lake --wfail build
python3 scripts/verify_theorem_map.py
lake env lean -E warning KomorebiPaperIII/KernelAudit.lean
```

Retain the exit code and the complete standard output and error stream for every command. The first command builds the project. The second checks the human map against the compiled environment and the original manuscript. The third regenerates the axiom reports and namespace result. A missing dependency, toolchain failure, timeout, or nonzero exit is not a passing proof run.

The release also supplies `bash VERIFY.sh --full`. That path creates a temporary copy, regenerates figures, runs Lean and its gates, rebuilds the original finite manuscript, and verifies the original artifact hashes. It requires its documented TeX, graphics, Python, and Lean dependencies. It verifies the preserved finite release, not the newly expanded companion PDF.

7.6 Rebuilding this guide

In the editable guide directory, `python3 build.py` runs XeLaTeX four times on the supplied source and generated data. It does not invoke Lean and does not regenerate data. This separation allows a typesetting replay without changing the preserved provenance tables.

To regenerate the guide's static data, run `build_guide_data.py --source PATH`, where `PATH` is the preserved finite release root containing `formalization/lean`, `kernel_audit_output.txt`, and `SHA256SUMS.txt`. In the delivered package, the default points to `../../source/finite`. The generator uses only Python's standard library and never edits the source release.

The data generator writes the complete map, lexical source locations, import list, pins, exact excerpts, and static audit. The locator table for the traditional paper provides editorial navigation supplied by the adjacent traditional source directory or the explicit `--traditional` argument. Changing a locator does not alter the theorem map or extend the formal contract.

7.7 The edition's verification boundary

The guide supplies exact source excerpts, the complete 41-row map, the preserved transcript, and current static reconciliation. Its PDF is newly compiled and inspected. The package records separate computational checks and the paper's earlier bounded mathematical review. These records concern different objects. They do not establish fresh Lean proofs of every ordinary theorem.

The finite v1.0.1 contract has explicit types and evidence. To expand it, add the required types and proofs, update the map, run the pinned or newly documented environment, and retain the new transcript.

8 The complete 41-row formal contract

This index preserves every row, target, status, and boundary of the authoritative v1.0.1 map. Fully qualified declaration names are printed in full. File and line locations are a current lexical navigation aid, not a second compiled type audit. Each target appears in exactly one row. The 143 names comprise 88 theorem declarations and 55 constructions.

The finite limits are part of the statements: edge sums use finite packet and edge types, arbitrary-length heterogeneous products keep one common finite packet type, and packet reconstruction uses a finite complete idempotent family. Rows 38–41 have the separate grammar and candidate-specific scopes explained in Section 6.

8.1 Row 1: Packet graph and scalar packet matrix

Status and boundary. Defined; row is target and column is source

`Komorebi.PaperIII.EdgeOperators.PacketGraph`

structure, `KomorebiPaperIII/EdgeOperators.lean`, line 27.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.packetMatrix`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 36.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.packetMatrix_apply`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 40.

8.2 Row 2: Typed edge system, elementary edge operator, and finite transfer operator

Status and boundary. Defined for the finite scalar matrix model

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem`

structure, `KomorebiPaperIII/EdgeOperators.lean`, line 47.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 55.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp_apply`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 58.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 162.

8.3 Row 3: Edge support outside the target-source block vanishes

Status and boundary. Proved

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp_eq_zero_off_support`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 65.

8.4 Row 4: Scalar packet multiplication and the two-edge composition dichotomy

Status and boundary. Proved; right factor acts first

`Komorebi.PaperIII.EdgeOperators.PacketGraph.packetMatrix_mul_packetMatrix`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 31.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp_mul_edgeOp_eq_zero`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 75.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.edgeOp_mul_edgeOp_of_composable`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 92.

8.5 Row 5: Composable edge words and their operators

Status and boundary. Defined and proved

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.Composable`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 106.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 112.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_nil`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 117.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_singleton`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 121.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_append`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 126.

8.6 Row 6: Broken and noncomposable words vanish

Status and boundary. Proved

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_eq_zero_of_first_break`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 135.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_eq_zero_of_not_composable`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 144.

8.7 Row 7: Recursive, tuple, explicit-word, and walk expansions

Status and boundary. Defined; all sums are finite

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordExpansion`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 170.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.tupleWordOp`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 177.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.explicitWordExpansion`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 182.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.walkExpansion`

def, `KomorebiPaperIII/EdgeOperators.lean`, line 190.

8.8 Row 8: Successor laws for explicit words

Status and boundary. Proved

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.tupleWordOp_snoc`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 198.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.explicitWordExpansion_succ`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 210.

8.9 Row 9: Transfer powers equal recursive, explicit-word, and composable-walk expansions

Status and boundary. Proved

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordExpansion_eq_explicitWordExpansion`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 226.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.explicitWordExpansion_eq_walkExpansion`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 237.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_eq_wordExpansion`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 252.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_eq_explicitWordExpansion`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 263.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_eq_walkExpansion`

theorem, `KomorebiPaperIII/EdgeOperators.lean`, line 274.

8.10 Row 10: Endpoint block and coefficient of a composable word

Status and boundary. Defined and proved; coefficient order follows operator composition

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.reverseCoefficient`

def, `KomorebiPaperIII/EdgeExtensions.lean`, line 49.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.terminalTarget`

def, `KomorebiPaperIII/EdgeExtensions.lean`, line 54.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.Connects`

def, `KomorebiPaperIII/EdgeExtensions.lean`, line 59.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_eq_packetMatrix_of_composable`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 75.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.wordOp_apply_eq_if_connects`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 98.

8.11 Row 11: Fixed-endpoint coefficient sum

Status and boundary. Defined and proved for fixed length

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.endpointWalkCoefficientSum`

def, `KomorebiPaperIII/EdgeExtensions.lean`, line 118.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_apply_eq_sum_walks`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 128.

8.12 Row 12: Fixed-length and global reachability obstructions

Status and boundary. Proved; reachability includes the empty path

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_apply_eq_zero_of_no_walk`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 142.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.Reachable`

def, `KomorebiPaperIII/EdgeExtensions.lean`, line 153.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_apply_eq_zero_of_not_reachable`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 157.

`Komorebi.PaperIII.EdgeOperators.PacketGraph.EdgeSystem.transferOp_pow_mulVec_apply_eq_zero_of_unreachable_support`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 169.

8.13 Row 13: Fully typed two- and three-step packet expansions

Status and boundary. Proved; intermediate packet types may differ

`Komorebi.PaperIII.EdgeOperators.heterogeneousPacketPathExpansion_two`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 195.

`Komorebi.PaperIII.EdgeOperators.heterogeneousPacketPathExpansion_three`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 205.

8.14 Row 14: Arbitrary-length heterogeneous factor expansion

Status and boundary. Defined and proved for varying factors on one common finite packet type

`Komorebi.PaperIII.EdgeOperators.heterogeneousProduct`

def, `KomorebiPaperIII/EdgeExtensions.lean`, line 221.

`Komorebi.PaperIII.EdgeOperators.heterogeneousPathSum`

def, `KomorebiPaperIII/EdgeExtensions.lean`, line 231.

`Komorebi.PaperIII.EdgeOperators.heterogeneousPacketPathExpansion`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 242.

8.15 Row 15: Supported heterogeneous paths and vanishing

Status and boundary. Defined and proved; support relations are explicit data

`Komorebi.PaperIII.EdgeOperators.SupportedStep`

structure, `KomorebiPaperIII/EdgeExtensions.lean`, line 258.

`Komorebi.PaperIII.EdgeOperators.SupportedPath`

def, `KomorebiPaperIII/EdgeExtensions.lean`, line 265.

`Komorebi.PaperIII.EdgeOperators.heterogeneousPathSum_eq_zero_of_no_supportedPath`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 273.

`Komorebi.PaperIII.EdgeOperators.supportedHeterogeneousPacketPathExpansion`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 296.

`Komorebi.PaperIII.EdgeOperators.heterogeneousProduct_apply_eq_zero_of_no_supportedPath`

theorem, `KomorebiPaperIII/EdgeExtensions.lean`, line 304.

8.16 Row 16: Canonical homogeneous linear coordinate transport

Status and boundary. Defined from one frame family

`Komorebi.PaperIII.Transport.exactTransport`

def, `KomorebiPaperIII/Transport.lean`, line 35.

`Komorebi.PaperIII.Transport.exactTransport_apply`

theorem, `KomorebiPaperIII/Transport.lean`, line 38.

8.17 Row 17: Exact self, cocycle, round-trip, and triangle laws

Status and boundary. Proved

`Komorebi.PaperIII.Transport.exactTransport_self`

theorem, `KomorebiPaperIII/Transport.lean`, line 44.

`Komorebi.PaperIII.Transport.exactTransport_cocycle`

theorem, `KomorebiPaperIII/Transport.lean`, line 51.

`Komorebi.PaperIII.Transport.exactTransport_roundTrip`

theorem, `KomorebiPaperIII/Transport.lean`, line 59.

`Komorebi.PaperIII.Transport.exactTransport_triangleLoop`

theorem, `KomorebiPaperIII/Transport.lean`, line 65.

8.18 Row 18: Exact finite path transport telescopes

Status and boundary. Defined and proved for arbitrary finite vertex lists

`Komorebi.PaperIII.Transport.pathEndpoint`

def, `KomorebiPaperIII/Transport.lean`, line 71.

`Komorebi.PaperIII.Transport.exactPathTransport`

def, `KomorebiPaperIII/Transport.lean`, line 76.

`Komorebi.PaperIII.Transport.exactPathTransport_eq_endpoint`

theorem, `KomorebiPaperIII/Transport.lean`, line 84.

`Komorebi.PaperIII.Transport.exactPathTransport_loop`

theorem, `KomorebiPaperIII/Transport.lean`, line 96.

8.19 Row 19: Set-level exact coordinate transport

Status and boundary. Defined and proved for arbitrary equivalences, without linearity assumptions

`Komorebi.PaperIII.Transport.exactCoordinateTransport`

def, `KomorebiPaperIII/CoordinateNaturality.lean`, line 24.

`Komorebi.PaperIII.Transport.exactCoordinateTransport_apply`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 29.

`Komorebi.PaperIII.Transport.exactCoordinateTransport_self`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 36.

`Komorebi.PaperIII.Transport.exactCoordinateTransport_symm`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 43.

`Komorebi.PaperIII.Transport.exactCoordinateTransport_cocycle`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 52.

`Komorebi.PaperIII.Transport.exactCoordinateTransport_triangle`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 61.

8.20 Row 20: Representation, composition, and naturality of arbitrary functions

Status and boundary. Defined and proved at set level

`Komorebi.PaperIII.Transport.exactCoordinateMap`

def, `KomorebiPaperIII/CoordinateNaturality.lean`, line 71.

`Komorebi.PaperIII.Transport.exactCoordinateMap_apply`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 77.

`Komorebi.PaperIII.Transport.exactCoordinateMap_id`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 86.

`Komorebi.PaperIII.Transport.exactCoordinateMap_comp`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 93.

`Komorebi.PaperIII.Transport.exactCoordinateMap_naturality`

theorem, `KomorebiPaperIII/CoordinateNaturality.lean`, line 109.

8.21 Row 21: Minimal homogeneous noncanonical connection and its defects

Status and boundary. Defined; only self-transport is structural

`Komorebi.PaperIII.Transport.Connection`

structure, `KomorebiPaperIII/Transport.lean`, line 113.

`Komorebi.PaperIII.Transport.Connection.CocycleAt`

def, `KomorebiPaperIII/Transport.lean`, line 123.

`Komorebi.PaperIII.Transport.Connection.Flat`

def, `KomorebiPaperIII/Transport.lean`, line 127.

`Komorebi.PaperIII.Transport.Connection.cocycleDefect`

def, `KomorebiPaperIII/Transport.lean`, line 133.

`Komorebi.PaperIII.Transport.Connection.triangleHolonomy`

def, `KomorebiPaperIII/Transport.lean`, line 138.

8.22 Row 22: Flat minimal connections have trivial defect and triangle holonomy

Status and boundary. Proved

`Komorebi.PaperIII.Transport.Connection.triangleHolonomy_eq_refl_of_flat`

theorem, `KomorebiPaperIII/Transport.lean`, line 142.

`Komorebi.PaperIII.Transport.Connection.cocycleDefect_eq_refl_of_flat`

theorem, `KomorebiPaperIII/Transport.lean`, line 147.

8.23 Row 23: Explicit three-point connection with nontrivial loop

Status and boundary. Constructed and proved over the rationals

`Komorebi.PaperIII.Transport.doubleEquiv`

def, `KomorebiPaperIII/Transport.lean`, line 160.

`Komorebi.PaperIII.Transport.doubleEquiv_apply`

theorem, `KomorebiPaperIII/Transport.lean`, line 170.

`Komorebi.PaperIII.Transport.twistedThreePointConnection`

def, `KomorebiPaperIII/Transport.lean`, line 177.

`Komorebi.PaperIII.Transport.twisted_triangleHolonomy_apply_one`

theorem, `KomorebiPaperIII/Transport.lean`, line 185.

`Komorebi.PaperIII.Transport.twisted_triangleHolonomy_ne_refl`

theorem, `KomorebiPaperIII/Transport.lean`, line 192.

8.24 Row 24: Heterogeneous reversible connection and gauges

Status and boundary. Defined over a commutative ring and a family of modules

`Komorebi.PaperIII.Transport.ReversibleConnection`

structure, `KomorebiPaperIII/ReversibleConnections.lean`, line 35.

`Komorebi.PaperIII.Transport.ReversibleConnection.Gauge`

abbrev, `KomorebiPaperIII/ReversibleConnections.lean`, line 49.

8.25 Row 25: Reversible cocycle, flatness, additive defect, holonomy, and obstruction

Status and boundary. Defined; theorem arguments are in traversal order

`Komorebi.PaperIII.Transport.ReversibleConnection.CocycleAt`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 52.

`Komorebi.PaperIII.Transport.ReversibleConnection.Flat`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 56.

`Komorebi.PaperIII.Transport.ReversibleConnection.cocycleDefect`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 62.

`Komorebi.PaperIII.Transport.ReversibleConnection.triangleHolonomy`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 67.

`Komorebi.PaperIII.Transport.ReversibleConnection.triangleObstruction`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 71.

8.26 Row 26: Defect-holonomy identity and equivalent flatness criteria

Status and boundary. Proved exactly

`Komorebi.PaperIII.Transport.ReversibleConnection.cocycleDefect_eq_zero_iff_cocycleAt`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 75.

`Komorebi.PaperIII.Transport.ReversibleConnection.triangleHolonomy_sub_id`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 96.

`Komorebi.PaperIII.Transport.ReversibleConnection.triangleObstruction_eq_zero_iff_cocycleDefect_eq_zero`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 107.

`Komorebi.PaperIII.Transport.ReversibleConnection.triangleObstruction_eq_zero_iff_triangleHolonomy_eq_refl`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 120.

`Komorebi.PaperIII.Transport.ReversibleConnection.cocycleDefect_eq_zero_iff_triangleHolonomy_eq_refl`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 139.

`Komorebi.PaperIII.Transport.ReversibleConnection.flat_iff_triangleHolonomy_eq_refl`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 146.

8.27 Row 27: Canonical reversible frame connection is flat

Status and boundary. Defined and proved

`Komorebi.PaperIII.Transport.ReversibleConnection.canonicalTransport`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 164.

`Komorebi.PaperIII.Transport.ReversibleConnection.canonicalConnection`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 171.

`Komorebi.PaperIII.Transport.ReversibleConnection.canonicalConnection_flat`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 181.

`Komorebi.PaperIII.Transport.ReversibleConnection.canonicalTransport_zero_connectionDefect`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 189.

`Komorebi.PaperIII.Transport.ReversibleConnection.canonicalTransport_triangleHolonomy_eq_refl`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 196.

8.28 Row 28: Every flat reversible connection is gauge-equivalent to canonical transport

Status and boundary. Gauge constructed and identity proved relative to a base vertex and frames

`Komorebi.PaperIII.Transport.ReversibleConnection.flatGauge`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 207.

`Komorebi.PaperIII.Transport.ReversibleConnection.flat_isGaugeOfCanonical`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 215.

8.29 Row 29: List-based reversible path transport is endpoint-only when flat

Status and boundary. Defined and proved for finite lists

`Komorebi.PaperIII.Transport.ReversibleConnection.pathTransport`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 241.

`Komorebi.PaperIII.Transport.ReversibleConnection.pathTransport_eq_direct_of_flat`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 248.

8.30 Row 30: Indexed finite path transport and flat loops

Status and boundary. Defined and proved for dependent fibers

`Komorebi.PaperIII.Transport.ReversibleConnection.VertexPath`

inductive, `KomorebiPaperIII/ReversibleConnections.lean`, line 262.

`Komorebi.PaperIII.Transport.ReversibleConnection.indexedPathTransport`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 267.

`Komorebi.PaperIII.Transport.ReversibleConnection.indexedPathTransport_eq_direct_of_flat`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 274.

`Komorebi.PaperIII.Transport.ReversibleConnection.indexedLoopTransport_eq_refl_of_flat`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 286.

8.31 Row 31: Gauge action, covariance, and invariance

Status and boundary. Defined and proved algebraically

`Komorebi.PaperIII.Transport.ReversibleConnection.gauge`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 294.

`Komorebi.PaperIII.Transport.ReversibleConnection.gauge_cocycleDefect`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 308.

`Komorebi.PaperIII.Transport.ReversibleConnection.gauge_triangleHolonomy`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 317.

`Komorebi.PaperIII.Transport.ReversibleConnection.gauge_cocycleAt_iff`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 326.

`Komorebi.PaperIII.Transport.ReversibleConnection.gauge_flat_iff`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 344.

`Komorebi.PaperIII.Transport.ReversibleConnection.gauge_triangleHolonomy_eq_refl_iff`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 353.

8.32 Row 32: Finite complete idempotent packet resolution and blocks

Status and boundary. Defined; orthogonality is not required

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution`

structure, `KomorebiPaperIII/ReversibleConnections.lean`, line 379.

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.block`

def, `KomorebiPaperIII/ReversibleConnections.lean`, line 391.

8.33 Row 33: Exact block reconstruction, zero detection, and packet witnesses

Status and boundary. Proved; witness extraction is classical

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.sum_blocks`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 395.

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.blocks_zero_iff`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 420.

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.exists_packetWitness_of_block_ne_zero`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 430.

8.34 Row 34: Triangle-obstruction block reconstruction and four-way detection

Status and boundary. Proved for each fixed triangle and finite packet resolution

`Komorebi.PaperIII.HolonomyBlocks.obstruction_sum_blocks`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 452.

`Komorebi.PaperIII.HolonomyBlocks.obstructionBlocks_zero_iff`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 464.

`Komorebi.PaperIII.HolonomyBlocks.obstructionDetection_equivalences`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 479.

`Komorebi.PaperIII.HolonomyBlocks.exists_packetWitness_of_obstructionBlock_ne_zero`

theorem, `KomorebiPaperIII/ReversibleConnections.lean`, line 495.

8.35 Row 35: Gauge conjugation and packet preservation

Status and boundary. Defined

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.gaugeConjugate`

def, `KomorebiPaperIII/HolonomyBlocks.lean`, line 51.

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.PacketPreserving`

def, `KomorebiPaperIII/HolonomyBlocks.lean`, line 55.

8.36 Row 36: Packet-preserving conjugation acts blockwise and preserves vanishing

Status and boundary. Proved

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.symm_commutes_of_packetPreserving`

theorem, `KomorebiPaperIII/HolonomyBlocks.lean`, line 60.

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.block_gauge_of_commutes`

theorem, `KomorebiPaperIII/HolonomyBlocks.lean`, line 70.

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.gaugeConjugate_eq_zero_iff`

theorem, `KomorebiPaperIII/HolonomyBlocks.lean`, line 99.

`Komorebi.PaperIII.HolonomyBlocks.PacketResolution.block_gauge_eq_zero_iff`

theorem, `KomorebiPaperIII/HolonomyBlocks.lean`, line 111.

8.37 Row 37: Gauge covariance of total and packetwise triangle obstructions

Status and boundary. Proved; packetwise result assumes the base gauge commutes with every projector

`Komorebi.PaperIII.Transport.ReversibleConnection.gauge_triangleObstruction`

theorem, `KomorebiPaperIII/HolonomyBlocks.lean`, line 27.

`Komorebi.PaperIII.HolonomyBlocks.obstructionBlock_gauge_of_commutes`

theorem, `KomorebiPaperIII/HolonomyBlocks.lean`, line 131.

`Komorebi.PaperIII.HolonomyBlocks.obstructionBlock_gauge_eq_zero_iff`

theorem, `KomorebiPaperIII/HolonomyBlocks.lean`, line 145.

8.38 Row 38: Nonlinear operation-tree grammar, evaluation, and input support

Status and boundary. Defined for arbitrary unary and binary semantics

`Komorebi.PaperIII.NonlinearObstructions.OperationTree`

inductive, `KomorebiPaperIII/NonlinearObstructions.lean`, line 34.

`Komorebi.PaperIII.NonlinearObstructions.OperationTree.eval`

def, `KomorebiPaperIII/NonlinearObstructions.lean`, line 51.

`Komorebi.PaperIII.NonlinearObstructions.OperationTree.inputs`

def, `KomorebiPaperIII/NonlinearObstructions.lean`, line 66.

8.39 Row 39: Operation-tree packet locality

Status and boundary. Proved without algebraic assumptions on operations

`Komorebi.PaperIII.NonlinearObstructions.OperationTree.eval_congr_on_inputs`

theorem, `KomorebiPaperIII/NonlinearObstructions.lean`, line 77.

`Komorebi.PaperIII.NonlinearObstructions.OperationTree.eval_eq_of_inputs_eq_empty`

theorem, `KomorebiPaperIII/NonlinearObstructions.lean`, line 102.

8.40 Row 40: Candidate-specific solution and obstruction packet

Status and boundary. Defined

`Komorebi.PaperIII.NonlinearObstructions.IsSolution`

def, `KomorebiPaperIII/NonlinearObstructions.lean`, line 126.

`Komorebi.PaperIII.NonlinearObstructions.ObstructionPacket`

structure, `KomorebiPaperIII/NonlinearObstructions.lean`, line 130.

8.41 Row 41: Candidate failure is equivalent to a local obstruction

Status and boundary. Proved; extraction from nonsolution is classical and candidate-specific

`Komorebi.PaperIII.NonlinearObstructions.ObstructionPacket.not_isSolution`

theorem, `KomorebiPaperIII/NonlinearObstructions.lean`, line 136.

`Komorebi.PaperIII.NonlinearObstructions.obstructionPacket_of_not_isSolution`

theorem, `KomorebiPaperIII/NonlinearObstructions.lean`, line 144.

`Komorebi.PaperIII.NonlinearObstructions.not_isSolution_iff_nonempty_obstructionPacket`

theorem, `KomorebiPaperIII/NonlinearObstructions.lean`, line 156.

A Monograph navigation

The following locations point into the monograph companion. Printed pages differ from a PDF viewer's physical page index because the cover is unnumbered and the opening pages use Roman numerals. A locator connects related presentations; it does not certify every neighboring ordinary statement in Lean.

Canonical label	Final printed locator
`sec:purpose`	Section 1, p. 1
`sec:edge-operators`	Section 2, p. 2
`def:edge-system`	Definition 2.1, p. 2
`lem:edge-support`	Lemma 2.2, p. 2
`thm:edge-annihilation`	Theorem 2.3, p. 2
`thm:edge-composable`	Theorem 2.4, p. 3
`sec:words-walks`	Section 3, p. 4
`def:word-op`	Definition 3.1, p. 4
`thm:first-break`	Theorem 3.2, p. 4
`thm:nonwalk-vanish`	Theorem 3.3, p. 4
`def:recursive-word-expansion`	Definition 3.4, p. 4
`thm:power-recursive-expansion`	Theorem 3.5, p. 4
`lem:word-endpoint`	Lemma 3.7, p. 5
`thm:word-to-walk`	Theorem 3.8, p. 5
`cor:word-to-walk-entry`	Corollary 3.9, p. 6
`cor:fixed-reachability`	Corollary 3.10, p. 6
`cor:orbit-reachability`	Corollary 3.11, p. 6
`sec:canonical-transport`	Section 4, p. 7
`thm:exact-frame-transport`	Theorem 4.1, p. 7
`thm:exact-path-transport`	Theorem 4.2, p. 7
`def:canonical-J`	Definition 4.3, p. 8
`thm:J-cocycle`	Theorem 4.4, p. 8
`thm:canonical-naturality`	Theorem 4.5, p. 9
`cor:J-flat`	Corollary 4.6, p. 9
`sec:connection`	Section 5, p. 9
`def:minimal-connection`	Definition 5.1, p. 9
`thm:minimal-flatness`	Theorem 5.2, p. 10
`prop:twisted-loop`	Proposition 5.3, p. 10
`def:connection`	Definition 5.5, p. 11

Canonical label	Final printed locator
def:calibration-flatness	Definition 5.6, p. 12
def:cocycle-defect	Definition 5.8, p. 12
def:triangle-holonomy	Definition 5.9, p. 12
thm:defect-holonomy	Theorem 5.10, p. 13
cor:canonical-zero-defect	Corollary 5.11, p. 13
def:path-transport	Definition 5.12, p. 13
prop:flat-endpoints	Proposition 5.13, p. 14
sec:gauge	Section 6, p. 14
def:gauge-change	Definition 6.1, p. 14
thm:gauge-covariance	Theorem 6.2, p. 14
thm:flat-gauge-canonical	Theorem 6.3, p. 14
sec:obstruction-packets	Section 7, p. 15
def:generic-obstruction	Definition 7.1, p. 15
thm:generic-obstruction	Theorem 7.2, p. 15
def:obstruction-packet	Definition 7.4, p. 16
thm:obstruction-detection	Theorem 7.5, p. 17
cor:packet-witness	Corollary 7.6, p. 18
prop:packet-gauge	Proposition 7.7, p. 18
sec:holonomy-walks	Section 8, p. 19
prop:heterogeneous-path	Proposition 8.1, p. 19
sec:example	Section 9, p. 20
sec:arbitrary-maps	Section 10, p. 20
prop:set-naturality	Proposition 10.1, p. 20
sec:operation-trees	Section 11, p. 22
def:compiled-operation-tree	Definition 11.1, p. 22
thm:tree-locality	Theorem 11.2, p. 22
def:signature	Definition 11.4, p. 22
def:operation-tree	Definition 11.5, p. 23
sec:ledger	Section 12, p. 24

B Inherited declaration names

The following twelve names are inherited citations to the separately verified Papers I–II v1.2.1 bundle. The standalone Paper III build does not recompile them. Their fourteen manuscript uses are checked against the frozen manifest.

1. Reachability support hull

`Komorebi.PacketNaturalityCompletion.Reachability.algebraicOrbitHull_le_reachable_support`

2. Unreachable-packet obstruction

`Komorebi.PacketNaturalityCompletion.Reachability.unreachable_nonzero_packet_obstruction`

3. Nested coordinate refinement

`Komorebi.PaperII.NestedRefinement.refineCoordinates`

4. Refinement equivalence

`Komorebi.PaperII.NestedRefinement.coordinateRefinementEquiv`

5. Identity refinement

`Komorebi.PaperII.NestedRefinement.refineCoordinates_self`

6. Coarsen-after-refine identity

`Komorebi.PaperII.NestedRefinement.coarsenCoordinates_refineCoordinates`

7. Refine-after-coarsen identity

`Komorebi.PaperII.NestedRefinement.refineCoordinates_coarsenCoordinates`

8. Refinement cocycle and functoriality

`Komorebi.PaperII.NestedRefinement.refineCoordinates_comp`

9. Transitivity of refinement equivalences

`Komorebi.PaperII.NestedRefinement.coordinateRefinementEquiv_trans`

10. Coordinate-operator composition

`Komorebi.PaperII.NestedRefinement.coordinateOperator_comp`

11. Coordinate-operator naturality

`Komorebi.PaperII.NestedRefinement.coordinateOperator_naturality`

12. Prior packet-path power identity

`Komorebi.PacketNaturalityCompletion.Packets.transferBlock_pow_eq_packetPathSum`

C Imports and exact dependency revisions

The import table lists direct local file dependencies, not a theorem-level dependency graph. The adjacent JSON includes direct external imports as well. The aggregate module imports seven mathematical modules; the audit module imports the aggregate.

Module	Direct local imports
`CoordinateNaturality`	External imports only.
`EdgeExtensions`	`EdgeOperators`
`EdgeOperators`	External imports only.
`HolonomyBlocks`	`ReversibleConnections`
`KernelAudit`	`KomorebiPaperIII`
`NonlinearObstructions`	External imports only.
`ReversibleConnections`	`Transport`
`Transport`	External imports only.
`KomorebiPaperIII`	`EdgeOperators`, `EdgeExtensions`, `Transport`, `ReversibleConnections`, `CoordinateNaturality`, `HolonomyBlocks`, `NonlinearObstructions`

The manifest's exact pinned revisions follow. They accompany the toolchain string `leanprover/lean4:v4.34.0-rc2`.

Package	Exact revision
mathlib	`aeb72f1e9981b6d35b426b16dca7759be3e7ba06`
plausible	`d9598f07b1bc701f1e3aae163d2681c1fd978793`
LeanSearchClient	`ba67e212be1197b84c1f1f6299488a10a3002713`
importGraph	`d8823026ac7ef130c253089d95685f9877b95323`
proofwidgets	`a8acbfd87375ff4abe14ce09db5b7664d383bc7f`
aesop	`18889deb9e83ea7420ef51c160d6f88552e744e3`
Qq	`507746ab8f4b643ccdacb2ec4cdb5853fa9f8ab3`
batteries	`4cac2177c37f5530c4da76aa8e4307f3fc9e4dcb`
Cli	`ab3a82db9fea14cf0fd7f5a2de650f4b534640af`

D Source and reproduction inventory

The preserved formal component has nine Lean files, totaling 1,948 source lines, and sixteen files including the map, manifest, gate, pins, and documentation. The guide stores 44 exact excerpts. For each, `generated/excerpts.json` records the source file, inclusive line range, full-source hash, exact-excerpt hash, and PDF-display-file hash.

`generated/theorem_map.json` contains all 41 rows and complete source locations. `generated/mapped_source_index.json` provides an alphabetical index. `generated/inherited_axiom_records.json` records the 143 saved transcript reports. `generated/static_audit.json` separates current checks from inherited counts and records the checksums used. The original report and transcript remain in the preserved finite release.

`build.py` is a compile-only XeLaTeX builder. `build_guide_data.py` regenerates static data from an explicitly supplied preserved release. `verify_pdf.py` renders the PDF, prepares contact sheets, and records page-level layout checks. `BUILD_NOTES.txt` records authoring failures and their repairs, rather than silently treating failed attempts as successful verification.

The guide uses P052 for body text, Nimbus Sans for headings, DejaVu Sans Mono for code, and Latin Modern Math for mathematical symbols. Code displays use `fvextra`, preserving token order. Display fallbacks are documented separately from the exact UTF-8 source excerpts.